Tablelist

Programmierhandbuch (deutsch)

2026-10-27

Inhalt

index **3**
tablelist **3**
tablelistBWidget **61**
tablelistBinding **63**
tablelistColSort **66**
tablelistCombobox **68**
tablelistCtext **70**
tablelistIwidgets **72**
tablelistMentry **76**
tablelistThemes **81**
tablelistTile **84**
tablelistTkCore **88**
tablelistTsw **91**
tablelistWidget **92**
Lizenz **248**

index

Stichwörter: index

Das Paket "Tablelist 7.12" mit mehrspaltigen Listbox- und Baum-Widgets

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Tablelist-Programmierhandbuch
Der Befehl tablelist::tablelist
Befehle zum interaktiven Sortieren nach einer oder mehreren Spalten
Befehle im Zusammenhang mit Bindungsskripten
Befehle im Zusammenhang mit Tile-Themes
Interaktive Bearbeitung von Tabellenzellen mit Tk-Core-Widgets
Interaktive Bearbeitung von Tabellenzellen mithilfe von Tile-Widgets
Interaktive Bearbeitung von Tabellenzellen mit dem BWidget-Paket
Interaktive Bearbeitung von Tabellenzellen mit dem Iwidgets-Paket
Interaktive Bearbeitung von Tabellenzellen mit dem combobox-Paket
Interaktive Bearbeitung von Tabellenzellen mit dem ctext-Paket
Interaktive Bearbeitung von Tabellenzellen mit dem Mentry-Paket
Interaktive Bearbeitung von Tabellenzellen mit dem Tsw-Paket

tablelist

Stichwörter: tablelist

Tablelist-Programmierhandbuch

Für Tablelist Version 7.12

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Überblick
- Was ist Tablelist?

- Wie bekomme ich es?
- Wie wird es installiert?
- Wie wird es angewendet?
- Mehr zu tablelist::scalingpct
- Die Prozedur tablelist::getScalingPct und der Skalierungsfaktor von Tk
- Mehr zu tablelist::svgfmt
- Mehr zu Tablelist_tile

Beispiele
- Ein Tablelist-Widget zum Anzeigen und Bearbeiten von Konfigurationsoptionen
- Zwei skalierbare Widget-Browser auf Basis einer Tablelist
- Ein skalierbarer Verzeichnis-Viewer auf Basis einer Tablelist
- Verbesserung des Erscheinungsbilds eines Tablelist-Widgets
- Erweiterte, skalierbare und interaktive Bearbeitung von Tabellenzellen
- Ein Tablelist-Widget mit skalierbaren eingebetteten Fenstern
- Demo-Skripte auf Tile-Basis

Überblick

Was ist Tablelist?

Tablelist ist ein Paket für Tcl/Tk ab Version 8.4, das in reinem Tcl/Tk-Code geschrieben ist. Es enthält:

- die Implementierung des tablelist-Megawidgets, einschließlich eines allgemeinen Dienstprogrammmoduls für Megawidgets;
- ein Demo-Skript, das eine nützliche Prozedur enthält, welche die Konfigurationsoptionen eines beliebigen Widgets in einer Tablelist anzeigt und es Ihnen ermöglicht, deren Werte interaktiv zu bearbeiten;
- ein Demo-Skript zur Implementierung eines Widget-Browsers auf Basis einer Tablelist, die als mehrspaltige listbox verwendet wird;
- ein Demo-Skript zur Implementierung eines Widget-Browsers auf Basis einer Tablelist, die als mehrspaltiges Baum-Widget verwendet wird;
- ein Demo-Skript zur Implementierung eines Verzeichnis-Viewers auf Basis einer Tablelist, die als mehrspaltiges Baum-Widget verwendet wird;
- ein Demo-Skript, das verschiedene Möglichkeiten zur Verbesserung der Darstellung eines Tablelist-Widgets zeigt;
- fünf weitere Demo-Skripte, die die interaktive Bearbeitung von Zellen mithilfe verschiedener Widgets aus dem Tk-Kern sowie aus den Paketen "tile", "BWidget", "Iwidgets", "combobox" (von Bryan Oakley), "Mentry" und "Tsw" veranschaulichen;
- ein weiteres Demo-Skript mit einem "Tablelist"-Widget, das eingebettete Fenster enthält;
- auf Tile aufbauende Entsprechungen der oben genannten Demo-Skripte;
- dieses Tutorial;
- Referenzseiten im HTML-Format.

Eine "Tablelist" ist ein mehrspaltiges listbox- und Baum-Widget. Die Breite jeder Spalte kann dynamisch sein (d. h. gerade so breit, dass alle Elemente einschließlich der Kopfzeile Platz finden) oder statisch (in Zeichen oder Pixeln festgelegt). Die Spalten sind standardmäßig in der Größe anpassbar. Die Ausrichtung jeder Spalte kann als `left`, `right` oder `center` festgelegt

werden.

Die Spalten, Zeilen und Zellen lassen sich individuell konfigurieren. Mehrere der globalen und spaltenspezifischen Optionen beziehen sich auf die Kopfzeilen, die als Beschriftungs-Widgets implementiert sind. So legt beispielsweise die Option `-labelcommand` einen Tcl-Befehl fest, der aufgerufen wird, wenn die linke Maustaste über einer Kopfzeilenbeschriftung losgelassen wird. Der häufigste Wert dieser Option sortiert die Elemente nach der jeweiligen Spalte.

Das "Tablelist"-Paket bietet eine große Auswahl an Baumstilen, mit denen sich das Erscheinungsbild der Spalte, in der die Baumhierarchie angezeigt wird, mithilfe von Einrückungen und Steuerelementen zum Ein- und Ausklappen anpassen lässt.

Die interaktive Bearbeitung der Elemente eines Tablelist-Widgets kann für einzelne Zellen und für ganze Spalten aktiviert werden. Eine große Vielfalt an Widgets aus dem Tk-Kern sowie aus den Paketen "tile", "BWidget", "Iwidgets", "combobox", "ctext" und "Mentry" (bzw. "Mentry_tile") wird zur Verwendung als eingebettetes Fenster zur Bearbeitung unterstützt. Darüber hinaus steht eine umfangreiche Auswahl an Tastatur-Bindungen zur Verfügung, die eine komfortable Navigation zwischen den bearbeitbaren Zellen ermöglichen.

Der Tcl-Befehl für ein Tablelist-Widget ist dem für eine normale listbox sehr ähnlich. Es gibt spalten-, zeilen- und zellenspezifische Entsprechungen der Unterbefehle `configure` und `cget` (`columnconfigure`, `rowconfigure`, `cellconfigure`, ...). Sie können unter anderem dazu verwendet werden, Bilder und eingebettete Fenster in die Zellen und die Kopfzeilenbeschriftungen einzufügen. Die Befehlsoptionen `index`, `nearest` und `see` beziehen sich auf die Zeilen, doch für die Spalten und Zellen stehen ähnliche Unterbefehle zur Verfügung (`columnindex`, `cellindex`, ...). Die Elemente können mit den Optionen `sort`, `sortbycolumn` und `sortbycolumnlist` sortiert werden.

Durch die für den Datenbereich eines Tablelist-Widgets definierten Bindungen verhält sich dieses genau wie eine normale listbox. Dazu gehört auch die Unterstützung des virtuellen Ereignisses `<<ListboxSelect>>` (das dem Ereignis `<<TablelistSelect>>` entspricht). Darüber hinaus können Versionen 2.3 oder höher des Widget-Callback-Pakets Wcb (ebenfalls in reinem Tcl/Tk-Code geschrieben) verwendet werden, um Rückruffunktionen für die Befehle `activate`, `selection set` und `selection clear` zu definieren; Wcb-Versionen ab 3.0 unterstützen zudem Rückruffunktionen für die Befehle `activatecell`, `cellselection set` und `cellselection clear`. Der Download-Ort von Wcb ist

https://www.nemethi.de

Wie bekomme ich es?

Tablelist steht unter derselben URL wie Wcb zum kostenlosen Download bereit. Die Distributionsdatei lautet `tablelist7.12.tar.gz` für UNIX und `tablelist7_12.zip` für Windows. Diese Dateien enthalten dieselben Informationen, mit Ausnahme des zusätzlichen Wagenrücklaufzeichens, das in den Textdateien für Windows dem Zeilenvorschub am Ende jeder Zeile vorangestellt ist.

"Tablelist" ist ebenfalls in "tklib" enthalten, dessen Adresse lautet

https://core.tcl-lang.org/tklib

Wie wird es installiert?

Installieren Sie das Paket als Unterverzeichnis eines der Verzeichnisse, die durch die Variable `auto_path` angegeben sind. Sie können es beispielsweise als Unterverzeichnis des

Verzeichnisses `lib` innerhalb Ihrer Tcl/Tk-Installation installieren.

Um Tablelist *unter UNIX* zu installieren, wechseln Sie mit `cd` in das gewünschte Verzeichnis und entpacken Sie die Distributionsdatei `tablelist7.12.tar.gz`:

```
gunzip -c tablelist7.12.tar.gz | tar -xf -
```

Auf den meisten UNIX-Systemen kann dies durch Folgendes ersetzt werden:

```
tar -zxf tablelist7.12.tar.gz
```

Beide Befehle erstellen ein Verzeichnis mit dem Namen `tablelist7.12`, das die Unterverzeichnisse `demos`, `doc` und `scripts` enthält.

Unter Windows verwenden Sie WinZip oder ein anderes Programm, mit dem Sie die Distributionsdatei `tablelist7_12.zip` in das Verzeichnis `tablelist7.12` entpacken können, einschließlich der Unterverzeichnisse `demos`, `doc` und `scripts`.

Die Datei `tablelistEdit.tcl` im Verzeichnis `scripts` wird nur für Anwendungen benötigt, die die interaktive Zellbearbeitung nutzen. Ebenso wird die Datei `tablelistMove.tcl` im selben Verzeichnis nur für Skripte benötigt, die den Befehl `move` oder `movecolumn` aufrufen. Schließlich wird die Datei `tablelistThemes.tcl` nur für Anwendungen benötigt, die das Paket "Tablelist_tile" verwenden (siehe nächster Abschnitt).

Beachten Sie, dass in tklib das Tablelist-Verzeichnis `demos` durch das Unterverzeichnis `tablelist` des Verzeichnisses `examples` ersetzt wird. Bitte berücksichtigen Sie dies, wenn Sie die folgenden Beispiele lesen.

Wie wird es angewendet?

Die Tablelist-Distribution enthält zwei Pakete: **Tablelist** und **Tablelist_tile**. Der Hauptunterschied zwischen den beiden besteht darin, dass Tablelist_tile die tilebasierte, themespezifische Darstellung von Tablelist-Widgets ermöglicht; dieses Paket erfordert Tile 0.6 oder höher. Es ist nicht möglich, beide Pakete in ein und derselben Anwendung zu verwenden, da beide im selben Namespace `tablelist` implementiert sind und identische Befehle bereitstellen.

Um auf die Befehle und Variablen des Tablelist-Pakets zugreifen zu können, müssen Ihre Skripte eine der folgenden Zeilen enthalten:

```
package require tablelist ?version?
package require Tablelist ?version?
```

Sie können eine der beiden oben genannten Anweisungen verwenden, da die Datei `tablelist.tcl` beide Zeilen enthält.

```
package provide tablelist ...
package provide Tablelist ...
```

Um auf die Befehle und Variablen des Pakets "Tablelist_tile" zugreifen zu können, müssen Ihre Skripte ebenfalls eine der folgenden Zeilen enthalten:

```
package require tablelist_tile ?version?
package require Tablelist_tile ?version?
```

Auch hier können Sie eine der beiden oben genannten Anweisungen verwenden, da die Datei `tablelist_tile.tcl` beide Zeilen enthält.

```
package provide tablelist_tile ...
package provide Tablelist_tile ...
```

Es steht Ihnen frei, jeweils eine der oben genannten Zeilen aus `tablelist.tcl` bzw. `tablelist_tile.tcl` zu entfernen, wenn Sie verhindern möchten, dass die entsprechenden Pakete jeweils unter zwei verschiedenen Namen erscheinen. Dadurch beschränken Sie natürlich das Argument von `package require` auf einen einzigen Namen pro Paket.

Da die Pakete "Tablelist" und "Tablelist_tile" im Namespace `tablelist` implementiert sind, müssen Sie entweder den

```
namespace import tablelist::pattern ?tablelist::pattern ...?
```

Befehl aufrufen, um die benötigten *Prozeduren* zu importieren, oder qualifizierte Namen wie `tablelist::tablelist` verwenden. In den folgenden Beispielen haben wir uns für den letzteren Ansatz entschieden.

Um auf Tablelist-*Variablen* zuzugreifen, *müssen* Sie qualifizierte Namen verwenden. Es gibt nur 6 Tablelist-Variablen (und eine weitere bei Verwendung von Tablelist_tile), die für den Zugriff außerhalb des Namespaces `tablelist` vorgesehen sind:

- Die Variable `tablelist::version` enthält die aktuelle Versionsnummer der Pakete "Tablelist" und "Tablelist_tile".
- Die Variable `tablelist::library` enthält den Pfad zum Verzeichnis der Tablelist-Installation.
- Die schreibgeschützte Variable `tablelist::scalingpct` wird beim Laden des Pakets "Tablelist" oder "Tablelist_tile" über `package require` auf den Skalierungsprozentsatz gesetzt, der der DPI-Skalierungsstufe des Bildschirms entspricht. Tablelist passt unter anderem den Standardstil der Sortierpfeile unter X11 und Windows Vista, 7, 8 und 10+ sowie die Größen der Baumstile `vistaAero`, `vistaClassic`, `win7Aero`, `win7Classic` und `win10` an den Wert dieser Variablen an. Die derzeit unterstützten Werte sind `100`, `125`, `150`, `175` und `200`. Sie können diese Variable beispielsweise verwenden, wenn Sie je nach DPI-Skalierungsstufe Bilder unterschiedlicher Größe erstellen möchten. Wenn Ihre Anwendung beispielsweise auf einem nicht skalierten Display Bilder mit der Größe 16 x 16 verwendet und `tablelist::scalingpct` den Wert `150` hat, sollte die Bildgröße für dieses Display 24 x 24 betragen.
- Wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) ist oder 8.6 und die Erweiterung "tksvg" in den Interpreter geladen werden kann, dann wird die Variable `tablelist::svgfmt` auf eine Tcl-Liste gesetzt, die Sie an die Befehle übergeben können, die SVG-Bilder erstellen oder bearbeiten, und zwar als Wert ihrer Option `-format`, um sicherzustellen, dass Ihre Bilder korrekt skaliert werden.
- Die boolesche Variable `tablelist::strictTk` (mit dem Standardwert `0`) steuert die strikte Listbox-Kompatibilität der Vorgabebindungen.
- Die schreibgeschützte boolesche Variable `tablelist::usingTile` hat im Paket "Tablelist" den Wert `0` und in "Tablelist_tile" den Wert `1`.
- In "Tablelist_tile" enthält das Array `tablelist::themeDefaults` die themespezifischen Standardwerte für eine Reihe von Tablelist-Konfigurationsoptionen.

Mehr zu tablelist::scalingpct

Der Tablelist-Code setzt die Variable `tablelist::scalingpct` auf den Wert, der von der

öffentlichen Prozedur `scaleutil::scalingPercentage` des Pakets "scaleutil" zurückgegeben wird, das im Lieferumfang von Tablelist enthalten ist. Die Art und Weise, wie dieser Wert berechnet wird, hängt vom Fenstersystem ab:

Unter Windows und Mac OS X/11+ wird der Skalierungsprozentsatz anhand des von Tk vorgegebenen Skalierungsfaktors `[tk scaling]` berechnet. HINWEIS: Unter Mac OS X/11+ ist das Ergebnis unabhängig von der Skalierungsstufe des Bildschirms immer `100`. Auf diesem System skaliert die Desktop-Engine alles automatisch nach Bedarf.

Unter X11 erfolgt die Berechnung des Skalierungsprozentsatzes aus `[tk scaling]` nur als Ausweichmethode, da die Implementierung der Bildschirmskalierung stark von der Desktop-Umgebung abhängt und größtenteils Systemressourcen nutzt, die außerhalb von Xlib liegen, auf der Tk basiert. (Auch herkömmliche X-Anwendungen wie `bitmap` und `xmag` sind davon betroffen.) Mit der teilweisen Ausnahme von Xfce und MATE (siehe unten) berechnet die Prozedur den Skalierungsprozentsatz anhand des Werts der X-Ressource `Xft.dpi`, indem die Anwendung `xrdb` ausgeführt wird. Auf GNOME-basierten Systemen, auf denen `xrdb` nicht standardmäßig installiert ist (z. B. Solus GNOME und Solus Budgie), werden stattdessen die Anwendung `xrandr` und die Datei `~/.config/monitors.xml` verwendet.

- Unter *Xfce* kann der Anzeigenskalierungsmodus, der als *Fensterskalierung* bezeichnet wird, auf "1x" oder "2x" eingestellt werden und im Dialogfeld *Darstellung* ausgewählt werden. Xfce-Versionen ab 4.16 unterstützen zudem eine fraktionierte Skalierung, deren Parameter im Dialogfeld *Anzeige* konfiguriert werden können. Wenn der Modus "2x" ausgewählt wurde, wird der Skalierungsprozentsatz durch die Prozedur `scaleutil::scalingPercentage` auf `200` festgelegt; ansonsten wird er anhand des Werts der Schriftart-DPI berechnet, der durch die X-Ressource `Xft.dpi` vorgegeben wird. Der Wert der Schriftart-DPI kann auf der Registerkarte *Schriftarten* des Dialogs *Darstellung* oder (unter Linux Lite 5+) über den Dialog *HiDPI-Einstellungen* festgelegt werden.
- Unter *MATE* kann der Anzeigenskalierungsmodus, der als *Fenster-Skalierungsfaktor* bezeichnet wird, auf "Automatische Erkennung", "Normal" oder "HiDPI" eingestellt werden und im Dialogfeld *MATE Tweak* ausgewählt werden. Der gleiche Effekt lässt sich erzielen, indem Sie im Dialogfeld *Monitor-Einstellungen* einen der möglichen Skalierungswerte "Auto-Detect", "100 %" und "200 %" auswählen. Wenn Bedarf besteht, erkennt die Prozedur `scaleutil::scalingPercentage` automatisch, welcher der Modi "Normal" ("100 %") und "HiDPI" ("200 %") dem voreingestellten Modus "Auto-detect" ("automatische Erkennung") entspricht. Wenn der Modus "HiDPI" ("200 %") vom Benutzer ausgewählt wurde oder von der Prozedur automatisch erkannt wurde, wird der Skalierungsprozentsatz auf `200` gesetzt; andernfalls wird er anhand des Werts der Schriftart-DPI berechnet, der durch die X-Ressource `Xft.dpi` vorgegeben ist. Der Wert der Schriftart-DPI kann über das Dialogfeld *Font Rendering Details* eingestellt werden, das wiederum über die Registerkarte *Schriftarten* im Dialogfeld *Darstellungseinstellungen* geöffnet werden kann.
- Bei *GNOME unter Xorg, Budgie und Cinnamon in Versionen vor 4.6* kann die Bildschirmskalierung entweder 100 % oder 200 % betragen, und die Prozedur legt den Skalierungsprozentsatz entsprechend auf `100` oder `200` fest. In neueren GNOME- und Budgie-Versionen unter Ubuntu können Sie `x11-randr-fractional-scaling` als experimentelles Feature aktivieren, wodurch (mindestens) 125 %, 150 % und 175 % zur Liste der unterstützten Skalierungsprozentsätze hinzugefügt werden. HINWEIS: Aufgrund der Art und Weise, wie diese fraktionierte Skalierung implementiert ist, gibt

`scaleutil::scalingPercentage` einen Wert zurück, der unabhängig von der gewählten Bildschirmskalierung `200` lautet. Cinnamon ab Version 4.6 unterstützt ebenfalls die fraktionierte Skalierung; ist diese aktiviert, lautet der von der Prozedur zurückgegebene Skalierungsprozentsatz `200`, unabhängig von der Auswahl der Bildschirmskalierung.

- *GNOME auf Wayland* unterstützt traditionell die Werte für die Bildschirmskalierung 100 % und 200 %, und der Skalierungsprozentsatz wird entsprechend auf `100` oder `200` festgelegt. In neueren GNOME-Versionen können Sie die experimentelle Funktion `scale-monitor-framebuffer` aktivieren, die (mindestens) 125 %, 150 % und 175 % zur Liste der unterstützten Skalierungsprozentsätze hinzufügt. Ist diese Funktion aktiviert, gibt `scaleutil::scalingPercentage` den Wert `100` bei älteren GNOME-Versionen zurück und `200` bei neueren Versionen, unabhängig vom Skalierungsgrad, da in diesem Fall nicht der Fensterinhalt, sondern die Monitor-Framebuffer in einem logischen Pixelkoordinatenraum skaliert werden.
- *KDE Plasma unter Xorg* bietet Unterstützung für die fraktionierte Skalierung. Auf diesem Desktop gibt die Prozedur je nach Skalierungsstufe des Bildschirms `100`, `125`, `150`, `175` oder `200` zurück.
- *KDE Plasma auf Wayland* unterstützt ebenfalls die fraktionelle Skalierung. In diesem Fall gibt die Prozedur bei älteren KDE-Versionen immer den Wert `100` zurück, unabhängig vom Skalierungsgrad des Bildschirms, während sie bei neueren KDE-Versionen den Wert `100`, `125`, `150`, `175` oder `200` zurückgibt.

Unter GNOME und Budgie lässt sich die Bildschirmskalierung auf der Seite *Bildschirme* des Dialogfelds *Einstellungen* einstellen, der Wert der X-Ressource `Xft.dpi`, die von der Prozedur `scaleutil::scalingPercentage` verwendet wird, kann jedoch auch durch Einstellen des Skalierungsfaktors der Schriftarten über die Anwendung *Tweaks* angepasst werden. Ebenso lässt sich unter Cinnamon die Bildschirmskalierung auf der Seite *Bildschirm* des Dialogfelds *Systemeinstellungen* einstellen, doch der Wert der X-Ressource `Xft.dpi` kann auch durch Einstellen des Textskalierungsfaktors über die Seite *Schriftartauswahl* des Dialogfelds *Systemeinstellungen* angepasst werden. Schließlich lässt sich unter KDE Plasma die Bildschirmskalierung auf der Seite *Bildschirmkonfiguration* des Dialogs *Systemeinstellungen* einstellen, und unter KDE Plasma auf Xorg kann der Wert der Schriftart-DPI, der durch die X-Ressource `Xft.dpi` vorgegeben wird, ebenfalls über die Seite *Schriftarten* im Dialog *Systemeinstellungen* eingestellt werden.

Neben der Berechnung des Skalierungsprozentsatzes führt die Prozedur `scaleutil::scalingPercentage` eine Reihe weiterer Aufgaben aus. Unter anderem ändert sie die Standardhöhe der ttk::treeview-Zeilen vom fest codierten Wert von 20 Pixeln auf einen sinnvolleren Wert, basierend auf den Schriftmetriken der vom `Treeview`-Stil verwendeten Schriftart (in der Regel `TkDefaultFont`), und stellt sicher, dass dieser Schritt jedes Mal wiederholt wird, wenn das virtuelle Ereignis `<<ThemeChanged>>` empfangen wird (z. B. weil sich der Wert der Option `-font` des Stils `Treeview` geändert hat), oder das virtuelle Ereignis `<<TkWorldChanged>>` empfangen wird, bei dem das Feld `user_data` (`%d`) auf `FontChanged` gesetzt ist. Wenn die Tk-Version höher als 8.6.11 oder 8.7a5 ist, wird dieses virtuelle Ereignis von der Tk-Engine an alle Widgets gesendet, wenn eine Schriftart geändert wird, beispielsweise durch den Aufruf von `font configure`(siehe TIP 608).

Die im weiteren Verlauf dieses Abschnitts beschriebenen zusätzlichen Schritte werden nur ausgeführt, wenn der Skalierungsprozentsatz größer ist als `100`.

Unter X11 synchronisiert die Prozedur den von Tk verwendeten Skalierungsfaktor für die Umrechnung zwischen physikalischen Einheiten und Pixeln mit dem Skalierungsprozentsatz, indem sie einen aus letzterem abgeleiteten Wert an den Befehl `tk scaling` übergibt. Während dies in den allermeisten Anwendungen das gewünschte und empfohlene Verhalten ist, kann es Fälle geben, in denen Sie diesen Schritt unterdrücken möchten, da Ihre Anwendung möglicherweise über eine eigene Logik zur Bestimmung des Werts des Skalierungsfaktors von Tk verfügt. Dies können Sie erreichen, indem Sie die Zeile

```
namespace eval ::scaleutil { set keepTkScaling 1 }
```

einfügen, bevor "Tablelist" oder "Tablelist_tile" über `package require` geladen wird.

Die Prozedur `scaleutil::scalingPercentage` korrigiert bei Bedarf auch die Größen der Standardschriften. Diese Schriftarten (`TkDefaultFont`, `TkTextFont` usw.) sind in der Datei `$tk_library/ttk/fonts.tcl` definiert. Lange Zeit waren die in dieser Datei angegebenen Schriftgrößen für X11 in Pixeln angegeben, was für die Verwendung auf HiDPI-Bildschirmen nicht geeignet war. Dies führte dazu, dass mehrere Linux-Distributionen gepatchte Versionen dieser Datei mitlieferten, in denen die Größenangaben in Pixeln durch Größenangaben in Punkten ersetzt wurden. Die gleiche Korrektur wurde im Februar 2020 in das Tk-Core-Repository committet und ist nun in Tk 8.7a5 und späteren Versionen enthalten. Um sicherzustellen, dass die Schriftgrößen unabhängig von der Tk-Version zum Skalierungsgrad des Bildschirms passen, überprüft die Prozedur diese Bibliotheksdatei und setzt, wenn diese Größen in Pixeln enthalten sind, die Option `-size` der Standardschriften auf entsprechende Größen in Punkten (ohne die Datei zu verändern). Zudem verdoppelt die Prozedur für den "2x"-Modus unter Xfce die Größen (in Punkten) der Standardschriften (die Funktionsweise der Bildschirmskalierung auf diesem Desktop macht dies erforderlich).

Die Prozedur skaliert außerdem:

- die Standardparameter des Trenngriffs (sash) des Tk-Core-panedwindow;
- die Standardgröße des Tk-Core-Scale-Widgets und seines Schiebereglers;
- die Standardbreite der Tk-Core-Scrollbars unter X11;
- die Standardlänge der Widgets ttk::scale und ttk::progressbar;
- für die eingebauten Themes `alt` und `default` einige Stiloptionen der Widgets ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Theme `clam` einige Stiloptionen der Widgets ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, ttk::treeview und ttk::labelframe;
- für das eingebaute Theme `classic` einige Stiloptionen der Widgets ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Theme `vista` einige Stiloptionen der Widgets ttk::combobox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Theme `winnative` einige Stiloptionen der Widgets ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, toolbutton, ttk::menubutton, ttk::checkbutton,

ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Theme `xpnative` einige Stiloptionen der Widgets ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview.

Darüber hinaus stellt die Prozedur `scaleutil::scalingPercentage` sicher, dass in den Themes `vista` und `xpnative` die Indikatoren der Widgets ttk::checkbutton und ttk::radiobutton korrekt skaliert angezeigt werden, unabhängig von der verwendeten Tk-Version. (Ein seit Langem bestehender Fehler in der Implementierung dieser Widgets wurde im Mai 2020 behoben und ist nun sowohl in Tk 8.6.11 und höher als auch in 8.7a5 und höher enthalten; die Prozedur bietet jedoch eine automatische Abhilfe für die Tk-Versionen, die noch von diesem Fehler betroffen sind.)

HINWEIS: In Tk-Versionen vor Version 9 sind die Tk-Core-Widgets "checkbutton" und "radiobutton" sowie die Indikatoren der Widgets "ttk::checkbutton" und "ttk::radiobutton" der Themes `alt` und `winnative` nicht skalierbar (in Tk 9 und höher werden sie automatisch skaliert). Das Gleiche gilt für das Widget "ttk::sizegrip" in den integrierten Themes.

Die Prozedur tablelist::getScalingPct und der Skalierungsfaktor von Tk

Das Paket "Tablelist" stellt die Hilfsprozedur `tablelist::getScalingPct` bereit, die keine Argumente erwartet und den Skalierungsprozentsatz der Anwendung zurückgibt, der aus dem *aktuellen* Skalierungsfaktor von Tk abgeleitet wird, der durch `[tk scaling]` angegeben wird. Der Wert, den die Prozedur zurückgibt, ist auf Vielfache von `25` beschränkt, beträgt mindestens `100` und kann größer als `200` sein, insbesondere wenn AndroWish auf einem Tablet oder Smartphone ausgeführt wird. Tablelist nutzt diese Prozedur, um sicherzustellen, dass die Größen seiner GUI-Komponenten dem aktuellen Skalierungsfaktor von Tk entsprechen (der möglicherweise über `tk scaling` geändert wurde), und es wird empfohlen, dies auch im Anwendungscode zu tun.

Wenn die Tk-Version 9 oder höher ist, ist die im vorherigen Abschnitt beschriebene Funktionalität der Prozedur `scaleutil::scalingPercentage` in den Kern integriert. Als Teil dieser Funktionalität setzt Tk 9+ die Variable `tk::scalingPct` bei der Initialisierung auf den Skalierungsprozentsatz, der der DPI-Skalierungsstufe des Bildschirms entspricht, und die Prozedur `scaleutil::scalingPercentage` gibt lediglich das Minimum aus dem aktuellen Wert von `tk::scalingPct` und `200` zurück. Zur Erinnerung: In Tk 9+ wird der Wert der Variablen `tk::scalingPct` automatisch aktualisiert, sobald der Skalierungsfaktor über `tk scaling` geändert wird. Folglich wird, wenn die Anwendung den Skalierungsfaktor von Tk *vor dem Laden von* "*Tablelist*" *oder* "*Tablelist_tile*" über `package require` ändert, die Variable `tablelist::scalingpct` auf den kleinsten Wert aus dem automatisch aktualisierten Wert von `tk::scalingPct` und `200` gesetzt. Diese Tatsachen führen zu Folgendem

Empfehlung: Wenn Ihre Anwendung den Skalierungsfaktor von Tk durch den Aufruf von `tk scaling` ändert, sollte dies *vor dem Laden von Tablelist oder Tablelist_tile* geschehen.

Weitere Informationen zu tablelist::svgfmt

Wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) oder 8.6 ist und das Paket "tksvg" in den Interpreter geladen werden kann, setzt "Tablelist" die Variable `tablelist::svgfmt` auf

```
[list svg -scale [expr {[tablelist::getScalingPct] / 100.0}]]
```

Typische Werte sind `{svg -scale 1.0}`, `{svg -scale 1.25}`, `{svg -scale 2.0}` usw.

Es wird empfohlen, den Wert der Variablen `tablelist::svgfmt` an die Befehle `image create photo`, `imageName configure`, `imageName put`und `imageName read` als Wert für deren Option `-format` zu übergeben, um sicherzustellen, dass deren Größen dem aktuellen Skalierungsprozentsatz der Anwendung entsprechen.

Mehr zu Tablelist_tile

Ein Tablelist-Widget besteht aus einem Datenbereich (der die Elemente enthält) und einem Kopfbereich (in der die Spaltentitel und optionale Kopfzeilenelemente angezeigt werden). Beide Komponenten sind in einem Rahmen enthalten, der als Frame implementiert ist. Die Kopfzeile weist eine recht komplexe Struktur auf und besteht unter anderem aus Frame- und Label-Widgets. Während im Tablelist-Paket alle diese Komponenten-Widgets Tk-Widgets sind, verwendet das Tablelist_tile-Paket sowohl Tk- als auch Tile-Frame- und Label-Widgets. Aufgrund mehrerer Inkompatibilitäten zwischen Tk und Tile ist es derzeit nicht möglich, alle Tk-Widgets, aus denen eine Tablelist besteht, durch ihre Tile-Entsprechungen zu ersetzen.

Daraus folgt, dass **das Paket "Tablelist_tile" nur dann wie erwartet funktioniert, wenn die Tk-Befehle "frame" und "label" nicht durch den Befehl namespace import -force ttk::* im globalen Geltungsbereich überschrieben wurden**. Während in früheren Tile-Versionen empfohlen wurde, diesen Befehl im globalen Geltungsbereich zu verwenden – für die wirklich Abenteuerlustigen –, gilt dies in neueren Tile-Versionen als wirklich schlechte Idee, die zu zahlreichen Fehlfunktionen führt. Stattdessen **sollten Sie ttk::frame, ttk::label usw. explizit aufrufen, wann immer Sie ein Tile-Widget verwenden möchten**.

Das Überschreiben einiger anderer Tk-Widgets im globalen Bereich kann ebenso gefährlich sein, wenn verschiedene Widgets aus dem Tk-Kern sowie aus den Paketen BWidget, Iwidgets, combobox (von Bryan Oakley), ctext und Mentry für die interaktive Zellenbearbeitung verwendet werden, da diese Pakete Tk-Widgets erwarten, die möglicherweise nicht immer durch ihre "Tile"-Entsprechungen ersetzt werden können.

Eine weitere Einschränkung, die zu berücksichtigen ist, ergibt sich daraus, dass in früheren Tile-Versionen der Befehl `(ttk::)style theme use` nur zum Festlegen des aktuellen Themes verwendet werden konnte, nicht jedoch zum Abrufen desselben. Aus diesem Grund greift das Tablelist_tile-Paket, wenn es das aktuelle Theme nicht mithilfe von `ttk::style theme use` abrufen kann, auf die Variable `ttk::currentTheme` oder `tile::currentTheme` (je nach Tile-Version), die durch den Befehl `ttk::setTheme` oder `tile::setTheme` gesetzt wird, zurück. Daraus folgt: **Wenn die verwendete Tile-Version den Befehl ttk::style theme use ohne Argument nicht unterstützt, haben die Tile-basierten Tablelist-Widgets nur dann das erwartete Erscheinungsbild, wenn das plattformspezifische Vorgabe-Theme entweder unverändert bleibt oder durch Aufruf der Prozedur ttk::setTheme oder tile::setTheme – je nach Tile-Version – durch ein anderes Theme ersetzt wird**. (Siehe auch den Befehl `tablelist::setTheme`.)

Nach diesen Hinweisen zur Verwendung von "Tile" werden im weiteren Verlauf dieses Abschnitts die Unterschiede zwischen den Paketen "Tablelist" und "Tablelist_tile" beschrieben.

Das Paket "Tablelist_tile" überprüft, ob die erforderlichen Tk- und Tile-Versionen vorhanden sind, indem es die folgenden Befehle ausführt:

```
package require Tk 8.4-
if {$::tk_version < 8.5 || [regexp {^8\.5a[1-5]$} $::tk_patchLevel]} {
    package require tile 0.6-
}
```

Der zweite Befehl oben spiegelt die Tatsache wider, dass "Tile" ab Tk 8.5a6 in den Tk-Kern integriert ist und daher nur bei Verwendung einer früheren Tk-Version explizit geladen werden sollte.

Abgesehen davon und vom Suffix `_tile` im Befehl `package require` besteht der einzige Unterschied (aus Sicht des Programmierers) zwischen den Paketen "Tablelist" und "Tablelist_tile" in den unterstützten Konfigurationsoptionen: `-highlightbackground`, `-highlightcolor`, `-highlightthickness`, `-labelbackground`, `-labelactivebackground`, `-labelactiveforeground`, `-labeldisabledforeground` und `-labelheight` (letztere sowie `-labelbackground` sowohl auf Widget- als auch auf Ebene der Spalten), die im Tablelist-Paket enthalten sind, werden von Tablelist_tile nicht unterstützt. Die ersten drei sind Standard-Tk-Widget-Optionen, die für Tile-Widgets nicht verfügbar sind. Die Option `-labelbackground` entspricht der Option `-background` für die Spaltenbeschriftungen, und aktuelle Versionen der Tile-Engine unterstützen keine Einstellung der Hintergrundfarbe für diese Widgets mit einem speziellen Header-Layout. Die übrigen Optionen entsprechen den Optionen `-activebackground`, `-activeforeground`, `-disabledforeground` und `-height` der Spaltenbeschriftungen, und diese Konfigurationsoptionen werden für Tile-Label-Widgets nicht unterstützt.

Beachten Sie außerdem, dass bei einigen Themes die Einstellung der Option `-labelborderwidth` (auf Widget- oder Spaltenebene) auf einen anderen Wert als den Standardwert möglicherweise von Tile ignoriert wird und somit zu Problemen bei der Ausrichtung führen kann. Dies liegt daran, dass der Rahmen von Tile-Widgets mit themespezifischen Methoden gezeichnet wird, die nicht immer zu den aus Tk-Widgets bekannten Ergebnissen führen.

Beachten Sie abschließend, dass wenn das Theme `tileqt` verwendet wird, die Versionsnummer des Pakets `tile::theme::tileqt` mindestens 0.4 betragen muss, und dass `tileqt` selbst nicht mit Tile-Versionen vor 0.7 kompatibel ist.

Beispiele

Ein Tablelist-Widget zum Anzeigen und Bearbeiten von Konfigurationsoptionen

Die Datei `config.tcl` im Verzeichnis `demos` enthält eine Prozedur `demo::displayConfig`, die die Konfigurationsoptionen eines beliebigen Widgets in einer Tablelist anzeigt, die in einem neu erstellten Toplevel-Fenster enthalten ist, und es Ihnen ermöglicht, diese Optionen zu bearbeiten. Diese Prozedur kann sich bei der interaktiven GUI-Entwicklung als sehr nützlich erweisen. Um sie zu testen, starten Sie `wish` und führen Sie die Auswertung der Datei mit dem Befehl `source` wie folgt durch:

- Wenn `wish` im Verzeichnis `demos` gestartet wurde, reicht es aus, Folgendes einzugeben:

```
source config.tcl
```

- Wenn `wish` in einem anderen Verzeichnis gestartet wurde, können Sie die Variable `tablelist::library` verwenden, um den Speicherort der Datei zu ermitteln. Angenommen,

Ihre Tablelist-Installation weist die im Abschnitt Wie wird es installiert? beschriebene Verzeichnisstruktur auf, lauten die erforderlichen Befehle wie folgt:

```
package require tablelist
source [file join $tablelist::library demos config.tcl]
```

In beiden Fällen gibt das Skript die folgende Meldung in `stdout` aus:

```
To display the configuration options of an arbitrary widget, enter

        demo::displayConfig <widgetName>
```

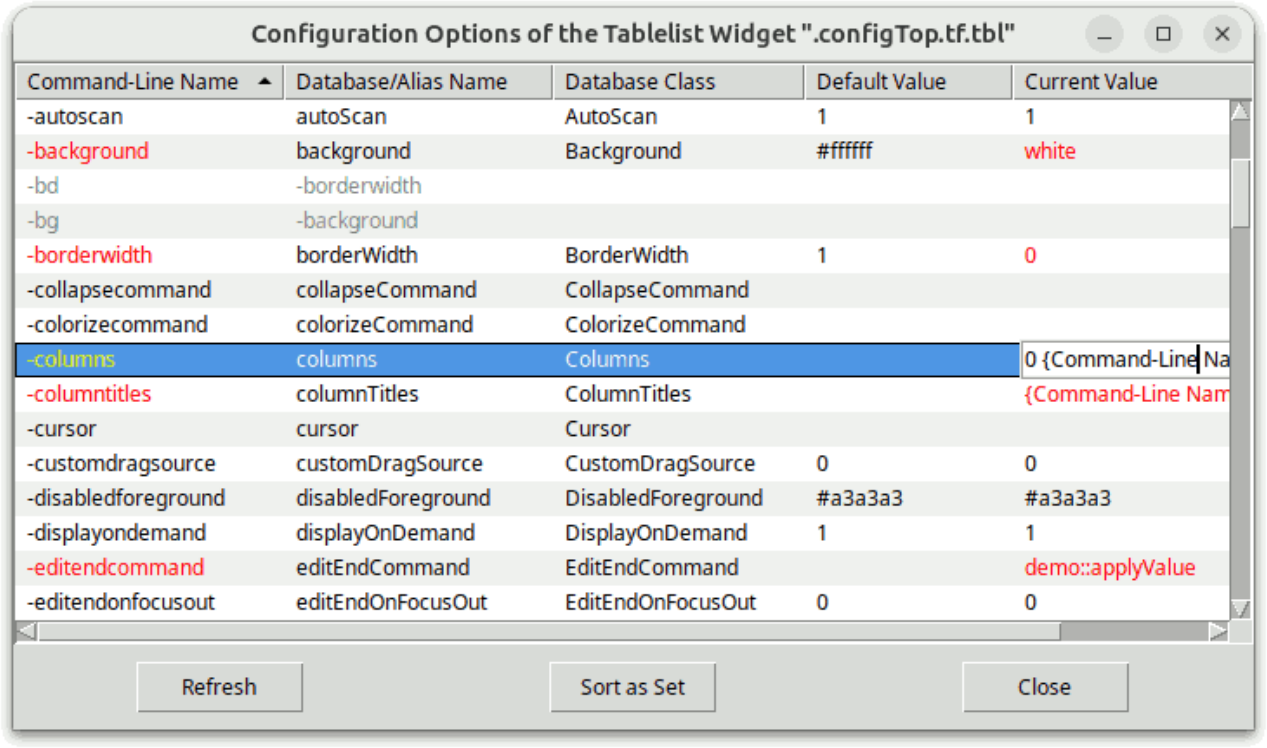
`<widgetName>` kann der Pfadname eines beliebigen bereits vorhandenen Widgets sein. Wenn Sie beispielsweise Folgendes eingeben:

```
demo::displayConfig .
```

Dann werden Sie sehen, dass ein Tablelist-Widget mit der Bezeichnung `.configTop.tf.tbl` in einem neuen Toplevel-Fenster erstellt wird und dessen Name auf `stdout` ausgegeben wird. Wenn Sie dann Folgendes eingeben

```
demo::displayConfig .configTop.tf.tbl
```

Dann erscheint ein weiteres Toplevel-Fenster, das das Tablelist-Widget `.configTop2.tf.tbl` enthält. Letzteres sieht wie in der folgenden Abbildung aus:



Konfigurationsoptionen

Es wird davon ausgegangen, dass der Tcl-Befehl, der dem durch `<widgetName>` angegebenen Widget zugeordnet ist, über einen Unterbefehl `configure` verfügt, der, wenn er ohne Argumente aufgerufen wird, eine Liste zurückgibt, die alle verfügbaren Konfigurationsoptionen für das Widget beschreibt, und zwar in dem von den Standard-Tk-Widgets bekannten Format. Die Prozedur `demo::displayConfig` fügt die Elemente dieser Liste in eine scrollbare Tablelist mit

5 Spalten dynamischer Breite und interaktiver Sortierfunktion ein und gibt den Namen des neu erstellten Tablelist-Widgets zurück:

```
package require tablelist

namespace eval demo {
    #
    # Add some entries to the Tk option database for the following
    # widget hierarchy within a toplevel widget of the class DemoTop:
    #
    # Name              Class
    # ----------------------------
    # tf                Frame
    #   tbl               Tabellist
    #   vsb, hsb          Scrollbar
    # bf                Frame
    #   b1, b2, b3        Button
    #
    if {[tk windowingsystem] eq "x11"} {
        #
        # Create the font TkDefaultFont if not yet present
        #
        catch {font create TkDefaultFont -family Helvetica -size 9}

        option add *DemoTop*Font                        TkDefaultFont
        option add *DemoTop*selectBackground            #5294e2
        option add *DemoTop*selectForeground            white
    }
    option add *DemoTop.tf.borderWidth                  1
    option add *DemoTop.tf.relief                       sunken
    option add *DemoTop.tf.tbl.borderWidth              0
    option add *DemoTop.tf.tbl.highlightThickness       0
    option add *DemoTop.tf.tbl.background               white
    option add *DemoTop.tf.tbl.stripeBackground         #f0f0f0
    option add *DemoTop.tf.tbl*Entry.background         white
    option add *DemoTop.bf.Button.width                 10
}

#------------------------------------------------------------------------------
# demo::displayConfig
#
# Displays the configuration options of the widget w in a tablelist widget
# contained in a newly created toplevel widget.  Returns the name of the
# tablelist widget.
#------------------------------------------------------------------------------
proc demo::displayConfig w {
    if {![winfo exists $w]} {
        bell
        tk_messageBox -title "Error" -icon error -message \
            "Bad window path name \"$w\""
        return ""
    }
```

```
    #
    # Create a toplevel widget of the class DemoTop
    #
    set top .configTop
    for {set n 2} {[winfo exists $top]} {incr n} {
        set top .configTop$n
    }
    toplevel $top -class DemoTop
    wm title $top "Configuration Options of the [winfo class $w] Widget \"$w\""

    #
    # Create a scrolled tablelist widget with 5 dynamic-width
    # columns and interactive sort capability within the toplevel
    #
    set tf $top.tf
    frame $tf
    set tbl $tf.tbl
    set vsb $tf.vsb
    set hsb $tf.hsb
    tablelist::tablelist $tbl \
        -columns {0 "Command-Line Name"
                  0 "Database/Alias Name"
                  0 "Database Class"
                  0 "Default Value"
                  0 "Current Value"} \
        -labelcommand tablelist::sortByColumn -sortcommand demo::compareAsSet \
        -editendcommand demo::applyValue -height 15 -width 100 -stretch all \
        -xscrollcommand [list $hsb set] -yscrollcommand [list $vsb set]
    if {[$tbl cget -selectborderwidth] == 0} {
        $tbl configure -spacing 1
    }
    $tbl columnconfigure 3 -maxwidth 30
    $tbl columnconfigure 4 -maxwidth 30 -editable yes
    scrollbar $vsb -orient vertical   -command [list $tbl yview]
    scrollbar $hsb -orient horizontal -command [list $tbl xview]

    #
    # Create three buttons within a frame child of the toplevel widget
    #
    set bf $top.bf
    frame $bf
    set b1 $bf.b1
    set b2 $bf.b2
    set b3 $bf.b3
    button $b1 -text "Refresh"     -command [list demo::putConfig $w $tbl]
    button $b2 -text "Sort as Set" -command [list $tbl sort]
    button $b3 -text "Close"       -command [list destroy $top]

    #
    # Manage the widgets
    #
    grid $tbl -row 0 -rowspan 2 -column 0 -sticky news
    if {[tk windowingsystem] eq "win32"} {
```

```
        grid $vsb -row 0 -rowspan 2 -column 1 -sticky ns
    } else {
        grid [$tbl cornerpath] -row 0 -column 1 -sticky ew
        grid $vsb              -row 1 -column 1 -sticky ns
    }
    grid $hsb -row 2 -column 0 -sticky ew
    grid rowconfigure    $tf 1 -weight 1
    grid columnconfigure $tf 0 -weight 1
    pack $b1 $b2 $b3 -side left -expand yes -pady 7p
    pack $bf -side bottom -fill x
    pack $tf -side top -expand yes -fill both

    #
    # Populate the tablelist with the configuration options of the given widget
    #
    putConfig $w $tbl
    return $tbl
}
```

Die Prozedur ruft den Befehl `tablelist::tablelist` auf, um ein Tablelist-Widget zu erstellen. Der an diesen Befehl übergebene Wert der Option `-columns` legt die Breiten, Titel und Ausrichtungen der 5 Spalten fest. Die Breite jeder Spalte wird als `0` angegeben, was festlegt, dass die Spaltenbreite gerade so groß sein soll, dass alle Elemente der Spalte Platz finden, einschließlich ihres Titels, der als Zeichenkette auf die Breitenangabe folgt. Wir haben die Angaben zur Ausrichtung (die optional auf die Titel folgen können) weggelassen, da alle Spalten linksbündig ausgerichtet werden sollen.

Da alle Spalten eine dynamische Breite haben und linksbündig ausgerichtet sind, hätten wir anstelle von `-columns` auch die Option `-columntitles` wie folgt verwenden können:

```
    tablelist::tablelist $tbl \
        -columntitles {"Command-Line Name"
                       "Database/Alias Name"
                       "Database Class"
                       "Default Value"
                       "Current Value"} \
        . . .
```

Der Befehl `tablelist::sortByColumn`, der als Wert der Option `-labelcommand` festgelegt wird, wird immer dann aufgerufen, wenn die Maustaste 1 über einer der Beschriftungen losgelassen wird. Dieser Befehl sortiert die Elemente anhand der Spalte, die dieser Beschriftung entspricht, in der richtigen Reihenfolge, indem er den Unterbefehl `sortbycolumn` des Tcl-Befehls aufruft, der dem Tablelist-Widget zugeordnet ist.

Wie aus der Erstellung der Schaltfläche mit dem Text `"Sort as Set"` ersichtlich ist, werden die Elemente auch durch Aufruf des Unterbefehls `sort` sortiert. Daher muss als Wert der Option `-sortcommand` ein Befehl angegeben werden, der für den Vergleich der Elemente verwendet werden soll. In unserem Beispiel ist dies die unten gezeigte Prozedur `demo::compareAsSet`.

Die Option `-editendcommand` legt den Befehl fest, der automatisch aufgerufen werden soll, sobald die interaktive Bearbeitung des Inhalts einer Zelle abgeschlossen ist und sich der endgültige Inhalt des für die Bearbeitung verwendeten temporären eingebetteten

Eingabe-Widgets vom ursprünglichen Inhalt unterscheidet. Standardmäßig können die Elemente eines "Tablelist"-Widgets nur programmgesteuert bearbeitet werden, doch wir aktivieren die interaktive Bearbeitung für die Zellen der letzten Spalte mithilfe der Spaltenkonfigurationsoption `-editable`.

Indem wir für die Konfigurationsoption `-stretch` den Wert `all` festlegen, stellen wir sicher, dass alle Spalten gestreckt werden, um den Leerraum zu beseitigen, der möglicherweise rechts neben der Tabelle erscheint.

Wenn der Standardwert der Option `-selectborderwidth 0` ist (dies ist auf Windows- und Macintosh-Plattformen sowie in einer X11-Umgebung mit Tk 8.5 oder höher der Fall), dann verwenden wir die Option `-spacing`, um oben und unten an den Zeilen etwas zusätzlichen Abstand zu schaffen.

Für die letzten beiden Spalten der Tablelist verwenden wir die Spaltenkonfigurationsoption `-maxwidth`, um sicherzustellen, dass die dynamischen Breiten dieser Spalten 30 Zeichen (Durchschnittsbreite) nicht überschreiten.

Neben den in der Befehlszeile angegebenen Optionen übernimmt unser Tablelist-Widget automatisch die Optionen, die in den Einträgen der Tk-Optionsdatenbank enthalten sind, die bei der Namespace-Initialisierung vor der Prozedur `demo::displayConfig` festgelegt wurden. Der Datenbankname `stripeBackground` entspricht der Konfigurationsoption `-stripebackground`. Gemäß diesem Eintrag wird jede zweite Zeile des Tablelist-Widgets in der Hintergrundfarbe `#f0f0f0` angezeigt, was die Lesbarkeit der Einträge verbessert und dem Widget ein ansprechendes Erscheinungsbild verleiht.

Die Einträge in der Datenbank für Optionen für `*DemoTop.tf.borderWidth`, `*DemoTop.tf.relief`, `*DemoTop.tf.tbl.borderWidth` und `*DemoTop.tf.tbl.highlightThickness` werden implizit verwendet, wenn das Tablelist-Widget und die beiden Scrollbars mithilfe von `grid` verwaltet werden. Beachten Sie, wie der Unterbefehl `cornerpath` es uns ermöglicht, ein natives Look & Feel in Bezug auf die vertikale Scrollbar auf anderen Fenstersystemen als `win32` (d. h. `aqua` und `x11`) zu erzielen.

Wir füllen die Tablelist, indem wir die im Folgenden beschriebene Prozedur `demo::putConfig` aufrufen. Das gleiche Skript ist der Schaltfläche "Aktualisieren" zugeordnet, und zwar als Wert der Konfigurationsoption `-command`. Diese Prozedur ist wie folgt implementiert:

```
#------------------------------------------------------------------------------
# demo::putConfig
#
# Outputs the configuration options of the widget w into the tablelist widget
# tbl.
#------------------------------------------------------------------------------
proc demo::putConfig {w tbl} {
    if {![winfo exists $w]} {
        bell
        tk_messageBox -title "Error" -icon error -message \
            "Bad window path name \"$w\"" -parent [winfo toplevel $tbl]
        return ""
    }

    #
    # Display the configuration options of w in the tablelist widget tbl
```

```
    #
    $tbl delete 0 end
    foreach configSet [$w configure] {
        #
        # Insert the list configSet into the tablelist widget
        #
        $tbl insert end $configSet

        if {[llength $configSet] == 2} {
            $tbl rowconfigure end -foreground gray50 -selectforeground gray75
            $tbl cellconfigure end -editable no
        } else {
            #
            # Change the colors of the first and last cell of the row
            # if the current value is different from the default one
            #
            set default [lindex $configSet 3]
            set current [lindex $configSet 4]
            if {$default ne $current} {
                foreach col {0 4} {
                    $tbl cellconfigure end,$col \
                         -foreground red -selectforeground yellow
                }
            }
        }
    }

    $tbl sortbycolumn 0
    $tbl activate 0
    $tbl attrib widget $w
}
```

Nach dem Löschen der aktuellen Elemente des Tablelist-Widgets `tbl` fügt die Prozedur die Einträge der Liste ein, die vom Unterbefehl `configure` des Tcl-Befehls zurückgegeben wird, der dem Widget `w` zugeordnet ist. Für jede Option, die lediglich eine abgekürzte Form einer anderen Option ist, verwenden wir den Tablelist-Unterbefehl `rowconfigure`, um die normalen und die Auswahl-Vordergrundfarben des gerade angehängten Eintrags zu ändern, und deaktivieren die interaktive Bearbeitung in der zuletzt eingefügten Zelle mithilfe der Zellenkonfigurationsoption `-editable`. Die "tablelist"-Operation `cellconfigure` wird ebenfalls für jede echte Option aufgerufen, deren aktueller Wert vom Standardwert abweicht, um die Werte der Optionen `-foreground` und `-selectforeground` der Zellen Nr. 0 und 4 zu ändern, die den Befehlszeilennamen der Option und ihren aktuellen Wert enthalten.

Jedes Tablelist-Widget kann eine beliebige Anzahl privater **Attribute** besitzen, die mithilfe des Unterbefehls `attrib` des dem Widget entsprechenden Tcl-Befehls gesetzt und abgerufen werden können. Die Prozedur `demo::putConfig` setzt das Attribut `widget` auf den Namen des Widgets, dessen Optionen in der Tablelist angezeigt werden.

Die Implementierung des oben erwähnten Vergleichsbefehls `demo::compareAsSet` ist recht einfach:

```
#------------------------------------------------------------------------------
# demo::compareAsSet
#
# Compares two items of a tablelist widget used to display the configuration
# options of an arbitrary widget.  The item in which the current value is
# different from the default one is considered to be less than the other; if
# both items fulfil this condition or its negation then string comparison is
# applied to the two option names.
#------------------------------------------------------------------------------
proc demo::compareAsSet {item1 item2} {
    foreach {opt1 dbName1 dbClass1 default1 current1} $item1 \
            {opt2 dbName2 dbClass2 default2 current2} $item2 {
        set changed1 [expr {$default1 ne $current1}]
        set changed2 [expr {$default2 ne $current2}]
        if {$changed1 == $changed2} {
            return [string compare $opt1 $opt2]
        } elseif {$changed1} {
            return -1
        } else {
            return 1
        }
    }
}
```

Abschließend folgt hier die Implementierung der Prozedur `demo::applyValue`, die als Wert der Option `-editendcommand` festgelegt ist:

```
#------------------------------------------------------------------------------
# demo::applyValue
#
# Applies the new value of the configuraton option contained in the given row
# of the tablelist widget tbl to the widget whose options are displayed in it,
# and updates the colors of the first and last cell of the row.
#------------------------------------------------------------------------------
proc demo::applyValue {tbl row col text} {
    #
    # Try to apply the new value of the option contained in
    # the given row to the widget whose options are displayed
    # in the tablelist; reject the value if the attempt fails
    #
    set w [$tbl attrib widget]
    set opt [$tbl cellcget $row,0 -text]
    if {[catch {$w configure $opt $text} result] != 0} {
        bell
        tk_messageBox -title "Error" -icon error -message $result \
            -parent [winfo toplevel $tbl]
        $tbl rejectinput
        return ""
    }

    #
    # Replace the new option value with its canonical form and
    # update the colors of the first and last cell of the row
```

```
    #
    set text [$w cget $opt]
    set default [$tbl cellcget $row,3 -text]
    if {$default eq $text} {
        foreach col {0 4} {
            $tbl cellconfigure $row,$col \
                 -foreground "" -selectforeground ""
        }
    } else {
        foreach col {0 4} {
            $tbl cellconfigure $row,$col \
                 -foreground red -selectforeground yellow
        }
    }

    return $text
}
```

Die Prozedur ruft den Namen des Widgets ab, dessen Optionen in der Tablelist angezeigt werden, und zwar als Wert seines Attributs `widget`, und ruft den Tablelist-Unterbefehl `cellcget` auf, um den Namen der Option abzurufen, die in der ersten Zelle der Zeile angegeben ist, deren letztes Element gerade bearbeitet wurde. Anschließend versucht sie, den neuen Wert der Option auf das Widget anzuwenden, und ruft den Unterbefehl `rejectinput` auf, wenn der Versuch fehlschlägt. Andernfalls ersetzt sie den neuen Optionswert durch seine kanonische Form und aktualisiert die normalen und die Auswahl-Vordergrundfarben der Zellen Nr. 0 und 4. Die kanonische Form des Optionswerts wird durch den Unterbefehl `cget` des diesem Widget zugeordneten Tcl-Befehls vorgegeben. Beispielsweise wird ein boolescher Wert immer durch `1` oder `0` ersetzt, selbst wenn die Eingabe die Zeichenkette `yes` oder `no` enthält. Die Prozedur gibt diesen kanonischen Wert für die Option zurück und stellt so sicher, dass dieser zum neuen Inhalt der soeben bearbeiteten Zelle wird.

Zwei skalierbare Widget-Browser auf Basis einer Tablelist

Die Dateien `browse.tcl` und `browseTree.tcl` im Verzeichnis `demos` enthalten eine Prozedur `demo::displayChildren`, die Informationen über die untergeordneten Elemente eines beliebigen Widgets in einer Tablelist anzeigt, die in einem neu erstellten Toplevel-Fenster enthalten ist. Um dies zu testen, starten Sie `wish` und führen Sie die Auswertung der ausgewählten Datei mit dem Befehl `source` durch, ähnlich wie im Fall des vorherigen Beispiels.

Beide Skripte geben die folgende Meldung in `stdout` aus:

```
To display information about the children of an arbitrary widget, enter

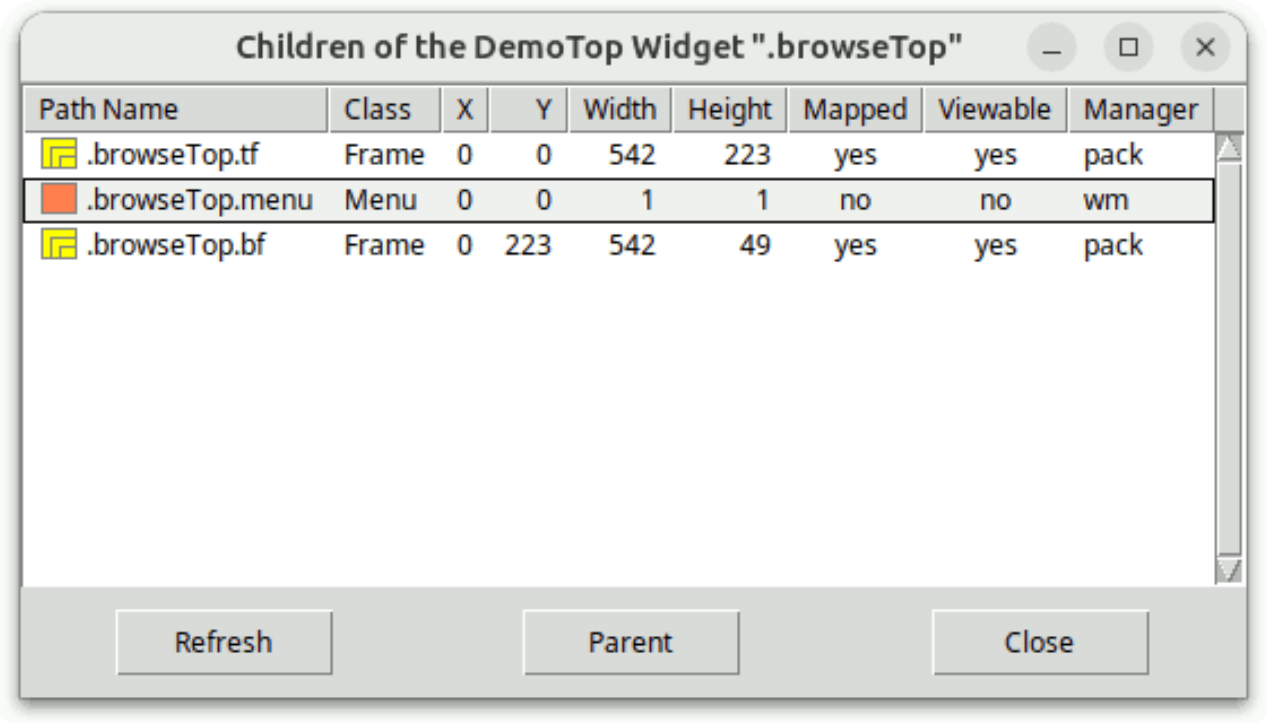
        demo::displayChildren <widgetName>
```

<`widgetName`> kann der Pfadname eines beliebigen bereits vorhandenen Widgets sein. Wenn Sie beispielsweise Folgendes eingeben:

```
demo::displayChildren .
```

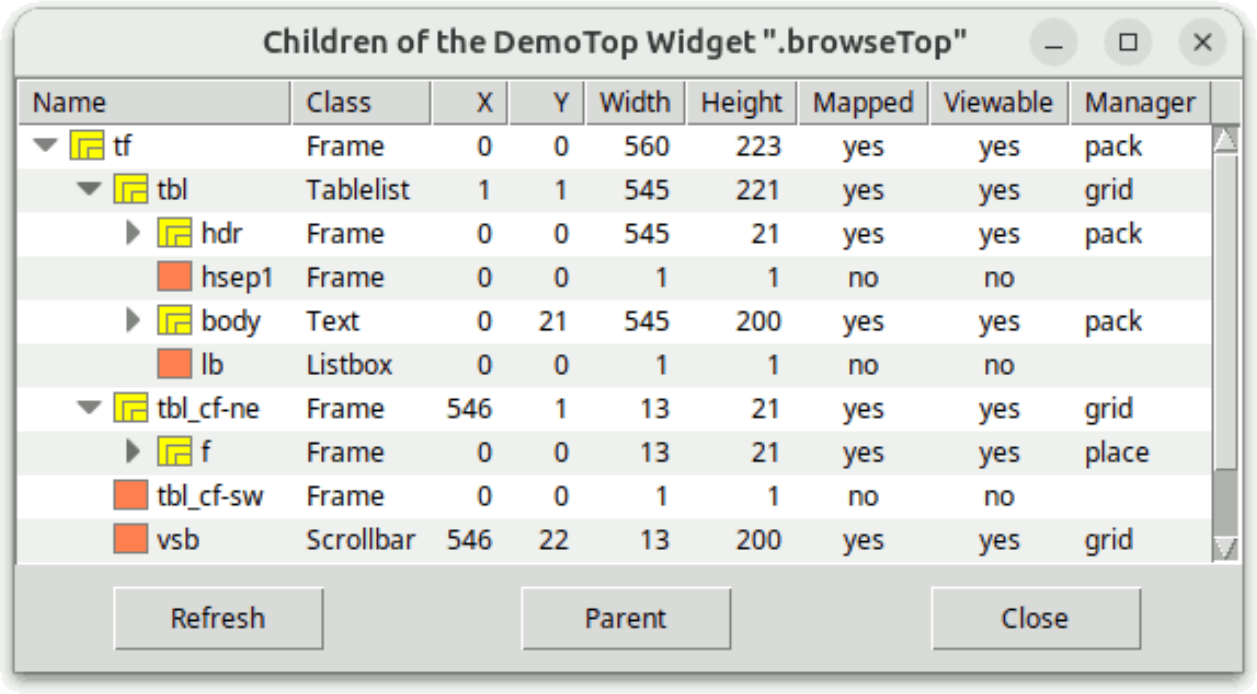
Dann werden Sie feststellen, dass ein Tablelist-Widget mit der Bezeichnung `.browseTop.tf.tbl` in einem neuen Toplevel-Fenster erstellt wird und dessen Name auf `stdout` ausgegeben wird.

Die von der Prozedur `demo::displayChildren` in der Datei `browse.tcl` erstellte Tablelist ist eine mehrspaltige listbox:



Widget Browser

Die Tablelist, die von der gleichnamigen Prozedur in der Datei `browseTree.tcl` erstellt wurde, ist ein mehrspaltiges Baum-Widget:



Widget Browser

Der Befehl `demo::displayChildren` fügt einige Daten der untergeordneten Elemente des durch `<widgetName>` festgelegten Widgets in eine vertikal scrollbare Tablelist mit 9 Spalten

dynamischer Breite und interaktiver Sortierfunktion ein und gibt den Namen des neu erstellten Tablelist-Widgets zurück. Durch Doppelklick auf ein Element (z. B. das Element, das den Pfadnamen `.browseTop` in `browse.tcl` und den Namen `browseTop` in `browseTree.tcl` enthält) oder durch Aufrufen des ersten Eintrags eines Popup-Menüs im Datenbereich der Tablelist können Sie die Daten der untergeordneten Elemente des Widgets anzeigen, das dem aktiven Eintrag entspricht, und mit dem zweiten Menüeintrag können Sie dessen Konfigurationsoptionen anzeigen (Einzelheiten finden Sie im vorherigen Beispiel). Um eine Ebene höher zu gelangen, klicken Sie auf die Schaltfläche "Übergeordnet".

Die Skripte `browse.tcl` und `browseTree.tcl` weisen viele gemeinsame Codeabschnitte auf. Wir beschränken uns in der folgenden Beschreibung auf das zweite Skript.

```
package require tablelist

namespace eval demo {
    variable dir [file dirname [info script]]

    #
    # Create two images corresponding to the display's DPI scaling level
    #
    variable leafImg [image create photo]
    variable compImg [image create photo]
    variable pct ""
    if {$::tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
        variable fmt $::tablelist::svgfmt
        $leafImg read [file join $dir leaf.svg] -format $fmt
        $compImg read [file join $dir comp.svg] -format $fmt
    } else {
        set pct $::tablelist::scalingpct
        $leafImg read [file join $dir leaf$pct.gif] -format gif
        $compImg read [file join $dir comp$pct.gif] -format gif
    }
}

source [file join $demo::dir config.tcl]
```

Wenn die Tk-Version mindestens 8.7 ist oder das Paket "tksvg" in den Interpreter geladen werden kann, werden die beiden Bilder, die ein "Compound"-Widget (mit untergeordneten Elementen) und ein "Leaf"-Widget (ohne untergeordnete Elemente) darstellen, unter Verwendung der öffentlichen Variablen `::tablelist::svgfmt` aus den Dateien `comp.svg` bzw. `leaf.svg` erstellt. Andernfalls werden sie aus Dateien erstellt, deren Namen den DPI-Skalierungsprozentsatz des Bildschirms enthalten, der durch die öffentliche Variable `::tablelist::scalingpct` angegeben wird. Die Dateien `comp100.gif` und `leaf100.gif` enthalten Bilder der Größe 16 x 14, die Dateien `*125.gif` enthalten Bilder der Größe 20 x 18, die Dateien `*150.gif` enthalten Bilder mit einer Größe von 24 x 21 und so weiter. Im ersten Fall werden nur zwei `*.svg`-Dateien benötigt, und die Bilder werden automatisch skaliert, entsprechend dem tatsächlichen Skalierungsprozentsatz des Displays, der größer sein kann als der Maximalwert `200` der Variablen `::tablelist::scalingpct`. Dieser Vorteil ist besonders praktisch, wenn AndroWish auf einem Tablet oder Smartphone ausgeführt wird. Im zweiten Fall werden insgesamt 10 `*.gif` (oder `*.png` oder `*.xbm`) Dateien benötigt, um sicherzustellen, dass die Größen der Bilder dem Wert der Variablen

`::tablelist::scalingpct` entsprechen.

```
#------------------------------------------------------------------------------
# demo::displayChildren
#
# Displays information on the children of the widget w in a tablelist widget
# contained in a newly created toplevel widget.  Returns the name of the
# tablelist widget.
#------------------------------------------------------------------------------
proc demo::displayChildren w {
    if {![winfo exists $w]} {
        bell
        tk_messageBox -title "Error" -icon error -message \
            "Bad window path name \"$w\""
        return ""
    }

    #
    # Create a toplevel widget of the class DemoTop
    #
    set top .browseTop
    for {set n 2} {[winfo exists $top]} {incr n} {
        set top .browseTop$n
    }
    toplevel $top -class DemoTop

    #
    # Create a vertically scrolled tablelist widget with 9 dynamic-width
    # columns and interactive sort capability within the toplevel
    #
    set tf $top.tf
    frame $tf
    set tbl $tf.tbl
    set vsb $tf.vsb
    tablelist::tablelist $tbl \
        -columns {0 "Name"      left
                  0 "Class"     left
                  0 "X"         right
                  0 "Y"         right
                  0 "Width"     right
                  0 "Height"    right
                  0 "Mapped"    center
                  0 "Viewable"  center
                  0 "Manager"   left} \
        -expandcommand demo::expandCmd -labelcommand demo::labelCmd \
        -yscrollcommand [list $vsb set] -width 0
    if {[$tbl cget -selectborderwidth] == 0} {
        $tbl configure -spacing 1
    }
    foreach col {2 3 4 5} {
        $tbl columnconfigure $col -sortmode integer
    }
    foreach col {6 7} {
```

```
        $tbl columnconfigure $col -formatcommand demo::formatBoolean
    }
    scrollbar $vsb -orient vertical -command [list $tbl yview]

    #
    # On X11 configure the tablelist according
    # to the display's DPI scaling level
    #
    variable winSys                                     ;# see config.tcl
    if {[tk windowingsystem] eq "x11"} {
        variable pct                                    ;# ""|100|125|...|200
        $tbl configure -treestyle bicolor$pct
    }

    #
    # When displaying the information about the children of any
    # ancestor of the label widgets, the widths of some of the
    # labels and thus also the widths and x coordinates of some
    # children may change.  For this reason, make sure the items
    # will be updated after any change in the sizes of the labels
    #
    foreach l [$tbl labels] {
        bind $l <Configure> [list demo::updateItemsDelayed $tbl]
    }
    bind $tbl <Configure> [list demo::updateItemsDelayed $tbl]

    #
    # Create a pop-up menu with two command entries; bind the script
    # associated with its first entry to the <Double-1> event, too
    #
    set menu $top.menu
    menu $menu -tearoff no
    $menu add command -label "Display Children" \
                      -command [list demo::putChildrenOfActiveWidget $tbl]
    $menu add command -label "Display Config" \
                      -command [list demo::dispConfigOfActiveWidget $tbl]
    set bodyTag [$tbl bodytag]
    bind $bodyTag <Double-1>   [list demo::putChildrenOfActiveWidget $tbl]
    bind $bodyTag <<Button3>>  [bind TablelistBody <Button-1>]
    bind $bodyTag <<Button3>> +[bind TablelistBody <ButtonRelease-1>]
    bind $bodyTag <<Button3>> +[list demo::postPopupMenu $top %X %Y]

    #
    # Create three buttons within a frame child of the toplevel widget
    #
    set bf $top.bf
    frame $bf
    set b1 $bf.b1
    set b2 $bf.b2
    set b3 $bf.b3
    button $b1 -text "Refresh"
    button $b2 -text "Parent"
    button $b3 -text "Close" -command [list destroy $top]
```

```
    #
    # Manage the widgets
    #
    . . .

    #
    # Populate the tablelist with the data of the given widget's children
    #
    putChildren $w $tbl root
    return $tbl
}
```

Die Prozedur ruft den Befehl `tablelist::tablelist` auf, um ein Tablelist-Widget zu erstellen. Der an diesen Befehl übergebene Wert der Option `-columns` legt die Breiten, Titel und Ausrichtungen der 9 Spalten fest. Die Breite jeder Spalte wird als `0` angegeben, was festlegt, dass die Breite der Spalte gerade so groß sein soll, dass alle Elemente der Spalte darin Platz finden, einschließlich ihres Titels, der als Zeichenkette auf die Breitenangabe folgt. Auf jeden Titel folgt eine Ausrichtung, die angibt, wie sowohl die Elemente als auch der Titel der jeweiligen Spalte ausgerichtet werden sollen.

Wir möchten nicht nur die Daten der untergeordneten Elemente des angegebenen Widgets anzeigen, sondern auch die seiner weiteren Nachkommen. Zu diesem Zweck benötigen wir einen Befehl, der immer dann aufgerufen wird, wenn ein Element, das einem Widget mit untergeordneten Elementen entspricht, erweitert wird. Dieser Befehl wird als Wert der Option `-expandcommand` angegeben. Wie später erläutert wird, fügt die Prozedur `demo::expandCmd` die untergeordneten Elemente der Zeile ein, die gerade erweitert werden soll, sofern diese noch keine untergeordneten Elemente hat.

Der Befehl `demo::labelCmd`, der als Wert der Option `-labelcommand` festgelegt wird, wird immer dann aufgerufen, wenn die linke Maustaste über einer der Beschriftungen losgelassen wird. Wir werden diese Prozedur später näher erläutern.

Wir legen für die Option `-width` des Widgets den Wert `0` fest, was bedeutet, dass die Breite der Tablelist gerade so groß sein soll, dass alle Spalten darin Platz finden.

Nachdem wir das Tablelist-Widget erstellt haben, stellen wir sicher, dass die Elemente der Spalten 2, 3, 4 und 5 (in denen die x- und y-Koordinaten sowie die Breiten und Höhen der untergeordneten Elemente angezeigt werden) beim Sortieren der Elemente nach einer dieser Spalten als Ganzzahlen verglichen werden. Dies erreichen wir mithilfe der Tablelist-Operation `columnconfigure`.

Mit demselben Unterbefehl `columnconfigure` können wir festlegen, dass bei der Anzeige der Elemente der Spalten 6 und 7 (mit den Titeln `"Mapped"` bzw. `"Viewable"`) die booleschen Werte `1` und `0` durch die Zeichenketten `"yes"` und `"no"` ersetzt werden, die der unten gezeigte Befehl `demo::formatBoolean` zurückgibt.

In einer skalierungsfähigen Anwendung sollte der Wert der Tablelistoption `-treestyle` den DPI-Skalierungsgrad des Bildschirms widerspiegeln. Unter Windows Vista, 7, 8 und 10+ erfüllt der Standardwert dieser Option diese Anforderung bereits, und unter Mac OS X/11+ skaliert die Desktop-Engine alles automatisch nach Bedarf. Unter X11 setzen wir die Option `-treestyle` bei vorhandener SVG-Unterstützung auf `bicolor` und andernfalls auf einen der Werte

`bicolor100`, ..., `bicolor200` (abhängig vom Wert der Variablen `::tablelist::scalingpct`).

Nachdem wir die Option `-treestyle` unter X11 konfiguriert haben, durchlaufen wir die Elemente der Liste, die die Pfadnamen aller Kopfzeilenbeschriftungen des Tablelist-Widgets enthält und vom Unterbefehl `labels` des dem Widget entsprechenden Tcl-Befehls zurückgegeben wird. Für jedes Element der Liste binden wir die Prozedur `demo::updateItemsDelayed` an das Ereignis `<Configure>`. Auf diese Weise stellen wir sicher, dass die Prozedur immer dann aufgerufen wird, wenn sich die Größe der durch dieses Listenelement angegebenen Kopfzeile ändert.

Die vier Aufrufe des Befehls `bind` im Anschluss an die Erstellung des Pop-up-Menüs verwenden ein Bindungs-Tag, dessen Name vom Pfadnamen des Tablelist-Widgets abhängt und vom Unterbefehl `bodytag` des mit dem Tablelist-Widget verknüpften Tcl-Befehls zurückgegeben wird. Der Vorteil der Verwendung dieses Tags anstelle des Pfadnamens des Datenbereichs der Tablelist besteht darin, dass dieses Bindungs-Tag nicht nur mit dem Datenbereich, sondern auch mit den Trennrahmen und den Beschriftungen, die eingebettete Bilder anzeigen, verknüpft ist. Dies ist in unserem Beispiel wichtig, da wir sicherstellen möchten, dass die Ereignisse `<<Button3>>` und `<Double-1>` innerhalb einer Beschriftung, die eingebettete Bilder enthält, auf dieselbe Weise behandelt werden wie im restlichen Datenbereich der Tablelist. Sowohl das virtuelle Ereignis `<<Button3>>` (verwendet in den ersten drei `bind`-Befehlen) als auch das Bindungs-Tag `TablelistBody` (verwendet im ersten Bindungsskript) werden vom Tablelist-Paket erstellt. Die ersten drei `bind`-Befehle stellen sicher, dass ein virtuelles Ereignis `<<Button3>>` das nächstgelegene Element auswählt und aktiviert und ein Popup-Menü mit zwei Befehlseinträgen anzeigt, die sich auf das durch dieses Element beschriebene Widget beziehen.

Wir füllen die Tablelist, indem wir die Prozedur `demo::putChildren` aufrufen, die wie folgt implementiert ist:

```
#------------------------------------------------------------------------------
# demo::putChildren
#
# Outputs the data of the children of the widget w into the tablelist widget
# tbl, as child items of the one identified by nodeIdx.
#------------------------------------------------------------------------------
proc demo::putChildren {w tbl nodeIdx} {
    . . .

    if {$nodeIdx eq "root"} {
        set top [winfo toplevel $tbl]
        wm title $top "Children of the [winfo class $w] Widget \"$w\""

        $tbl resetsortinfo
        $tbl delete 0 end
        set row 0
    } else {
        set row [expr {$nodeIdx + 1}]
    }

    #
    # Display the data of the children of the
    # widget w in the tablelist widget tbl
```

```
    #
    variable leafImg
    variable compImg
    foreach c [winfo children $w] {
        #
        # Insert the data of the current child into the tablelist widget
        #
        set item {}
        lappend item \
                [winfo name $c] [winfo class $c] [winfo x $c] [winfo y $c] \
                [winfo width $c] [winfo height $c] [winfo ismapped $c] \
                [winfo viewable $c] [winfo manager $c]
        $tbl insertchild $nodeIdx end $item

        #
        # Embed an image into the first cell of the row; mark the
        # row as collapsed if the child widget has children itself
        #
        if {[llength [winfo children $c]] == 0} {
            $tbl cellconfigure end,0 -image $leafImg
        } else {
            $tbl cellconfigure end,0 -image $compImg
            $tbl collapse $row
        }

        $tbl rowattrib $row pathName $c
        incr row
    }

    if {$nodeIdx eq "root"} {
        #
        # Configure the "Refresh" and "Parent" buttons
        #
        $top.bf.b1 configure -command [list demo::refreshView $w $tbl]
        set b2 $top.bf.b2
        set p [winfo parent $w]
        if {$p eq ""} {
            $b2 configure -state disabled
        } else {
            $b2 configure -state normal -command \
                [list demo::putChildren $p $tbl root]
        }
    }
}
```

Das letzte Argument dieser Prozedur gibt den Baumknoten an, der zum übergeordneten Knoten der Elemente werden soll, die die Daten der als erstes Argument übergebenen Widget-Kinder anzeigen. Wenn dieser übergeordnete Knoten der unsichtbare Knoten `root` ist, setzen wir zunächst die Sortierinformationen zurück, indem wir den Unterbefehl `resetsortinfo` für die Tablelist aufrufen, und löschen die aktuellen Elemente des Tablelist-Widgets `tbl`. Anschließend durchläuft die Prozedur die Kinder des angegebenen Widgets und fügt die Elemente ein, die aus Daten erstellt wurden, die mithilfe des Befehls `winfo` abgerufen wurden. Jedes neue Element

wird mithilfe des Unterbefehls `insertchild(ren)` am Ende der Liste der untergeordneten Elemente des übergeordneten Knotens angefügt.

Für jedes untergeordnete Widget rufen wir die Tablelistoperation `cellconfigure` auf, um den Wert der Option `-image` in der ersten Zelle der entsprechenden Zeile festzulegen, die den Namen des untergeordneten Elements enthält. Auf diese Weise bettet die Prozedur das Bild `$leafImg` oder `$compImg` in die erste Zelle ein, je nachdem, ob es sich bei dem betreffenden untergeordneten Widget um ein Blatt- oder ein zusammengesetztes Widget handelt. (Zur Erinnerung: Beide Bilder wurden außerhalb dieser Prozedur im Rahmen der Initialisierung des Namespaces `demo` erstellt.)

Wir kennzeichnen jede neu erstellte Zeile, die einem untergeordneten Widget entspricht, das selbst untergeordnete Elemente hat, als "zusammengeklappt", indem wir den Unterbefehl `collapse` aufrufen. Dadurch wird dem Inhalt der ersten Spalte ein Steuerelement zum Ein- und Ausblenden vorangestellt, dessen Spaltenindex `0` dem Standardwert der Konfigurationsoption `-treecolumn` entspricht.

Wir verwenden den Unterbefehl `rowattrib` von "tablelist", um den vollständigen Pfadnamen jedes untergeordneten Widgets als privates Attribut zu speichern, das der entsprechenden Zeile in der Tablelist zugeordnet ist, da dieser später an mehreren Stellen benötigt wird.

Wie oben erwähnt, wird die Prozedur `demo::expandCmd` automatisch aufgerufen, wenn eine Zeile erweitert wird, die ein Steuerelement zum Erweitern/Reduzieren enthält:

```
#------------------------------------------------------------------------------
# demo::expandCmd
#
# Outputs the data of the children of the widget whose leaf name is displayed
# in the first cell of the specified row of the tablelist widget tbl, as child
# items of the one identified by row.
#------------------------------------------------------------------------------
proc demo::expandCmd {tbl row} {
    if {[$tbl childcount $row] == 0} {
        set w [$tbl rowattrib $row pathName]
        putChildren $w $tbl $row

        #
        # Apply the last sorting (if any) to the new items
        #
        $tbl refreshsorting $row
    }
}
```

Die Prozedur ruft den Unterbefehl `childcount` auf, um zu prüfen, ob die untergeordneten Elemente der Zeile, die gerade erweitert werden soll, bereits in das Tablelist-Widget eingefügt wurden, und fügt sie ein, falls dies nicht der Fall ist. Außerdem stellt sie mithilfe des Tablelist-Unterbefehls `refreshsorting` sicher, dass die untergeordneten Elemente in der Reihenfolge angezeigt werden, die der letzten Sortierung (falls vorhanden) entspricht. Eine etwaige Sortierung (falls erforderlich) wird ausschließlich auf die untergeordneten Elemente angewendet, die gerade in das Tablelist-Widget eingefügt wurden.

Die oben erwähnten Prozeduren `demo::formatBoolean` und `demo::labelCmd` sind trivial:

```
#------------------------------------------------------------------------------
# demo::formatBoolean
#
# Returns "yes" or "no", according to the specified boolean value.
#------------------------------------------------------------------------------
proc demo::formatBoolean val {
    return [expr {$val ? "yes" : "no"}]
}

#------------------------------------------------------------------------------
# demo::labelCmd
#
# Sorts the content of the tablelist widget tbl by its col'th column and makes
# sure the items will be updated 500 ms later (because one of the items might
# refer to a canvas containing the arrow that displays the sorting order).
#------------------------------------------------------------------------------
proc demo::labelCmd {tbl col} {
    tablelist::sortByColumn $tbl $col
    updateItemsDelayed $tbl
}
```

Der Befehl `tablelist::sortByColumn` sortiert die Elemente des Tablelist-Widgets nach der angegebenen Spalte in der richtigen Reihenfolge, indem er den Unterbefehl `sortbycolumn` des dem Tablelist-Widget zugeordneten Tcl-Befehls aufruft.

Die Umsetzung des Befehls `demo::updateItemsDelayed`, der in dieser Prozedur aufgerufen wird und bereits in der oben genannten Prozedur `demo::displayChildren` vorkam, ist recht einfach:

```
#------------------------------------------------------------------------------
# demo::updateItemsDelayed
#
# Arranges for the items of the tablelist widget tbl to be updated 500 ms later.
#------------------------------------------------------------------------------
proc demo::updateItemsDelayed tbl {
    #
    # Schedule the demo::updateItems command for execution
    # 500 ms later, but only if it is not yet pending
    #
    if {[$tbl attrib afterId] eq ""} {
        $tbl attrib afterId [after 500 [list demo::updateItems $tbl]]
    }
}

#------------------------------------------------------------------------------
# demo::updateItems
#
# Updates the items of the tablelist widget tbl.
#------------------------------------------------------------------------------
proc demo::updateItems tbl {
    #
    # Reset the tablelist's "afterId" attribute
    #
```

```
    $tbl attrib afterId ""

    #
    # Update the items
    #
    set rowCount [$tbl size]
    for {set row 0} {$row < $rowCount} {incr row} {
        set c [$tbl cellcget $row,0 -text]
        if {![winfo exists $c]} {
            continue
        }

        set item {}
        lappend item \
                [winfo name $c] [winfo class $c] [winfo x $c] [winfo y $c] \
                [winfo width $c] [winfo height $c] [winfo ismapped $c] \
                [winfo viewable $c] [winfo manager $c]
        $tbl rowconfigure $row -text $item
    }

    #
    # Repeat the last sort operation (if any)
    #
    $tbl refreshsorting
}
```

Wie bereits im vorherigen Beispiel erwähnt, kann jedes Tablelist-Widget eine beliebige Anzahl privater Attribute besitzen, die mithilfe des Unterbefehls `attrib` des dem Widget entsprechenden Tcl-Befehls gesetzt und abgerufen werden können. Das Attribut `afterId` wird von der Prozedur `demo::updateItemsDelayed` gesetzt, wenn der Befehl `demo::updateItems` für die Ausführung 500 ms später eingeplant wird, jedoch nur, wenn sein Wert eine leere Zeichenkette ist. Aus diesem Grund setzt die Prozedur `demo::updateItems` dieses Attribut zurück. Außerdem nutzt sie den Tablelist-Unterbefehl `cellcget`, um die in der ersten Zelle jeder Zeile enthaltenen Pfadnamen abzurufen, und aktualisiert die Daten der untergeordneten Elemente mithilfe des Unterbefehls `rowconfigure`. Nach der Aktualisierung der Elemente wiederholt die Prozedur die zuletzt durchgeführte Sortierung nach einer Spalte (wenn vorhanden) mithilfe des Unterbefehls `refreshsorting`.

Die Prozeduren `demo::putChildrenOfActiveWidget`, `demo::dispConfigOfActiveWidget` und `demo::postPopupMenu` (siehe `demo::displayChildren`) sind ebenfalls unkompliziert. Der unten gezeigte Befehl `demo::putChildrenOfActiveWidget` nutzt beispielsweise den Index der aktiven Zeile `active`, um die aktive Zeile zu identifizieren.

```
#------------------------------------------------------------------------------
# demo::putChildrenOfActiveWidget
#
# Outputs the data of the children of the active widget into the tablelist
# widget tbl.
#------------------------------------------------------------------------------
proc demo::putChildrenOfActiveWidget tbl {
    set w [$tbl rowattrib active pathName]
```

```
    if {![winfo exists $w]} {
        bell
        tk_messageBox -title "Error" -icon error -message \
            "Bad window path name \"$w\"" -parent [winfo toplevel $tbl]
        return ""
    }

    if {[llength [winfo children $w]] == 0} {
        bell
    } else {
        putChildren $w $tbl
    }
}
```

Die Prozedur `demo::refreshView`, die der Schaltfläche "Aktualisieren" zugeordnet ist, ist wie folgt implementiert:

```
#------------------------------------------------------------------------------
# demo::refreshView
#
# Redisplays the data of the children of the widget w in the tablelist widget
# tbl and restores the expanded states of the items as well as the vertical
# view.
#------------------------------------------------------------------------------
proc demo::refreshView {w tbl} {
    #
    # Save the vertical view and get the path names of
    # the child widgets displayed in the expanded rows
    #
    set yView [$tbl yview]
    foreach key [$tbl expandedkeys] {
        set pathName [$tbl rowattrib $key pathName]
        set expandedWidgets($pathName) 1
    }

    #
    # Redisplay the data of the widget's (possibly changed) children and
    # restore the expanded states of the children, along with the vertical view
    #
    putChildren $w $tbl root
    restoreExpandedStates $tbl root expandedWidgets
    $tbl yview moveto [lindex $yView 0]
}
```

Bevor der Inhalt der Tablelist über `demo::putChildren` erneut angezeigt wird, holen wir mithilfe des Tablelist-Unterbefehls `expandedkeys` die vollständigen Schlüssel der aktuell erweiterten Elemente und fügen die entsprechenden Widget-Pfade in das Array `expandedWidgets` ein. Nachdem die Daten der (möglicherweise geänderten) untergeordneten Elemente des als erstes Argument angegebenen Widgets neu angezeigt wurden, übergeben wir dieses Array an die unten gezeigte Prozedur `demo::restoreExpandedStates`:

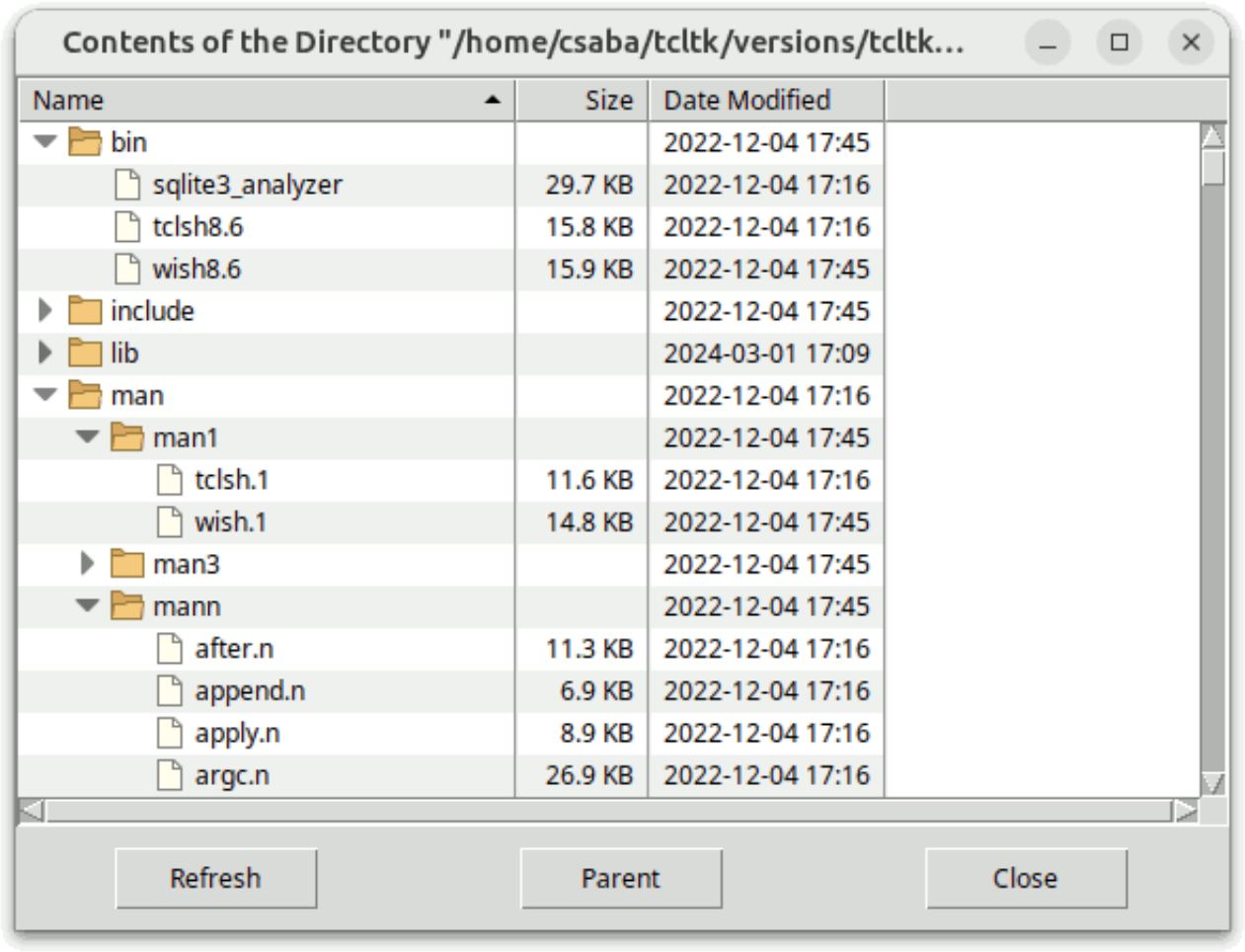
```
#------------------------------------------------------------------------------
# restoreExpandedStates
#
# Expands those children of the parent identified by nodeIdx that display the
# data of child widgets whose path names are the names of the elements of the
# array specified by the last argument.
#------------------------------------------------------------------------------
proc demo::restoreExpandedStates {tbl nodeIdx expandedWidgetsName} {
    upvar $expandedWidgetsName expandedWidgets

    foreach key [$tbl childkeys $nodeIdx] {
        set pathName [$tbl rowattrib $key pathName]
        if {[info exists expandedWidgets($pathName)]} {
            $tbl expand $key -partly
            restoreExpandedStates $tbl $key expandedWidgets
        }
    }
}
```

Die Prozedur ruft mithilfe des Unterbefehls `childkeys` die Liste der vollständigen Schlüssel der untergeordneten Knoten des durch `nodeIdx` angegebenen übergeordneten Knotens ab. Anschließend durchläuft sie diese Liste schrittweise und ruft für jeden Schlüssel, dessen entsprechende Zeile zuvor erweitert wurde, den Unterbefehl `expand` der Tablelist auf; anschließend ruft sie sich selbst rekursiv auf, um den erweiterten Zustand der untergeordneten Elemente dieser Zeile wiederherzustellen.

Ein skalierbarer Verzeichnis-Viewer auf Basis einer Tablelist

Das Skript `dirViewer.tcl` im Verzeichnis `demos` zeigt den Inhalt der im System eingebundenen Volumes an (z. B. das Stammverzeichnis `/` unter UNIX und die lokalen Laufwerke unter Windows) in einer Tablelist an, die als mehrspaltiges Baum-Widget verwendet wird:



Verzeichnis-Viewer

Durch Doppelklicken auf einen Eintrag oder durch Aufrufen des einzigen Eintrags eines Popup-Menüs im Datenbereich der Tablelist können Sie den Inhalt des Ordners anzeigen, der dem aktiven Eintrag entspricht. Um eine Ebene höher zu gelangen, klicken Sie auf die Schaltfläche "Übergeordneter Ordner".

Es gibt viele Ähnlichkeiten zwischen diesem Skript und dem im vorherigen Abschnitt behandelten Skript. Im Folgenden werden wir nur einige wenige Prozeduren vorstellen, die Tablelist-Befehle aufrufen, die in den obigen Beispielen nicht vorkommen:

```
package require tablelist

#
# Add some entries to the Tk option database
#
set dir [file dirname [info script]]
source [file join $dir option.tcl]

#
# Create three images corresponding to the display's DPI scaling level
#
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set pct ""; set sfx "svg"; set fmt $tablelist::svgfmt
} else {
```

```
    set pct $tablelist::scalingpct; set sfx "gif"; set fmt "gif"
}
foreach name {clsdFolder openFolder file} {
    set imgFile $name$pct.$sfx          ;# e.g., "file.svg" or "file150.gif"
    image create photo ${name}Img -file [file join $dir $imgFile] -format $fmt
}

#
# Helper procedures needed to work around the fact that
# on Android the directory "/" is normally not readable
#
set onAndroid [expr {[info exists ::tk::android] && $::tk::android}]
proc isReadable name {
    return [expr {$::onAndroid && $name eq "/" ? 1 : [file readable $name]}]
}
proc dirContent dir {
    return [expr {$::onAndroid && $dir eq "/" ? [list /sdcard] :
            [glob -nocomplain -types {d f} -directory $dir *]}]
}
```

Wenn die Tk-Version mindestens 8.7 ist oder das Paket "tksvg" in den Interpreter geladen werden kann, werden die drei Bilder, die jeweils einen geschlossenen Ordner, einen offenen Ordner und eine Datei darstellen, aus den Dateien `clsdFolder.svg`, `openFolder.svg` und `file.svg` unter Verwendung der öffentlichen Variablen `tablelist::svgfmt` erstellt. Andernfalls werden sie aus Dateien erstellt, deren Namen den DPI-Skalierungsprozentsatz des Bildschirms enthalten, der durch die öffentliche Variable `tablelist::scalingpct` vorgegeben ist. Die Dateien `clsdFolder100.gif`, `openFolder100.gif` und `file100.gif` enthalten Bilder der Größe 16 × 16, die Dateien `*125.gif` enthalten Bilder mit einer Größe von 20 x 20, die Dateien `*150.gif` enthalten Bilder mit einer Größe von 24 x 24 und so weiter. Im ersten Fall werden nur drei `*.svg`-Dateien benötigt, und die Bilder werden automatisch skaliert, entsprechend dem tatsächlichen Skalierungsprozentsatz des Displays, der größer sein kann als der Maximalwert `200` der Variablen `tablelist::scalingpct`. Dieser Vorteil ist besonders praktisch, wenn AndroWish auf einem Tablet oder Smartphone ausgeführt wird. Im zweiten Fall werden insgesamt 15 `*.gif` (oder `*.png`) Dateien benötigt, um sicherzustellen, dass die Größen der Bilder dem Wert der Variablen `tablelist::scalingpct` entsprechen.

```
#--------------------------------------------------------------------------
# displayContents
#
# Displays the content of the directory dir in a tablelist widget.
#--------------------------------------------------------------------------
proc displayContents dir {
    #
    # Create a scrolled tablelist widget with 3 dynamic-
    # width columns and interactive sort capability
    #
    set tf .tf
    frame $tf -class ScrollArea
    set tbl $tf.tbl
    set vsb $tf.vsb
```

```
    set hsb $tf.hsb
    tablelist::tablelist $tbl \
        -columns {0 "Name"          left
                  0 "Size"          right
                  0 "Date Modified" left} \
        -height 16 -width 80 -movablecolumns no -showseparators yes \
        -expandcommand expandCmd -xscrollcommand [list $hsb set] \
        -yscrollcommand [list $vsb set]
    if {[$tbl cget -selectborderwidth] == 0} {
        $tbl configure -spacing 1
    }
    $tbl columnconfigure 0 -formatcommand formatString -sortmode dictionary
    $tbl columnconfigure 1 -formatcommand formatSize -sortmode integer
    $tbl columnconfigure 2 -formatcommand formatString
    scrollbar $vsb -orient vertical   -command [list $tbl yview]
    scrollbar $hsb -orient horizontal -command [list $tbl xview]

    #
    # On X11 configure the tablelist according
    # to the display's DPI scaling level
    #
    if {[tk windowingsystem] eq "x11"} {
        global pct                                      ;# ""|100|125|...|200
        $tbl configure -treestyle bicolor$pct
    }

    bind $tbl <<TablelistYViewChanged>> [list addImages $tbl]

    . . .

    #
    # Populate the tablelist with the content of the given directory
    #
    $tbl sortbycolumn 0
    putContents $dir $tbl root
}
```

Die Prozedur `displayContents` erstellt das Tablelist-Widget und die beiden Scrollbars als untergeordnete Elemente eines Frames der Klasse `ScrollArea`. Für diese Klasse enthält die Datei `option.tcl`, die in das Hauptskript mittels `source` eingebunden ist, einige Einstellungen zum Look & Feel, die denen in unserem ersten Beispiel ähneln:

```
option add *ScrollArea.borderWidth                    1
option add *ScrollArea.relief                         sunken
option add *ScrollArea.Tablelist.borderWidth          0
option add *ScrollArea.Tablelist.highlightThickness   0
```

In einer skalierungsfähigen Anwendung sollte der Wert der Tablelistoption `-treestyle` den DPI-Skalierungsgrad des Bildschirms widerspiegeln. Unter Windows Vista, 7, 8 und 10+ erfüllt der Standardwert dieser Option diese Anforderung bereits, und unter Mac OS X/11+ skaliert die Desktop-Engine alles automatisch nach Bedarf. Unter X11 setzen wir die Option `-treestyle` bei vorhandener SVG-Unterstützung auf `bicolor` und andernfalls auf einen der Werte `bicolor100`, ..., `bicolor200` (abhängig vom Wert der Variablen `tablelist::scalingpct`

).

Immer wenn sich die vertikale Ansicht im Fenster der Tablelist ändert, erhält das Widget ein virtuelles Ereignis vom Typ `<<TablelistYViewChanged>>`. Unsere Bindung für dieses Ereignis ruft die Prozedur `addImages` auf, die wir gegen Ende dieses Abschnitts vorstellen und erläutern.

Schauen wir uns einmal an, wie das Skript die Tablelist mit dem Inhalt des angegebenen Verzeichnisses füllt:

```
#------------------------------------------------------------------------------
# putContents
#
# Outputs the content of the directory dir into the tablelist widget tbl, as
# child items of the one identified by nodeIdx.
#------------------------------------------------------------------------------
proc putContents {dir tbl nodeIdx} {
    . . .

    if {$nodeIdx eq "root"} {
        if {$dir eq ""} {
            if {[llength [file volumes]] == 1} {
                wm title . "Contents of the File System"
            } else {
                wm title . "Contents of the File Systems"
            }
        } else {
            wm title . "Contents of the Directory \"[file nativename $dir]\""
        }

        $tbl delete 0 end
        set row 0
    } else {
        set row [expr {$nodeIdx + 1}]
    }

    #
    # Build a list from the data of the subdirectories and
    # files of the directory dir.  Prepend a "D" or "F" to
    # each entry's name and modification date & time, for
    # sorting purposes (it will be removed by formatString).
    #
    set itemList {}
    if {$dir eq ""} {
        foreach volume [file volumes] {
            lappend itemList [list D[file nativename $volume] -1 D $volume]
        }
    } else {
        foreach entry [dirContent $dir] {
            if {[catch {file mtime $entry} modTime] != 0} {
                continue
            }

            if {[file isdirectory $entry]} {
```

```
                lappend itemList [list D[file tail $entry] -1 \
                    D[clock format $modTime -format "%Y-%m-%d %H:%M"] $entry]
            } else {
                lappend itemList [list F[file tail $entry] [file size $entry] \
                    F[clock format $modTime -format "%Y-%m-%d %H:%M"] ""]
            }
        }
    }

    #
    # Sort the above list and insert it into the tablelist widget
    # tbl as list of children of the row identified by nodeIdx
    #
    set itemList [$tbl applysorting $itemList]
    $tbl insertchildlist $nodeIdx end $itemList

    foreach item $itemList {
        set name [lindex $item end]
        if {$name ne ""} {                                      ;# directory
            $tbl rowattrib $row pathName $name

            #
            # Mark the row as collapsed if the directory is non-empty
            #
            if {[isReadable $name] && [llength [dirContent $name]] != 0} {
                $tbl collapse $row
            }
        }

        incr row
    }

    . . .
}
```

Der Hauptunterschied zwischen der oben beschriebenen Prozedur `putContents` und der im vorherigen Abschnitt beschriebenen Prozedur `demo::putChildren` betrifft die Art und Weise, wie untergeordnete Elemente in das Tablelist-Widget eingefügt werden. Anstatt sie einzeln mit Hilfe des Tablelist-Unterbefehls `insertchild(ren)` einzufügen, fügen wir hier die entsprechenden Daten zu einer Liste von Elementen hinzu und rufen dann den wesentlich leistungsfähigeren Unterbefehl `insertchildlist` auf. Anstatt die Elemente zunächst einzufügen und sie dann über `refreshsorting` zu sortieren, führen wir außerdem zunächst die erforderlichen Sortierungen an der oben genannten Liste von Elementen durch, indem wir den Unterbefehl `applysorting` aufrufen. Auch dies ist wesentlich schneller als das Sortieren der bereits eingefügten untergeordneten Elemente.

Diese Prozedur veranschaulicht zudem eine effektive Technik, die auf der Spaltenkonfigurationsoption `-formatcommand` basiert: In der internen Liste des Tablelist-Widgets werden den Namen und Änderungszeiten der Verzeichnisse und Dateien jeweils ein `D` bzw. `F` vorangestellt. Dadurch wird sichergestellt, dass die Verzeichnisse vor den Dateien sortiert werden (bei aufsteigender Sortierung). Bei der Anzeige der Elemente ruft der Tablelist-Code automatisch die Prozedur `formatString` auf, die das erste Zeichen entfernt.

Ebenso wird in der internen Liste des Widgets die Größe eines Verzeichnisses auf `-1` gesetzt, wodurch es vor den Größen der Dateien sortiert wird. Die Prozedur `formatSize`, die bei der Anzeige der Elemente automatisch aufgerufen wird, ersetzt diesen Wert durch eine leere Zeichenkette:

```
#------------------------------------------------------------------------------
# formatString
#
# Returns the substring obtained from the specified value by removing its first
# character.
#------------------------------------------------------------------------------
proc formatString val {
    return [string range $val 1 end]
}

#------------------------------------------------------------------------------
# formatSize
#
# Returns an empty string if the specified value is negative and the value
# itself in user-friendly format otherwise.
#------------------------------------------------------------------------------
proc formatSize val {
    if {$val < 0} {
        return ""
    } elseif {$val < 1024} {
        return "$val bytes"
    } elseif {$val < 1048576} {
        return [format "%.1f KB" [expr {$val / 1024.0}]]
    } elseif {$val < 1073741824} {
        return [format "%.1f MB" [expr {$val / 1048576.0}]]
    } else {
        return [format "%.1f GB" [expr {$val / 1073741824.0}]]
    }
}

#------------------------------------------------------------------------------
# expandCmd
#
# Outputs the content of the directory whose leaf name is displayed in the
# first cell of the specified row of the tablelist widget tbl, as child items
# of the one identified by row.
#------------------------------------------------------------------------------
proc expandCmd {tbl row} {
    if {[$tbl childcount $row] == 0} {
        set dir [$tbl rowattrib $row pathName]
        putContents $dir $tbl $row
    }
}

#------------------------------------------------------------------------------
# addImages
#
# Embeds an image into the first cell of each row in the current view.
```

```
#------------------------------------------------------------------------------
proc addImages tbl {
    set topRow [$tbl index top]
    set btmRow [$tbl index bottom]
    for {set row $topRow} {$row <= $btmRow} {incr row} {
        if {[$tbl hasrowattrib $row pathName]} {        ;# directory item
            set img [expr {[$tbl isexpanded $row] ?
                    "openFolderImg" : "clsdFolderImg"}]
        } else {                                        ;# file item
            set img fileImg
        }
        $tbl cellconfigure $row,0 -image $img
    }
}

. . .

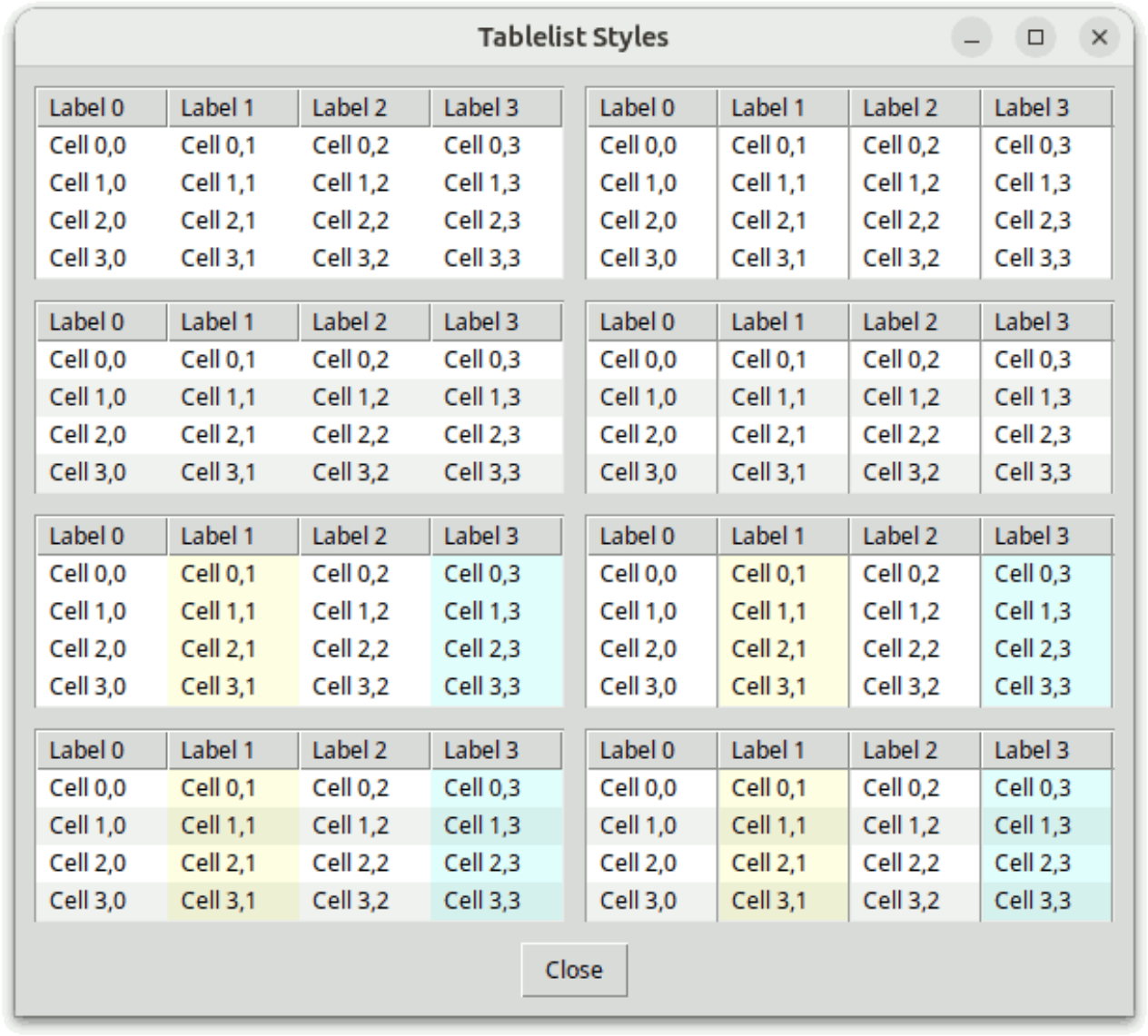
displayContents ""
```

Wie bereits erwähnt, wird die Prozedur `addImages` immer dann aufgerufen, wenn sich die vertikale Ansicht im Fenster der Tablelist ändert. Sie fügt in die erste Zelle jeder Zeile der aktuellen Ansicht ein Bild ein, das einen geöffneten oder geschlossenen Ordner bzw. eine Datei darstellt. Anders als im vorherigen Beispiel fügen wir hier die Bilder nur zu den Elementen hinzu, die in der aktuellen Ansicht enthalten sind, und sparen so erheblich Zeit und Speicherplatz.

Die letzte Zeile des Skripts ruft die Prozedur `displayContents` mit einer leeren Zeichenkette als Argument auf, d. h., sie zeigt die auf dem System eingebundenen Volumes an.

Das Erscheinungsbild eines Tablelist-Widgets verbessern

Das Skript `styles.tcl` im Verzeichnis `demos` veranschaulicht einige Möglichkeiten, wie Sie Tablelist-Widgets intelligenter gestalten und die Lesbarkeit ihrer Einträge verbessern können. Es erstellt 8 Tablelist-Widgets, die in der folgenden Abbildung dargestellt sind:



Styles

Hier ist der entsprechende Codeausschnitt:

```
#
# Create, configure, and populate 8 tablelist widgets
#
frame .f
for {set n 0} { $n < 8} {incr n} {
    set tbl .f.tbl$n
    tablelist::tablelist $tbl \
        -columntitles {"Label 0" "Label 1" "Label 2" "Label 3"} \
        -background white -height 4 -width 40 -stretch all
    if {[$tbl cget -selectborderwidth] == 0} {
        $tbl configure -spacing 1
    }

    switch $n {
        1 {
            $tbl configure -showseparators yes
```

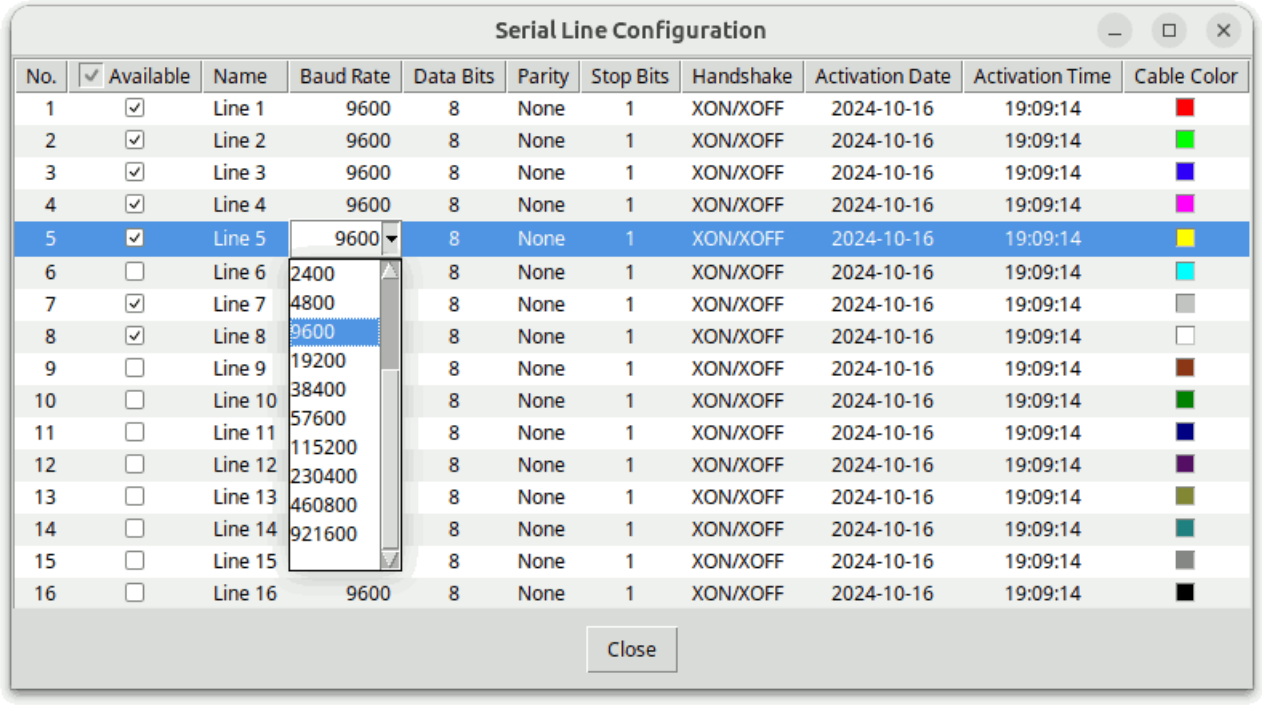
```
        }
        2 {
            $tbl configure -stripebackground #f0f0f0
        }
        3 {
            $tbl configure -stripebackground #f0f0f0 -showseparators yes
        }
        4 {
            $tbl columnconfigure 1 -background LightYellow
            $tbl columnconfigure 3 -background LightCyan
        }
        5 {
            $tbl configure -showseparators yes
            $tbl columnconfigure 1 -background LightYellow
            $tbl columnconfigure 3 -background LightCyan
        }
        6 {
            $tbl configure -stripebackground #f0f0f0
            $tbl columnconfigure 1 -background LightYellow \
                -stripebackground #f0f0d2
            $tbl columnconfigure 3 -background LightCyan \
                -stripebackground #d2f0f0
        }
        7 {
            $tbl configure -stripebackground #f0f0f0 -showseparators yes
            $tbl columnconfigure 1 -background LightYellow \
                -stripebackground #f0f0d2
            $tbl columnconfigure 3 -background LightCyan \
                -stripebackground #d2f0f0
        }
    }

    foreach row {0 1 2 3} {
        $tbl insert end \
             [list "Cell $row,0" "Cell $row,1" "Cell $row,2" "Cell $row,3"]
    }
}
```

Die einzige Konfigurationsoption, die hier verwendet wird, aber in den ersten drei Beispielen nicht behandelt wurde (obwohl sie bereits im vorherigen Beispiel vorkam), ist `-showseparators`. Der dadurch erzielte visuelle Effekt sieht sowohl für sich allein als auch in Kombination mit horizontalen oder vertikalen Streifen ansprechend aus, die mithilfe der Option `-stripebackground` bzw. des Unterbefehls `columnconfigure` erzeugt werden.

Erweiterte, skalierbare und interaktive Bearbeitung von Tabellenzellen

Die Skripte `tileWidgets.tcl`, `bwidget.tcl`, `iwidgets.tcl`, sowie `miscWidgets.tcl` im Verzeichnis `demos` erstellen ein Tablelist-Widget, das einige Parameter von 16 seriellen Leitungen anzeigt, und veranschaulichen die Verwendung verschiedener Widgets aus dem Tk-Kern sowie aus den Paketen "tile", BWidget, Iwidgets, combobox (von Bryan Oakley), ctext und Mentry (oder Mentry_tile) für die interaktive Bearbeitung von Zellen. Die folgende Abbildung zeigt das Tablelist-Widget zusammen mit einer BWidget-ComboBox, die zur Bearbeitung des Inhalts einer seiner Zellen verwendet wird:



Konfiguration der seriellen Schnittstelle

Hier ist der relevante Codeausschnitt aus dem Skript `bwidget.tcl` (die Skripte `tileWidgets.tcl`, `iwidgets.tcl` und `miscWidgets.tcl` sind ähnlich). Einige Teile des Codes sind in der Farbe "red" dargestellt – darauf werden wir gegen Ende dieses Abschnitts noch einmal zurückkommen.

```
package require tablelist
package require BWidget

wm title . "Serial Line Configuration"

#
# Add some entries to the Tk option database
#
set dir [file dirname [info script]]
source [file join $dir option.tcl]
option add *Tablelist*Entry.background white

#
# Create the images "checkedImg" and "uncheckedImg", as well as 16 images of
# names like "img#FF0000", displaying colors identified by names like "red"
#
source [file join $dir images.tcl]

#
# Register some widgets from the BWidget package for interactive cell editing
#
tablelist::addBWidgetEntry
tablelist::addBWidgetSpinBox
tablelist::addBWidgetComboBox
```

```
#
# Create a tablelist widget with editable columns (except the first one)
#
set tbl .tbl
tablelist::tablelist $tbl \
    -columns {0 "No."             right
              0 "Available"       center
              0 "Name"            left
              0 "Baud Rate"       right
              0 "Data Bits"       center
              0 "Parity"          left
              0 "Stop Bits"       center
              0 "Handshake"       left
              0 "Activation Date" center
              0 "Activation Time" center
              0 "Cable Color"     center} \
    -editstartcommand editStartCmd -editendcommand editEndCmd \
    -aftercopycommand afterCopyCmd -height 0 -width 0
if {[$tbl cget -selectborderwidth] == 0} {
    $tbl configure -spacing 1
}
$tbl columnconfigure 0 -sortmode integer
$tbl columnconfigure 1 -name available -editable yes -editwindow checkbutton \
    -formatcommand emptyStr -labelwindow checkbutton
$tbl columnconfigure 2 -name lineName  -editable yes -editwindow Entry \
    -allowduplicates 0 -sortmode dictionary
$tbl columnconfigure 3 -name baudRate  -editable yes -editwindow ComboBox \
    -sortmode integer
$tbl columnconfigure 4 -name dataBits  -editable yes -editwindow SpinBox
$tbl columnconfigure 5 -name parity    -editable yes -editwindow ComboBox
$tbl columnconfigure 6 -name stopBits  -editable yes -editwindow ComboBox
$tbl columnconfigure 7 -name handshake -editable yes -editwindow ComboBox
$tbl columnconfigure 8 -name actDate   -editable yes -editwindow Entry \
    -formatcommand formatDate -sortmode integer
$tbl columnconfigure 9 -name actTime   -editable yes -editwindow Entry \
    -formatcommand formatTime -sortmode integer
$tbl columnconfigure 10 -name color    -editable yes -editwindow menubutton \
    -formatcommand emptyStr

proc emptyStr   val { return "" }
proc formatDate val { return [clock format $val -format "%Y-%m-%d"] }
proc formatTime val { return [clock format $val -format "%H:%M:%S"] }

#
# Populate the tablelist widget and configure the checkbutton
# embedded into the header label of the column "available"
#
source [file join $dir serialParams.tcl]

. . .
```

Die Größe der Bilder, die in der Datei `images.tcl` erstellt werden, die in das Hauptskript mittels `source` eingebunden ist, hängt vom Skalierungsgrad des Bildschirms ab. Beispiel:

```
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set fmt $tablelist::svgfmt
    image create photo checkedImg   -file [file join $dir checked.svg] \
        -format $fmt
    image create photo uncheckedImg -file [file join $dir unchecked.svg] \
        -format $fmt
} else {
    set pct $tablelist::scalingpct
    image create photo checkedImg   -file [file join $dir checked$pct.gif] \
        -format gif
    image create photo uncheckedImg -file [file join $dir unchecked$pct.gif] \
        -format gif
}
```

Wir rufen die Befehle `tablelist::addBWidgetEntry`, `tablelist::addBWidgetSpinBox` und `tablelist::addBWidgetComboBox` auf, um die Widgets "Entry", "SpinBox" und "ComboBox" aus dem BWidget-Paket für die interaktive Zellbearbeitung zu registrieren. Diese Befehle geben die Werte `"Entry"`, `"SpinBox"` und `"ComboBox"` zurück, die wir anschließend in der Spaltenkonfigurationsoption `-editwindow` verwenden, um das Bearbeitungsfenster für die Spalten Nr. 2, ..., 10 festzulegen. In den Spalten Nr. 1 und 10 verwenden wir die Tk-Core-Widgets "checkbutton" und "menubutton", die automatisch für die interaktive Zellbearbeitung registriert werden.

Beachten Sie die Verwendung der Spaltenkonfigurationsoption `-name`, die es uns ermöglicht, auf die Spalten anhand ihrer Namen statt anhand numerischer Spaltenindizes zuzugreifen. Dies ist wichtig, da die Datei `option.tcl`, die in das Hauptskript mittels `source` eingebunden ist, die folgende Zeile enthält:

```
option add *Tablelist.movableColumns    yes
```

Die unten aufgeführten Prozeduren `editStartCmd` und `editEndCmd` verwenden den Unterbefehl `columncget`, um den Namen der Spalte anhand des numerischen Spaltenindex abzurufen.

Übrigens sind in der Datei `option.tcl` zwei weitere Einstellungen der Optionsdatenbank enthalten:

```
option add *Tablelist.labelCommand      tablelist::sortByColumn
option add *Tablelist.labelCommand2     tablelist::addToSortColumns
```

Die in diesen Einstellungen festgelegten Befehle `tablelist::sortByColumn` und `tablelist::addToSortColumns` ermöglichen es dem Benutzer, die Elemente mithilfe der linken Maustaste und der Taste `Shift` nach einer oder mehreren Spalten zu sortieren.

Beachten Sie außerdem, dass wir die Option `-allowduplicates` der Spalte, die die Namen der seriellen Leitungen enthält, auf "false" gesetzt haben, um sicherzustellen, dass diese Namen bei der interaktiven Bearbeitung eindeutig bleiben.

Die Datei `serialParams.tcl`, die in das Hauptskript mittels `source` eingebunden ist, enthält den folgenden Code, der sich hauptsächlich auf die Einstellung `-labelwindow checkbutton` für Spalte Nr. 1 sowie die Konfigurationsoption `-aftercopycommand` bezieht:

```
#
# Populate the tablelist widget; set the activation
# date & time to 10 minutes past the current clock value
#
set clock [expr {[clock seconds] + 600}]
for {set row 0; set line 1} {$row < 16} {set row $line; incr line} {
    $tbl insert end [list $line [expr {$row < 8}] "Line $line" 9600 8 None 1 \
                     XON/XOFF $clock $clock [lindex $colorNames $row]]

    set availImg [expr {($row < 8) ? "checkedImg" : "uncheckedImg"}]
    $tbl cellconfigure $row,available -image $availImg
    $tbl cellconfigure $row,color -image img[lindex $colorValues $row]
}

#
# Configure the "-command" option of the checkbutton embedded into the
# header label of the column "available", and make sure that it will be
# reconfigured whenever any column is moved interactively to a new position
#
proc configCkbtn {tbl col} {
    set ckbtn [$tbl labelwindowpath $col]
    $ckbtn configure -command [list onCkbtnToggle $tbl $col $ckbtn]
}
proc onCkbtnToggle {tbl col ckbtn} {
    upvar #0 [$ckbtn cget -variable] var
    $tbl fillcolumn $col -text $var
    $tbl fillcolumn $col -image [expr {$var ? "checkedImg" : "uncheckedImg"}]
}
configCkbtn $tbl available
bind $tbl <<TablelistColumnMoved>> { configCkbtn %W available }
bind $tbl <<ThemeChanged>>         { configCkbtn %W available }

#
# Make sure that the checkbutton will appear in tri-state mode
#
set ckbtn [$tbl labelwindowpath available]
set varName [$ckbtn cget -variable]
if {[winfo class $ckbtn] eq "Checkbutton"} {
    set $varName ""
} else {
    unset $varName
}

#
# Selects/deselects the checkbutton embedded into the header label
# of the specified column or sets it into the tri-state mode.
#
proc updateHdrCkbtn {tbl col} {
    set lst [$tbl getcolumns $col]
    set ckbtn [$tbl labelwindowpath $col]
    upvar #0 [$ckbtn cget -variable] var

    if {[lsearch -exact $lst 1] < 0} {                  ;# all 0
```

```
        set var 0                                           ;# deselect
    } elseif {[lsearch -exact $lst 0] < 0} {                ;# all 1
        set var 1                                           ;# select
    } elseif {[winfo class $ckbtn] eq "Checkbutton"} {
        set var ""                                          ;# tri-state mode
    } else {
        unset -nocomplain var                               ;# tri-state mode
    }
}

#
# For the columns "available" and "color", updates
# the images contained in the column's cells.
#
proc afterCopyCmd {tbl col} {
    switch [$tbl columncget $col -name] {
        available {
            #
            # Update the images contained in the column's cells and
            # the checkbutton embedded into the column's header label
            #
            for {set row 0} {$row < 16} {incr row} {
                set text [$tbl cellcget $row,$col -text]
                set img [expr {$text ? "checkedImg" : "uncheckedImg"}]
                $tbl cellconfigure $row,$col -image $img
            }
            updateHdrCkbtn $tbl $col
        }

        color {
            #
            # Update the images contained in the column's cells
            #
            for {set row 0} {$row < 16} {incr row} {
                set text [$tbl cellcget $row,$col -text]
                $tbl cellconfigure $row,$col -image img$::colors($text)
            }
        }
    }
}
```

Zu der oben genannten Prozedur `configCkbtn` siehe HINWEIS 2 in der Beschreibung der Spaltenkonfigurationsoption `-labelwindow`.

Die Prozedur `editStartCmd`, die als Wert der Konfigurationsoption `-editstartcommand` festgelegt wird, benötigt den Pfadnamen des Bearbeitungsfensters, um das Widget auf verschiedene Weise konfigurieren zu können. Dies ist ein häufiger Fall, und Tablelist stellt zu diesem Zweck den Unterbefehl `editwinpath` bereit:

```
#------------------------------------------------------------------------------
# editStartCmd
#
# Applies some configuration options to the edit window; if the latter is a
```

```
# ComboBox, the procedure populates it.
#------------------------------------------------------------------------
proc editStartCmd {tbl row col text} {
    set w [$tbl editwinpath]

    switch [$tbl columncget $col -name] {
        lineName {
            #
            # Set an upper limit of 20 for the number of characters
            #
            $w configure -invalidcommand bell -validate key \
                         -validatecommand {expr {[string length %P] <= 20}}
        }

        baudRate {
            #
            # Populate the ComboBox and allow no more
            # than 6 digits in its Entry component
            #
            $w configure -values {50 75 110 300 1200 2400 4800 9600 19200 38400
                                  57600 115200 230400 460800 921600}
            $w configure -invalidcommand bell -validate key -validatecommand \
                {expr {[string length %P] <= 6 && [regexp {^[0-9]*$} %S]}}
        }

        dataBits {
            #
            # Configure the SpinBox
            #
            $w configure -range {5 8 1} -editable no
        }

        parity {
            #
            # Populate the ComboBox and make it non-editable
            #
            $w configure -values {None Even Odd Mark Space} -editable no
        }

        . . .

        color {
            #
            # Populate the menu and make sure the menubutton will display the
            # color name rather than $text, which is "", due to -formatcommand
            #
            set menu [$w cget -menu]
            foreach name $::colorNames {
                $menu add radiobutton -compound left \
                    -image img$::colors($name) -label $name
            }
            $menu entryconfigure 8 -columnbreak 1
            return [$tbl cellcget $row,$col -text]
```

```
        }
    }

    return $text
}
```

Die Prozedur `editEndCmd`, die als Wert der Konfigurationsoption `-editendcommand` festgelegt wird, ist für eine abschließende Validierung des Textes im Bearbeitungsfenster zuständig. Ein weiterer Zweck dieses Befehls besteht darin, den im Bearbeitungsfenster enthaltenen Text in den neuen *internen* Inhalt der Zelle umzuwandeln, was notwendig ist, da der interne Wert des Aktivierungsdatums und der Aktivierungszeit ein Zeitwert in Sekunden ist:

```
#------------------------------------------------------------------------------
# editEndCmd
#
# Performs a final validation of the text contained in the edit window and gets
# the cell's internal content.
#------------------------------------------------------------------------------
proc editEndCmd {tbl row col text} {
    switch [$tbl columncget $col -name] {
        available {
            #
            # Update the image contained in the cell and the checkbutton
            # embedded into the header label of the column "available"
            #
            set img [expr {$text ? "checkedImg" : "uncheckedImg"}]
            $tbl cellconfigure $row,$col -image $img
            after idle [list updateHdrCkbtn $tbl $col]
        }

        baudRate {
            #
            # Check whether the baud rate is an integer in the range 50..921600
            #
            if {![regexp {^[0-9]+$} $text] || $text < 50 || $text > 921600} {
                bell
                tk_messageBox -title "Error" -icon error -message \
                    "The baud rate must be an integer in the range 50..921600"
                $tbl rejectinput
            }
        }

        actDate {
            #
            # Get the activation date in seconds from the last argument
            #
            if {[catch {clock scan $text} actDate] != 0} {
                bell
                tk_messageBox -title "Error" -icon error -message "Invalid date"
                $tbl rejectinput
                return ""
            }
```

```
            #
            # Check whether the activation clock value is later than the
            # current one; if this is the case then make sure the cells
            # "actDate" and "actTime" will have the same internal value
            #
            set actTime [$tbl cellcget $row,actTime -text]
            set actClock [clock scan [formatTime $actTime] -base $actDate]
            if {$actClock <= [clock seconds]} {
                bell
                tk_messageBox -title "Error" -icon error -message \
                    "The activation date & time must be in the future"
                $tbl rejectinput
            } else {
                $tbl cellconfigure $row,actTime -text $actClock
                return $actClock
            }
        }

        . . .

        color {
            #
            # Update the image contained in the cell
            #
            $tbl cellconfigure $row,$col -image img$::colors($text)
        }
    }

    return $text
}
```

Anstatt die Spalte "Verfügbar" über einen *temporären* Checkbutton bearbeitbar zu machen und die Bilder `"checkedImg"` und `"uncheckedImg"` in ihren Zellen anzuzeigen, können wir den Unterbefehl `embedcheckbuttons` verwenden, um die Spalte mit *dauerhaft* eingebetteten checkbuttons zu füllen. Die erforderlichen Änderungen lauten wie folgt:

- Entfernen Sie die Teile des obigen Codes, die in der Farbe "red" dargestellt sind.
- Implementieren Sie die Prozedur `ckbtnCmd` wie folgt:

```
proc ckbtnCmd {tbl row col} { return [updateHdrCkbtn $tbl $col] }
```

- Nachdem Sie das "tablelist"-Widget gefüllt haben, rufen Sie Folgendes auf:

```
$tbl embedcheckbuttons 1 ckbtnCmd
```

- Fügen Sie anstelle der beiden gelöschten Zeilen der Prozedur `onCkbtnToggle` die folgende Schleife ein:

```
    for {set row 0} {$row < 16} {incr row} {
        set ckbtn2 [$tbl windowpath $row,available].ckbtn
        set [$ckbtn2 cget -variable] $var
    }
```

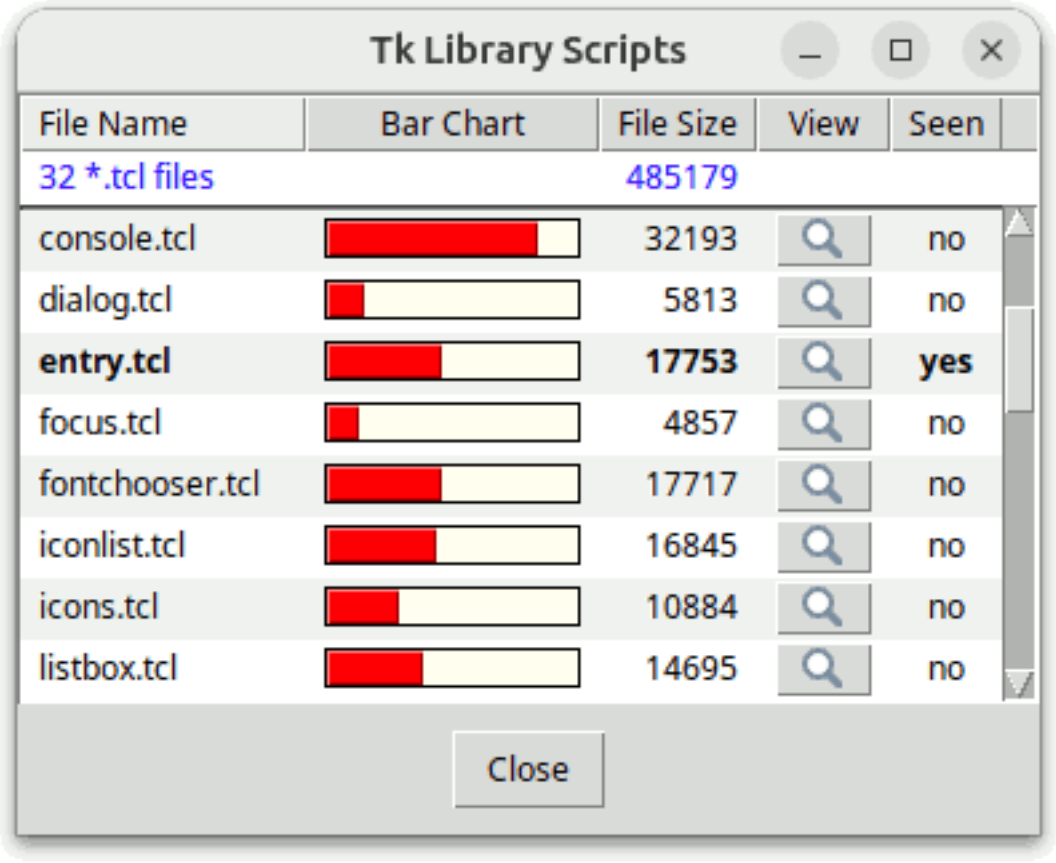
Wie oben erwähnt, ähneln die Skripte `tileWidgets.tcl`, `iwidgets.tcl` und `miscWidgets.tcl` dem Skript `bwidget.tcl`. Das erste Skript nutzt die Widgets "Tile-Entry",

"Spinbox", "Combobox", "Checkbutton" und "Menubutton". Das zweite Skript verwendet (neben dem checkbutton und dem Menubutton aus dem Tk-Kern) die Widgets "entryfield", "spinint", "combobox", "dateentry" und "timeentry" aus dem Iwidgets-Paket sowie die für diese Bibliothek spezifischen Validierungsfunktionen. Das dritte Skript nutzt die Widgets "entry", "spinbox", "checkbutton" und "menubutton" aus dem Tk-Kern, Bryan Oakleys "combobox" sowie die "mentry"-Widgets vom Typ `"Date"` und `"Time"`, und führt die Validierung der Eingaben mit Hilfe des Wcb-Pakets durch (das ohnehin für die Mentry-Bibliothek erforderlich ist).

Ein Tablelist-Widget mit skalierbaren eingebetteten Fenstern

Das Skript `embeddedWindows.tcl` im Verzeichnis `demos` erstellt ein Tablelist-Widget, dessen Elemente den Skripten der Tk-Bibliothek entsprechen. Die Größe jeder Datei (in Byte) wird nicht nur als Zahl angezeigt, sondern auch mithilfe eines Rahmens mit rotem Hintergrund veranschaulicht, der als untergeordnetes Element eines eingebetteten Rahmens mit elfenbeinfarbenem Hintergrund erstellt wird. Die Dateien können durch Klicken auf die entsprechenden eingebetteten Button-Widgets angezeigt werden.

Der folgende Screenshot zeigt das Tablelist-Widget, wobei sich der Mauszeiger über der ersten Kopfzeile befindet, wodurch diese Kopfzeile im Zustand `active` angezeigt wird:



Eingebettete Fenster

Zunächst erstellen wir das "tablelist"-Widget und füllen es mit Daten:

```
package require tablelist

wm title . "Tk Library Scripts"
```

```
#
# Add some entries to the Tk option database
#
set dir [file dirname [info script]]
source [file join $dir option.tcl]

#
# Create the font TkFixedFont if not yet present
#
catch {font create TkFixedFont -family Courier -size 9}

#
# Create an image corresponding to the display's DPI scaling
# level, to be displayed in buttons embedded in a tablelist widget
#
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set fmt $tablelist::svgfmt
    image create photo viewImg -file [file join $dir view.svg] -format $fmt
} else {
    set pct $tablelist::scalingpct
    image create photo viewImg -file [file join $dir view$pct.gif] -format gif
}

#
# Create a vertically scrolled tablelist widget with 5
# dynamic-width columns and interactive sort capability
#
set tf .tf
frame $tf -class ScrollArea
set tbl $tf.tbl
set vsb $tf.vsb
tablelist::tablelist $tbl \
    -columns {0 "File Name" left
              0 "Bar Chart" center
              0 "File Size" right
              0 "View"      center
              0 "Seen"      center} \
    -yscrollcommand [list $vsb set] -width 0
if {[$tbl cget -selectborderwidth] == 0} {
    $tbl configure -spacing 1
}
$tbl columnconfigure 0 -name fileName
$tbl columnconfigure 1 -formatcommand emptyStr -sortmode integer \
    -stretchwindow yes
$tbl columnconfigure 2 -name fileSize -sortmode integer
$tbl columnconfigure 4 -name seen
scrollbar $vsb -orient vertical -command [list $tbl yview]

proc emptyStr val { return "" }

#
# Create a bold font
#
```

```
set tblFont [$tbl cget -font]
set size [font actual $tblFont -size]
if {$size == 0} {                                       ;# e.g., on Ubuntu
    set size 9
}
eval font create BoldFont [font actual $tblFont] -size $size -weight bold

#
# Populate the tablelist widget
#
cd $tk_library
set totalSize 0
set maxSize 0
foreach fileName [lsort [glob *.tcl]] {
    set fileSize [file size $fileName]
    $tbl insert end [list $fileName $fileSize $fileSize "" no]

    incr totalSize $fileSize
    if {$fileSize > $maxSize} {
        set maxSize $fileSize
    }
}
if {$tk_version >= 8.5} {
    $tbl header insert 0 [list "[$tbl size] *.tcl files" "" $totalSize "" ""]
    $tbl header rowconfigure 0 -foreground blue
}
```

Das Bild, das die Aktion "Anzeigen" darstellt, wird entweder aus der Datei `view.svg` oder aus einer GIF-Bilddatei erstellt, deren Name den DPI-Skalierungsprozentsatz des Bildschirms enthält, der durch die öffentliche Variable `tablelist::scalingpct` vorgegeben ist.

Wir fügen die Größe jeder Datei nicht nur in die Spalte mit dem Titel `"File Size"`, sondern auch in die Spalte `"Bar Chart"` ein. Da wir diese Spalte mit `-formatcommand emptyStr` konfiguriert haben, bleibt der Text darin verborgen. Er wird jedoch benötigt, wenn die Einträge nach dieser Spalte sortiert werden.

Nachdem wir den Datenbereich der Tablelist gefüllt haben, erstellen wir ein Kopfzeilenelement, das die Gesamtanzahl und die Größe der Dateien der Bibliothek anzeigt, indem wir den Unterbefehl `header insert` aufrufen, und ändern dessen Vordergrundfarbe mithilfe des Unterbefehls `header rowconfigure`.

Um die eingebetteten Fenster erstellen zu können, müssen wir zunächst die entsprechenden Erstellungsskripte implementieren, wie in der Beschreibung der Zellenkonfigurationsoption `-window` angegeben. Hier ist das Skript, das einen Rahmen erstellt, der in die Spalte mit dem Balkendiagramm eingebettet werden soll:

```
#------------------------------------------------------------------------------
# createFrame
#
# Creates a frame widget w to be embedded into the specified cell of the
# tablelist widget tbl, as well as a child frame representing the size of the
# file whose name is diplayed in the first column of the cell's row.
#------------------------------------------------------------------------------
```

```
proc createFrame {tbl row col w} {
    #
    # Create the frame and replace the binding tag "Frame"
    # with "TablelistBody" in the list of its binding tags
    #
    set height [expr {[font metrics $::tblFont -linespace] * 9 / 10}]
    frame $w -width 72p -height $height -background ivory -borderwidth 1 \
             -relief solid
    bindtags $w [lreplace [bindtags $w] 1 1 TablelistBody]

    #
    # Create the child frame and replace the binding tag "Frame"
    # with "TablelistBody" in the list of its binding tags
    #
    frame $w.f -background red -borderwidth 1 -relief raised
    bindtags $w.f [lreplace [bindtags $w] 1 1 TablelistBody]

    #
    # Manage the child frame
    #
    set fileSize [$tbl cellcget $row,fileSize -text]
    place $w.f -relheight 1.0 -relwidth [expr {double($fileSize) / $::maxSize}]
}
```

Beachten Sie, dass wir die Werte der Optionen `-width` und `-height` für den Rahmen skalierungsbewusst festlegen.

Da der Frame in den Datenbereich der Tablelist eingebettet wird, möchten wir, dass Mausereignisse im Frame und in dessen untergeordneten Frames genauso behandelt werden wie im restlichen Datenbereich der Tablelist. Zu diesem Zweck ersetzen wir das Bindungs-Tag `Frame` (das ohnehin keine eigenen Bindungen enthält) durch `TablelistBody` und stellen so sicher, dass die diesem Tag zugeordneten Standard-Bindungsskripte auch für den übergeordneten Frame und dessen untergeordneten Frame gültig sind.

Den roten untergeordneten Rahmen platzieren wir mit `place` in seinem Elternrahmen, und zwar mit den Optionen `-relheight` und `-relwidth`, damit er die Fläche innerhalb des Rahmens seines Elternteils vertikal ausfüllt und seine Breite beim Ändern der Größe des Elternrahmens proportional zur Größe der zugehörigen Datei bleibt (das geschieht beim Ändern der Spaltenbreite, wie unten zu sehen).

Das Skript zur Erstellung der Schaltflächen, mit denen die Dateien der Tk-Bibliothek angezeigt werden können, ist recht einfach:

```
#------------------------------------------------------------------------------
# createButton
#
# Creates a button widget w to be embedded into the specified cell of the
# tablelist widget tbl.
#------------------------------------------------------------------------------
proc createButton {tbl row col w} {
    set key [$tbl getkeys $row]
    button $w -image viewImg -highlightthickness 0 -takefocus 0 \
              -command [list viewFile $tbl $key]
```

```
}

#------------------------------------------------------------------------------
# viewFile
#
# Displays the content of the file whose name is contained in the row with the
# given key of the tablelist widget tbl.
#------------------------------------------------------------------------------
proc viewFile {tbl key} {
    set top .top$key
    if {[winfo exists $top]} {
        raise $top
        focus $top
        return ""
    }

    toplevel $top
    set fileName [$tbl cellcget k$key,fileName -text]
    wm title $top "File \"$fileName\""

    #
    # Create a vertically scrolled text widget as a grandchild of the toplevel
    #
    set tf $top.tf
    frame $tf -class ScrollArea
    set txt $tf.txt
    set vsb $tf.vsb
    text $txt -background white -font TkFixedFont \
              -yscrollcommand [list $vsb set]
    catch {$txt configure -tabstyle wordprocessor}
    scrollbar $vsb -orient vertical -command [list $txt yview]

    #
    # Insert the file's content into the text widget
    #
    set chan [open $fileName]
    $txt insert end [read -nonewline $chan]
    close $chan

    . . .

    #
    # Mark the file as seen
    #
    $tbl rowconfigure k$key -font BoldFont
    $tbl cellconfigure k$key,seen -text yes
}
```

Jede Datei wird in einem Text-Widget angezeigt, das sich in einem Toplevel-Fenster mit dem Namen `.top$key` befindet, wobei `$key` mithilfe des Unterbefehls `getkeys` ermittelt wird. Durch die Verwendung des Schlüssels anstelle der Zeilennummer erhalten wir einen eindeutigen Namen für das Toplevel-Fenster, selbst wenn sich die Reihenfolge der Elemente aufgrund einer interaktiven Sortierung nach einer Spalte ändert. (Beachten Sie, dass die eingebetteten Fenster

beim Sortieren der Elemente oder beim Verschieben der Spalten gelöscht und automatisch neu erstellt werden.)

Nachdem wir die Skripte zum Erstellen der Rahmen und Schaltflächen implementiert haben, können wir nun den Unterbefehl `cellconfigure` verwenden, um diese Widgets effektiv als eingebettete Fenster zu erstellen:

```
#
# Create embedded windows in the columns no. 1 and 3
#
set rowCount [$tbl size]
for {set row 0} {$row < $rowCount} {incr row} {
    $tbl cellconfigure $row,1 -window createFrame
    $tbl cellconfigure $row,3 -window createButton
}
```

Tatsächlich erzielt das Skript dasselbe Ergebnis mit weniger Code, indem es den etwas effizienteren Unterbefehl `fillcolumn` verwendet:

```
#
# Create embedded windows in the columns no. 1 and 3
#
$tbl fillcolumn 1 -window createFrame
$tbl fillcolumn 3 -window createButton
```

Beachten Sie, dass aufgrund der Einstellung `-stretchwindow yes` für Spalte Nr. 1 die Breite der eingebetteten Rahmen an die Breite der Spalte angepasst wird, wenn deren Größe interaktiv geändert wird.

Tile-basierte Demo-Skripte

Die Tablelist-Distribution enthält auch tilebasierte Entsprechungen der oben besprochenen Demo-Skripte. Wie im Abschnitt Mehr zu Tablelist_tile dieses Tutorials beschrieben, ist es recht einfach, eine Anwendung, die das Tablelist-Paket verwendet, auf eine auf Tablelist_tile basierende Anwendung zu portieren. Schauen wir uns zum Beispiel an, wie Sie das Demo-Skript `bwidget.tcl` in ein tilebasiertes Skript namens `bwidget_tile.tcl` umwandeln. Die Änderungen sind unten in der Farbe red dargestellt:

Zunächst ersetzen wir die erste Zeile

```
package require tablelist
```

mit

```
package require tablelist_tile
```

und den Befehl

```
source [file join $dir option.tcl]
```

mit

```
source [file join $dir option_tile.tcl]
```

Um sicherzustellen, dass das Gesamterscheinungsbild der Benutzeroberfläche dem aktuell verwendeten Theme entspricht, erstellen wir einen themespezifischen Container für unsere Widgets:

```
#
# Improve the window's appearance by using a tile
# frame as a container for the other widgets
#
set f [ttk::frame .f]
```

Das bedeutet, dass wir die Anweisung

```
set tbl .tbl
```

die den Pfadnamen unseres Tablelist-Widgets festlegt, durch Folgendes ersetzen müssen:

```
set tbl $f.tbl
```

Ebenso verwenden wir anstelle einer Tk-Schaltfläche, die mit dem folgenden Befehl erstellt wird,

```
set btn [button .btn -text "Close" -command exit]
```

eine Tile-Schaltfläche, die ein untergeordnetes Element des oben genannten Tile-Rahmens ist:

```
set btn [ttk::button $f.btn -text "Close" -command exit]
```

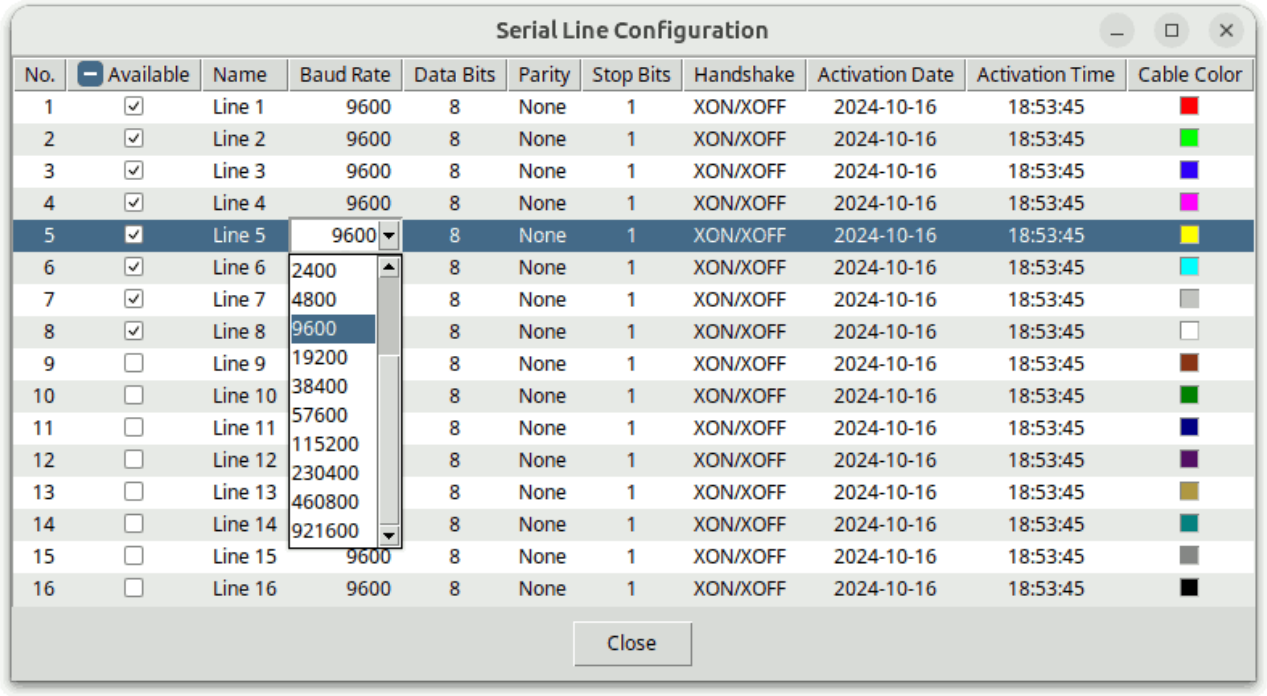
Wir verwalten diesen Frame wie gewohnt:

```
pack $f -expand yes -fill both
```

Das Skript `option_tile.tcl` ist nahezu identisch mit `option.tcl`. Sein tilespezifischer Teil verwendet die Werte, die der Befehl `tablelist::setThemeDefaults` in das Array `tablelist::themeDefaults` schreibt, um sicherzustellen, dass die Elemente des Datenbereichs mit Streifen angezeigt werden, und in X11 erhält die Auswahl in allen von der Anwendung erstellten Widgets das gleiche themespezifische Erscheinungsbild:

```
set currentTheme [tablelist::getCurrentTheme]
if {$tablelist::themeDefaults(-stripebackground) eq "" &&
    $currentTheme ne "black"} {
    option add *Tablelist.background            white
    option add *Tablelist.stripeBackground      #f0f0f0
}
if {[tk windowingsystem] eq "x11"} {
    option add *Font              TkDefaultFont
    option add *selectBackground  $tablelist::themeDefaults(-selectbackground)
    option add *selectForeground  $tablelist::themeDefaults(-selectforeground)
}
option add *selectBorderWidth     $tablelist::themeDefaults(-selectborderwidth)
```

Das Demo-Skript `tileWidgets.tcl` nutzt nicht nur das Paket "Tablelist_tile" zur Erstellung eines Tablelist-Widgets mit einem modernen, themespezifischen Erscheinungsbild, sondern auch die Widgets "Tile Entry", "Spinbox", "Combobox", "Checkbutton" und "Menubutoon" für die interaktive Bearbeitung von Zellen. Das resultierende Fenster hat ein ansprechendes, themespezifisches Erscheinungsbild:



Konfiguration der seriellen Schnittstelle

Das Skript `tileWidgets.tcl` beginnt damit, das Theme `default` unter X11 bei Bedarf anzupassen, indem es die öffentliche Prozedur `themepatch::patch` des Pakets "themepatch" aufruft, das im Lieferumfang von Tablelist enthalten ist (im Gegensatz zum Paket "scaleutil" wird es jedoch nicht vom Tablelist-Code verwendet und muss daher explizit über `package require themepatch` geladen werden):

```
if {[tk windowingsystem] eq "x11" &&
    ($::tk_version < 8.7 || [package vcompare $::tk_patchLevel "8.7a5"] <= 0)} {
    #
    # Patch the default theme's styles TCheckbutton and TRadiobutton
    #
    package require themepatch
    themepatch::patch default
}
```

Aufgrund dieses (optionalen) Schritts haben die skalierungsfähigen ttk::checkbutton-Widgets, die in der Spalte "Verfügbar" verwendet werden, ein modernes Erscheinungsbild und Verhalten, genau wie die der Themes `aqua` und `vista`:



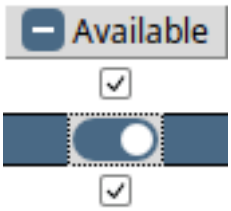
ttk::checkbutton

Das Demo-Skript `tileWidgets2.tcl` unterscheidet sich von `tileWidgets.tcl` vor allem dadurch, dass es das Widget ttk::toggleswitch oder tsw::toggleswitch anstelle von ttk::checkbutton verwendet, um die Elemente der Spalte "Verfügbar" zu bearbeiten. (Das Widget ttk::toggleswitch wurde in Tk 9.1a1 zum Kern hinzugefügt.) Hier ist der entsprechende Code:

```
if {[llength [info commands ::ttk::toggleswitch]] == 1} {
    set tglswEditWin ttk::toggleswitch
} else {
    #
    # Register the tsw::toggleswitch widget for interactive cell editing
    #
    package require tsw
    tablelist::addToggleswitch
    set tglswEditWin toggleswitch
}

. . .

$tbl columnconfigure 1 -name available -editable yes -editwindow $tglswEditWin \
    -formatcommand emptyStr -labelwindow ttk::checkbutton
```



toggleswitch

Außerdem weist in `tileWidgets2.tcl` das Widget ttk::spinbox, das zur Bearbeitung der Elemente der Spalte "Data Bits" verwendet wird, den Stil `Wide.TSpinbox` statt `TSpinbox` auf:

```
catch {package require wsb}

. . .

proc editStartCmd {tbl row col text} {
    set w [$tbl editwinpath]

    switch [$tbl columncget $col -name] {

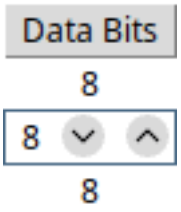
        . . .

        dataBits {
            #
            # Configure the spinbox
            #
            $w configure -style Wide.TSpinbox -from 5 -to 8 -state readonly
        }
```

Das kleine **w**ide **s**pin**b**ox-Paket "wsb" ist im Lieferumfang der Tablelist-Distribution enthalten, und der Versuch, es über `package require wsb` zu laden, ist erfolgreich, wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) oder 8.6 ist und die Erweiterung "tksvg" in den Interpreter geladen werden kann. Wenn es geladen ist, erstellt das wsb-Paket automatisch das Layout `Wide.TSpinbox`. Andernfalls hat die Einstellung `-style Wide.TSpinbox` für ein ttk::spinbox-Widget keine Wirkung.

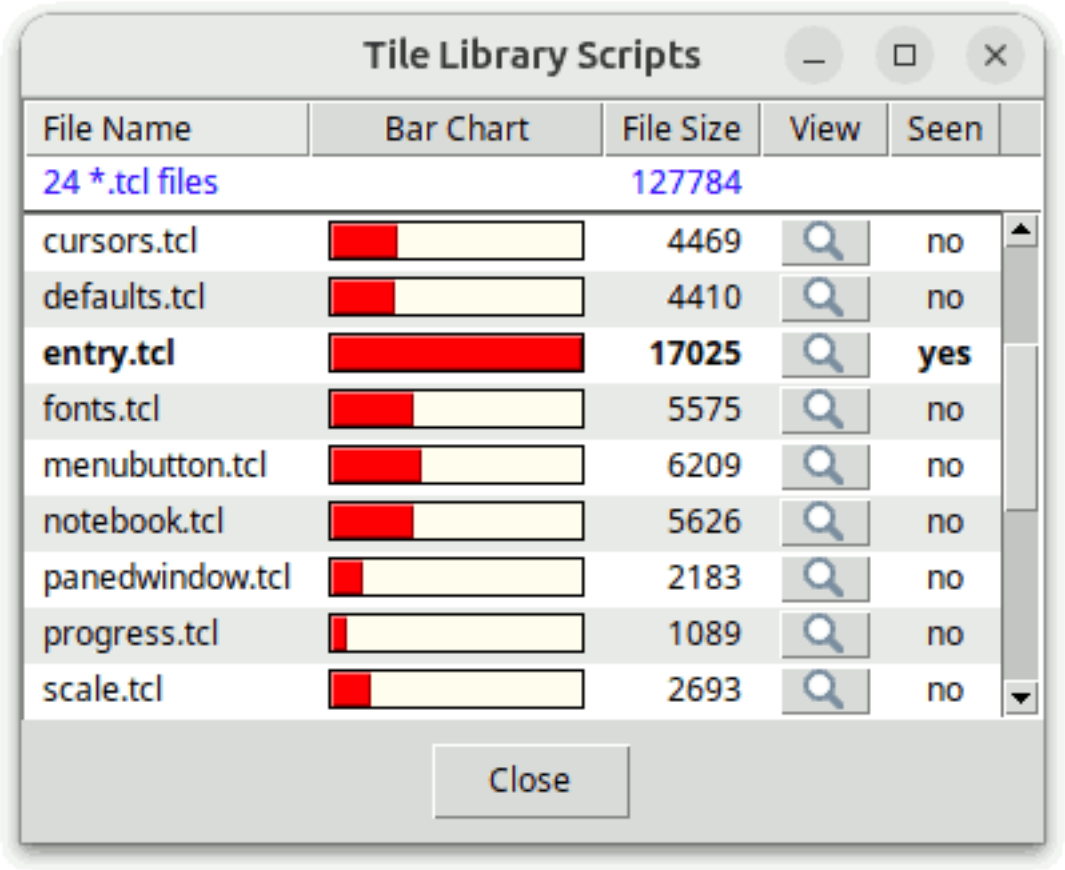
Der Hauptvorteil des Layouts `Wide.TSpinbox` gegenüber dem Stil `TSpinbox` liegt in der

benutzerfreundlichen Größe und Anordnung der skalierungsabhängigen Auf- und Ab-Pfeile, was sich insbesondere bei der Nutzung von AndroWish auf einem Tablet oder Smartphone als nützlich erweist:



Wide.TSpinbox

Die tilebasierte Version des Demo-Skripts `embeddedWindows.tcl` enthält etwas mehr Änderungen, von denen die meisten jedoch nicht spezifisch für Tablelist sind. Einzelheiten finden Sie in der Datei `embeddedWindows_tile.tcl` im Verzeichnis `demos`. Hier ist ein Screenshot des resultierenden Fensters:



Eingebettete Fenster

Beachten Sie abschließend, dass die folgenden tilebasierten Demo-Skripte *vollständige* Unterstützung für den Dark-Mode-Modus unter macOS 10.14 (Mojave) und höher mit Tk 8.6.10 oder höher sowie für die Themes `awdark`, `awlight` sowie die sechs weiteren Themes, die im plattformübergreifenden Paket awthemes 10.3 oder höher von Brad Lanam enthalten sind, bieten: `browse_tile.tcl`, `browseTree_tile.tcl`, `config_tile.tcl`, `dirViewer_tile.tcl`, `embeddedWindows_tile.tcl`, `tileWidgets.tcl` und `tileWidgets2.tcl`. Zwar ist die Unterstützung für den Aqua-Dark-Mode bereits seit Version 8.6.10 in Tk vorhanden, doch für ein optimales Benutzererlebnis auf dem Mac wird empfohlen,

diese Skripte mit Tk 8.6.11 oder höher auszuführen, da in Tk 8.6.11 einige Verbesserungen hinsichtlich der Darstellung vorgenommen wurden.

tablelistBWidget

Stichwörter: tablelistBWidget

Interaktive Bearbeitung von Tabellenzellen mit dem BWidget-Paket

Für Tablelist Version 7.12

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::addBWidgetEntry
- Der Befehl tablelist::addBWidgetSpinBox
- Der Befehl tablelist::addBWidgetComboBox

Überblick

Das BWidget-Paket ist eine Bibliothekserweiterung für Tcl/Tk ab Version 8.1.1, die in reinem Tcl/Tk-Code geschrieben ist. Es kann hier heruntergeladen werden:

https://sourceforge.net/projects/tcllib/files

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe der Widgets "Entry", "SpinBox" und "ComboBox" aus dem BWidget-Paket. Die folgenden Schritte sind erforderlich, um eines dieser Widgets zur Bearbeitung der Zellen einer bestimmten Spalte zu verwenden:

1. Registrieren Sie das gewünschte Widget für die interaktive Zellenbearbeitung, indem Sie einen der auf dieser Referenzseite beschriebenen Befehle aufrufen.
2. Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf den vom oben genannten Befehl zurückgegebenen Wert zu setzen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

HINWEIS: Das temporär eingebettete BWidget-Widget, das für die interaktive Zellbearbeitung verwendet wird, wird entsprechend der DPI-Skalierung des Bildschirms und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk korrekt skaliert angezeigt.

Der Befehl tablelist::addBWidgetEntry

NAME

`tablelist::addBWidgetEntry` – Das "Entry"-Widget aus dem BWidget-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addBWidgetEntry ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das "Entry"-Widget aus dem BWidget-Paket für die interaktive Zellenbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das "Entry"-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `Entry`. Der Befehl gibt sein Argument `name` zurück.

SCHLÜSSELWÖRTER

tablelist, editing, BWidget, Entry

Der Befehl tablelist::addBWidgetSpinBox

NAME

`tablelist::addBWidgetSpinBox` – Das SpinBox-Widget aus dem BWidget-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addBWidgetSpinBox ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das SpinBox-Widget aus dem BWidget-Paket für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das SpinBox-Widget als Wert der Spalten- oder Zellkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `SpinBox`. Der Befehl gibt sein Argument `name` zurück.

Das temporäre, eingebettete SpinBox-Widget, das mit dem oben genannten `name` verknüpft ist, wird erstellt, wobei die Option `-editable` auf `1` gesetzt ist. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um die SpinBox nicht editierbar zu machen oder Validierungen dafür zu definieren sowie deren Werte (bzw. Wertebereich) und die Option `-wrap` festzulegen.

SCHLÜSSELWÖRTER

tablelist, editing, BWidget, SpinBox

Der Befehl tablelist::addBWidgetComboBox

NAME

`tablelist::addBWidgetComboBox` – Das ComboBox-Widget aus dem BWidget-Paket für die interaktive Zellbearbeitung registrieren

ÜBERSICHT

```
tablelist::addBWidgetComboBox ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das ComboBox-Widget aus dem BWidget-Paket für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das ComboBox-Widget als Wert der Spalten- oder Zellkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `ComboBox`. Der Befehl gibt sein Argument `name` zurück.

Das temporäre, eingebettete ComboBox-Widget, das mit dem oben genannten `name` verknüpft ist, wird erstellt, wobei die Option `-editable` auf `1` gesetzt ist. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` für die Tablelist entspricht, um die ComboBox nicht bearbeitbar zu machen oder Validierungen dafür zu definieren, sowie zum Befüllen ihrer listbox-Komponente (mithilfe der Option `-values` des ComboBox-Widgets).

SCHLÜSSELWÖRTER

tablelist, editing, BWidget, ComboBox

tablelistBinding

Stichwörter: tablelistBinding

Befehle im Zusammenhang mit Bindungsskripten

Für Tablelist Version 7.12

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::convEventFields
- Der Befehl tablelist::getTablelistPath
- Der Befehl tablelist::getTablelistColumn
- Der Befehl tablelist::delaySashPosUpdates

Überblick

Die ersten drei in dieser Referenzseite beschriebenen Befehle sind für die Verwendung in Bindungs-Skripten vorgesehen, die mit den Bindungs-Tags verknüpft sind, deren Namen von den Unterbefehlen `bodytag` und `labeltag` des Tcl-Befehls zurückgegeben werden, der einem Tablelist-Widget zugeordnet ist. Die ersten beiden dieser Befehle werden auch in den Skripten der Vorgabebindungen verwendet, die mit dem Bindungs-Tag `TablelistBody` verknüpft sind. Einzelheiten und Beispiele finden Sie in den Abschnitten VORGABE- UND EINZELBINDUNGEN FÜR DEN DATENBEREICH und VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFBESCHRIFTUNGEN.

Der letzte Befehl bezieht sich auf die Vorgabebindungsskripte für die Tk-Core-Widgets

"panedwindow" und "ttk::panedwindow". Er kann verwendet werden, um die Häufigkeit der Aktualisierungen der Sash-Position zu steuern, was erforderlich sein kann, um bestimmte Artefakte zu beseitigen, die beim Verschieben eines Sash entstehen, wenn ein oder mehrere der Fensterbereiche ein "Tablelist"-Widget mit eingebetteten Bildern enthalten.

Der Befehl tablelist::convEventFields

NAME

`tablelist::convEventFields` – Ereignisfelder relativ zu einem Nachkommen eines "Tablelist"-Widgets konvertieren

ÜBERSICHT

```
tablelist::convEventFields descendantPathName x y
```

BESCHREIBUNG

Dieser Befehl ermittelt den Pfadnamen eines Tablelist-Widgets sowie die x- und y-Koordinaten relativ zu diesem Widget anhand des Pfadnamens `descendantPathName` eines seiner Nachkommen und anhand der x- und y-Koordinaten `x`, `y`, die relativ zu diesem Nachkommen sind. Der Befehl gibt diese Werte als Komponenten einer Liste zurück, die aus drei Elementen besteht.

In einem Bindungsskript entspricht das untergeordnete Widget dem Ereignisfeld `%W`. Dabei kann es sich um den Datenbereich der Tablelist, einen der Trennrahmen, ein Label-Widget, das ein eingebettetes Bild anzeigt, oder um (einen Nachkommen) eines eingebetteten Fensters handeln. Es kann sich auch um ein Kopfzeilen-Label oder einen in ein Kopfzeilen-Label eingebetteten Tk-Core- oder Tile-Checkbutton oder um einen Sortierpfeil handeln. Ebenso entsprechen die Argumente `x` und `y` den Ereignisfeldern `%x` und `%y`. Die drei Elemente der von dem Befehl zurückgegebenen Liste werden üblicherweise den Hilfevariablen `tablelist::W`, `tablelist::x` und `tablelist::y` zugewiesen, und zwar mithilfe der Anweisung

```
foreach {tablelist::W tablelist::x tablelist::y} \
    [tablelist::convEventFields %W %x %y] {}
```

SCHLÜSSELWÖRTER

tablelist, event fields, binding script

Der Befehl tablelist::getTablelistPath

NAME

`tablelist::getTablelistPath` – Den Pfadnamen eines "Tablelist"-Widgets anhand des Pfadnamens eines seiner Nachkommen ermitteln

ÜBERSICHT

```
tablelist::getTablelistPath descendantPathName
```

BESCHREIBUNG

Dieser Befehl ruft den Pfadnamen eines Tablelist-Widgets anhand des Pfadnamens `descendantPathName` eines seiner Nachkommen ab.

In einem Bindungsskript entspricht das untergeordnete Widget dem Ereignisfeld `%W`, bei dem es sich um den Datenbereich der Tablelist, einen der Trennrahmen, ein Label-Widget mit eingebettetem Bild oder (einen Nachkommen) eines eingebetteten Fensters handeln kann. Es kann sich auch um eine Kopfzeilenbeschriftung oder einen in eine Kopfzeilenbeschriftung eingebetteten Tk-Core- oder Tile-Checkbutton oder um einen Sortierpfeil handeln. Der Wert wird häufig der Hilfevariablen `tablelist::W` zugewiesen.

SCHLÜSSELWÖRTER

tablelist, path name, binding script

Der Befehl tablelist::getTablelistColumn

NAME

`tablelist::getTablelistColumn` – Die Spaltennummer aus dem Pfadnamen einer Tablelist-Kopfbeschriftung ermitteln

ÜBERSICHT

```
tablelist::getTablelistColumn headerLabelPathName
```

BESCHREIBUNG

Dieser Befehl ermittelt die Spaltennummer anhand des Pfadnamens `headerLabelPathName` einer Tablelist-Kopfbeschriftung, eines in eine Kopfbeschriftung eingebetteten Tk-Core- oder Tile-Checkbuttons oder eines Sortierpfeils. Gibt bei Erfolg die Spaltennummer und bei Fehlschlag den Wert `-1` zurück.

In einem Bindungsskript entspricht das Argument dem Ereignisfeld `%W`. Der Rückgabewert wird häufig der Hilfsvariablen `tablelist::col` zugewiesen.

SCHLÜSSELWÖRTER

tablelist, path name, binding script

Der Befehl tablelist::delaySashPosUpdates

NAME

`tablelist::delaySashPosUpdates` – Verzögerung für die Aktualisierung der Sash-Position von panedwindow-Widgets festlegen

ÜBERSICHT

```
tablelist::delaySashPosUpdates panedwindowPathName ms
```

BESCHREIBUNG

Dieser Befehl legt eine Verzögerung für die Aktualisierungen der Sash-Position fest, die durch das Ziehen eines Sash des durch das erste Argument angegebenen "panedwindow"-Widgets verursacht werden. Das erste Argument muss der Pfadname eines Tk-Core-"panedwindow"-Widgets oder eines ttk::panedwindow-Widgets sein. `ms` muss eine Ganzzahl sein, die die Verzögerung in Millisekunden angibt. Wenn diese Zahl nicht negativ ist, wird die Aktualisierung der Sash-Position über `sash place ...` (für ein panedwindow-Widget des Tk-Kerns) oder `sashpos ...` (bei einem ttk::panedwindow-Widget) `ms` Millisekunden später so eingeplant, dass bis zum Ablauf der

Zeit für eine bereits eingeplante Aktualisierung neue Zieh-Ereignisse keine neuen Aktualisierungsvorgänge einplanen (lediglich deren Daten werden gespeichert). Dadurch wird die Häufigkeit der Aktualisierungen der Sash-Position, die durch das Ziehen eines Sash des angegebenen Widgets verursacht werden, je nach dem Wert von `ms` verringert. Ist diese Zahl negativ, löst jedes Ziehen eines Sash des angegebenen Widgets sofort eine Aktualisierung der Sash-Position aus, wie es in den Standard-Skripten für die Tk-Kern-Widgets panedwindow und ttk::panedwindow implementiert ist.

Mit diesem Befehl lässt sich die Häufigkeit der Aktualisierungen der Sash-Position für ein bestimmtes "panedwindow"-Widget steuern. Dies kann erforderlich sein, um bestimmte Artefakte zu beseitigen, die beim Ziehen eines Sashs entstehen, wenn ein oder mehrere der Fensterbereiche ein "tablelist"-Widget mit eingebetteten Bildern enthalten. Der Wert `ms` (in der Regel eine Zahl zwischen 10 und 100) ist anwendungs- und plattformspezifisch (unter Windows und Mac OS X/11+ ist ein höherer Wert erforderlich als unter X11).

SCHLÜSSELWÖRTER

panedwindow, tablelist, path name, binding script

tablelistColSort

Stichwörter: tablelistColSort

Befehle zum interaktiven Sortieren nach einer oder mehreren Spalten

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::sortByColumn
- Der Befehl tablelist::addToSortColumns

Überblick

Die auf dieser Referenzseite beschriebenen Befehle ermöglichen es dem Benutzer, die Elemente eines Tablelist-Widgets mithilfe der linken Maustaste nach einer oder mehreren Spalten zu sortieren.

Wenn der Befehl `tablelist::sortByColumn` als Wert der Konfigurationsoption `-labelcommand` angegeben wurde, werden die Elemente anhand der Elemente der entsprechenden Spalte sortiert, wenn Sie die linke Maustaste über einer der Kopfzeilenbeschriftungen drücken und sie später über derselben Beschriftung wieder loslassen.

Wenn der Befehl `tablelist::addToSortColumns` als Wert der Konfigurationsoption

`-labelcommand2` angegeben wurde, wird durch Drücken der Maustaste 1 in Verbindung mit der Taste `Shift` über einer der Kopfzeilenbezeichnungen und anschließendes Loslassen über derselben Bezeichnung die entsprechende Spalte an die Liste der Sortierspalten angehängt oder (falls sie bereits in dieser Liste enthalten war) die entsprechende Sortierreihenfolge umgeschaltet; In beiden Fällen werden die Einträge anhand der aktualisierten Listen der Sortierspalten und Sortierreihenfolgen sortiert.

Der Befehl tablelist::sortByColumn

NAME

`tablelist::sortByColumn` – Die Elemente eines Tablelist-Widgets nach einer seiner Spalten sortieren

ÜBERSICHT

```
tablelist::sortByColumn pathName columnIndex
```

BESCHREIBUNG

Dieser Befehl sortiert die Einträge des Widgets tablelist `pathName` anhand der Elemente der durch `columnIndex` angegebenen Spalte. Dies geschieht durch Aufruf des Unterbefehls `sortbycolumn` des Tcl-Befehls, der dem angegebenen Tablelist-Widget zugeordnet ist. Wenn die Elemente zuletzt in aufsteigender Reihenfolge sortiert wurden, basierend auf derselben Spalte, die durch `columnIndex` angegeben wurde, und kein nachfolgender Aufruf des Unterbefehls `resetsortinfo` erfolgte, ist das letzte an `sortbycolumn` übergebene Argument `-decreasing`, andernfalls `-increasing`.

Nach dem Sortieren der Einträge wird das virtuelle Ereignis `<<TablelistColumnSorted>>` generiert. Bei Tk-Versionen 8.5 oder höher wird dieses virtuelle Ereignis so generiert, dass seine Option `-data` auf eine Liste gesetzt ist, die aus dem numerischen Spaltenindex und der Sortierreihenfolge besteht (`decreasing` oder `increasing`). Der Befehl gibt die Sortierreihenfolge zurück, und zwar als `decreasing` oder `increasing`.

Wenn die Option `-showlinenumbers` für die angegebene Spalte auf "true" gesetzt wurde, werden die oben beschriebenen Aktionen nicht ausgeführt und der Rückgabewert ist eine leere Zeichenkette.

`tablelist::sortByColumn` wird in der Regel als Wert für die Konfigurationsoption `-labelcommand` eines Tablelist-Widgets angegeben.

SCHLÜSSELWÖRTER

tablelist, sort, column, widget

Der Befehl tablelist::addToSortColumns

NAME

`tablelist::addToSortColumns` – Einen Spaltenindex zur Liste der Sortierspalten hinzufügen und die mehrspaltige Sortierung durchführen

ÜBERSICHT

```
tablelist::addToSortColumns pathName columnIndex
```

BESCHREIBUNG

Dieser Befehl fügt den durch `columnIndex` festgelegten Spaltenindex zur Liste der Sortierspalten des tablelist-Widgets `pathName` hinzu und sortiert die Elemente anhand der Elemente der Spalten, die durch die geänderte Liste angegeben werden. Dies geschieht durch Aufruf des Unterbefehls `sortbycolumnlist` des Tcl-Befehls, der dem angegebenen Tablelist-Widget zugeordnet ist. Die beiden an `sortbycolumnlist` übergebenen Argumente werden wie folgt aufgebaut: (a) Wenn der angegebene Spaltenindex bereits in der Liste der Sortierspalten des Widgets enthalten ist (der vom Unterbefehl `sortcolumnlist` zurückgegeben wird) dann bleibt diese Liste unverändert, und die Sortierreihenfolge-Liste (die vom Unterbefehl `sortorderlist` zurückgegeben wird) wird aktualisiert, indem das entsprechende Element von `increasing` auf `decreasing` umgeschaltet wird und umgekehrt; (b) andernfalls wird der Spaltenindex an die Liste der Sortierspalten angehängt und der Wert `increasing` an die Liste der Sortierreihenfolgen angehängt.

Nach dem Sortieren der Elemente durch Übergabe der beiden aktualisierten Listen an den Unterbefehl `sortbycolumnlist` wird das virtuelle Ereignis `<<TablelistColumnsSorted>>` (beachten Sie die Pluralform!) generiert. Bei Tk-Versionen 8.5 oder höher wird dieses virtuelle Ereignis so erzeugt, dass seine Option `-data` auf eine Liste gesetzt ist, die aus den beiden oben genannten aktualisierten Listen besteht, die als Argumente an `sortbycolumnlist` übergeben wurden. Der Befehl gibt die neue Sortierreihenfolge für die angegebene Spalte zurück, und zwar als `decreasing` oder `increasing`.

Wenn die Option `-showlinenumbers` für die angegebene Spalte auf "true" gesetzt wurde, werden die oben beschriebenen Aktionen nicht ausgeführt und der Rückgabewert ist eine leere Zeichenkette.

`tablelist::addToSortColumns` wird in der Regel als Wert der Konfigurationsoption `-labelcommand2` für ein Tablelist-Widget angegeben.

SCHLÜSSELWÖRTER

tablelist, sort, column list, widget

tablelistCombobox

Stichwörter: tablelistCombobox

Interaktive Bearbeitung von Tabellenzellen mithilfe des combobox-Pakets

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::addOakleyCombobox

Überblick

Das "Combobox"-Paket von Bryan Oakley ist eine Bibliothekserweiterung für Tcl/Tk ab Version 8.0, die vollständig in Tcl/Tk geschrieben ist. Der Download-Link lautet

http://www1.clearlight.com/~oakley/tcl/combobox

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe des Combobox-Widgets, das in dem oben genannten Paket implementiert ist. Die folgenden Schritte sind erforderlich, um dieses Widget zur Bearbeitung der Zellen einer bestimmten Spalte zu verwenden:

1. Registrieren Sie das Combobox-Widget für die interaktive Zellenbearbeitung, indem Sie den unten beschriebenen Befehl `tablelist::addOakleyCombobox` aufrufen.
2. Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf den vom oben genannten Befehl zurückgegebenen Wert zu setzen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

HINWEIS: Die Pfeil- und Scrollbar-Komponenten des temporär eingebetteten Combobox-Widgets, das für die interaktive Zellbearbeitung verwendet wird, werden entsprechend der DPI-Skalierung des Bildschirms und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk korrekt skaliert angezeigt.

Der Befehl tablelist::addOakleyCombobox

NAME

`tablelist::addOakleyCombobox` – Das Combobox-Widget von Bryan Oakley für die interaktive Zellbearbeitung registrieren

ÜBERSICHT

```
tablelist::addOakleyCombobox ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das Combobox-Widget von Bryan Oakley für die interaktive Zellenbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das Combobox-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `combobox`. Der Befehl gibt sein Argument `name` zurück.

Das temporäre, eingebettete Combobox-Widget, das mit dem oben genannten `name` verknüpft ist, wird erstellt, wobei die Option `-editable` auf `1` gesetzt ist. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um die Combobox nicht editierbar zu machen oder Validierungen für ihr untergeordnetes Eingabefeld zu definieren (dessen Pfadname durch Aufruf des Unterbefehls

`entrypath` des Tablelist-Widgets, durch Anhängen von `.entry` an den Pfadnamen des Combobox-Widgets oder – bei Verwendung von Combobox Version 2.3 oder höher – mithilfe des Unterbefehls `subwidget entry` des Combobox-Widgets ermittelt werden kann) sowie zum Befüllen ihrer listbox-Komponente (mithilfe des Unterbefehls `list insert` des Combobox-Widgets).

SCHLÜSSELWÖRTER

tablelist, editing, combobox

tablelistCtext

Stichwörter: tablelistCtext

Interaktive Bearbeitung von Tabellenzellen mit dem ctext-Paket

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::addCtext

Überblick

Das ctext-Paket von George Peter Staplin ist eine Bibliothekserweiterung für Tcl/Tk ab Version 8.0, die in reinem Tcl/Tk-Code geschrieben ist. Es ist Teil von tklib, dessen Adresse lautet

https://core.tcl.tk/tklib

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe des im oben genannten Paket implementierten ctext-Widgets. Die folgenden Schritte sind erforderlich, um dieses Widget zur Bearbeitung der Zellen einer bestimmten Spalte zu verwenden:

1. Registrieren Sie das ctext-Widget für die interaktive Zellenbearbeitung, indem Sie den unten beschriebenen Befehl `tablelist::addCtext` aufrufen.
2. Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf den vom oben genannten Befehl zurückgegebenen Wert zu setzen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

Der Befehl tablelist::addCtext

NAME

`tablelist::addCtext` – Das ctext-Widget für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addCtext ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das ctext-Widget für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das ctext-Widget als Wert der Spalten- oder Zellkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `ctext`. Der Befehl gibt sein Argument `name` zurück.

Das temporäre, eingebettete ctext-Widget, das für die interaktive Zellenbearbeitung verwendet wird, wird so erstellt, dass seine Optionen `-padx` und `-pady` auf `2` gesetzt sind, die Option `-wrap` auf `none` gesetzt und die anfängliche Höhe auf die Anzahl der darin enthaltenen Zeilen festgelegt. Es gibt jedoch eine Ausnahme von dieser Regel: Wenn die Option `-wrap` der Spalte der Zelle auf "true" gesetzt wurde und Tk Version 8.5 oder höher verwendet wird, dann wird die Option `-wrap` des ctext-Widgets auf `word` gesetzt und seine Anfangshöhe entspricht der Anzahl der darin enthaltenen *angezeigten* Zeilen (unter Berücksichtigung der Zeilenumbrüche). Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um die Anfangseinstellungen (mit Ausnahme der Höhe) nach Ihren Bedürfnissen zu überschreiben.

Wenn die Option `-wrap` des ctext-Widgets auf `word` oder `char` gesetzt wurde (entweder durch "Tablelist" oder innerhalb des oben genannten Skripts) und Tk Version 8.5 oder höher verwendet wird, wird bei jeder Änderung der Breite (z. B. durch interaktive Anpassung der Spalten) die Höhe automatisch auf die Anzahl der darin enthaltenen Anzeigezeilen gesetzt. (Die Anzahl der Anzeigezeilen wird mithilfe des in Tk 8.5 eingeführten Text-Widget-Unterbefehls `count -displaylines` ermittelt.)

Wenn das Widget-Callback-Paket "Wcb" in den Interpreter geladen wurde (über `package require Wcb` oder `package require wcb`), wird die Höhe des ctext-Widgets automatisch aktualisiert, sobald Text eingefügt oder gelöscht wird, was die Bearbeitung wesentlich benutzerfreundlicher macht. Dies wird durch die Verwendung einer entsprechend definierten Rückruffunktion "after-`insert`" und "after-`delete`" für das Bearbeitungsfenster erreicht. Sie können das Skript verwenden, das der Tablelist-Konfigurationsoption `-editstartcommand` entspricht, um weitere Rückruffunktionen für das ctext-Widget zu definieren. (Die oben genannte Rückruffunktion wird über `wcb::cbappend` erstellt, *nach* der Rückkehr aus diesem Skript.)

Die Taste `Tab` ist für die Navigation zwischen den bearbeitbaren Zellen reserviert, der Benutzer kann jedoch durch Drücken von `Control-i` ein Tabulatorzeichen in das ctext-Widget einfügen.

Anders als bei den anderen Widgets, die für die interaktive Zellenbearbeitung verwendet werden (mit Ausnahme des Tk-Core-Text-Widgets), fügen die Tasten `Return` und `KP_Enter` ein Zeilenumbruchzeichen in das ctext-Widget ein. Auch `Control-j` kann zum Einfügen

eines Zeilenumbruchs verwendet werden. `Control-Return` und `Control-KP_Enter` beenden die Bearbeitung und löschen das Bearbeitungsfenster.

`Control-Home` und `Control-End` verfügen über ihre bekannten, für Text-Widgets spezifischen Bindungen, genau wie `Meta-<` und `Meta->`, wenn `tk_strictMotif` auf "false" gesetzt ist. Auch dies unterscheidet sich vom Verhalten der anderen Widgets, die für die interaktive Zellbearbeitung verwendet werden (mit Ausnahme des Tk-Core-Text-Widgets). Um zur ersten bzw. letzten bearbeitbaren Zelle zu springen, kann der Benutzer `Alt-Home`/`Alt-End` oder `Meta-Home`/`Meta-End` (`Command-Home`/`Command-End` unter Mac OS X/11+ Aqua) drücken.

SCHLÜSSELWÖRTER

tablelist, editing, ctext

tablelistIwidgets

Stichwörter: tablelistIwidgets

Interaktive Bearbeitung von Tabellenzellen mit dem Iwidgets-Paket

Für Tablelist Version 7.12

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::addIncrEntryfield
- Der Befehl tablelist::addIncrDateTimeWidget
- Der Befehl tablelist::addIncrSpinner
- Der Befehl tablelist::addIncrSpinint
- Der Befehl tablelist::addIncrCombobox

Überblick

Das Iwidgets-Paket (auch bekannt als [incr Widgets]) ist eine Bibliothekserweiterung für Tcl/Tk ab Version 8.0, die in reinem Tcl/Tk-Code geschrieben ist. Es benötigt die kompilierten Pakete Itcl und Itk (auch bekannt als [incr Tcl] bzw. [incr Tk]). Der Download-Ort dieser drei Bibliotheken ist

https://sourceforge.net/projects/incrtcl/files

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe der Widgets "entryfield", "datefield", "dateentry", "timefield", "timeentry", "spinner", "spinint" und "combobox" aus dem Iwidgets-Paket. Die folgenden Schritte sind erforderlich, um eines dieser Widgets zur Bearbeitung der Zellen einer bestimmten Spalte zu verwenden:

1. Registrieren Sie das gewünschte Widget für die interaktive Zellenbearbeitung, indem Sie

einen der auf dieser Referenzseite beschriebenen Befehle aufrufen.
2. Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf den vom oben genannten Befehl zurückgegebenen Wert zu setzen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

HINWEIS: Das temporär eingebettete "Iwidgets"-Widget, das für die interaktive Bearbeitung von Zellen verwendet wird, wird entsprechend der DPI-Skalierung des Bildschirms und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk korrekt skaliert angezeigt.

Der Befehl tablelist::addIncrEntryfield

NAME

`tablelist::addIncrEntryfield` – Das "entryfield"-Widget aus dem Iwidgets-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIncrEntryfield ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das "entryfield"-Widget aus dem Iwidgets-Paket für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das "entryfield"-Widget als Wert der Spalten- oder Zellkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `entryfield`. Der Befehl gibt sein Argument `name` zurück.

SCHLÜSSELWÖRTER

tablelist, editing, Iwidgets, entryfield

Der Befehl tablelist::addIncrDateTimeWidget

NAME

`tablelist::addIncrDateTimeWidget` – Das Widget "datefield", "dateentry", "timefield" oder "timeentry" aus dem Iwidgets-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIncrDateTimeWidget datefield|dateentry|timefield|timeentry ?-seconds?
    ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das Widget "datefield", "dateentry", "timefield" oder "timeentry" aus dem Iwidgets-Paket für die interaktive Zellbearbeitung in Tablelist-Widgets. Wenn das Argument `-seconds` vorhanden ist, ruft der Unterbefehl `finishediting` des mit dem Tablelist-Widget verknüpften Tcl-Befehls den Inhalt des für die interaktive Zellbearbeitung verwendeten eingebetteten Fensters als ganzzahligen Zeitwert (in Sekunden) ab, ansonsten als Zeichenkette. Verwenden Sie diese Option für Tablelist-Widgets, deren interne Liste Datums- oder Zeitangaben in Sekunden enthält (die mithilfe von Befehlen angezeigt werden, die durch die Spaltenkonfigurationsoption `-formatcommand` vorgegeben sind). Das zweite

optionale Argument legt den Namen fest, der für das "datefield"-"dateentry"-"timefield"- oder "timeentry"-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `datefield`, `dateentry`, `timefield` oder `timeentry`, wie durch das erste Argument vorgegeben. Der Befehl gibt sein Argument `name` zurück.

Wenn Sie eine Zelle mithilfe des temporär eingebetteten Widgets "datefield", "dateentry", "timefield" oder "timeentry" bearbeiten, das mit dem oben genannten `name` verknüpft ist, können Sie das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um beliebige Optionen des Widgets festzulegen, wie beispielsweise `-gmt`, `-iq`, `-int` (internationales Datumsformat, unterstützt von den Widgets "datefield" und "dateentry") oder `-format` (für "timefield"- und "timeentry"-Widgets, mit den Werten `civilian` und `military`). Für das "dateentry"-Widget können Sie außerdem die Option `-startday` auf `monday` und die Option `-days` beispielsweise auf `{Mo Tu We Th Fr Sa Su}` setzen.

SCHLÜSSELWÖRTER

tablelist, editing, Iwidgets, datefield, dateentry, timefield, timeentry

---

Der Befehl tablelist::addIncrSpinner

NAME

`tablelist::addIncrSpinner` – Das Spinner-Widget aus dem Iwidgets-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIncrSpinner ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das Spinner-Widget aus dem Iwidgets-Paket für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das Spinner-Widget als Wert der Spalten- oder Zellkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `spinner`. Der Befehl gibt sein Argument `name` zurück.

Wenn Sie eine Zelle mithilfe des temporär eingebetteten Spinner-Widgets bearbeiten, das mit dem oben genannten `name` verknüpft ist, können Sie das Skript verwenden, das der Konfigurationsoption `-editstartcommand` für die Tablelist entspricht, um Validierungen für das Widget zu definieren oder andere Optionen festzulegen, wie beispielsweise `-decrement` und `-increment`.

SCHLÜSSELWÖRTER

tablelist, editing, Iwidgets, spinner

Der Befehl tablelist::addIncrSpinint

NAME

`tablelist::addIncrSpinint` – Das "spinint"-Widget aus dem "Iwidgets"-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIncrSpinint ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das "spinint"-Widget aus dem "Iwidgets"-Paket für die interaktive Zellbearbeitung in "tablelist"-Widgets. Das optionale Argument legt den Namen fest, der für das "spinint"-Widget als Wert der Spalten- oder Zellkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `spinint`. Der Befehl gibt sein Argument `name` zurück.

Wenn Sie eine Zelle mithilfe des temporären, eingebetteten "spinint"-Widgets bearbeiten, das mit dem oben genannten `name` verknüpft ist, können Sie das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um Validierungen für das Widget zu definieren oder andere Optionen festzulegen, wie beispielsweise `-range`, `-step` und `-wrap`.

SCHLÜSSELWÖRTER

tablelist, editing, Iwidgets, spinint

Der Befehl tablelist::addIncrCombobox

NAME

`tablelist::addIncrCombobox` – Das Combobox-Widget aus dem Iwidgets-Paket für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIncrCombobox ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das Combobox-Widget im Dropdown-Stil aus dem Iwidgets-Paket für die interaktive Zellenbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das Combobox-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `combobox`. Der Befehl gibt sein Argument `name` zurück.

Das temporäre, eingebettete Combobox-Widget, das mit dem oben genannten `name` verknüpft ist, wird erstellt, wobei die Option `-editable` auf `1` und die Option `-grab` auf `global` gesetzt ist. Sie können das Skript verwenden, das der Tablelist-Konfigurationsoption `-editstartcommand` entspricht, um die Combobox nicht bearbeitbar zu machen oder Validierungen dafür zu definieren, sowie zum Befüllen ihrer listbox-Komponente (mithilfe des Unterbefehls `insert list` und der Option `-listheight` des Kombinationsfeld-Widgets).

SCHLÜSSELWÖRTER

tablelist, editing, Iwidgets, combobox

tablelistMentry

Stichwörter: tablelistMentry

Interaktive Bearbeitung von Tabellenzellen mit dem Mentry-Paket

Für Tablelist Version 7.12

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::addDateMentry
- Der Befehl tablelist::addTimeMentry
- Der Befehl tablelist::addDateTimeMentry
- Der Befehl tablelist::addFixedPointMentry
- Der Befehl tablelist::addIPAddrMentry
- Der Befehl tablelist::addIPv6AddrMentry

Überblick

Das Multi-Entry-Paket "Mentry" ist eine Bibliothekserweiterung für Tcl/Tk ab Version 8.4, die vollständig in Tcl/Tk geschrieben ist. Es kann hier heruntergeladen werden:

https://www.nemethi.de

Ab Version 3.0 enthält die Mentry-Distribution nicht nur das Paket "Mentry", sondern auch dessen auf tile basierendes Pendant "Mentry_tile", das eine themespezifische Darstellung der mentry-Widgets ermöglicht; dieses Paket erfordert Tcl/Tk 8.4 oder höher sowie Tile 0.6 oder höher.

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe von Mentry-Widgets der Typen `"Date"`, `"Time"`, `"DateTime`, `"FixedPoint"`, `"IPAddr"` und `"IPv6Addr"`. Die erforderlichen Schritte zur Verwendung eines dieser Widgets für die Bearbeitung der Zellen einer bestimmten Spalte lauten wie folgt:

1. Registrieren Sie das gewünschte Widget für die interaktive Zellenbearbeitung, indem Sie einen der auf dieser Referenzseite beschriebenen Befehle aufrufen.
2. Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf den vom oben genannten Befehl zurückgegebenen Wert zu setzen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

Der Befehl tablelist::addDateMentry

NAME

`tablelist::addDateMentry` – Das Mentry-Widget vom Typ `"Date"` für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addDateMentry format separator ?-gmt? ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das mit dem Befehl `mentry::dateMentry` aus dem Paket "Mentry" (oder "Mentry_tile") erstellte Widget für die interaktive Zellenbearbeitung in Tablelist-Widgets. Die Argumente `format` und `separator` haben dieselbe Bedeutung wie im Befehl `mentry::dateMentry`. Wenn das Argument `-gmt` vorhanden ist, werden sowohl der interne Uhrzeitwert als auch dessen externe Datumsdarstellung im Mentry-Widget als Greenwich-Zeit (GMT) betrachtet, andernfalls als Ortszeit. Das zweite optionale Argument legt den Namen fest, der für das "mentry"-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann eine beliebige Zeichenkette sein, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `dateMentry`. Der Befehl gibt sein `name`-Argument zurück.

Eine Spalte in einer Tablelist, deren Option `-editwindow` den oben genannten Wert `name` aufweist, muss als interne Zellenwerte Datumsangaben in Sekunden enthalten (die mithilfe eines Befehls angezeigt werden, der durch die Spaltenkonfigurationsoption `-formatcommand` vorgegeben wird). Der Unterbefehl `finishediting` des mit dem Tablelist-Widget verknüpften Tcl-Befehls ruft den Inhalt des eingebetteten Fensters ab, das für die interaktive Zellenbearbeitung verwendet wird, indem er den Befehl `mentry::getClockVal` aufruft. Der von diesem Befehl zurückgegebene Wert (ein Zeitwert in Sekunden oder eine der Fehlerzeichenketten `"EMPTY"`, `"BAD"`, `"BAD_DATE"` oder `"BAD_YEAR"`) wird als letztes Argument an das Skript übergeben, das der Tablelist-Konfigurationsoption `-editendcommand` entspricht (sofern vorhanden). Ist dieser Wert eine der oben genannten Fehlerzeichenketten, sollte das Skript den Inhalt des Mentry durch Aufruf des Unterbefehls `rejectinput` ablehnen.

SCHLÜSSELWÖRTER

tablelist, editing, Mentry, date

Der Befehl tablelist::addTimeMentry

NAME

`tablelist::addTimeMentry` – Das Mentry-Widget vom Typ `"Time"` für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addTimeMentry format separator ?-gmt? ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das mit dem Befehl `mentry::timeMentry` aus dem Paket "Mentry" (oder "Mentry_tile") erstellte Widget für die interaktive Zellenbearbeitung in Tablelist-Widgets. Die Argumente `format` und `separator` haben dieselbe Bedeutung wie im Befehl `mentry::timeMentry`. Wenn das Argument `-gmt` vorhanden ist, werden sowohl

der interne Uhrwert als auch dessen externe Zeitdarstellung im Mentry-Widget als Greenwich-Zeit (GMT) betrachtet, andernfalls als Ortszeit. Das zweite optionale Argument legt den Namen fest, der für das "mentry"-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann eine beliebige Zeichenkette sein, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `timeMentry`. Der Befehl gibt sein `name`-Argument zurück.

Eine Spalte in einer Tablelist, deren Option `-editwindow` den oben genannten Wert `name` aufweist, muss als interne Zellenwerte Zeitangaben in Sekunden enthalten (die mithilfe eines durch die Spaltenkonfigurationsoption `-formatcommand` vorgegebenen Befehls angezeigt werden). Der Unterbefehl `finishediting` des mit dem Tablelist-Widget verknüpften Tcl-Befehls ruft den Inhalt des eingebetteten Fensters ab, das für die interaktive Zellenbearbeitung verwendet wird, indem er den Befehl `mentry::getClockVal` aufruft. Der von diesem Befehl zurückgegebene Wert (ein Zeitwert in Sekunden oder eine der Fehlerzeichenketten `"EMPTY"` oder `"BAD"`) wird als letztes Argument an das Skript übergeben, das der Tablelist-Konfigurationsoption `-editendcommand` entspricht (sofern vorhanden). Ist dieser Wert eine der oben genannten Fehlerzeichenketten, sollte das Skript den Inhalt des Mentry durch Aufruf des Unterbefehls `rejectinput` ablehnen.

SCHLÜSSELWÖRTER

tablelist, editing, Mentry, time

Der Befehl tablelist::addDateTimeMentry

NAME

`tablelist::addDateTimeMentry` – Das Mentry-Widget vom Typ `"DateTime"` für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addDateTimeMentry format dateSeparator timeSeparator ?-gmt? ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das vom Befehl `mentry::dateTimeMentry` aus dem Paket "Mentry" (oder "Mentry_tile") erstellte Widget für die interaktive Zellbearbeitung in Tablelist-Widgets. Die Argumente `format`, `dateSeparator` und `timeSeparator` haben dieselbe Bedeutung wie im Befehl `mentry::dateTimeMentry`. Wenn das Argument `-gmt` vorhanden ist, werden sowohl der Wert der internen Uhr als auch dessen externe Datums- und Zeitdarstellung im Mentry-Widget als Greenwich-Zeit (GMT) betrachtet, andernfalls als Ortszeit. Das zweite optionale Argument legt den Namen fest, der für das "mentry"-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann eine beliebige Zeichenkette sein, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `dateTimeMentry`. Der Befehl gibt sein `name`-Argument zurück.

Eine Spalte in einer Tablelist, deren Option `-editwindow` den oben genannten Wert `name` aufweist, muss als interne Zellenwerte Datums- und Zeitangaben in Sekunden enthalten (die mithilfe eines Befehls angezeigt werden, der durch die Spaltenkonfigurationsoption

`-formatcommand` vorgegeben wird). Der Unterbefehl `finishediting` des mit dem Tablelist-Widget verknüpften Tcl-Befehls ruft den Inhalt des eingebetteten Fensters ab, das für die interaktive Zellenbearbeitung verwendet wird, indem er den Befehl `mentry::getClockVal` aufruft. Der von diesem Befehl zurückgegebene Wert (ein Zeitwert in Sekunden oder eine der Fehlerzeichenketten `"EMPTY"`, `"BAD"`, `"BAD_DATE"` oder `"BAD_YEAR"`) wird als letztes Argument an das Skript übergeben, das der Tablelist-Konfigurationsoption `-editendcommand` entspricht (sofern vorhanden). Ist dieser Wert eine der oben genannten Fehlerzeichenketten, sollte das Skript den Inhalt des "mentry" durch Aufruf des Unterbefehls `rejectinput` ablehnen.

SCHLÜSSELWÖRTER

tablelist, editing, Mentry, date, time

Der Befehl tablelist::addFixedPointMentry

NAME

`tablelist::addFixedPointMentry` – Das Mentry-Widget vom Typ `"FixedPoint"` für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addFixedPointMentry count1 count2 ?-comma? ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das vom Befehl `mentry::fixedPointMentry` aus dem Paket "Mentry" (oder "Mentry_tile") erstellte Widget für die interaktive Zellbearbeitung in Tablelist-Widgets. Die Argumente `count1`, `count2` und `-comma` haben dieselbe Bedeutung wie im Befehl `mentry::fixedPointMentry`. Das zweite optionale Argument legt den Namen fest, der für das Mentry-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann eine beliebige Zeichenkette sein, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `fixedPointMentry_count1.count2` (z. B. `fixedPointMentry_6.2`) oder `fixedPointMentry_count1,count2` (z. B. `fixedPointMentry_6,2`), je nachdem, ob das optionale Argument `-comma` vorhanden ist. Der Befehl gibt sein Argument `name` zurück.

Eine Spalte der Tablelist, deren Option `-editwindow` den oben genannten Wert `name` aufweist, muss reelle Zahlen als interne Zellenwerte enthalten. Der Unterbefehl `finishediting` des Tcl-Befehls, der dem Tablelist-Widget zugeordnet ist, ruft den Inhalt des eingebetteten Fensters ab, das für die interaktive Zellbearbeitung verwendet wird, indem er den Befehl `mentry::getReal` aufruft. Der von diesem Befehl zurückgegebene Wert (eine reelle Zahl oder die Fehlerzeichenkette `"EMPTY"`) wird als letztes Argument an das Skript übergeben, das der Tablelist-Konfigurationsoption `-editendcommand` entspricht (sofern vorhanden). Ist dieser Wert die oben genannte Fehlerzeichenkette, sollte das Skript den Inhalt des Mentry durch Aufruf des Unterbefehls `rejectinput` ablehnen.

SCHLÜSSELWÖRTER

tablelist, editing, Mentry, real number

Der Befehl tablelist::addIPAddrMentry

NAME

`tablelist::addIPAddrMentry` – Das Mentry-Widget vom Typ `"IPAddr"` für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIPAddrMentry ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das vom Befehl `mentry::ipAddrMentry` aus dem Paket "Mentry" (oder "Mentry_tile") erstellte Widget für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das Mentry-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann eine beliebige Zeichenkette sein, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `ipAddrMentry`. Der Befehl gibt sein `name` -Argument zurück.

Eine Spalte der Tablelist, deren Option `-editwindow` den oben genannten Wert `name` aufweist, muss IP-Adressen als interne Zellenwerte enthalten. Der Unterbefehl `finishediting` des Tcl-Befehls, der dem Tablelist-Widget zugeordnet ist, ruft den Inhalt des eingebetteten Fensters ab, das für die interaktive Zellbearbeitung verwendet wird, indem er den Befehl `mentry::getIPAddr` aufruft. Der von diesem Befehl zurückgegebene Wert (eine IP-Adresse oder die Fehlerzeichenkette `"EMPTY"`) wird als letztes Argument an das Skript übergeben, das der Tablelist-Konfigurationsoption `-editendcommand` entspricht (sofern vorhanden). Ist dieser Wert die oben genannte Fehlerzeichenkette, sollte das Skript den Inhalt des Mentry durch Aufruf des Unterbefehls `rejectinput` ablehnen.

SCHLÜSSELWÖRTER

tablelist, editing, Mentry, IP address

Der Befehl tablelist::addIPv6AddrMentry

NAME

`tablelist::addIPv6AddrMentry` – Das Mentry-Widget vom Typ `"IPv6Addr"` für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addIPv6AddrMentry ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das mit dem Befehl `mentry::ipv6AddrMentry` aus dem Paket "Mentry" (oder "Mentry_tile") erstellte Widget für die interaktive Zellbearbeitung in Tablelist-Widgets. Das optionale Argument legt den Namen fest, der für das Mentry-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann eine beliebige Zeichenkette sein, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `ipv6AddrMentry`. Der Befehl gibt sein `name`-Argument zurück.

Eine Spalte der Tablelist, deren Option `-editwindow` den oben genannten Wert `name` aufweist, muss IPv6-Adressen als interne Zellenwerte enthalten. Der Unterbefehl `finishediting` des mit dem Tablelist-Widget verknüpften Tcl-Befehls ruft den Inhalt des eingebetteten Fensters ab, das für die interaktive Zellenbearbeitung verwendet wird, indem er den Befehl `mentry::getIPv6Addr` aufruft. Der von diesem Befehl zurückgegebene Wert (eine IPv6-Adresse oder die Fehlerzeichenkette `"EMPTY"`) wird als letztes Argument an das Skript übergeben, das der Tablelist-Konfigurationsoption `-editendcommand` entspricht (sofern vorhanden). Ist dieser Wert die oben genannte Fehlerzeichenkette, sollte das Skript den Inhalt des Mentry durch Aufruf des Unterbefehls `rejectinput` ablehnen.

SCHLÜSSELWÖRTER

tablelist, editing, Mentry, IPv6 address

tablelistThemes

Stichwörter: tablelistThemes

Befehle im Zusammenhang mit Tile-Themes

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::setTheme
- Der Befehl tablelist::getCurrentTheme
- Der Befehl tablelist::getThemes
- Der Befehl tablelist::setThemeDefaults

Überblick

Die auf dieser Referenzseite beschriebenen Befehle sollten nur bei Verwendung des Pakets "Tablelist_tile" aufgerufen werden. Mit ihnen können Sie das aktuelle Theme festlegen und abfragen, eine Liste der verfügbaren Themes abrufen und sicherstellen, dass Ihre Widgets ein themespezifisches Erscheinungsbild erhalten.

Der Befehl tablelist::setTheme

NAME

`tablelist::setTheme` – Aktuelles Theme festlegen

ÜBERSICHT

```
tablelist::setTheme theme
```

BESCHREIBUNG

Dieser Befehl legt das aktuelle Theme auf `theme` fest und lädt es gegebenenfalls. Es handelt sich dabei lediglich um einen Alias für `ttk::setTheme` oder `tile::setTheme`, je nachdem, welche Tile-Version in den Interpreter geladen wurde. (Der Befehl `tile::setTheme` wurde in der Tile-Version 0.8 in `ttk::setTheme` umbenannt.)

Da es sich lediglich um einen Alias für eine Prozedur der Tile-Bibliothek handelt, führt der Befehl `tablelist::setTheme` genau dasselbe aus wie der ursprüngliche Befehl: Er lädt bei Bedarf das Paket, das das angegebene Theme implementiert, legt das angegebene Theme fest und speichert dieses je nach aktueller Tile-Version in der Variable `ttk::currentTheme` oder `tile::currentTheme`.

SCHLÜSSELWÖRTER

tablelist, theme, tile

Der Befehl tablelist::getCurrentTheme

NAME

`tablelist::getCurrentTheme` – Aktuelles Theme abrufen

ÜBERSICHT

```
tablelist::getCurrentTheme
```

BESCHREIBUNG

Dieser Befehl gibt den Namen des aktuellen Themes zurück, indem er den Befehl `ttk::style theme use` aufruft. Wird eine ältere Tile-Version verwendet, die diese Methode noch nicht unterstützt, gibt der Befehl je nach Tile-Version den Wert der Variablen `ttk::currentTheme` oder `tile::currentTheme` zurück. (Der Namespace, der die Variable `currentTheme` enthält, wurde in der Tile-Version 0.8 von `tile` in `ttk` geändert.)

SCHLÜSSELWÖRTER

tablelist, theme, tile

Der Befehl tablelist::getThemes

NAME

`tablelist::getThemes` – Die in der Paketdatenbank registrierten Themes abrufen

ÜBERSICHT

```
tablelist::getThemes
```

BESCHREIBUNG

Dieser Befehl gibt eine Liste der in der Paketdatenbank registrierten Themes zurück. Es handelt sich dabei lediglich um einen Alias für `ttk::themes` oder `tile::availableThemes`, je nachdem, welche Tile-Version in den Interpreter geladen wurde. (Der Befehl `tile::availableThemes` wurde in der Tile-Version 0.8 in `ttk::themes` umbenannt.)

SCHLÜSSELWÖRTER

tablelist, theme, tile

Der Befehl tablelist::setThemeDefaults

NAME

`tablelist::setThemeDefaults` – Festlegen themespezifischer Standardwerte für bestimmte Konfigurationsoptionen der Tablelist

ÜBERSICHT

```
tablelist::setThemeDefaults
```

BESCHREIBUNG

Dieser Befehl füllt das Array `tablelist::themeDefaults` mit themespezifischen Standardwerten für eine Reihe von Tablelist-Konfigurationsoptionen. Die Array-Namen entsprechen den Befehlszeilennamen der Optionen, und die entsprechenden Array-Werte sind die Standardwerte dieser Konfigurationsoptionen für das aktuell eingestellte Tile-Theme.

Die Optionen, deren Namen und Werte in das Array `tablelist::themeDefaults` geschrieben werden, sind: `-arrowcolor`, `-arrowdisabledcolor`, `-arrowstyle`, `-background`, `-disabledforeground`, `-font`, `-foreground`, `-labelborderwidth`, `-labelfont`, `-labelforeground`, `-labelpady`, `-selectbackground`, `-selectborderwidth`, `-selectforeground`, `-stripebackground` und `-treestyle`. Darüber hinaus setzt der Befehl einige weitere Array-Elemente auf themespezifische Standardwerte für die Hintergrund- und Vordergrundfarben der Spaltenbeschriftungen in den Zuständen `normal`, `disabled`, `active` und `pressed`. (Tablelist benötigt die Beschriftungsfarben für die Darstellung von Sortierpfeilen und Bildern mit transparentem Hintergrund in den Spaltenbeschriftungen.)

Der Befehl `tablelist::setThemeDefaults` wird von Tablelist_tile automatisch beim Start der Anwendung sowie dann aufgerufen, wenn das virtuelle Ereignis `<<ThemeChanged>>` an das Hauptfenster gesendet wird. Er wird außerdem unter macOS 10.14 (Mojave) und höher immer dann aufgerufen, wenn das Hauptfenster ein virtuelles Ereignis vom Typ `<<LightAppearance>>` (früher: `<<LightAqua>>`) oder `<<DarkAppearance>>` (früher: `<<DarkAqua>>`) erhält, wenn das aktuelle Theme `aqua` ist. In diesen Fällen sendet "Tablelist_tile" nach dem Aufruf des Befehls `tablelist::setThemeDefaults` das virtuelle Ereignis `<<TablelistThemeDefaultsChanged>>` an das Hauptfenster und konfiguriert alle Tablelist-Widgets der Anwendung neu, wobei die neuen Standardwerte jener Optionen verwendet werden, die nicht explizit auf andere Werte als die entsprechenden Standardwerte gesetzt wurden.

Unter macOS 10.14 (Mojave) und höher werden die virtuellen Ereignisse `<<LightAppearance>>` und `<<DarkAppearance>>` (bzw. `<<LightAqua>>` und `<<DarkAqua>>`) von Tk 8.6.10 und höher an alle Toplevel-Fenster gesendet, sobald das Systemdesign zwischen hellem und dunklem Modus wechselt oder sich die Akzentfarbe in den allgemeinen Einstellungen ändert. Tk-Versionen ab 8.6.11 senden diese Ereignisse unter macOS 10.15 (Catalina) und höher auch wenn die Hervorhebungsfarbe in den allgemeinen Einstellungen geändert wird.

Dieser Befehl wird nicht nur von Tablelist_tile verwendet, sondern kann auch im Anwendungscode aufgerufen werden, obwohl es bei den aktuellen Versionen von Tablelist_tile

nicht notwendig ist, ihn explizit aufzurufen.

Durch die Verwendung der im Array `tablelist::themeDefaults` enthaltenen Standardwerte können Sie sicherstellen, dass klassische Tk-Widgets, zum Beispiel listbox und text, genau wie die Tile-Widgets ein themespezifisches Erscheinungsbild erhalten. Beispielsweise können Sie der Optionsdatenbank wie folgt einige gängige Konfigurationsoptionen hinzufügen:

```
if {[tk windowingsystem] eq "x11"} {
    option add *selectBackground  $tablelist::themeDefaults(-selectbackground)
    option add *selectForeground  $tablelist::themeDefaults(-selectforeground)
}
option add *selectBorderWidth     $tablelist::themeDefaults(-selectborderwidth)
```

Sie können außerdem das virtuelle Ereignis `<<TablelistThemeDefaultsChanged>>` nutzen, um das Erscheinungsbild Ihrer Tk-Widgets entsprechend den neuen Werten im Array `tablelist::themeDefaults` anzupassen, indem Sie eine Bindung an das dafür vorgesehene Bindungs-Tag `TablelistMain` herstellen, das dem Hauptfenster zugeordnet ist:

```
bind TablelistMain <<TablelistThemeDefaultsChanged>> ...
```

Das Bindungs-Tag `TablelistMain` wird vom Tablelist_tile dem Hauptfenster zugewiesen und intern zur Verarbeitung der virtuellen Ereignisse `<<ThemeChanged>>`, `<<LightAppearance>>` (oder `<<LightAqua>>`) sowie `<<DarkAppearance>>` (oder `<<DarkAqua>>`) verwendet. Wie oben gezeigt, können Anwendungen dies nutzen, um beispielsweise das virtuelle Ereignis `<<TablelistThemeDefaultsChanged>>` zu verarbeiten.

SCHLÜSSELWÖRTER

tablelist, theme, tile

tablelistTile

Stichwörter: tablelistTile

Interaktive Bearbeitung von Tabellenzellen mithilfe von Tile-Widgets

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Interaktive Zellbearbeitung mit dem Widget ttk::entry
- Interaktive Zellbearbeitung mit dem Widget ttk::spinbox

- Interaktive Zellbearbeitung mit dem Widget ttk::combobox
- Interaktive Zellbearbeitung mit dem Widget ttk::checkbutton
- Interaktive Zellbearbeitung mit dem Widget ttk::menubutton
- Interaktive Zellbearbeitung mit dem Widget ttk::toggleswitch

---

Überblick

Die Tile-Themes-Engine ist eine kompilierte Bibliothekserweiterung für Tcl/Tk ab Version 8.4. Ab Tk 8.5a6 ist "tile" in den Tk-Kern integriert. Für frühere Tk-Versionen kann sie unter folgender Adresse heruntergeladen werden:

https://sourceforge.net/projects/tktable//files/tile

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe der Widgets ttk::entry, ttk::spinbox, ttk::combobox, ttk::checkbutton, ttk::menubutton und ttk::toggleswitch, auf die als `ttk::entry`, `ttk::spinbox`, `ttk::combobox`, `ttk::checkbutton`, `ttk::menubutton` und `ttk::toggleswitch` zugegriffen wird. Die vollqualifizierten Namen `::ttk::entry`, `::ttk::spinbox`, `::ttk::combobox`, `::ttk::checkbutton`, `::ttk::menubutton` und `::ttk::toggleswitch` werden ebenfalls unterstützt. Die Version des "tile"-Pakets muss 0.6 oder höher sein (das Widget ttk::spinbox erfordert sogar "tile" 0.8.3 oder höher oder alternativ Tk 8.5.9 oder höher). Das Widget ttk::toggleswitch wurde in Tk 9.1a1 eingeführt. Diese Widgets werden automatisch für die Zellbearbeitung registriert, daher ist zur Verwendung eines dieser Widgets für die Bearbeitung der Zellen einer bestimmten Spalte lediglich Folgendes erforderlich:

Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf `ttk::entry`, `ttk::spinbox`, `ttk::combobox`, `ttk::checkbutton`, `ttk::menubutton` oder `ttk::toggleswitch` (bzw. auf den entsprechenden vollqualifizierten Namen) festzulegen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

HINWEIS 1: Wenn eines der integrierten Themes verwendet wird, wird das temporäre, eingebettete Tile-Widget, das für die interaktive Zellenbearbeitung verwendet wird, entsprechend der DPI-Skalierungsstufe des Bildschirms und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk korrekt skaliert angezeigt. Die einzige Ausnahme bildet das Tile-Checkbutton-Widget, das bei den Themes `aqua`, `clam`, `classic`, `default`, `vista`, und `xpnative` wie erwartet skaliert wird, aus technischen Gründen jedoch nicht skaliert werden kann, wenn das Theme `alt` oder `winnative` lautet und die Tk-Version älter als 9 ist.

HINWEIS 2: Eine bekannte Einschränkung früherer Tile-Versionen bestand darin, dass der Befehl `(ttk::)style theme use` nur zum Festlegen des aktuellen Themes verwendet werden konnte, nicht jedoch zum Abrufen desselben. Aus diesem Grund greift Tablelist, wenn es das aktuelle Theme nicht mithilfe von `ttk::style theme use` abrufen kann, auf die Variable `ttk::currentTheme` oder `tile::currentTheme` (je nach Tile-Version), die durch den Befehl `ttk::setTheme` oder `tile::setTheme` gesetzt wird, zurück. Daraus folgt: Wenn die verwendete Tile-Version den Befehl `ttk::style theme use` ohne Argument nicht unterstützt, werden die für die interaktive Zellbearbeitung verwendeten Tile-Widgets nur dann wie erwartet verwaltet, wenn das plattformspezifische Vorgabe-Theme entweder unverändert bleibt oder durch Aufruf der Prozedur der Bibliothek `ttk::setTheme` oder `tile::setTheme` – je nach aktueller Tile-Version – durch ein anderes Theme ersetzt wird. (Siehe auch den Befehl

`tablelist::setTheme`.)

Interaktive Zellbearbeitung mit dem Widget "ttk::entry"

BESCHREIBUNG

Das temporäre, eingebettete ttk::entry-Widget, das für die interaktive Bearbeitung von Zellen verwendet wird, wird mit einem explizit festgelegten Wert für die Option `-style` erstellt. Abgesehen von seinem themespezifischen Erscheinungsbild verhält es sich genau wie sein Tk-Core-Pendant.

Wenn eine Anwendung das Widget "ttk::entry" für die interaktive Zellenbearbeitung sowie das Wcb-Paket verwendet (auch wenn dies nicht für dieses Widget gilt), muss die Version von Wcb 3.1 oder höher sein (da frühere Wcb-Versionen keine Tile-Widgets unterstützten).

SCHLÜSSELWÖRTER

tablelist, editing, ttk::entry

Interaktive Zellbearbeitung mit dem Widget "ttk::spinbox"

BESCHREIBUNG

Das temporäre, eingebettete ttk::spinbox-Widget, das für die interaktive Zellbearbeitung verwendet wird, wird mit einem explizit festgelegten Wert für die Option `-style` erstellt, wobei die Option `-state` auf `normal` gesetzt ist, wodurch das Widget bearbeitbar wird. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um den Zustand der Spinbox auf `readonly` zu setzen oder Validierungen dafür zu definieren, sowie zur Festlegung ihrer Werte (bzw. ihres Wertebereichs) und ihrer Option `-wrap`. Abgesehen von ihrem themespezifischen Erscheinungsbild verhält sie sich genau wie ihr Tk-Core-Pendant.

Wenn eine Anwendung das Widget "ttk::spinbox" für die interaktive Zellbearbeitung sowie das Wcb-Paket verwendet (auch wenn dies nicht für dieses Widget gilt), dann muss die Version von Wcb 3.2 oder höher sein (da die Unterstützung für das neue "Tile Spinbox"-Widget erst in Version 3.2 von Wcb hinzugefügt wurde).

SCHLÜSSELWÖRTER

tablelist, editing, ttk::spinbox

Interaktive Zellbearbeitung mit dem Widget "ttk::combobox"

BESCHREIBUNG

Das temporäre, eingebettete ttk::combobox-Widget, das für die interaktive Zellenbearbeitung verwendet wird, wird mit einem explizit festgelegten Wert für die Option `-style` erstellt, wobei die Option `-state` auf `normal` gesetzt ist, wodurch das Widget bearbeitbar wird. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` für die Tablelist entspricht, um den Zustand der Combobox auf `readonly` zu setzen oder Validierungen dafür zu definieren, sowie zum Befüllen ihrer listbox-Komponente (mithilfe der Option `-values` des Combobox-Widgets).

Wenn eine Anwendung das Widget "ttk::combobox" für die interaktive Zellenbearbeitung sowie das Wcb-Paket verwendet (auch wenn dies nicht für dieses Widget gilt), muss die Version von Wcb 3.1 oder höher sein (da frühere Wcb-Versionen keine Tile-Widgets unterstützten).

SCHLÜSSELWÖRTER

tablelist, editing, ttk::combobox

Interaktive Zellbearbeitung mit dem Widget "ttk::checkbutton".

BESCHREIBUNG

Das temporäre, eingebettete ttk::checkbutton-Widget, das für die interaktive Zellenbearbeitung verwendet wird, wird mit explizit festgelegten Werten für die Optionen `-style` und `-variable` erstellt. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um beliebige andere Konfigurationsoptionen wie `-offvalue` und `-onvalue` entsprechend den *internen* Werten der Zellen festzulegen. Da die Standardwerte der ttk::checkbutton-Optionen `-offvalue` und `-onvalue` `0` und `1` lauten, müssen Sie diese Optionen nicht ändern, wenn die Zellen die gleichen internen Werte `0` und `1` aufweisen.

SCHLÜSSELWÖRTER

tablelist, editing, ttk::checkbutton

Interaktive Zellbearbeitung mit dem Widget "ttk::menubutton".

BESCHREIBUNG

Das temporär eingebettete ttk::menubutton-Widget für die interaktive Zellenbearbeitung wird mit ausdrücklich gesetzten Werten seiner Optionen `-style` und `-textvariable` erzeugt. Zusätzlich wird ein Menü erzeugt, dessen Option `-tearoff` auf `0` gesetzt ist und das ein passendes Skript als Wert der Option `-postcommand` erhält, und dieses Menü wird als Wert der Option `-menu` der menubutton gesetzt. Unter X11 wird das Aussehen des Menüs an das Tablelist-Widget angepasst, indem die Optionen `-background`, `-foreground`, `-activebackground`, `-activeforeground` und `-activeborderwidth` auf passende Werte gesetzt werden. Mit dem Skript zur Tablelist-Konfigurationsoption `-editstartcommand` können Sie weitere Optionen der menubutton oder des zugehörigen Menüs setzen. Dieses Skript brauchen Sie vor allem, um das Menü zu füllen, vorzugsweise mit radiobutton-Einträgen. Für jeden dem Menü hinzugefügten radiobutton-Eintrag sorgt die Tablelist-Implementierung dafür, dass sein Wert (angegeben über die Option `-value` oder `-label` des Eintrags) beim Auswählen als Text der menubutton angezeigt wird. (Tablelist erreicht das, indem die Option `-variable` des Menüeintrags auf den Wert der Option `-textvariable` der menubutton gesetzt wird.) Bei Menüeinträgen eines anderen Typs als radiobutton (zum Beispiel command-Einträge) muss die Anwendung dafür sorgen, dass der Text des ausgewählten Eintrags in der menubutton erscheint, zum Beispiel über die Option `-command` des Menüeintrags.

SCHLÜSSELWÖRTER

tablelist, editing, ttk::menubutton

Interaktive Zellbearbeitung mit dem Widget "ttk::toggleswitch".

BESCHREIBUNG

Das temporäre, eingebettete ttk::toggleswitch-Widget, das für die interaktive Zellenbearbeitung verwendet wird, wird erstellt, wobei die Option `-size` auf `1` gesetzt ist. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um die ursprünglichen Einstellungen nach Ihren Bedürfnissen zu überschreiben.

SCHLÜSSELWÖRTER

tablelist, editing, ttk::toggleswitch

---

tablelistTkCore

Stichwörter: tablelistTkCore

Interaktive Bearbeitung von Tabellenzellen mithilfe von Tk-Core-Widgets

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Interaktive Zellbearbeitung mithilfe des Eingabe-Widgets
- Interaktive Zellbearbeitung mit dem Text-Widget
- Interaktive Zellbearbeitung mit dem Spinbox-Widget
- Interaktive Zellbearbeitung mit dem checkbutton-Widget
- Interaktive Zellbearbeitung mit dem menubutton-Widget

Überblick

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe der Tk-Core-Widgets "entry", "text", "spinbox", "checkbutton" und "menubutton". Diese Widgets werden automatisch für die Zellenbearbeitung registriert, sodass zur Verwendung eines dieser Widgets zur Bearbeitung der Zellen einer bestimmten Spalte lediglich folgender Schritt erforderlich ist:

Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf `entry`, `text`, `spinbox`, `checkbutton` oder `menubutton` festzulegen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.) Da der Standardwert der Spaltenkonfigurationsoption `-editwindow entry` lautet, ist es nicht erforderlich, ihn explizit festzulegen, wenn die Bearbeitung mithilfe eines eingebetteten Eingabe-Widgets erfolgen soll.

HINWEIS: Das temporär eingebettete Tk-Core-Widget, das für die interaktive Bearbeitung von Zellen verwendet wird, wird entsprechend der DPI-Skalierung des Bildschirms und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk korrekt skaliert angezeigt.

Interaktive Zellbearbeitung mithilfe des Eingabe-Widgets

BESCHREIBUNG

Wie oben erwähnt, erfolgt die interaktive Zellbearbeitung in einem Tablelist-Widget standardmäßig mithilfe eines eingebetteten Eingabe-Widgets. Einzelheiten zum Bearbeitungsprozess finden Sie im Abschnitt INTERAKTIVE ZELLENBEARBEITUNG der Referenzseite, in dem der Befehl `tablelist::tablelist` beschrieben wird.

SCHLÜSSELWÖRTER

tablelist, editing, entry

Interaktive Zellbearbeitung mit dem Text-Widget

BESCHREIBUNG

Das temporäre eingebettete Text-Widget, das für die interaktive Zellbearbeitung verwendet wird, wird so erstellt, dass seine Optionen `-padx` und `-pady` auf 2 gesetzt sind, die Option `-wrap` auf `none` und die anfängliche Höhe auf die Anzahl der darin enthaltenen Zeilen. Es gibt jedoch eine Ausnahme von dieser Regel: Wenn die Option `-wrap` der Spalte der Zelle auf "true" gesetzt wurde und Tk Version 8.5 oder höher verwendet wird, dann wird die Option `-wrap` des Text-Widgets auf `word` gesetzt und seine Anfangshöhe entspricht der Anzahl der darin enthaltenen *angezeigten* Zeilen (unter Berücksichtigung der Zeilenumbrüche). Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um die Anfangseinstellungen (mit Ausnahme der Höhe) nach Ihren Bedürfnissen zu überschreiben.

Wenn die Option `-wrap` des Text-Widgets auf `word` oder `char` gesetzt wurde (entweder durch "Tablelist" oder innerhalb des oben genannten Skripts) und Tk Version 8.5 oder höher verwendet wird, wird bei jeder Änderung seiner Breite (z. B. durch interaktive Anpassung der Spalten) seine Höhe automatisch auf die Anzahl der darin enthaltenen Anzeigezeilen gesetzt. (Die Anzahl der Anzeigezeilen wird mithilfe des in Tk 8.5 eingeführten Text-Widget-Unterbefehls `count -displaylines` ermittelt.)

Wenn das Widget-Rückruffunktion-Paket "Wcb" in den Interpreter geladen wurde (über `package require Wcb` oder `package require wcb`), wird die Höhe des Text-Widgets automatisch aktualisiert, sobald Text eingefügt oder gelöscht wird, was die Bearbeitung wesentlich benutzerfreundlicher macht. Dies wird durch die Verwendung einer entsprechend definierten Rückruffunktion "after-`insert`" und "after-`delete`" für das Bearbeitungsfenster erreicht. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um weitere Rückruffunktionen für das Text-Widget zu definieren. (Die oben erwähnte Rückruffunktion wird über `wcb::cbappend` erstellt, *nach* der Rückkehr aus diesem Skript.)

Die Taste `Tab` ist für die Navigation zwischen den bearbeitbaren Zellen reserviert, der

Benutzer kann jedoch durch Drücken der Taste `Control-i` ein Tabulatorzeichen in das Text-Widget einfügen.

Im Gegensatz zu den anderen Widgets, die für die interaktive Zellenbearbeitung verwendet werden, fügen die Tasten `Return` und `KP_Enter` ein Zeilenumbruchzeichen in das Text-Widget ein. Auch `Control-j` kann zum Einfügen eines Zeilenumbruchs verwendet werden. `Control-Return` und `Control-KP_Enter` beenden die Bearbeitung und schließen das Bearbeitungsfenster.

`Control-Home` und `Control-End` verfügen über ihre bekannten, für Text-Widgets spezifischen Bindungen, genau wie `Meta-<` und `Meta->`, wenn `tk_strictMotif` auf "false" gesetzt ist. Auch dies unterscheidet sich vom Verhalten der anderen Widgets, die für die interaktive Zellbearbeitung verwendet werden. Um zur ersten bzw. letzten bearbeitbaren Zelle zu springen, kann der Benutzer `Alt-Home`/`Alt-End` oder `Meta-Home`/`Meta-End` (`Command-Home`/`Command-End` unter Mac OS X/11+ Aqua) drücken.

SCHLÜSSELWÖRTER

tablelist, editing, text

Interaktive Zellbearbeitung mithilfe des Spinbox-Widgets

BESCHREIBUNG

Das temporäre, eingebettete Spinbox-Widget, das für die interaktive Bearbeitung von Zellen verwendet wird, wird so erstellt, dass die Option `-state` auf `normal` gesetzt ist, wodurch das Widget bearbeitbar wird. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um den Status der Spinbox auf `readonly` zu setzen oder Validierungen dafür zu definieren, sowie zur Festlegung ihres Wertebereichs und ihrer Option `-wrap`.

SCHLÜSSELWÖRTER

tablelist, editing, spinbox

Interaktive Zellbearbeitung mit dem checkbutton-Widget

BESCHREIBUNG

Unter Windows und Mac OS X/11+ Aqua wird das temporäre, eingebettete checkbutton-Widget, das für die interaktive Zellbearbeitung verwendet wird, mit explizit festgelegten Werten für die Optionen `-borderwidth`, `-font`, `-padx`, `-pady` und `-variable` erstellt. In einer X11-Umgebung wird es mit explizit festgelegten Werten für die Optionen `-borderwidth`, `-indicatoron`, `-image`, `-selectimage`, `-background`, `-activebackground`, `-selectcolor` und `-variable` erstellt. Sie können das Skript verwenden, das der Tablelist-Konfigurationsoption `-editstartcommand` entspricht, um beliebige andere Optionen wie `-offvalue` und `-onvalue` entsprechend den *internen* Werten der Zellen festzulegen. Da die Standardwerte der checkbutton-Optionen `-offvalue` und `-onvalue` `0` und `1` lauten, müssen Sie diese Optionen nicht ändern, wenn die Zellen die gleichen internen Werte `0` und `1` haben.

SCHLÜSSELWÖRTER

tablelist, editing, checkbutton

Interaktive Zellbearbeitung mit dem Menüschaltflächen-Widget

BESCHREIBUNG

Das temporär eingebettete menubutton-Widget für die interaktive Zellenbearbeitung wird mit ausdrücklich gesetzten Werten seiner Optionen `-anchor`, `-indicatoron`, `-justify`, `-padx`, `-pady`, `-relief` und `-textvariable` erzeugt. Zusätzlich wird ein Menü erzeugt, dessen Option `-tearoff` auf `0` gesetzt ist und das ein passendes Skript als Wert der Option `-postcommand` erhält, und dieses Menü wird als Wert der Option `-menu` der menubutton gesetzt. Unter X11 wird das Aussehen des Menüs an das Tablelist-Widget angepasst, indem die Optionen `-background`, `-foreground`, `-activebackground`, `-activeforeground` und `-activeborderwidth` auf passende Werte gesetzt werden. Mit dem Skript zur Tablelist-Konfigurationsoption `-editstartcommand` können Sie weitere Optionen der menubutton oder des zugehörigen Menüs setzen. Dieses Skript brauchen Sie vor allem, um das Menü zu füllen, vorzugsweise mit radiobutton-Einträgen. Für jeden dem Menü hinzugefügten radiobutton-Eintrag sorgt die Tablelist-Implementierung dafür, dass sein Wert (angegeben über die Option `-value` oder `-label` des Eintrags) beim Auswählen als Text der menubutton angezeigt wird. (Tablelist erreicht das, indem die Option `-variable` des Menüeintrags auf den Wert der Option `-textvariable` der menubutton gesetzt wird.) Bei Menüeinträgen eines anderen Typs als radiobutton (zum Beispiel command-Einträge) muss die Anwendung dafür sorgen, dass der Text des ausgewählten Eintrags in der menubutton erscheint, zum Beispiel über die Option `-command` des Menüeintrags.

SCHLÜSSELWÖRTER

tablelist, editing, menubutton

tablelistTsw

Stichwörter: tablelistTsw

Interaktive Bearbeitung von Tabellenzellen mit dem Tsw-Paket

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Überblick
- Der Befehl tablelist::addToggleswitch

Überblick

Das Widget-Paket "Tsw" für Kippschalter ist eine Bibliothekserweiterung für Tcl/Tk ab Version 8.6, die in reinem Tcl/Tk-Code geschrieben ist. Der Download-Link lautet

https://www.nemethi.de

Tablelist unterstützt die interaktive Bearbeitung von Zellen mithilfe des im oben genannten Paket implementierten ToggleSwitch-Widgets. Die folgenden Schritte sind erforderlich, um dieses Widget zur Bearbeitung der Zellen einer bestimmten Spalte zu verwenden:

1. Registrieren Sie das Toggle-Switch-Widget für die interaktive Zellenbearbeitung, indem Sie den unten beschriebenen Befehl `tablelist::addToggleswitch` aufrufen.
2. Verwenden Sie den Unterbefehl `columnconfigure` des Tablelist-Widgets, um die Option `-editable` der angegebenen Spalte auf "true" und die Option `-editwindow` auf den vom oben genannten Befehl zurückgegebenen Wert zu setzen. (Diese Optionen werden mithilfe des Unterbefehls `cellconfigure` auch auf Zellenebene unterstützt.)

HINWEIS: Das temporär eingebettete Kippschalter-Widget, das für die interaktive Bearbeitung von Zellen verwendet wird, wird entsprechend der DPI-Skalierung des Bildschirms und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk korrekt skaliert angezeigt.

Der Befehl tablelist::addToggleswitch

NAME

`tablelist::addToggleswitch` – Das Kippschalter-Widget für die interaktive Bearbeitung von Zellen registrieren

ÜBERSICHT

```
tablelist::addToggleswitch ?name?
```

BESCHREIBUNG

Dieser Befehl registriert das "toggleswitch"-Widget für die interaktive Zellenbearbeitung in "tablelist"-Widgets. Das optionale Argument legt den Namen fest, der für das "toggleswitch"-Widget als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` verwendet werden soll. Es kann sich um eine beliebige Zeichenkette handeln, die sich von den Namen der Bearbeitungsfenster im Tk-Kern und in Tile unterscheidet. Der Standardwert ist `toggleswitch`. Der Befehl gibt sein Argument `name` zurück.

Das temporäre, eingebettete Kippschalter-Widget, das für die interaktive Zellenbearbeitung verwendet wird, wird so erstellt, dass die Option `-size` auf `1` gesetzt ist. Sie können das Skript verwenden, das der Konfigurationsoption `-editstartcommand` der Tablelist entspricht, um die ursprünglichen Einstellungen nach Ihren Bedürfnissen zu überschreiben.

SCHLÜSSELWÖRTER

tablelist, editing, Tsw, toggleswitch

tablelistWidget

Stichwörter: tablelistWidget

Der Befehl `tablelist::tablelist`

Für Tablelist Version 7.12

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Kurzreferenz
- Ausführliche Referenz

Kurzreferenz

NAME

`tablelist::tablelist` – Tablelist-Widgets erstellen und bearbeiten

ÜBERSICHT

```
tablelist::tablelist pathName ?options?
```

STANDARDOPTIONEN

```
-borderwidth          -highlightthickness  -setgrid
-cursor               -relief              -xscrollcommand
-exportselection      -selectbackground    -yscrollcommand
-highlightbackground  -selectborderwidth
-highlightcolor       -selectforeground
```

OPTIONEN FÜR DATENBEREICH UND KOPFBEREICH

```
-background  -disabledforeground  -font  -foreground
```

WIDGET-SPEZIFISCHE OPTIONEN

`-acceptchildcommand command`

`-acceptdropcommand command`

`-activestyle frame|none|underline`

`-aftercopycommand command`

`-arrowcolor color`

`-arrowdisabledcolor color`

-arrowstyle	flat7x4|flat7x7|flat8x4|flat8x5|flat9x5|flat10x5|
	flat11x6|flat12x6|flat13x7|flat14x7|flat15x8|flat16x8|
	flatAngle7x4|flatAngle7x5|flatAngle9x5|flatAngle9x6|
	flatAngle10x6|flatAngle11x6|flatAngle13x7|flatAngle15x8|
	photo0x0|photo7x4|photo7x7|photo9x5|photo11x6|
	photo13x7|photo15x8|sunken8x7|sunken10x9|sunken12x11

```
-autofinishediting boolean

-autoscan boolean

-button2window window

-collapsecommand command

-colorizecommand command

-columns {width title ?left|right|center? width title
?left|right|center? ...}

-columntitles {title title ...}

-customdragsource boolean

-displayondemand boolean

-editendcommand command

-editendonfocusout boolean

-editendonmodclick boolean

-editselectedonly boolean

-editstartcommand command

-expandcommand command

-forceeditendcommand boolean

-fullseparators boolean

-height units

-inactiveselectbackground color

-inactiveselectforeground color

-incrarrowtype up|down

-instanttoggle boolean

-itembackground coloror -itembg color
```

```
-labelactivebackground color

-labelactiveforeground color

-labelbackground coloror -labelbg color

-labelborderwidth screenDistanceor -labelbd screenDistance

-labelcommand command

-labelcommand2 command

-labeldisabledforeground color

-labelfont font

-labelforeground coloror -labelfg color

-labelheight lines

-labelpady screenDistance

-labelrelief raised|sunken|flat|ridge|solid|groove

-listvariable variable

-movablecolumns boolean

-movablerows boolean

-movecolumncursor cursor

-movecursor cursor

-populatecommand command

-protecttitlecolumns boolean

-resizablecolumns boolean

-resizecursor cursor

-selectfiltercommand command

-selectmode single|browse|multiple|extended
```

```
-selecttype row|cell

-setfocus boolean

-showarrow boolean

-showbusycursor boolean

-showeditcursor boolean

-showhorizseparator boolean

-showlabels boolean

-showseparators boolean

-snipstring string

-sortcommand command

-spacing screenDistance

-state normal|disabled

-stretch all|columnIndexList

-stripebackground coloror -stripebg color

-stripeforeground coloror -stripefg color

-stripeheight lines

-takefocus 0|1|""|command

-targetcolor color

-tight boolean

-titlecolumns number

-tooltipaddcommand command

-tooltipdelcommand command

-treecolumn columnIndex
```

-treestyle	adwaita|ambiance|aqua|aqua11|arc|baghira|bicolor|bicolor100|
	bicolor125|bicolor150|bicolor175|bicolor200|blueMenta|classic|
	classic100|classic125|classic150|classic175|classic200|dust|dustSand|
	gtk|klearlooks|mate|menta|mint|mint2|newWave|oxygen1|oxygen2|phase|
	plain100|plain125|plain150|plain175|plain200|plastik|plastique|
	radiance|ubuntu|ubuntu2|ubuntu3|ubuntuMate|vistaAero|vistaClassic|
	white|white100|white125|white150|white175|white200|win7Aero|
	win7Classic|win10|winnative|winxpBlue|winxpOlive|winxpSilver|yuyo

```
  -width characters

  -xmousewheelwindow window

  -ymousewheelwindow window
```

SPALTENKONFIGURATIONSOPTIONEN

```
  -align left|right|center

  -allowduplicates boolean

  -background coloror -bg color

  -changesnipside boolean

  -changetitlesnipside boolean

  -editable boolean

  -editwindow name

  -font font

  -foreground coloror -fg color

  -formatcommand command

  -hide boolean

  -labelalign left|right|center

  -labelbackground coloror -labelbg color
```

```
-labelborderwidth screenDistanceor -labelbd screenDistance

-labelcommand command

-labelcommand2 command

-labelfont font

-labelforeground coloror -labelfg color

-labelheight lines

-labelimage image

-labelpady screenDistance

-labelrelief raised|sunken|flat|ridge|solid|groove

-labelvalign center|top|bottom

-labelwindow checkbutton|ttk::checkbutton

-maxwidth width

-name name

-resizable boolean

-selectbackground color

-selectfiltercommand command

-selectforeground color

-showarrow boolean

-showlinenumbers boolean

-sortcommand command

-sortmode ascii|asciinocase|command|dictionary|integer|real

-stretchable boolean

-stretchwindow boolean
```

```
  -stripebackground color

  -stripeforeground color

  -text list

  -title title

  -valign center|top|bottom

  -width width

  -windowdestroy command

  -windowupdate command

  -wrap boolean
```

ZEILENKONFIGURATIONSOPTIONEN IM DATENBEREICH

```
  -background coloror -bg color

  -font font

  -foreground coloror -fg color

  -hide boolean

  -name name

  -selectable boolean

  -selectbackground color

  -selectforeground color

  -text list
```

ZEILENKONFIGURATIONSOPTIONEN IM KOPFBEREICH

```
  -background coloror -bg color

  -font font

  -foreground coloror -fg color

  -name name

  -text list
```

ZELLENKONFIGURATIONSOPTIONEN IM DATENBEREICH

```
-background coloror -bg color

-editable boolean

-editwindow name

-font font

-foreground coloror -fg color

-image image

-imagebackground color

-selectbackground color

-selectforeground color

-stretchwindow boolean

-text text

-valign center|top|bottom

-window command

-windowdestroy command

-windowupdate command
```

ZELLENKONFIGURATIONSOPTIONEN IM KOPFBEREICH

```
-background coloror -bg color

-font font

-foreground coloror -fg color

-image image

-imagebackground color

-stretchwindow boolean

-text text

-valign center|top|bottom
```

```
-window command

-windowdestroy command

-windowupdate command
```

ZEILENINDIZES

```
number  knumber  active  anchor  end  last  top  bottom  @x,y  name
```

KNOTENINDIZES

```
root  number  knumber  active  anchor  end  last  top  bottom  @x,y  name
```

SPALTENINDIZES

```
number  active  anchor  end  last  left  right  @x,y  name
```

ZELLENINDIZES

```
row,col  active  anchor  end  last  @x,y

    row: number  knumber  active  anchor  end  last  top  bottom  name
    col: number  active  anchor  end  last  left  right  name
```

KOPFZEILENINDIZES

```
number  hknumber  end  last  @x,y  name
```

KOPFZELLENINDIZES

```
headerRow,col  end  last  @x,y

    headerRow: number  hknumber  end  last  name
    col:       number  active  anchor  end  last  left  right  name
```

WIDGET-BEFEHL

```
pathName activate index

pathName activatecell cellIndex

pathName applysorting itemList

pathName attrib ?name? ?value name value ...?

pathName autoscrolltarget event x y

pathName bbox index

pathName bodypath

pathName bodytag
```

```
pathName canceledediting

pathName cancelediting

pathName cellattrib cellIndex ?name? ?value name value ...?

pathName cellbbox cellIndex

pathName cellcget cellIndex option

pathName cellconfigure cellIndex ?option? ?value option value ...?

pathName cellindex cellIndex

pathName cellselection option args

pathName cellselection anchor cellIndex

pathName cellselection clear firstCell lastCell

pathName cellselection clear cellIndexList

pathName cellselection includes cellIndex

pathName cellselection set firstCell lastCell

pathName cellselection set cellIndexList
pathName cget option

pathName childcount nodeIndex

pathName childindex index

pathName childkeys nodeIndex

pathName collapse indexList ?-fully|-partly?

pathName collapseall ?-fully|-partly?

pathName columnattrib columnIndex ?name? ?value name value ...?

pathName columncget columnIndex option

pathName columnconfigure columnIndex ?option? ?value option value ...?
```

```
pathName columncount

pathName columnindex columnIndex

pathName columnwidth columnIndex ?-requested|-stretched|-total?

pathName configcelllist {cellIndex option value cellIndex option value
...}

pathName configcells ?cellIndex option value cellIndex option value
...?

pathName configcolumnlist {columnIndex option value columnIndex option
value ...}

pathName configcolumns ?columnIndex option value columnIndex option
value ...?

pathName configrowlist {index option value index option value ...}

pathName configrows ?index option value index option value ...?

pathName configure ?option? ?value option value ...?

pathName containing y

pathName containingcell x y

pathName containingcolumn x

pathName cornerlabelpath

pathName cornerpath ?-ne|-sw?

pathName curcellselection ?-all|-nonhidden|-viewable?

pathName curselection ?-all|-nonhidden|-viewable?

pathName delete firstIndex lastIndex

pathName delete indexList

pathName deletecolumns firstColumn lastColumn

pathName deletecolumns columnIndexList
```

```
pathName depth nodeIndex

pathName descendantcount nodeIndex

pathName dicttoitem dictionary

pathName dumptofile fileName

pathName dumptostring

pathName editcell cellIndex

pathName editinfo

pathName editwinpath

pathName editwintag

pathName embedcheckbutton cellIndex ?command?

pathName embedcheckbuttons columnIndex ?command?

pathName embedttkcheckbutton cellIndex ?command?

pathName embedttkcheckbuttons columnIndex ?command?

pathName entrypath

pathName expand indexList ?-fully|-partly?

pathName expandall ?-fully|-partly?

pathName expandedkeys

pathName fillcolumn columnIndex ?-text|-image|-window? value

pathName findcolumnname name

pathName findrowname name ?-descend? ?-parent nodeIndex?

pathName finishediting

pathName formatinfo

pathName get firstIndex lastIndex ?-all|-nonhidden|-viewable?
```

```
pathName get indexList

pathName getcells firstCell lastCell ?-all|-nonhidden|-viewable?

pathName getcells cellIndexList

pathName getcolumns firstColumn lastColumn

pathName getcolumns columnIndexList

pathName getformatted firstIndex lastIndex ?-all|-nonhidden|-viewable?

pathName getformatted indexList

pathName getformattedcells firstCell lastCell
?-all|-nonhidden|-viewable?

pathName getformattedcells cellIndexList

pathName getformattedcolumns firstColumn lastColumn

pathName getformattedcolumns columnIndexList

pathName getfullkeys firstIndex lastIndex ?-all|-nonhidden|-viewable?

pathName getfullkeys indexList

pathName getkeys firstIndex lastIndex ?-all|-nonhidden|-viewable?

pathName getkeys indexList

pathName hasattrib name

pathName hascellattrib cellIndex name

pathName hascolumnattrib columnIndex name

pathName hasrowattrib index name

pathName header option ?arg arg ...?

pathName headerpath

pathName headertag

pathName hidetargetmark
```

```
pathName imagelabelpath cellIndex

pathName index index

pathName insert index ?item item ...?

pathName insertchildlist parentNodeIndex childIndex itemList

pathName insertchild(ren) parentNodeIndex childIndex ?item item ...?

pathName insertcolumnlist columnIndex {width title ?left|right|center?
width title ?left|right|center? ...}

pathName insertcolumns columnIndex ?width title ?left|right|center?
width title ?left|right|center? ...?

pathName insertlist index itemList

pathName iselemsnipped cellIndex fullTextName

pathName isexpanded index

pathName istitlesnipped columnIndex fullTextName

pathName isviewable index

pathName itemlistvar

pathName itemtodict item ?excludedColumnIndexList?

pathName labelpath columnIndex

pathName labels

pathName labeltag

pathName labelwindowpath columnIndex

pathName loadfromfile fileName ?-fully|-partly?

pathName loadfromstring string ?-fully|-partly?

pathName move sourceIndex targetIndex

pathName move sourceIndex targetParentNodeIndex targetChildIndex
```

```
pathName movecolumn sourceColumn targetColumn

pathName nearest y

pathName nearestcell x y

pathName nearestcolumn x

pathName noderow parentNodeIndex childIndex

pathName parentkey nodeIndex

pathName refreshsorting ?parentNodeIndex?

pathName rejectinput

pathName resetsortinfo

pathName restorecursor

pathName rowattrib index ?name? ?value name value ...?

pathName rowcget index option

pathName rowconfigure index ?option? ?value option value ...?

pathName rowcount

pathName scan mark|dragto x y

pathName searchcolumn columnIndex pattern ?options?

pathName see index

pathName seecell cellIndex

pathName seecolumn columnIndex

pathName selection option args

pathName selection anchor index

pathName selection clear firstIndex lastIndex

pathName selection clear indexList
```

```
pathName selection includes index

pathName selection set firstIndex lastIndex

pathName selection set indexList
pathName separatorpath ?columnIndex?

pathName separators

pathName setbusycursor

pathName showtargetmark before|inside index

pathName size

pathName sort ?-increasing|-decreasing?

pathName sortbycolumn columnIndex ?-increasing|-decreasing?

pathName sortbycolumnlist columnIndexList ?sortOrderList?

pathName sortcolumn

pathName sortcolumnlist

pathName sortorder

pathName sortorderlist

pathName stopautoscroll

pathName targetmarkpath

pathName targetmarkpos y ?-any|-horizontal|-vertical?

pathName togglecolumnhide firstColumn lastColumn

pathName togglecolumnhide columnIndexList

pathName togglerowhide firstIndex lastIndex

pathName togglerowhide indexList

pathName toplevelkey index
```

```
pathName unsetattrib name

pathName unsetcellattrib cellIndex name

pathName unsetcolumnattrib columnIndex name

pathName unsetrowattrib index name

pathName viewablerowcount ?firstIndex lastIndex?

pathName windowpath cellIndex

pathName xview ?args?

pathName xview

pathName xview units

pathName xview moveto fraction

pathName xview scroll number units|pages
pathName yview ?args?

pathName yview

pathName yview units

pathName yview moveto fraction

pathName yview scroll number units|pages
```

KOPFBEREICHSBEZOGENE UNTERBEFEHLE

```
pathName header bbox headerIndex

pathName header cellattrib headerCellIndex ?name? ?value name value
...?

pathName header cellbbox headerCellIndex

pathName header cellcget headerCellIndex option

pathName header cellconfigure headerCellIndex ?option? ?value option
value ...?

pathName header cellindex headerCellIndex
```

```
pathName header configcelllist {headerCellIndex option value
headerCellIndex option value ...}

pathName header configcells ?headerCellIndex option value
headerCellIndex option value ...?

pathName header configrowlist {headerIndex option value headerIndex
option value ...}

pathName header configrows ?headerIndex option value headerIndex
option value ...?

pathName header containing y

pathName header containingcell x y

pathName header delete firstHeaderIndex lastHeaderIndex

pathName header delete headerIndexList

pathName header embedcheckbutton headerCellIndex ?command?

pathName header embedcheckbuttons columnIndex ?command?

pathName header embedttkcheckbutton headerCellIndex ?command?

pathName header embedttkcheckbuttons columnIndex ?command?

pathName header fillcolumn columnIndex ?-text|-image|-window? value

pathName header findrowname name

pathName header get firstHeaderIndex lastHeaderIndex

pathName header get headerIndexList

pathName header getcells firstHeaderCell lastHeaderCell

pathName header getcells headerCellIndexList

pathName header getcolumns firstColumn lastColumn

pathName header getcolumns columnIndexList

pathName header getformatted firstHeaderIndex lastHeaderIndex
```

```
pathName header getformatted headerIndexList

pathName header getformattedcells firstHeaderCell lastHeaderCell

pathName header getformattedcells headerCellIndexList

pathName header getformattedcolumns firstColumn lastColumn

pathName header getformattedcolumns columnIndexList

pathName header getfullkeys firstHeaderIndex lastHeaderIndex

pathName header getfullkeys headerIndexList

pathName header getkeys firstHeaderIndex lastHeaderIndex

pathName header getkeys headerIndexList

pathName header hascellattrib headerCellIndex name

pathName header hasrowattrib headerIndex name

pathName header imagelabelpath headerCellIndex

pathName header index headerIndex

pathName header insert headerIndex ?item item ...?

pathName header insertlist headerIndex itemList

pathName header iselemsnipped headerCellIndex fullTextName

pathName header itemlistvar

pathName header nearest y

pathName header nearestcell x y

pathName header rowattrib headerIndex ?name? ?value name value ...?

pathName header rowcget headerIndex option

pathName header rowconfigure headerIndex ?option? ?value option value
...?

pathName header rowcount
```

```
pathName header size

pathName header unsetcellattrib headerCellIndex name

pathName header unsetrowattrib headerIndex name

pathName header windowpath headerCellIndex
```

SCHLÜSSELWÖRTER

tablelist, multi-column, listbox, tree, widget

Ausführliche Referenz

NAME

`tablelist::tablelist` – Tablelist-Widgets erstellen und bearbeiten

ÜBERSICHT

```
tablelist::tablelist pathName ?options?
```

BESCHREIBUNG

Der Befehl `tablelist::tablelist` erstellt ein neues Fenster mit dem Namen `pathName` und der Klasse `Tablelist` und wandelt es in ein **tablelist**-Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank festgelegt werden, um Aspekte der Tablelist wie Farben, Schriftart und Spalten zu konfigurieren. Der Befehl `tablelist::tablelist` gibt sein Argument `pathName` zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen `pathName` vorhanden sein, das übergeordnete Fenster von `pathName` muss jedoch vorhanden sein.

Eine Tablelist ist ein mehrspaltiges listbox- und Baum-Widget, das als Megawidget implementiert ist und aus einem **Datenbereich** und einem **Kopfbereich** besteht. Der Datenbereich zeigt eine Liste von **Einträgen** an, einen pro Zeile. Jeder Eintrag ist eine Liste von **Elementen**, die in Spalten ausgerichtet sind. Mit anderen Worten: Ein Eintrag ist der Inhalt einer Zeile, und ein Element ist der in einer Zelle enthaltene Text. Die Kopfzeile besteht aus Label-Widgets, die die Spaltentitel anzeigen. Sie können unter anderem für die interaktive Anpassung der Spaltengrößen und die spaltenbasierte Sortierung der Einträge des Datenbereichs verwendet werden, wie im Folgenden beschrieben. Bei Tk-Versionen 8.5 und höher kann die Kopfbereichskomponente ebenso wie der Datenbereich (in der Regel eine geringe Anzahl von) Einträgen enthalten. Die Reihenfolge der Einträge des Kopfbereichs wird durch sortierbezogene Befehle oder Bindungen nicht beeinflusst. Das Ende des Kopfzeilenbereichs wird mithilfe eines horizontalen Trennstrichs visualisiert, der direkt unterhalb der letzten Kopfzeile (sofern vorhanden) platziert ist.

Der Grund dafür, dass die Unterstützung für Einträge im Kopfbereich auf Tk-Versionen ab 8.5 beschränkt ist, liegt darin, dass die Höhe der Kopfzeile so festgelegt ist, dass sie gerade groß genug ist, um alle Elemente aufzunehmen. Dies geschieht mithilfe des Befehls `count -ypixels` für Text-Widgets, der in Tk 8.5 eingeführt wurde.

Jede Zelle (sowohl im Hauptbereich als auch in der Kopfzeile) und jede Kopfzeilenbeschriftung eines Tablelist-Widgets kann zudem ein Bild oder ein eingebettetes Fenster enthalten, das je nach Ausrichtung der Spalte links oder rechts vom Text platziert wird. Das in eine Kopfzeilenbezeichnung eingebettete Fenster kann nur ein Tk-Core-Checkbutton oder ein ttk::checkbutton-Widget sein.

Wenn eine Tablelist als Baum-Widget verwendet wird, zeigt eine ihrer Spalten die Baumhierarchie mithilfe von Einrückungen und Steuerelementen zum Ein- und Ausklappen an. Das Tablelist-Paket bietet eine große Auswahl an Baumstilen, die das Erscheinungsbild dieser Spalte steuern, und wählt je nach Fenstersystem, Betriebssystemversion und Tile-Theme den passenden Standardstil aus. In einer Tablelist, die als mehrspaltiges Baum-Widget verwendet wird, ist jede Zeile ihres Datenbereichs (nicht jedoch ihres Kopfbereichs) gleichzeitig ein **Baumknoten**, der genau einen **übergeordneten** Knoten und eine beliebige Anzahl von **untergeordneten** Knoten besitzt. Der Ursprung der Baumstruktur ist der unsichtbare **Stammknoten**, der selbst keinen übergeordneten Knoten hat und dessen untergeordnete Knoten die Knoten der **obersten Ebene** sind.

Die Elemente eines Tablelist-Widgets können standardmäßig nur programmgesteuert bearbeitet werden. Die interaktive Bearbeitung kann jedoch für einzelne Zellen und für ganze Spalten des Datenbereichs aktiviert werden. Standardmäßig nutzt die interaktive Zellenbearbeitung ein temporär eingebettetes "Entry"-Widget, wodurch sichergestellt wird, dass alle für "Entry"-Widgets verfügbaren Validierungsfunktionen während des Bearbeitungsvorgangs genutzt werden können. Eine große Vielfalt an Widgets aus den Paketen "BWidget", "Iwidgets", "combobox" (von Bryan Oakley), "ctext", Mentry (oder Mentry_tile) und Tsw sowie die Tk-Kern-Widgets "text", "spinbox", "checkbutton" und "menubutton" sowie die tile-Widgets "entry", "spinbox", "combobox", "checkbutton" und "menubutton" werden ebenfalls als temporäre eingebettete Widgets für die Zellenbearbeitung unterstützt. Darüber hinaus steht eine umfangreiche Auswahl an Tastatur-Bindungen zur Verfügung, die eine komfortable Navigation zwischen den bearbeitbaren Zellen ermöglichen.

Bei seiner Erstellung enthält ein neues Tablelist-Widget keine Einträge. Einträge (sowohl im Datenbereich als auch im Kopfbereich) können mithilfe der unten beschriebenen WIDGET-BEFEHLE hinzugefügt, gelöscht oder aktualisiert werden. Zudem können ein oder mehrere Einträge oder Elemente des Hauptteils des Widgets ausgewählt werden. Wenn ein Tablelist-Widget seine Auswahl exportiert (siehe die Option `-exportselection`), befolgt es die Standard-X11-Protokolle für die Behandlung der Auswahl. Auswahlen aus Tablelist-Widgets stehen als Typen `STRING` und `UTF8_STRING` zur Verfügung; der Wert der Auswahl ist ein Text, der gebildet wird, indem alle Zeilen mit mindestens einem sichtbaren ausgewählten Element genommen, diese Elemente durch Tabulatoren miteinander verbunden und die resultierenden Zeichenketten wiederum durch Zeilenumbrüche verbunden werden. Wenn ein Tablelist-Widget, das seine Auswahl exportiert, der Eigentümer der Auswahl ist und ein anderes Fenster die Eigentumsrechte an der Auswahl beansprucht, wird das virtuelle Ereignis `<<TablelistSelectionLost>>` ausgelöst.

Es ist nicht erforderlich, dass alle Elemente gleichzeitig im Tablelist-Widget angezeigt werden; mit den unten beschriebenen Befehlen lassen sich die horizontale Ansicht im Fenster und die vertikale Ansicht im Datenbereich (jedoch nicht im Kopfbereich) ändern. Tablelist-Widgets ermöglichen das Scrollen in beide Richtungen (jedoch kein vertikales Scrollen in der Kopfzeilenkomponente) unter Verwendung der Standardoptionen `-xscrollcommand` und `-yscrollcommand`. Sie unterstützen außerdem das Scannen im Hauptbereich des Widgets (jedoch nicht in der Kopfzeile), wie im Folgenden beschrieben.

Jedes Tablelist-Widget sowie jede Spalte, Zeile und Zelle der Tablelist (sowohl im Datenbereich als auch im Kopfbereich) kann eine beliebige Anzahl von **Attributen** aufweisen, die in Befehlen verwendet werden können, mit denen Tablelist-Widgets für bestimmte Zwecke erstellt oder bearbeitet werden.

STANDARDOPTIONEN

```
-borderwidth          -highlightthickness  -setgrid
-cursor               -relief              -xscrollcommand
-exportselection      -selectbackground    -yscrollcommand
-highlightbackground  -selectborderwidth
-highlightcolor       -selectforeground
```

Weitere Informationen zu den Standardoptionen finden Sie im Handbucheintrag **options**. Die Optionen `-highlightbackground`, `-highlightcolor` und `-highlightthickness` werden nur vom Paket "Tablelist" unterstützt, nicht jedoch von "Tablelist_tile". Wenn das Paket "Tablelist_tile" verwendet wird, haben die Optionen `-selectbackground`, `-selectborderwidth` und `-selectforeground` themespezifische Standardwerte.

HINWEIS 1: Wenn der Wert der Konfigurationsoption `-width` null oder kleiner ist und die Tablelist dehnbare Spalten enthält, wird die Option `-setgrid` ignoriert. Diese geringfügige Einschränkung hat technische Gründe und ist nur unter X11 relevant.

HINWEIS 2: Anstatt die Scrollbar(n) manuell zu erstellen und die Optionen `-xscrollcommand` und/oder `-yscrollcommand` entsprechend einzustellen, könnten Sie die Verwendung des scrollarea-Widgets aus dem Scrollutil-Paket in Betracht ziehen, das die Erstellung beliebiger scrollbarer Widgets erheblich vereinfacht, sowohl statische als auch dynamische Scrollbars unterstützt und die Kopfzeilenkomponente sowie die Titelspalten von Tablelist-Widgets berücksichtigt.

OPTIONEN FÜR DATENBEREICH UND KOPFBEREICH

```
-background  -disabledforeground  -font  -foreground
```

Diese Optionen (beschrieben im Handbucheintrag **options**) gelten nur für die Komponenten Datenbereich und Kopfbereich des Tablelist-Widgets. Wie weiter unten erläutert, können sich die Farben und Schriftarten, die beim Zeichnen der *Beschriftungen* der Kopfzeile verwendet werden, von denen unterscheiden, die für die *Einträge* des Datenbereichs und des Kopfbereichs festgelegt wurden. Wenn das Paket "Tablelist_tile" verwendet wird, haben diese Optionen themespezifische Standardwerte.

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-acceptchildcommand
Name der Datenbank:	acceptChildCommand
Datenbankklasse:	AcceptChildCommand

Legt einen Tcl-Befehl fest, mit dem entschieden wird, ob ein bestimmter Knoten der Tablelist ein angegebenes Element, das interaktiv verschoben wird, als untergeordnetes Element akzeptieren darf. Der angegebene Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets, dem Knotenindex des potenziellen neuen übergeordneten Knotens und dem Zeilenindex des gezogenen Elements verkettet. Das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert (der ein Boolescher Wert sein muss) bestimmt, ob das Quellelement als untergeordnetes Element des angegebenen übergeordneten Knotens an die aktuelle Mausposition verschoben werden darf.

Beispielsweise könnte bei einem Tablelist-Widget, das als Dateimanager verwendet wird und bei dem die Elemente der obersten Ebene die im System eingebundenen Volumes darstellen, der durch diese Option angegebene Befehl wie im folgenden (Pseudo-)Code aussehen:

```
proc acceptChildCmd {tbl targetParentNodeIdx sourceRow} {
    if {[string compare $targetParentNodeIdx "root"] == 0} {
        # Allow only volumes as top-level items
        return [expr {[$tbl depth $sourceRow] == 1}]
    } else {
        # Allow only directories as parent items
        return [$targetParentNodeIdx represents a directory]
    }
}
```

Befehlszeilenname:	-acceptdropcommand
Name der Datenbank:	acceptDropCommand
Datenbankklasse:	AcceptDropCommand

Legt einen Tcl-Befehl fest, mit dem entschieden wird, ob eine bestimmte Zeile der Tablelist ein angegebenes Element akzeptieren darf, das interaktiv verschoben wird, um direkt vor dieser Zeile als gleichrangiges Element abgelegt zu werden. Der angegebene Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets, dem Zeilenindex des potenziellen neuen Ziels und dem Zeilenindex des gezogenen Elements verkettet. Das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert (der ein Boolescher Wert sein muss) bestimmt, ob das Quellelement an die aktuelle Mausposition direkt vor die angegebene Zeile verschoben werden darf.

Um beispielsweise sicherzustellen, dass die Position der letzten Zeile eines Tablelist-Widgets durch das interaktive Verschieben von Zeilen nicht beeinflusst wird, könnte der durch diese Option festgelegte Befehl wie im folgenden Code aussehen:

```
proc acceptDropCmd {tbl targetRow sourceRow} {
    set rowCount [[$tbl size]
    return [expr {$sourceRow != $rowCount - 1 && $targetRow < $rowCount}]
}
```

Befehlszeilenname:	-activestyle
Name der Datenbank:	activeStyle
Datenbankklasse:	ActiveStyle

Legt fest, wie der aktive Eintrag oder das aktive Element (abhängig vom Wert der Konfigurationsoption `-selecttype`) angezeigt wird, wenn die Tablelist den Tastaturfokus hat. Die zulässigen Werte sind `frame`, `none` und `underline`. Der Standardwert `frame` zeigt einen dünnen Rahmen um den aktiven Eintrag oder das aktive Element an, was in den meisten Fällen gut aussieht. Es sieht weniger ansprechend aus, wenn es auf das aktive Element angewendet wird, sofern die Hintergrundfarbe einiger seiner Zellen mithilfe des Widget-Befehls `columnconfigure` oder `cellconfigure` geändert wurde und keine Spaltentrennzeichen angezeigt werden. Der Wert `none` legt fest, dass keine besondere Hervorhebung des aktiven Eintrags oder Elements erfolgen soll. Der Wert `underline` erzeugt denselben visuellen Effekt wie bei der Tk-Core-Listbox.

Befehlszeilenname:	-aftercopycommand
Name der Datenbank:	afterCopyCommand
Datenbankklasse:	AfterCopyCommand

Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn die interaktive Bearbeitung des Inhalts einer Zelle mit einer der Tastenkombinationen `Alt-Return`, `Alt-KP_Enter`, `Alt-a`, `Alt-d` oder `Alt-u` beendet wurde und das Kopieren des neuen Werts in andere Zellen der Spalte abgeschlossen ist. Unter Mac OS X/11+ Aqua wird die Tastenkombination `Alt-a` durch `Control-Command-a` ersetzt, und in den anderen Tastenfolgen erfolgt die Ersetzung des Modifikators `Alt` durch `Command`. Der Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Index der gerade bearbeiteten Spalte verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Diese Option ist beispielsweise nützlich, wenn die in die Zellen der Spalte eingebetteten Bilder an die aktualisierten Werte der Zellen angepasst werden sollen. Einzelheiten zum Bearbeitungsprozess finden Sie im Abschnitt INTERAKTIVE ZELLENBEARBEITUNG.

Befehlszeilenname:	-arrowcolor
Name der Datenbank:	arrowColor
Datenbankklasse:	ArrowColor

Legt die Farbe fest, die für den Aufwärts- oder Abwärtspfeil verwendet werden soll, der durch

den Unterbefehl `sortbycolumn` oder `sortbycolumnlist` des mit dem Widget verknüpften Tcl-Befehls in eine Spaltenbezeichnung eingefügt wird. Diese Option ist nur relevant, wenn der Wert der Option `-showarrow` "true" ist. Der Standardwert hängt vom Fenstersystem im Tablelist-Paket und vom aktuellen Theme in Tablelist_tile ab. Wenn das Fenstersystem beispielsweise `x11` ist, lautet die Standardfarbe für den Pfeil `black`. Das Gleiche gilt für einige Tile-Themes wie `alt`, `clam` und `default`. Beim Fenstersystem `win32` ist die Standardfarbe für Pfeile `#595959` für Windows 10 und höher, `#569bc0` für Windows Vista, 7 und 8, `#aca899` für Windows XP und eine leere Zeichenkette für ältere Windows-Versionen. Schließlich hängt beim Fenstersystem `aqua` auf dem Macintosh und beim Theme `aqua` die Standardfarbe der Pfeile von der Version des Betriebssystems ab: Ab Version 10.10 (Yosemite) von Mac OS X ist die Standardfarbe der Pfeile `#404040` unter Mac OS X und `#878787` unter macOS 11+, während bei früheren Betriebssystemversionen die Standardfarbe `#777777` ist.

HINWEIS: Wenn die Pfeilfarbe eine leere Zeichenkette ist, übernimmt der Sortierpfeil die Hintergrundfarbe der Kopfbeschriftung, in der er platziert ist. In diesem Fall sollte die Option `-arrowstyle` auf einen der Werte `sunken8x7`, `sunken10x9` oder `sunken12x11` gesetzt werden, wodurch Sortierpfeile mit 3D-Rahmen und vertieftem Relief erzeugt werden.

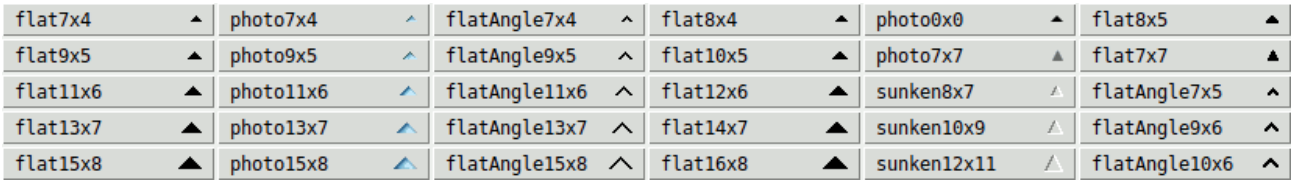
Befehlszeilenname:	-arrowdisabledcolor
Name der Datenbank:	arrowDisabledColor
Datenbankklasse:	ArrowDisabledColor

Legt die Farbe fest, die für den Aufwärts- oder Abwärtspfeil verwendet werden soll, der durch den Unterbefehl `sortbycolumn` oder `sortbycolumnlist` des mit dem Widget verknüpften Tcl-Befehls in eine Spaltenbezeichnung eingefügt wird, wenn der `state` der Tablelist auf `disabled` gesetzt ist. Diese Option ist nur relevant, wenn der Wert der Option `-showarrow` "true" ist. Wenn der Standardwert der Option `-arrowcolor` eine leere Zeichenkette ist, gilt dies auch als Standardwert für die Option `-arrowdisabledcolor`; ansonsten entspricht der Standardwert der letzteren der Standard-Vordergrundfarbe der Kopfzeilenbeschriftungen im Zustand `disabled`.

Befehlszeilenname:	-arrowstyle
Name der Datenbank:	arrowStyle
Datenbankklasse:	ArrowStyle

Legt das Relief, die Form, die Breite und die Höhe des Aufwärts- oder Abwärtspfeils fest, der durch den Unterbefehl `sortbycolumn` oder `sortbycolumnlist` des mit dem Widget verknüpften Tcl-Befehls in eine Spaltenbezeichnung eingefügt wird. Diese Option ist nur relevant, wenn der Wert der Option `-showarrow` "true" ist. Die derzeit unterstützten Werte sind `flat7x4`, `flat7x7`, `flat8x4`, `flat8x5`, `flat9x5`, `flat10x5`, `flat11x6`, `flat12x6`, `flat13x7`, `flat14x7`, `flat15x8`, `flat16x8`, `flatAngle7x4`, `flatAngle7x5`, `flatAngle9x5`, `flatAngle9x6`, `flatAngle10x6`, `flatAngle11x6`, `flatAngle13x7`, `flatAngle15x8`, `photo0x0`, `photo7x4`, `photo7x7`, `photo9x5`, `photo11x6`, `photo13x7`, `photo15x8`, `sunken8x7`, `sunken10x9` und `sunken12x11`, wie in der folgenden Abbildung

dargestellt:



Pfeilstile

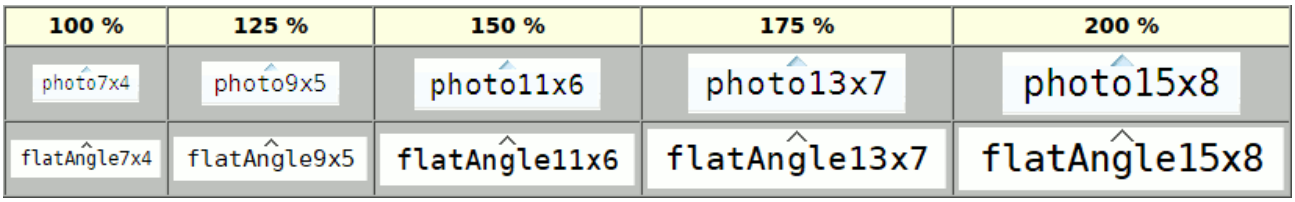
Während die Sortierpfeile der Stile `flat*` und `sunken*` aus Bitmaps erstellt werden, verwenden die Pfeilstile `photo7x4`, `photo9x5`, `photo11x6`, `photo13x7` und `photo15x8` GIF-Bilder, die den nativen Sortierpfeilen unter Windows Vista, 7 und 8 sehr ähnlich sehen. Wenn einer dieser Stile verwendet wird, haben die Optionen `-arrowcolor` und `-arrowdisabledcolor` keine Wirkung.

Der Pfeilstil `photo0x0` verwendet SVG-Bilder, die sich *automatisch* entsprechend der DPI-Skalierung des Displays und dem durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk anpassen. Dies ist besonders praktisch, wenn Androwish auf einem Tablet oder Smartphone ausgeführt wird. Dieser Pfeilstil wird nur unterstützt, wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) beträgt oder 8.6 und die Erweiterung "tksvg" im Interpreter geladen werden kann. Die Sortierpfeile dieses Pfeilstils ähneln stark denen der Stile `flat8x4`, `flat10x5`, `flat12x6`, `flat14x7` und `flat16x8`, mit dem Unterschied, dass sie mit Anti-Aliasing gerendert werden und unbegrenzt skalierbar sind.

Schließlich verwendet der Pfeilstil `photo7x7` PNG-Bilder, die in Aussehen und Verhalten den nativen Sortierpfeilen in Mac OS X Aqua-Versionen vor 10.10 (Yosemite) sehr ähnlich sind. Dieser Pfeilstil wird nur unterstützt, wenn die Tk-Version entweder mindestens 8.6 (mit integrierter PNG-Unterstützung) ist oder 8.5 und das Paket `img::png` in den Interpreter geladen werden kann. Bei Verwendung dieses Pfeilstils haben die Optionen `-arrowcolor` und `-arrowdisabledcolor` keine Wirkung, doch aufgrund der in den PNG-Bildern enthaltenen Transparenzinformationen passen die Pfeile ihre Farbe automatisch an die verschiedenen Zustände der Kopfzeilenbeschriftungen an, genau wie die nativen Sortierpfeile.

Der Standardwert hängt vom Fenstersystem im Tablelist-Paket und vom aktuellen Theme in Tablelist_tile ab. Unter X11 und in einigen Tile-Themes ist der Standard-Pfeilstil `photo0x0`, sofern SVG-Unterstützung vorhanden ist, und andernfalls (abhängig vom Skalierungsgrad des Bildschirms) einer der folgenden: `flat8x4`, `flat10x5`, `flat12x6`, `flat14x7` oder `flat16x8` (siehe die 4. Spalte der obigen Abbildung). Unter Windows 10 und höher ist die Standardeinstellung `flatAngle7x4`, `flatAngle9x5`, `flatAngle11x6`, `flatAngle13x7`, oder `flatAngle15x8` (dargestellt in der dritten Spalte), und unter Windows Vista, 7 und 8 wählt Tablelist standardmäßig `photo7x4`, `photo9x5`, `photo11x6`, `photo13x7` oder `photo15x8` (siehe zweite Spalte der Abbildung), je nach Skalierungsstufe des Bildschirms. Im Classic-Theme von Windows Vista, 7 und 8 ist der Standard-Pfeilstil einer der in der ersten Spalte der Abbildung gezeigten Werte. Schließlich hängt beim Fenstersystem `aqua` auf dem Macintosh und beim Theme `aqua` der Standardpfeilstil von der Version des Betriebssystems ab: Ab Version 10.10 (Yosemite) von Mac OS X ist der Standard-Pfeilstil `flatAngle7x4`, während bei früheren Betriebssystemversionen der Standard `photo7x7` ist, sofern PNG-Unterstützung vorhanden ist, und andernfalls `flat7x7`.

Unter Windows Vista, 7, 8 und 10+ werden die Sortierpfeile horizontal zentriert in den Kopfzeilen angezeigt, direkt unter deren oberem Rand:



Pfeil-Stile Vista

Befehlszeilenname:	-autofinishediting
Name der Datenbank:	autoFinishEditing
Datenbankklasse:	AutoFinishEditing

Legt einen booleschen Wert fest, der steuert, ob die interaktive Zellbearbeitung mithilfe einer Tile-Combobox, eines BWidget-ComboBox-Widgets, eines IWidgets-Combobox-Widgets, eines Oakley-Kombinationsfelds, eines Tk-Core-Menüknopfs oder eines Tile-Menüknopf-Widgets automatisch durch die Auswahl eines Eintrags aus dem Kombinationsfeld oder aus dem mit dem Menüknopf verknüpften Menü beendet wird. Der Standardwert ist `0`.

Befehlszeilenname:	-autoscan
Name der Datenbank:	autoScan
Datenbankklasse:	AutoScan

Legt einen booleschen Wert fest, der steuert, ob das automatische Scrollen ausgelöst wird, wenn die Maus das Tablelist-Fenster verlässt, während die erste Maustaste gedrückt ist. Der Standardwert ist `1`, was bedeutet, dass das automatische Scrollen aktiviert ist, genau wie beim Tk-Listbox-Widget. Wenn Sie das TkDND-Paket oder eine andere Implementierung für Ziehen und Ablegen verwenden, empfiehlt es sich jedoch, diese Option auf `0` zu setzen, um Konflikte zwischen der Drag-Operation und dem automatischen Scrollen zu vermeiden.

HINWEIS: Ab Tablelist-Version 5.12 ist diese Option obsolet geworden, da die Vorgabebindungen nun das oben erwähnte automatische Scrollen unterdrücken, wenn die Datenbereich-Komponente der Tablelist über den Befehl `tkdnd::drag_source register` oder den BWidget-Befehl `DragSite::register` als Ziehquelle für Maustaste 1 registriert wurde oder die Option `-customdragsource` der Tablelist auf "true" gesetzt wurde.

Befehlszeilenname:	-button2window
Name der Datenbank:	button2Window
Datenbankklasse:	Button2Window

Legt den Pfadnamen des Fensters fest, zu dem die Vorgabebindungen für die Bindungs-Tags `TablelistBody` und `TablelistEdit` die Ereignisse `<Button-2>`, `<B2-Motion>` und `<ButtonRelease-2>` über `event generate` umleiten sollen, wenn sich das Fokusfenster auf dem Bildschirm, der das Tablelist-Widget enthält, außerhalb der Tablelist befindet. Die

Standardeinstellung ist eine leere Zeichenkette, was bedeutet, dass Ereignisse der Maustaste 2, die von einem Widget mit einem der beiden oben genannten Bindungs-Tags empfangen werden, zu einem Durchlauf innerhalb der Tablelist oder ihres Bearbeitungsfensters führen, unabhängig davon, ob sich der Fokus innerhalb oder außerhalb des Tablelist-Widgets befindet.

HINWEIS 1: Wenn sich die Tablelist in einem scrollutil::scrollableframe befindet und Sie diese Option auf den Pfadnamen des umgebenden Toplevel-Fensters setzen, werden die oben genannten Ereignisse der Maustaste 2 über dem Datenbereich der Tablelist wie folgt behandelt:

- Wenn der Fokus innerhalb des "tablelist"-Widgets liegt, lösen die Ereignisse ein Scannen des Inhalts der "tablelist" oder ihres Bearbeitungsfensters aus, und es erfolgt keine weitere Verarbeitung der Ereignisse.
- Wenn der Fokus außerhalb des Tablelist-Widgets liegt, findet kein Scannen innerhalb des Tablelist-Datenbereichs oder des Bearbeitungsfensters statt. Stattdessen wird das Ereignis mithilfe des Befehls `event generate` an das Toplevel-Fenster weitergeleitet. Dies führt wiederum dazu, dass der Inhalt des "ScrollableFrame" gescannt wird.

HINWEIS 2: Wenn Sie das "tablelist"-Widget an den Befehl `scrollutil::adaptBtn2EventHandling` übergeben, setzt dieser Befehl diese Option auf den Pfadnamen des umgebenden Toplevel-Fensters.

Befehlszeilenname:	-collapsecommand
Name der Datenbank:	collapseCommand
Datenbankklasse:	CollapseCommand

Legt einen Tcl-Befehl fest, der beim Ausblenden einer Zeile einer als Baum-Widget verwendeten Tablelist (mithilfe des Unterbefehls `collapse` oder `collapseall`) aufgerufen werden soll. Der angegebene Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet, bevor die untergeordneten Elemente der betreffenden Zeile ausgeblendet werden.

Befehlszeilenname:	-colorizecommand
Name der Datenbank:	colorizeCommand
Datenbankklasse:	ColorizeCommand

Legt einen Tcl-Befehl fest, mit dem die Farben beliebiger Zeichenbereiche innerhalb der Zellen der aktuell sichtbaren Zeilen geändert werden können. Immer wenn das Tablelist-Paket die verschiedenen Vorder- und Hintergrundfarben (einschließlich der streifen- und auswahlspezifischen) anzeigen muss, durchläuft es die Zellen der Zeilen im Daten- und Kopfbereich, die derzeit im Tablelist-Fenster sichtbar sind, und stellt die Farben dar, indem es den Text-Widget-Bereichen von Haupt- und Kopfzeilen, aus denen sich der formatierte Inhalt der Zellen zusammensetzt, entsprechende Tags hinzufügt. Nach der Aktivierung der Farben für eine Zelle ruft der Tablelist-Code den als Wert dieser Konfigurationsoption angegebenen Befehl (vorausgesetzt, es handelt sich um eine nicht leere Zeichenkette) im globalen Geltungsbereich und mit den folgenden, für die gerade verarbeitete Zelle im Daten- oder Kopfbereich

spezifischen Argumenten auf:

- der Pfadname des Tablelist-Widgets;
- der Pfadname des Text-Widgets für den Datenbereich oder den Kopfbereich der Tablelist;
- der vollständige Schlüssel des Eintrags, der das gerade verarbeitete Element enthält (beachten Sie, dass dieser bei einem Eintrag des Datenbereichs mit dem Präfix `k` und bei einem Eintrag des Kopfbereichs mit `hk` beginnt);
- den numerischen Index der Zeile oder der Kopfzeile dieses Eintrags;
- der numerische Index der Spalte der Zelle;
- der Text-Index (in der Form `line.char`) des ersten Zeichens `\t`, das die Zelle im zugrunde liegenden Text-Widget abgrenzt;
- der Textindex (in der Form `line.char`) des zweiten Zeichens `\t`, das die Zelle im zugrunde liegenden Text-Widget abgrenzt;
- der boolesche Wert `0` oder `1`, der angibt, ob die gerade verarbeitete Zelle in einem Streifen enthalten ist (siehe z. B. die Beschreibung der Option `-stripebackground`);
- der boolesche Wert `0` oder `1`, der angibt, ob die gerade verarbeitete Zelle derzeit ausgewählt ist.

Daraus folgt, dass der Wert dieser Konfigurationsoption in der Regel der Name einer Prozedur ist, wie im folgenden Beispiel gezeigt:

```
proc myColorizeCmd {tbl textWidget key row col tabIdx1 tabIdx2 inStripe selected} {
    ... }
```

HINWEIS 1: Um die Farben bestimmter Teile des angezeigten Inhalts einer Zelle zu ändern, muss der als Wert dieser Konfigurationsoption angegebene Befehl Text-Widget-Tags verwenden, die die gewünschten Hintergrund- und/oder Vordergrundfarben haben. Neben `-background` und `-foreground` kann der Befehl auch einige andere Tag-Konfigurationsoptionen verwenden, wie beispielsweise `-overstrike` und `-underline`. Andererseits beachten Sie, dass die Verwendung einiger anderer Tag-Konfigurationsoptionen (insbesondere `-elide` und `-font`) zu Konflikten mit der Art und Weise führen kann, wie Tablelist die Elemente darstellt, was wiederum verschiedene Layoutprobleme im Datenbereich oder im Kopfbereich Ihres Tablelist-Widgets zur Folge haben kann!

HINWEIS 2: Da die mehrzeiligen Elemente mithilfe eingebetteter Nachrichten-Widgets angezeigt werden, gibt es derzeit keine Möglichkeit, die Farben der Textbereiche innerhalb mehrzeiliger Zellen zu ändern.

Befehlszeilenname:	-columns
Name der Datenbank:	columns
Datenbankklasse:	Columns

Legt die Breiten, Überschriften und Ausrichtungen der Spalten fest. Der Wert der Option muss eine Liste in folgender Form sein:

```
width title ?alignment? width title ?alignment? ...
```

Jeder `width`-Wert muss eine Zahl sein. Ein positiver Wert legt die Spaltenbreite in Zeichen durchschnittlicher Größe der Widget-Schriftart fest. Wenn `width` negativ ist, wird sein

Absolutwert als Spaltenbreite in Pixeln interpretiert. Schließlich legt der Wert Null fest, dass die Spaltenbreite gerade so groß sein soll, dass alle Elemente der Spalte, einschließlich der Spaltenüberschrift, Platz finden (jedoch nicht größer als die durch die Spaltenkonfigurationsoption `-maxwidth` angegebene maximale Breite). In allen drei Fällen ist die effektive Breite der Spalte etwas größer, da links und rechts der Spalte automatisch Außenabstände erstellt werden.

Jedes `title` legt den Text fest, der in der Überschrift der Spalte angezeigt werden soll, und kann optional im nächsten Listenelement von einem `alignment` gefolgt werden, das festlegt, wie die Elemente der Spalte ausgerichtet werden sollen. Jedes `alignment` muss einer der folgenden Werte sein: `left`, `right` oder `center`. Der Standardwert ist `left`. Das `alignment` bezieht sich ebenfalls auf den Titel der Spalte, wenn für diese Spalte nicht die Option `-labelalign` festgelegt wurde oder deren Wert eine leere Zeichenkette ist.

Der Standardwert dieser Option ist eine leere Liste, was festlegt, dass das Widget anfangs keine Spalten hat.

HINWEIS: Spalten, die eine Breite von Null aufweisen, werden als Spalten mit **dynamischer Breite** bezeichnet. Im Allgemeinen sind sie benutzerfreundlicher als ihre Gegenstücke mit **statischer Breite**, da ihre Breite automatisch an den Inhalt angepasst wird. Wenn die Option `-displayondemand` auf "true" gesetzt ist (was die Standardeinstellung ist), schneiden die Spalten mit statischer Breite beim Einfügen und Sortieren von Elementen deutlich besser ab als die Spalten mit dynamischer Breite.

Befehlszeilenname:	-columntitles
Name der Datenbank:	columnTitles
Datenbankklasse:	ColumnTitles

Diese Option bietet eine vereinfachte Möglichkeit, Spalten in einer Tablelist mit dynamischer Breite und linksbündiger Ausrichtung festzulegen. Ihr Wert wird als Liste von Spaltenüberschriften interpretiert. Standardmäßig ist dies eine leere Liste.

Im einfachsten Fall, in dem noch keine Spalten angegeben wurden, ist das Setzen dieser Option auf den in der Liste angegebenen Wert

```
title title ...
```

gleichbedeutend mit dem Setzen der Option `-columns` auf den in der Liste angegebenen Wert

```
0 title left 0 title left ...
```

Wenn die Spalten bereits festgelegt wurden, aktualisiert diese Option deren Titel (so viele wie möglich), und wenn die Anzahl der Elemente ihres Werts größer ist als die Anzahl der Spalten, werden die verbleibenden Elemente als Titel für zusätzliche Spalten mit dynamischer Breite und linksbündiger Ausrichtung verwendet. Wenn das Widget beispielsweise 3 Spalten hat und der Wert der Option eine Liste der Länge 5 ist, aktualisiert die Option die Überschriften der 3 Spalten und fügt 2 neue Spalten mit dynamischer Breite und linksbündiger Ausrichtung an, deren Überschriften die letzten 2 Elemente der Liste sind. Wenn das Widget über 3 Spalten verfügt und die Option nur 2 Texte festlegt, werden lediglich die Titel der ersten beiden Spalten aktualisiert.

Befehlszeilenname:	-customdragsource
Name der Datenbank:	customDragSource
Datenbankklasse:	CustomDragSource

Legt einen booleschen Wert fest, der angibt, ob das Tablelist-Widget eine Ziehquelle für eine andere Implementierung des Ziehen-und-Ablegen-Verfahrens ist, die weder TkDND noch der in BWidget enthaltene Rahmen für das Ziehen-und-Ablegen-Verfahren ist. Ist der Wert "true", führen die Vorgabebindungen eine automatische drag-freundliche Verarbeitung der Auswahl durch und unterdrücken das automatische Scrollen, wenn die Maus bei gedrückter Taste 1 das Tablelist-Fenster verlässt. Der Standardwert ist `0`.

Befehlszeilenname:	-displayondemand
Name der Datenbank:	displayOnDemand
Datenbankklasse:	DisplayOnDemand

Legt einen booleschen Wert fest, der steuert, wie die Texte neu eingefügter Zeilen im Datenbereich der Tablelist im zugrunde liegenden Text-Widget angezeigt werden. Wenn der Wert "true" ist, werden die Texte der neuen Zeilen erst dann in das Text-Widget des Datenbereichs eingefügt, wenn die entsprechenden Zellen sichtbar werden (z. B. infolge von horizontalem oder vertikalem Scrollen). Andernfalls werden die Texte der neuen Zeilen unmittelbar nach dem Einfügen der Zeilen im zugrunde liegenden Text-Widget angezeigt. Der Standardwert ist `1`.

HINWEIS 1: Der Standardwert "true" dieser Option beschleunigt das Einfügen und Sortieren von Elementen erheblich, insbesondere bei Spalten mit fester Breite, da das Abschneiden von Elementen, die nicht in ihre Zellen passen, auf den Zeitpunkt verschoben wird, zu dem sie im Text-Widget des Datenbereichs angezeigt werden. Andererseits führt diese bedarfsgesteuerte Anzeige zu gewissen Leistungseinbußen beim Scrollen, insbesondere wenn das Tablelist-Widget viele Spalten enthält. Um das Scrollen zu beschleunigen, sollten Sie in Erwägung ziehen, diese Option auf "false" zu setzen.

HINWEIS 2: Neu eingefügte Kopfzeilen werden unabhängig vom Wert dieser Option sofort im zugrunde liegenden Text-Widget angezeigt.

Befehlszeilenname:	-editendcommand
Name der Datenbank:	editEndCommand
Datenbankklasse:	EditEndCommand

Legt einen Tcl-Befehl fest, der bei normalem Abschluss der interaktiven Bearbeitung des Zellinhalts aufgerufen werden soll, wenn sich der endgültige Text des für die Bearbeitung verwendeten temporären eingebetteten Widgets von seinem ursprünglichen Text unterscheidet. Der Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets, den Zeilen- und Spaltenindizes der Zelle sowie dem endgültigen Inhalt des Bearbeitungsfensters verkettet; das

resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert wird nach dem Löschen des temporären eingebetteten Fensters zum neuen Inhalt der Zelle. Der Hauptzweck dieses Skripts besteht darin, eine abschließende Validierung des Inhalts des Bearbeitungsfensters durchzuführen. Weitere Informationen zum Aufruf des oben genannten Befehls finden Sie in der Beschreibung der Option `-forceeditendcommand` sowie im Abschnitt INTERAKTIVE ZELLENBEARBEITUNG, der Einzelheiten zum Bearbeitungsprozess enthält.

Befehlszeilenname:	-editendonfocusout
Name der Datenbank:	editEndOnFocusOut
Datenbankklasse:	EditEndOnFocusOut

Legt einen booleschen Wert fest, der steuert, ob die interaktive Zellbearbeitung automatisch beendet wird, wenn das Tablelist-Widget den Fokus verliert. Wenn diese Option auf "true" gesetzt ist, löst der Verlust des Fokus während der interaktiven Zellenbearbeitung automatisch die Ausführung des Unterbefehls `finishediting` aus. Die Standardeinstellung ist `0`.

Befehlszeilenname:	-editendonmodclick
Name der Datenbank:	editEndOnModClick
Datenbankklasse:	EditEndOnModClick

Legt einen booleschen Wert fest, der steuert, ob im Zusammenhang mit dem Beenden der interaktiven Zellenbearbeitung die Ereignisse `<Shift-Button-1>` und `<Control-Button-1>` (die üblicherweise zum Erweitern bzw. Umschalten der Auswahl verwendet werden) genauso behandelt werden wie `<Button-1>`. Der Standardwert ist `1`, was bedeutet, dass die interaktive Zellbearbeitung automatisch beendet wird, wenn mit der linken Maustaste an einer beliebigen Stelle im Datenbereich der Tablelist außerhalb der gerade bearbeiteten Zelle geklickt wird, unabhängig davon, ob die Taste `Shift` oder `Control` gedrückt ist oder nicht.

Befehlszeilenname:	-editselectedonly
Name der Datenbank:	editSelectedOnly
Datenbankklasse:	EditSelectedOnly

Legt einen booleschen Wert fest, der steuert, ob die interaktive Zellbearbeitung gestartet wird, wenn die linke Maustaste in einer bearbeitbaren Zelle gedrückt wird. Ist dieser Wert "true", wird die Bearbeitung nur gestartet, wenn die Zelle zuvor (interaktiv oder programmgesteuert) ausgewählt wurde. In diesem Fall wird durch einen ersten Linksklick in der Regel zunächst nur die Zelle (oder deren Zeile, abhängig vom Wert der Option `-selecttype`) ausgewählt, und ein zweiter Mausklick startet die Bearbeitungssitzung. Der Standardwert ist `0`, was bedeutet, dass die Bearbeitung unabhängig davon gestartet wird, ob die Zelle ausgewählt ist oder nicht.

Befehlszeilenname:	-editstartcommand
Name der Datenbank:	editStartCommand
Datenbankklasse:	EditStartCommand

Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn die interaktive Bearbeitung des Inhalts einer Zelle gestartet wird. Der Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets, den Zeilen- und Spaltenindizes der Zelle sowie dem in der Zelle angezeigten Text verkettet; das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert wird zum Anfangsinhalt des temporären eingebetteten Widgets, das für die Bearbeitung verwendet wird. Der Hauptzweck dieses Skripts besteht darin, Validierungen für den Inhalt des Bearbeitungsfensters zu definieren. Einzelheiten zum Bearbeitungsprozess finden Sie im Abschnitt INTERAKTIVE ZELLENBEARBEITUNG.

Befehlszeilenname:	-expandcommand
Name der Datenbank:	expandCommand
Datenbankklasse:	ExpandCommand

Legt einen Tcl-Befehl fest, der beim Erweitern einer Zeile einer als Baum-Widget verwendeten Tablelist (mithilfe des Unterbefehls `expand` oder `expandall`) aufgerufen werden soll. Der angegebene Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Index der betreffenden Zeile verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet, bevor die untergeordneten Elemente der betreffenden Zeile angezeigt werden.

HINWEIS: Es ist gängige Praxis, den als Wert dieser Option angegebenen Befehl zu verwenden, um die untergeordneten Elemente der Zeile einzufügen, die gerade erweitert werden soll, sofern diese noch keine untergeordneten Elemente enthält. Beispielsweise verwendet das Demo-Skript Directory Viewer den folgenden Befehl:

```
proc expandCmd {tbl row} {
    if {[$tbl childcount $row] == 0} {
        # Get the name of the directory whose leaf name is
        # displayed in the first cell of the specified row
        set dir [$tbl rowattrib $row pathName]

        # Display the content of the directory $dir
        # as child items of the one identified by $row
        putContents $dir $tbl $row
    }

    # The rest is just eye candy:
    if {[$tbl childcount $row] != 0} {
        # Update the image displayed in the row's first cell
        $tbl cellconfigure $row,0 -image openFolderImg
    }
}
```

Befehlszeilenname:	-forceeditendcommand
Name der Datenbank:	forceEditEndCommand
Datenbankklasse:	ForceEditEndCommand

Legt einen booleschen Wert fest, der den Aufruf des durch die Option `-editendcommand` angegebenen Befehls steuert. Ist dieser Wert "true", wird der Befehl bei normalem Beenden des Bearbeitungsvorgangs aufgerufen, selbst wenn der endgültige Text des für die Bearbeitung verwendeten temporären eingebetteten Widgets mit seinem ursprünglichen Text übereinstimmt, und er wird auch aufgerufen, wenn die interaktive Zellbearbeitung abgebrochen wird (im letzteren Fall ist der als letztes Argument an den Befehl übergebene Text der ursprüngliche Inhalt des Bearbeitungsfensters und nicht dessen endgültiger Inhalt). Der Standardwert dieser Option ist `0`, was bedeutet, dass der Befehl bei normalem Abschluss des Bearbeitungsvorgangs nur dann aufgerufen wird, wenn sich der endgültige Text des temporären eingebetteten Widgets von seinem ursprünglichen Text unterscheidet. Einzelheiten zum Bearbeitungsprozess finden Sie im Abschnitt INTERAKTIVE ZELLENBEARBEITUNG.

Wenn der Wert der Option "true" ist und kein Wert für die Option `-editendcommand` angegeben wurde, wird bei normalem Beenden des Bearbeitungsvorgangs der neue Inhalt der Zelle auf den im Bearbeitungsfenster enthaltenen Text gesetzt, auch wenn dieser nicht interaktiv geändert wurde (sondern möglicherweise durch den Befehl zurückgegeben wurde, der durch die Option `-editstartcommand` vorgegeben wurde).

Wenn Sie diese Option auf "true" setzen, können Sie eine beliebige Aktion ausführen, sobald die interaktive Zellbearbeitung abgeschlossen ist. Es reicht nicht aus, einfach ein Skript an das Ereignis `<Destroy>` des temporären, für die Bearbeitung verwendeten eingebetteten Widgets zu binden, da dieses Widget unter verschiedenen Umständen automatisch gelöscht und neu erstellt werden kann. Alternativ können Sie die virtuellen Ereignisse `<<TablelistCellUpdated>>` und `<<TablelistCellRestored>>` verwenden, die jeweils von den Unterbefehlen `finishediting` und `canceledlting` generiert werden.

Befehlszeilenname:	-fullseparators
Name der Datenbank:	fullSeparators
Datenbankklasse:	FullSeparators

Legt einen booleschen Wert fest, der bestimmt, ob sich die Trennlinien (sofern vorhanden) bis zum unteren Rand des Datenbereichs erstrecken sollen. Der Standardwert ist `0`. Das bedeutet, dass die Höhe der Trennlinien an den Inhalt des Widgets angepasst wird, d. h., sie reichen nicht bis zum unteren Rand des Datenbereichs, wenn unterhalb der letzten Zeile des Widgets noch vertikaler Freiraum vorhanden ist.

Befehlszeilenname:	-height
Name der Datenbank:	height

Datenbankklasse:	Height

Legt die gewünschte Höhe für den Datenbereich der Tablelist fest, gemessen in Zeichen der Schriftart, die durch die Option `-font` vorgegeben ist. Dies entspricht gleichzeitig der gewünschten Höhe in Zeilen, vorausgesetzt, dass keine spalten-, zeilen- oder zellspezifischen Schriftarten, mehrzeilige Elemente oder eingebettete Bilder oder Fenster dazu führen, dass die Höhe eines der Hauptinhaltselemente von der Höhe abweicht, die der oben genannten Schriftart entspricht. Wenn der Wert der Option null oder kleiner ist, wird die gewünschte Höhe des Hauptbereichs in Zeichen der Widget-Schriftart mit der Anzahl der viewable Einträge im Datenbereich synchron gehalten; auch hier gilt: Wenn alle Einträge im Datenbereich der Tablelist die der Schriftart des Widgets entsprechende Höhe haben, bedeutet dies, dass die gewünschte Höhe des Datenbereichs gerade so groß gemacht wird, dass alle sichtbaren Einträge des Datenbereichs Platz finden.

HINWEIS: Eingebettete Bilder und Fenster führen häufig dazu, dass die Zeilen der Tablelist höher werden, was zu einer Diskrepanz zwischen dem Wert dieser Option und der Anzahl der im Fenster des Datenbereichs sichtbaren Zeilen führt. In den meisten Fällen können Sie dieses Problem umgehen, indem Sie den Wert der Option `-font` des Widgets speichern, diese Option auf eine ausreichend große Schriftart einstellen und anschließend die Zeilenkonfigurationsoption `-font` jedes Eintrags im Daten- und Kopfbereich auf den gespeicherten Wert der Schriftart des Widgets einstellen.

Befehlszeilenname:	-inactiveselectbackground
Name der Datenbank:	inactiveSelectBackground
Datenbankklasse:	Foreground

Legt die Hintergrundfarbe fest, die für die Auswahl im Datenbereich der Tablelist verwendet werden soll, wenn diese keinen Fokus hat. Ist der Wert leer, wird der Wert der Option **-selectbackground** auf der Ebene widget, column, row oder cell verwendet, wenn der Datenbereich des Widgets keinen Fokus hat.

Der Standardwert hängt vom Fenstersystem im Tablelist-Paket und vom aktuellen Theme in Tablelist_tile ab: Im Tablelist-Paket ist er bei den Fenstersystemen `aqua` und `win32` nicht leer und bei `x11` eine leere Zeichenkette; in Tablelist_tile ist er für die Themes `aqua`, `clam`, `vista`, `winnative` und `xpnative` nicht leer, bei allen anderen Themes hingegen eine leere Zeichenkette. Die Wahl des Standardwerts stellt sicher, dass die Auswahl auch dann in den nativen Farben angezeigt wird, wenn der Fokus nicht auf dem Datenbereich liegt.

Befehlszeilenname:	-inactiveselectforeground
Name der Datenbank:	inactiveSelectForeground
Datenbankklasse:	Background

Legt die Vordergrundfarbe fest, die für die Auswahl im Datenbereich der Tablelist verwendet

werden soll, wenn diese keinen Fokus hat. Diese Option ist nur relevant, wenn der Wert der Option **-inactiveselectbackground** nicht leer ist; in diesem Fall wird, wenn der Wert dieser Option eine leere Zeichenkette ist, die Option **-selectforeground** auf der Ebene widget (und nur dort) verwendet, wenn der Datenbereich der Tablelist keinen Fokus hat.

Der Standardwert hängt vom Fenstersystem im Tablelist-Paket und vom aktuellen Theme in Tablelist_tile ab: Im Tablelist-Paket ist er bei den Fenstersystemen `aqua` und `win32` nicht leer und bei `x11` eine leere Zeichenkette; in Tablelist_tile ist er für die Themes `aqua`, `clam`, `vista`, `winnative` und `xpnative` nicht leer, bei allen anderen Themes hingegen eine leere Zeichenkette. Die Wahl des Standardwerts stellt sicher, dass die Auswahl auch dann in den nativen Farben angezeigt wird, wenn der Fokus nicht auf dem Datenbereich liegt.

Befehlszeilenname:	-incrarrowtype
Name der Datenbank:	incrArrowType
Datenbankklasse:	IncrArrowType

Legt den Typ des Pfeils fest, der in eine Spaltenbezeichnung eingefügt wird, wenn die Elemente anhand dieser Spalte in aufsteigender Reihenfolge sortiert werden, und zwar mithilfe des Unterbefehls `sortbycolumn` oder `sortbycolumnlist` des Tcl-Befehls, der dem Widget zugeordnet ist. Der Wert dieser Option muss entweder `up` oder `down` lauten. Der Standardwert ist `up`. Diese Option ist nur relevant, wenn der Wert der Option `-showarrow` "true" ist.

Befehlszeilenname:	-instanttoggle
Name der Datenbank:	instantToggle
Datenbankklasse:	InstantToggle

Legt einen booleschen Wert fest, der steuert, ob die interaktive Zellbearbeitung mithilfe eines Tk- oder Tile-Checkbuttons oder eines Toggleswitch-Widgets – sofern sie mit der linken Maustaste gestartet wurde – unmittelbar nach dem automatischen Umschalten des Kontrollzustands des temporär eingebetteten Checkbuttons oder des Schaltzustands des temporär eingebetteten Toggleswitches beendet wird. Der Standardwert ist `0`, was bedeutet, dass ein Mausklick in die Zelle lediglich die Bearbeitungssitzung startet (und dabei automatisch den Markierungsstatus des checkbuttons oder den Schaltzustand des Toggleswitch-Widgets umschaltet), diese jedoch nicht beendet.

Befehlszeilenname:	-itembackground or -itembg
Name der Datenbank:	itemBackground
Datenbankklasse:	Background

Legt die Hintergrundfarbe fest, die bei der Anzeige der Einträge im Daten- und Kopfbereich verwendet werden soll, d. h. die Hintergrundfarbe der mit Einträgen gefüllten Fläche des

Datenbereichs und die der mit Einträgen gefüllten Fläche des Kopfbereichs (wenn vorhanden). Der Standardwert ist eine leere Zeichenkette, was bedeutet, dass diese Bereiche die durch die Konfigurationsoption `-background` festgelegte Hintergrundfarbe übernehmen. Beachten Sie, dass unabhängig vom Wert dieser Option der Leerraum, der möglicherweise rechts neben den Haupt- und Kopfzeilenelementen sowie unterhalb des letzten Hauptelements erscheint, immer die durch die Option `-background` festgelegte Farbe aufweist.

Befehlszeilenname:	-labelactivebackground
Name der Datenbank:	labelActiveBackground
Datenbankklasse:	Foreground

Legt die Option `-activebackground` für die Kopfzeilenbeschriftungen fest, d. h. die Hintergrundfarbe, die verwendet wird, wenn der Mauszeiger über einer Kopfzeilenbeschriftung positioniert ist und der Wert von `tk_strictMotif` "false" ist. Das Paket "Tablelist_tile" unterstützt die Option `-labelactivebackground` nicht.

Befehlszeilenname:	-labelactiveforeground
Name der Datenbank:	labelActiveForeground
Datenbankklasse:	Background

Legt die Option `-activeforeground` für die Kopfzeilenbeschriftungen fest, d. h. die Vordergrundfarbe, die verwendet wird, wenn sich der Mauszeiger über einer Kopfzeilenbeschriftung befindet und der Wert von `tk_strictMotif` "false" ist. Das Paket "Tablelist_tile" unterstützt die Option `-labelactiveforeground` nicht.

Befehlszeilenname:	-labelbackground or -labelbg
Name der Datenbank:	labelBackground
Datenbankklasse:	Background

Legt die Option `-background` für die Kopfzeilenbeschriftungen fest. Diese Option wird nur vom Tablelist-Paket unterstützt, nicht jedoch von Tablelist_tile.

Befehlszeilenname:	-labelborderwidth or -labelbd
Name der Datenbank:	labelBorderWidth
Datenbankklasse:	BorderWidth

Legt die Option `-borderwidth` für die Kopfzeilenbeschriftungen fest. Diese Option unterscheidet sich von der Standardoption `-borderwidth`, die für das Tablelist-Widget selbst definiert ist. Im Paket "Tablelist_tile" hat diese Option einen themespezifischen Standardwert.

HINWEIS: Bitte beachten Sie, dass bei einigen Themes die Einstellung dieser Option auf einen anderen Wert als den Standardwert möglicherweise von "tile" ignoriert wird und somit zu Problemen bei der Ausrichtung führen kann. Dies liegt daran, dass der Rahmen von "tile"-Widgets mit themespezifischen Methoden gezeichnet wird, die nicht immer zu den aus Tk-Widgets bekannten Ergebnissen führen.

Befehlszeilenname:	-labelcommand
Name der Datenbank:	labelCommand
Datenbankklasse:	LabelCommand

Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn die Maustaste 1 über einer der Kopfzeilenbeschriftungen gedrückt und später über derselben Beschriftung wieder losgelassen wird. Sofern der Wert der Option eine nicht leere Zeichenkette ist, wird bei Eintreten des Ereignisses `<ButtonRelease-1>` der Befehl automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Spaltenindex der jeweiligen Beschriftung verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Wenn auf Spaltenebene als Wert der gleichnamigen Option ein anderer nicht leerer Befehl festgelegt wird, wird dieser spaltenspezifische Befehl anstelle des globalen Befehls verwendet. Wenn der Wert von `state` der Tablelist `disabled` lautet, findet diese Aktion nicht statt. Der häufigste Wert dieser Option ist `tablelist::sortByColumn`; dieser Befehl sortiert die Elemente anhand der Spalte, deren Index ihm als zweites Argument übergeben wurde.

Befehlszeilenname:	-labelcommand2
Name der Datenbank:	labelCommand2
Datenbankklasse:	LabelCommand2

Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn die Maustaste 1 zusammen mit der Taste `Shift` über einer der Kopfzeilenbeschriftungen gedrückt und später über derselben Beschriftung wieder losgelassen wird. Sofern der Wert der Option eine nicht leere Zeichenkette ist, wird wenn das Ereignis `<ButtonRelease-1>` eintritt, der Befehl automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Spaltenindex der jeweiligen Beschriftung verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Wenn auf Spaltenebene ein anderer nicht leerer Befehl als Wert der gleichnamigen Option angegeben wird, wird dieser spaltenspezifische Befehl anstelle des globalen Befehls verwendet. Wenn der Wert von `state` der Tablelist `disabled` lautet, findet diese Aktion nicht statt. Der häufigste Wert dieser Option ist `tablelist::addToSortColumns`; dieser Befehl fügt den als zweites Argument übergebenen Spaltenindex zur Liste der Sortierspalten hinzu und sortiert die Elemente anhand der durch die geänderte Liste angegebenen Spalten.

Befehlszeilenname:	-labeldisabledforeground
Name der Datenbank:	labelDisabledForeground

Datenbankklasse:	DisabledForeground

Legt die Option `-disabledforeground` für die Kopfzeilenbeschriftungen fest, d. h. die Vordergrundfarbe, die für die Beschriftungen verwendet werden soll, wenn die Option `state` der Tablelist auf `disabled` gesetzt ist. Das Paket "Tablelist_tile" unterstützt die Option `-labeldisabledforeground` nicht.

Befehlszeilenname:	-labelfont
Name der Datenbank:	labelFont
Datenbankklasse:	Font

Legt die Option `-font` für die Kopfzeilenbeschriftungen fest. Im Paket "Tablelist_tile" hat diese Option einen themespezifischen Standardwert.

Befehlszeilenname:	-labelforeground or -labelfg
Name der Datenbank:	labelForeground
Datenbankklasse:	Foreground

Legt die Option `-foreground` für die Kopfzeilenbeschriftungen fest. Im Paket "Tablelist_tile" hat diese Option einen themespezifischen Standardwert.

Befehlszeilenname:	-labelheight
Name der Datenbank:	labelHeight
Datenbankklasse:	Height

Legt die Option `-height` für die Kopfzeilenbeschriftungen fest. Diese Option wird nur vom Tablelist-Paket unterstützt, nicht jedoch von Tablelist_tile.

Befehlszeilenname:	-labelpady
Name der Datenbank:	labelPadY
Datenbankklasse:	Pad

Im Tablelist-Paket legt diese Option die Konfigurationsoption `-pady` für die Kopfzeilenbeschriftungen fest. Im Tablelist_tile-Paket wird der Wert der Option `-labelpady` den entsprechenden Komponenten des Werts der Konfigurationsoption `-padding` für die Kopfzeilenbeschriftungen zugeordnet, und die Option `-labelpady` hat einen themespezifischen Standardwert.

Befehlszeilenname:	-labelrelief
Name der Datenbank:	labelRelief
Datenbankklasse:	Relief

Legt die Option `-relief` für die Kopfzeilenbeschriftungen fest. Diese Option unterscheidet sich von der Standardoption `-relief`, die für das Tablelist-Widget selbst definiert ist. Der Standardwert ist `raised`.

Befehlszeilenname:	-listvariable
Name der Datenbank:	listVariable
Datenbankklasse:	Variable

Legt den Namen einer globalen Variablen oder den qualifizierten Namen einer Namespace-Variablen fest. Der Wert der Variablen ist eine Liste, die im Datenbereich des Widgets angezeigt werden soll; ändert sich der Wert der Variablen, aktualisiert sich das Widget automatisch, um den neuen Wert widerzuspiegeln. Der Wert der Variablen muss eine gültige Liste sein. Jedes Listenelement entspricht einer Zeile im Datenbereich des Widgets und muss selbst eine gültige Liste sein; ihre Elemente entsprechen den Zellen innerhalb der jeweiligen Zeile. Versuche, eine Variable, deren Wert diese Bedingungen nicht erfüllt, der Option `-listvariable` zuzuweisen, führen zu einem Fehler. Versuche, eine als `-listvariable` verwendete Variable mit `unset` zu löschen, schlagen fehl, führen jedoch nicht zu einer Fehlermeldung.

HINWEIS 1: Um die Effizienz zu steigern, erfolgt die Aktualisierung des Widgets entsprechend einem geänderten Wert der mit dieser Option angegebenen Variablen, wann immer möglich, in einer Leerlaufphase (d. h. wenn keine Ereignisse zu verarbeiten sind). Andererseits erfordern die meisten "tablelist"-Unterbefehle eine *sofortige* Aktualisierung der internen Liste des Widgets entsprechend dem Wert dieser Variable, bevor der betreffende Unterbefehl ausgeführt wird. Wird dies wiederholt durchgeführt, kann dies zu erheblichen Effizienzverlusten führen. Um Leistungsprobleme zu vermeiden, sollten Sie stets versuchen, die Operationen, die den Wert der durch diese Option festgelegten Variablen aufbauen, von anderen Befehlen zu trennen. Anstelle von

```
tablelist::tablelist .tbl ... -listvariable var
set var {}
for {set row 0} {$row < 1000} {incr row} {
    lappend var ...
    .tbl cellconfigure $row,3 -image ...
}
```

Schreiben Sie stattdessen

```
tablelist::tablelist .tbl ... -listvariable var
set var {}
for {set row 0} {$row < 1000} {incr row} {
    lappend var ...
}
for {set row 0} {$row < 1000} {incr row} {
    .tbl cellconfigure $row,3 -image ...
}
```

Die erste der oben genannten Methoden ist ziemlich ineffizient, da sie 1000 Aktualisierungen der internen Liste des Widgets erfordert. Die zweite Methode läuft unvergleichlich schneller, da sie nur eine Synchronisation benötigt (zu Beginn der zweiten Schleife).

HINWEIS 2: Es wird nicht empfohlen, diese Option für eine Tablelist zu setzen, die als Baum-Widget verwendet wird, da das Hinzufügen neuer Elemente zu der als Wert angegebenen Liste dazu führt, dass diese Listenelemente in das Widget eingefügt werden, ohne die interne Struktur des Baums zu berücksichtigen. Es ist kein Problem, wenn Sie die Variable nur zum Lesen aufrufen; für diesen Zweck ist es jedoch effizienter, den Unterbefehl `itemlistvar` anstelle der Option `-listvariable` zu verwenden.

Befehlszeilenname:	-movablecolumns
Name der Datenbank:	movableColumns
Datenbankklasse:	MovableColumns

Legt einen booleschen Wert fest, der festlegt, ob die Spalten interaktiv verschoben werden können. Informationen zum interaktiven Verschieben einer Spalte finden Sie im Abschnitt VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFBESCHRIFTUNGEN weiter unten. Der Standardwert ist `0`.

Befehlszeilenname:	-movablerows
Name der Datenbank:	movableRows
Datenbankklasse:	MovableRows

Legt einen booleschen Wert fest, der festlegt, ob die im Datenbereich der Tablelist angezeigten Zeilen interaktiv verschoben werden können. Weitere Informationen zum interaktiven Verschieben einer Zeile finden Sie im Unterabschnitt LOCAL DRAG & DROP des Abschnitts VORGABE- UND EINZELBINDUNGEN FÜR DEN DATENBEREICH weiter unten. Der Standardwert ist `0`.

HINWEIS: Die Möglichkeit, eine Zeile zu verschieben, ist auf die Datenbereich-Komponente des Widgets beschränkt. Das Verschieben von Kopfzeilen wird nicht unterstützt.

Befehlszeilenname:	-movecolumncursor
Name der Datenbank:	moveColumnCursor

Datenbankklasse:	MoveColumnCursor

Legt den Mauszeiger fest, der beim interaktiven Verschieben einer Spalte verwendet werden soll. Der Standardwert lautet `icon` auf den Fenstersystemen `x11` und `win32` und der native Cursor `closedhand` im Macintosh-Fenstersystem `aqua`.

Befehlszeilenname:	-movecursor
Name der Datenbank:	moveCursor
Datenbankklasse:	MoveCursor

Legt den Mauszeiger fest, der beim interaktiven Verschieben einer Zeile verwendet werden soll. Der Standardwert lautet `hand2` auf den Fenstersystemen `x11` und `win32`, und der native Cursor `pointinghand` im Macintosh-Fenstersystem `aqua`.

Befehlszeilenname:	-populatecommand
Name der Datenbank:	populateCommand
Datenbankklasse:	PopulateCommand

Legt einen Tcl-Befehl fest, der vom Unterbefehl `searchcolumn` aufgerufen werden soll, bevor die untergeordneten Elemente (oder Nachkommen, bei Verwendung der Option `-descend`) einer Zeile der Tablelist überprüft werden, deren untergeordnete Elemente noch nicht eingefügt wurden. Der angegebene Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Es wird erwartet, dass dieses Skript die untergeordneten Elemente der betreffenden Zeile einfügt, ohne den Knoten zu erweitern oder dessen Darstellung in irgendeiner anderen Weise zu verändern.

HINWEIS: Es gibt viele Ähnlichkeiten zwischen dieser Option und `-expandcommand`. Beide Optionen dienen in erster Linie dazu, untergeordnete Elemente nach Bedarf einzufügen. Die wichtigsten Unterschiede zwischen ihnen sind folgende:

1. Der durch die Option `-populatecommand` festgelegte Befehl wird nur für Zeilen aufgerufen, deren untergeordnete Elemente noch nicht eingefügt wurden, während der durch die Option `-expandcommand` festgelegte Befehl immer vor dem Erweitern einer Zeile aufgerufen wird, unabhängig davon, ob die untergeordneten Elemente dieser Zeile bereits im Widget vorhanden sind oder nicht.
2. Der durch die Option `-expandcommand` festgelegte Befehl kann visuelle Änderungen am betreffenden Knoten vornehmen, während die Funktion des durch `-populatecommand` festgelegten Befehls darauf beschränkt ist, die untergeordneten Elemente einzufügen, ohne das Erscheinungsbild des Knotens in irgendeiner Weise zu verändern.

Eine logische Folge des Vorstehenden ist, dass der Wert von `-populatecommand` in der Regel lediglich eine vereinfachte Version des Befehls ist, der durch die Option `-expandcommand` festgelegt wird. Beispielsweise könnte das Demo-Skript Directory Viewer einen Befehl

verwenden, der wie folgt implementiert ist:

```
proc populateCmd {tbl row} {
    # Get the name of the directory whose leaf name is
    # displayed in the first cell of the specified row
    set dir [$tbl rowattrib $row pathName]

    # Display the content of the directory $dir
    # as child items of the one identified by $row
    putContents $dir $tbl $row
}
```

Zudem kann die Option `-expandcommand` auf einen Befehl gesetzt werden, der den durch die Option `-populatecommand` angegebenen Befehl aufruft:

```
proc expandCmd {tbl row} {
    if {[$tbl childcount $row] == 0} {
        populateCmd $tbl $row
    }

    # The rest is just eye candy:
    if {[$tbl childcount $row] != 0} {
        # Update the image displayed in the row's first cell
        $tbl cellconfigure $row,0 -image openFolderImg
    }
}
```

Befehlszeilenname:	-protecttitlecolumns
Name der Datenbank:	protectTitleColumns
Datenbankklasse:	ProtectTitleColumns

Legt einen booleschen Wert fest, der bestimmt, ob die Begrenzung des Titel-Spaltenbereichs beim interaktiven Verschieben einer Spalte vor dem Überschreiten geschützt werden soll. Siehe den Abschnitt VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFBESCHRIFTUNGEN weiter unten für Informationen zum interaktiven Verschieben einer Spalte. Der Standardwert ist `0`; dies legt fest, dass Nicht-Titel-Spalten in den Titel-Spaltenbereich verschoben werden können und umgekehrt.

Befehlszeilenname:	-resizablecolumns
Name der Datenbank:	resizableColumns
Datenbankklasse:	ResizableColumns

Legt einen booleschen Wert fest, der festlegt, ob die Spalten interaktiv in der Größe angepasst werden können. Informationen zur interaktiven Spaltengrößenanpassung finden Sie im Abschnitt VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFBESCHRIFTUNGEN weiter unten. Der Standardwert ist `1`.

Befehlszeilenname:	-resizecursor
Name der Datenbank:	resizeCursor
Datenbankklasse:	ResizeCursor

Legt den Mauszeiger fest, der bei der interaktiven Anpassung der Spaltengrößen verwendet werden soll. Der Standardwert lautet `sb_h_double_arrow` auf den Fenstersystemen `x11` und `win32`, und der native Cursor `resizeleftright` im Macintosh-Fenstersystem `aqua`.

Befehlszeilenname:	-selectfiltercommand
Name der Datenbank:	selectFilterCommand
Datenbankklasse:	SelectFilterCommand

Legt einen Tcl-Befehl fest, der zum Filtern der Zellen verwendet werden kann, die ausgewählt werden, wenn die Vorgabebindungen für die Kopfzeilenbezeichnungen eine spaltenweise Zellenauswahl durchführen. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird dieser Befehl mit dem Pfadnamen des Tablelist-Widgets und dem numerischen Index der jeweiligen Spalte verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Wenn auf Spaltenebene ein anderer nicht leerer Befehl als Wert der gleichnamigen Option angegeben wird, wird dieser spaltenspezifische Befehl anstelle des globalen Befehls verwendet. In beiden Fällen wird erwartet, dass das resultierende Skript eine Liste zurückgibt, die aus den Zeilennummern jener Zellen besteht, die in der betreffenden Spalte enthalten sind und durch die spaltenweise Auswahl der Zellen ausgewählt werden sollen.

Befehlszeilenname:	-selectmode
Name der Datenbank:	selectMode
Datenbankklasse:	SelectMode

Legt einen von mehreren Stilen für die Bearbeitung der Auswahl fest. Der Wert der Option kann beliebig sein, die Vorgabebindungen erwarten jedoch, dass er entweder `single`, `browse`, `multiple` oder `extended` ist. Der Standardwert ist `browse`.

Befehlszeilenname:	-selecttype
Name der Datenbank:	selectType
Datenbankklasse:	SelectType

Legt einen von zwei Auswahltypen für das Tablelist-Widget fest: `row` oder `cell`. Wenn der Auswahltyp `row` ist, wählen die Vorgabebindungen ganze Einträge aus bzw. heben deren Auswahl auf, und die gesamte Zeile, in der sich der Positionscursor befindet, wird als aktiv angezeigt, wenn die Tablelist den Tastaturfokus hat. Wenn der Auswahltyp `cell` ist, wählen die

Vorgabebindungen einzelne Elemente aus bzw. heben deren Auswahl auf, und die einzelne Zelle, in der sich der Positionscursor befindet, wird als aktiv angezeigt, wenn die Tablelist den Tastaturfokus hat. Der Standardwert ist `row`.

Befehlszeilenname:	-setfocus
Name der Datenbank:	setFocus
Datenbankklasse:	SetFocus

Legt einen booleschen Wert fest, der bestimmt, ob ein Klick mit der linken Maustaste an einer beliebigen Stelle im Datenbereich der Tablelist – einschließlich der Trennlinien und der eingebetteten Bilder (genauer gesagt, alle untergeordneten Elemente des Tablelist-Widgets, die das Bindungs-Tag `TablelistBody` tragen), den Fokus auf den Datenbereich des Tablelist-Widgets setzen soll, wenn dessen `state` den Wert `normal` hat. Der Standardwert ist 1.

Befehlszeilenname:	-showarrow
Name der Datenbank:	showArrow
Datenbankklasse:	ShowArrow

Legt einen booleschen Wert fest, der festlegt, ob die Unterbefehle `sortbycolumn` und `sortbycolumnlist` des mit dem Widget verknüpften Tcl-Befehls einen Pfeil, der die Sortierreihenfolge angibt, in die Kopfzeilenbeschriftung(en) der Spalte(n) setzen sollen, die durch ihr erstes Argument festgelegt sind. Der Standardwert ist `1`.

Befehlszeilenname:	-showbusycursor
Name der Datenbank:	showBusyCursor
Datenbankklasse:	ShowBusyCursor

Legt einen booleschen Wert fest, der angibt, ob die Vorgabebindungen bei Vorgängen, deren Ausführung längere Zeit in Anspruch nehmen kann, mithilfe der Unterbefehle `setbusycursor` und `restorecursor` einen für das Fenstersystem spezifischen "Beschäftigt"-Cursor anzeigen sollen. Der Standardwert ist `1`.

Die möglicherweise länger andauernden Vorgänge, die bei aktivierter Option den Mauszeiger vorübergehend verändern, sind folgende:

- die Größe einer Spalte ändern oder eine Spalte verschieben;
- Aufruf des durch die Option `-labelcommand` oder `-labelcommand2` auf Widget- oder Spaltenebene angegebenen Befehls (z. B. Sortieren der Elemente nach einer oder mehreren Spalten);
- eine Zeile ein- oder ausblenden;
- Erweitern, Umschalten oder Aufheben der (Zellen-)Auswahl, wenn die Option `-selectmode` auf `extended` gesetzt wurde;

- Erweiterung der spaltenweisen Auswahl der Zellen.

HINWEIS: Wenn Sie nicht möchten, dass bei potenziell lang andauernden Vorgängen standardmäßig der Lade-Cursor angezeigt wird, können Sie die Standardeinstellung für alle Tablelist-Instanzen überschreiben, indem Sie folgende Zeile einfügen:

```
option add *Tablelist.showBusyCursor 0
```

irgendwo am Anfang Ihres Skripts (bevor Sie irgendwelche Tablelist-Widgets erstellen).

Befehlszeilenname:	-showeditcursor
Name der Datenbank:	showEditCursor
Datenbankklasse:	ShowEditCursor

Legt einen booleschen Wert fest, der angibt, ob der Mauszeiger automatisch in die Form eines Bleistifts wechseln soll, sobald ein Klick mit Maustaste 1 eine Bearbeitungssitzung startet. Der Standardwert lautet `1`.

Unter X11 und Mac OS X/11+ ist der von Tablelist festgelegte Cursor, der bei einem Mausklick die interaktive Zellbearbeitung auslöst, der in Tk integrierte Cursor `pencil`. Da dieser Cursor unter Windows unansehnlich und nicht wirklich brauchbar ist, wird auf dieser Plattform der von Tablelist angezeigte Bearbeitungscursor aus der Datei `pencil.cur` geladen, die sich im Verzeichnis `scripts` des Tablelist-Installationsverzeichnisses befindet. Wenn die Anwendung als Starpack gestartet wurde, das das Tablelist-Paket enthält, wird der Bearbeitungscursor aus einer Kopie dieser Datei geladen, die automatisch von Tablelist im Verzeichnis `Temp` des Benutzers erstellt wurde. Die Datei `pencil.cur` ist eine Kopie der Datei `Ubuntu Handwriting.cur`, die von folgender Adresse heruntergeladen wurde:

http://www.rw-designer.com/cursor-detail/46922

HINWEIS: Wenn Sie nicht möchten, dass standardmäßig der Bearbeitungscursor angezeigt wird, sobald ein Klick mit Maustaste 1 die Bearbeitung auslöst, können Sie die Standardeinstellung für alle Tablelist-Instanzen überschreiben, indem Sie folgende Zeile einfügen:

```
option add *Tablelist.showEditCursor 0
```

irgendwo am Anfang Ihres Skripts (bevor Sie irgendwelche Tablelist-Widgets erstellen).

Befehlszeilenname:	-showhorizseparator
Name der Datenbank:	showHorizSeparator
Datenbankklasse:	ShowHorizSeparator

Legt einen booleschen Wert fest, der steuert, ob direkt unterhalb der letzten Zeile ein horizontaler Trennstrich eingefügt wird, wenn der Wert der Option `-showseparators` "true" und der Wert der Option `-fullseparators` "false" ist, und zwischen der letzten Zeile und dem unteren Rand des Tablelist-Fensters noch freier Platz vorhanden ist. Der Standardwert ist `1`.

HINWEIS: Wenn Sie unter den oben genannten Umständen nicht möchten, dass standardmäßig ein horizontaler Trennstrich angezeigt wird, können Sie die Standardeinstellung für alle

Tablelist-Instanzen überschreiben, indem Sie folgende Zeile einfügen:

```
option add *Tablelist.showHorizSeparator 0
```

irgendwo am Anfang Ihres Skripts (bevor Sie irgendwelche Tablelist-Widgets erstellen).

Befehlszeilenname:	-showlabels
Name der Datenbank:	showLabels
Datenbankklasse:	ShowLabels

Legt einen booleschen Wert fest, der festlegt, ob die Kopfzeilenbeschriftungen angezeigt werden sollen oder nicht. Der Standardwert lautet `1`.

Befehlszeilenname:	-showseparators
Name der Datenbank:	showSeparators
Datenbankklasse:	ShowSeparators

Legt einen booleschen Wert fest, der bestimmt, ob die Spalten durch Rahmen voneinander getrennt werden sollen. Der Standardwert lautet `0`. Die Trennlinien sind im Paket "Tablelist" als dünne Rahmen mit vertiefter Prägung und im Paket "Tablelist_tile" als Tile-Trennlinien-Widgets implementiert. Sie werden an den rechten Rändern der Kopfzeilenbeschriftungen angebracht und nur erstellt, wenn der Wert dieser Option "true" ist. In diesem Fall wird zudem ein horizontaler Trennbalken erstellt und direkt unterhalb der letzten Zeile platziert, vorausgesetzt, der Wert der Option `-showhorizseparator` ist "true", der der Option `-fullseparators` ist "false" und zwischen der letzten Zeile und dem unteren Rand des Tablelist-Fensters noch freier Platz vorhanden ist.

HINWEIS: Tablelist unterstützt keine horizontalen Trennlinien (mit Ausnahme der oben genannten und einer weiteren, die direkt unterhalb der letzten Kopfzeile platziert ist), aber mit Hilfe der unten beschriebenen Option `-stripebackground` lässt sich ein ansprechender Effekt zur Unterscheidung der Zeilen im Datenbereich des Widgets erzielen.

Befehlszeilenname:	-snipstring
Name der Datenbank:	snipString
Datenbankklasse:	SnipString

Legt die Zeichenkette fest, die als Ausschnittanzeige verwendet werden soll, wenn Elemente angezeigt werden, die nicht in ihre Zellen passen. Standardmäßig wird eine Ellipse verwendet (`"..."`).

Befehlszeilenname:	-sortcommand

Name der Datenbank:	sortCommand
Datenbankklasse:	SortCommand

Legt einen Befehl fest, der beim Aufruf des Unterbefehls `sort` des mit dem Tablelist-Widget verknüpften Tcl-Befehls zum Vergleich der Elemente verwendet werden soll. Um zwei Elemente (die jeweils als Listen von Zelleninhalten innerhalb einer Zeile betrachtet werden) während der Operation `sort` zu vergleichen, wird der Befehl automatisch mit den beiden Elementen verkettet und das resultierende Skript ausgewertet. Das Skript sollte eine Ganzzahl zurückgeben, die kleiner als, gleich oder größer als Null ist, je nachdem, ob das erste Element als kleiner als, gleich oder größer als das zweite angesehen werden soll.

Befehlszeilenname:	-spacing
Name der Datenbank:	spacing
Datenbankklasse:	Spacing

Legt den zusätzlichen Abstand fest, der oberhalb und unterhalb jeder Zeile des Widgets sowohl im Datenbereich als auch im Kopfbereich bereitgestellt werden soll. Der Wert dieser Option kann eine der Standardangaben für Bildschirmabstände sein. Der Standardwert ist `0`. Siehe auch die Option `-tight`.

Befehlszeilenname:	-state
Name der Datenbank:	state
Datenbankklasse:	State

Legt einen von zwei Zuständen für das Tablelist-Widget fest: `normal` oder `disabled`. Wenn das Widget deaktiviert ist, dürfen weder Elemente noch Spalten eingefügt, gelöscht, aktualisiert oder verschoben werden; die Elemente, die Kopfzeilenbeschriftungen sowie die Aufwärts- und Abwärtspfeile werden jeweils in den Farben `-disabledforeground`, `-labeldisabledforeground` bzw. `-arrowdisabledcolor` in Farbe dargestellt; die Auswahl kann nicht geändert werden und wird nicht angezeigt (die Auswahlinformationen bleiben jedoch erhalten); die Kopfzeilenbeschriftungen sind vollständig unempfindlich, und es kann keine interaktive Zellenbearbeitung durchgeführt werden. Zudem werden im deaktivierten Zustand alle auf Spalten-, Zeilen- oder Zellenebene festgelegten Farboptionen ignoriert.

Befehlszeilenname:	-stretch
Name der Datenbank:	stretch
Datenbankklasse:	Stretch

Legt die **dehnbaren** Spalten fest, d. h. die Spalten, die bei Bedarf gedehnt werden sollen, um

das Tablelist-Fenster auszufüllen. Der Wert der Option kann `all` oder eine Liste von Spaltenindizes in einer der im Abschnitt SPALTENINDIZES unten beschriebenen Formen sein. Im zweiten Fall werden die angegebenen Spaltenindizes durch ihre numerischen Entsprechungen ersetzt, mit Ausnahme von `end` und `last`, die als dynamische Spaltenindizes betrachtet werden, deren numerisches Äquivalent (d. h. der Index der letzten Spalte der Tablelist) sich während der Programmausführung ändern kann und daher bei jeder Streckung der Spalten neu berechnet wird. Die Liste wird automatisch aktualisiert, sobald Spalten eingefügt, gelöscht oder verschoben werden. Die Anzahl der Pixel, um die eine Spalte gestreckt wird, ist proportional zu ihrer Breite in Pixeln. Der Standardwert dieser Option ist eine leere Liste, was bedeutet, dass keine Spalte gestreckt wird, um den Leerraum zu beseitigen, der möglicherweise rechts neben der Tabelle erscheint. (HINWEIS: Der Leerraum hinter den Kopfzeilenbeschriftungen wird mit einer nicht reagierenden Dummy-Beschriftung gefüllt, die denselben Hintergrund, dieselbe Rahmenbreite und denselben Relief-Effekt wie die "normalen" Kopfzeilenbeschriftungen aufweist.)

HINWEIS: Wenn der Wert der Konfigurationsoption `-width` null oder kleiner ist und die Tablelist dehnbare Spalten enthält, wird die Option `-setgrid` ignoriert. Diese geringfügige Einschränkung hat technische Gründe und ist nur unter X11 relevant.

Befehlszeilenname:	-stripebackground or -stripebg
Name der Datenbank:	stripeBackground
Datenbankklasse:	Background

Legt die Farbe des Hintergrundes fest, die bei der Anzeige der zu einem Streifen gehörenden Einträge des Datenbereichs verwendet werden soll. Jeder Streifen besteht aus der gleichen Anzahl `stripeHeight` aufeinanderfolgender viewable Elemente, entsprechend dem Wert der Konfigurationsoption `-stripeheight`. Die ersten `stripeHeight` sichtbaren Elemente sind "normale" Elemente; darauf folgt ein Streifen, der aus den nächsten `stripeHeight` sichtbaren Elementen besteht, worauf wiederum die gleiche Anzahl "normaler" sichtbarer Elemente folgt, und so weiter. Im Tablelist-Paket und in den meisten von Tablelist_tile unterstützten Themes ist der Standardwert eine leere Zeichenkette, was bedeutet, dass die Streifen die durch die Konfigurationsoption `-background` festgelegte Hintergrundfarbe übernehmen. Wenn Sie Tablelist_tile mit dem `tileqt`-Theme verwenden, wird der Standardwert durch die globale KDE-Option `alternateBackground` vorgegeben, die wiederum vom aktuellen Farbschema abhängt. In diesem Fall wird empfohlen, entweder diesen von KDE übernommenen Standardwert beizubehalten oder explizit einen leeren Wert festzulegen, wenn keine Streifen angezeigt werden sollen. Die Option `-stripebackground` hat eine höhere Priorität als die Konfigurationsoption für Spalten `-background`, jedoch eine niedrigere Priorität als die auf Zeilen- oder Zellenebene festgelegte Option `-background`.

Befehlszeilenname:	-stripeforeground or -stripefg
Name der Datenbank:	stripeForeground
Datenbankklasse:	Foreground

Legt die Vordergrundfarbe fest, die bei der Anzeige der zu einem Streifen gehörenden Einträge des Datenbereichs verwendet werden soll. Jeder Streifen besteht aus der gleichen Anzahl `stripeHeight` aufeinanderfolgender viewable Elemente, entsprechend dem Wert der Konfigurationsoption `-stripeheight`. Die ersten `stripeHeight` sichtbaren Elemente sind "normale" Elemente; darauf folgt ein Streifen, der aus den nächsten `stripeHeight` sichtbaren Elementen besteht, worauf wiederum die gleiche Anzahl "normaler" sichtbarer Elemente folgt, und so weiter. Der Standardwert ist eine leere Zeichenkette, was bedeutet, dass die Streifen die durch die Konfigurationsoption `-foreground` festgelegte Vordergrundfarbe übernehmen. Die Option `-stripeforeground` hat eine höhere Priorität als die Spaltenkonfigurationsoption `-foreground`, jedoch eine niedrigere Priorität als die auf Zeilen- oder Zellenebene festgelegte Option `-foreground`.

Befehlszeilenname:	-stripeheight
Name der Datenbank:	stripeHeight
Datenbankklasse:	StripeHeight

Legt die Anzahl der Elemente in jedem Streifen fest. Wenn der Wert Null oder weniger ist, werden keine Streifen angezeigt. Der Standardwert lautet `1`.

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das Widget selbst, sondern sein "body"-Kind den Fokus erhält, wenn die Standardtasten (`Tab` und `Shift-Tab`) bei der Tastaturnavigation gedrückt werden.

Befehlszeilenname:	-targetcolor
Name der Datenbank:	targetColor
Datenbankklasse:	TargetColor

Legt die Farbe des Abstands fest, der im Datenbereich oder im Kopfbereich der Tablelist angezeigt wird, um die Zielposition beim interaktiven Verschieben einer Zeile oder einer Spalte anzuzeigen. Die Zielanzeige für den Datenbereich der Tablelist kann auch explizit mithilfe des Unterbefehls `showtargetmark` angezeigt werden, und ihr Pfadname kann über `targetmarkpath` abgerufen werden. Der Standardwert ist `black`.

Befehlszeilenname:	-tight

Name der Datenbank:	tight
Datenbankklasse:	Tight

Legt einen booleschen Wert fest, der bestimmt, ob der zusätzliche Abstand von einem Pixel, der aus Gründen der Kompatibilität mit Tk-Listboxen unter jeder Zeile der Tablelist verbleibt, sowohl im Datenbereich als auch im Kopfbereich entfernt werden soll. Der Standardwert ist `0`, wodurch unter jeder Zeile ein zusätzlicher Abstand von einem Pixel eingezeichnet wird, genau wie bei einer Tk-Core-Listbox.

HINWEIS: Sie können diese Option auf "true" setzen und gleichzeitig mithilfe der Konfigurationsoption `-spacing` zusätzlichen Abstand oberhalb und unterhalb jeder Zeile festlegen. Dadurch entspricht der Abstand unter jeder Zeile dem Abstand über der Zeile (und ist nicht einen Pixel höher).

Befehlszeilenname:	-titlecolumns
Name der Datenbank:	titleColumns
Datenbankklasse:	TitleColumns

Legt die Anzahl der nicht scrollbaren Spalten am linken Rand des Fensters fest, die auch als **Titelspalten** bezeichnet werden. Die Positionen dieser Spalten ändern sich nicht, wenn die horizontale Ansicht durch Aufruf des Unterbefehls `scan`, `seecell`, `seecolumn` oder `xview` angepasst wird. Der Standardwert ist `0`. Der Wert dieser Option bestimmt außerdem die von den oben genannten Befehlen beim Verschieben der horizontalen Ansicht verwendete Scrolleinheit: Ist er positiv, erfolgt der horizontale Bildlauf spaltenweise, andernfalls zeichenweise (entsprechend der Breite des Zeichens `0`).

Das Ende des Titel-Spaltenbereichs wird mithilfe einer Trennlinie dargestellt, das am rechten Rand der Kopfzeilenbezeichnung der letzten nicht ausgeblendeten Titelspalte angebracht ist. Diese spezielle Trennlinie wird immer angezeigt, um das Ende der Titelspalten (sofern vorhanden) zu markieren, unabhängig vom Wert der Option `-showseparators`. Der Benutzer kann sie anhand ihrer Hintergrundfarbe, die sich von der der anderen Trennlinien unterscheidet, leicht von diesen unterscheiden.

Befehlszeilenname:	-tooltipaddcommand
Name der Datenbank:	tooltipAddCommand
Datenbankklasse:	TooltipAddCommand

Legt einen Tcl-Befehl fest, der zur Anzeige von zellen- und spaltenbezeichnerspezifischen Tooltip-Hilfen verwendet werden soll. Wenn der Mauszeiger in eine Zelle im Datenbereich der Tablelist gelangt, wird der Befehl automatisch mit dem Pfadnamen des Tablelist-Widgets sowie den Zeilen- und Spaltenindizes der Zelle verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Ähnlich wird, wenn der Mauszeiger in eine Kopfzeilenbezeichnung

gelangt, der Befehl automatisch mit dem Pfadnamen des Tablelist-Widgets, der Nummer `-1` und dem Spaltenindex der jeweiligen Bezeichnung verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Wenn schließlich der Mauszeiger in eine Zelle im Kopfbereich der Tablelist gelangt, wird der Befehl automatisch mit dem Pfadnamen des Tablelist-Widgets, dem Index der Zeile der Kopfzeilenzelle, dem der Buchstabe `h` vorangestellt ist, sowie dem Spaltenindex der Zelle verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. In allen drei Fällen wird die oben beschriebene Aktion nur ausgelöst, wenn sowohl der Wert dieser Option als auch der von `-tooltipdelcommand` nichtleere Zeichenketten sind.

Betrachten Sie beispielsweise die unten gezeigte Prozedur `tooltipAddCmd`, die den Befehl `DynamicHelp::add` aus dem Paket BWidget verwendet, um die vollständigen Zell- und Beschriftungstexte als Tooltips für die Zellen und Kopfzeilenbeschriftungen mit gekürztem Inhalt anzuzeigen.

```
proc tooltipAddCmd {tbl row col} {
    if {($row >= 0 && [$tbl iselemsnipped $row,$col fullText]) ||
        ($row <  0 && [$tbl istitlesnipped $col fullText])} {
        DynamicHelp::add $tbl -text $fullText
    }
}
```

Wenn das Widget auch Kopfzeilenelemente enthalten kann, ist eine etwas kompliziertere Prozedur erforderlich:

```
proc tooltipAddCmd {tbl row col} {
    if {[string is integer $row]} {
        if {($row >= 0 && [$tbl iselemsnipped $row,$col fullText]) ||
            ($row <  0 && [$tbl istitlesnipped $col fullText])} {
            DynamicHelp::add $tbl -text $fullText
        }
    } else {
        set row [string range $row 1 end]
        if {[$tbl header iselemsnipped $row,$col fullText]} {
            DynamicHelp::add $tbl -text $fullText
        }
    }
}
```

Ein "Tablelist"-Widget kann diese Prozedur nutzen, indem es Folgendes festlegt:

```
... -tooltipaddcommand tooltipAddCmd -tooltipdelcommand DynamicHelp::delete
```

Wenn Sie den Befehl `tooltip::tooltip` aus dem in tklib enthaltenen Tooltip-Paket anstelle von `DynamicHelp::add` verwenden möchten, müssen Sie die oben aufgeführten Aufrufe des letzteren durch Folgendes ersetzen:

```
        tooltip::tooltip $tbl $fullText
```

und die resultierende Prozedur verwenden, indem Sie Folgendes angeben:

```
... -tooltipaddcommand tooltipAddCmd -tooltipdelcommand "tooltip::tooltip clear"
```

Bitte beachten Sie, dass in dem eher seltenen Fall, dass der Name Ihres Tablelist-Widgets Leerzeichen enthält, der Befehl `tooltip::tooltip clear` nicht wie erwartet funktioniert,

wenn die Version des Tooltip-Pakets 1.4.6 oder älter ist. Als Workaround für dieses Tooltip-Problem (behoben in Version 1.4.7) können Sie den unten gezeigten, leicht abgewandelten Ansatz verwenden:

```
proc tooltipDelCmd tbl { tooltip::tooltip $tbl "" }

... -tooltipaddcommand tooltipAddCmd -tooltipdelcommand tooltipDelCmd
```

Eine weitere Alternative zu `DynamicHelp::add` ist der Befehl `baltip::tip` aus dem baltip-Paket von Alex Plotnikov, der unter folgender Adresse heruntergeladen werden kann:

https://chiselapp.com/user/aplsimple/repository/baltip/download/

Ersetzen Sie einfach die Aufrufe von `DynamicHelp::add` in der oben genannten Prozedur `tooltipAddCmd` durch

```
        baltip::tip $tbl $fullText
```

und verwenden Sie die daraus resultierende Prozedur, indem Sie Folgendes angeben:

```
... -tooltipaddcommand tooltipAddCmd -tooltipdelcommand baltip::clear
```

Beachten Sie, dass die Version des baltip-Pakets 1.3 oder höher sein muss.

In den obigen Beispielen werden die Unterbefehle `iselemsnipped`, `header iselemsnipped` und `istitlesnipped` als Unterbefehle verwendet, um sicherzustellen, dass die vollständigen Zell- und Beschriftungstexte nur für jene Zellen und Kopfzeilenbeschriftungen angezeigt werden, deren Inhalt abgeschnitten ist.

Befehlszeilenname:	-tooltipdelcommand
Name der Datenbank:	tooltipDelCommand
Datenbankklasse:	TooltipDelCommand

Legt einen Tcl-Befehl fest, der zum Entfernen der zellen- oder spaltenüberschriftenspezifischen Tooltip-Hilfe verwendet werden soll. Wenn der Mauszeiger eine Zelle oder eine Überschrift verlässt, wird der durch diese Option angegebene Befehl automatisch mit dem Pfadnamen des Tablelist-Widgets verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Diese Aktion wird nur ausgelöst, wenn sowohl der Wert dieser Option als auch der von `-tooltipaddcommand` nichtleere Zeichenketten sind. Gängige Werte für diese Option sind `DynamicHelp::delete` (wofür das BWidget-Paket erforderlich ist), `"tooltip::tooltip clear"` (wofür das in tklib enthaltene tooltip-Paket erforderlich ist) und `baltip::clear` (das baltip-Paket von Alex Plotnikov erfordert). Ihre Verwendung wird in den obigen Beispielen gezeigt.

Befehlszeilenname:	-treecolumn
Name der Datenbank:	treeColumn
Datenbankklasse:	TreeColumn

Legt die Spalte fest, in der die Einrückungen und die Steuerelemente zum Ein- und Ausblenden

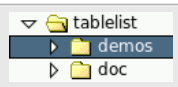
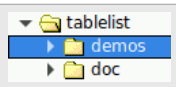
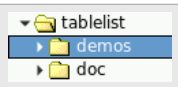
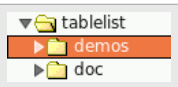
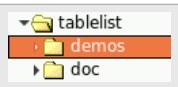
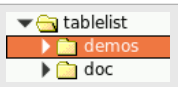
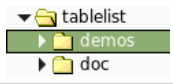
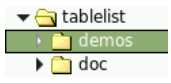
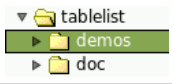
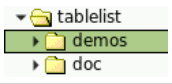
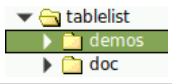
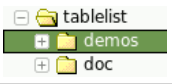
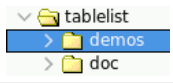
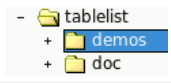
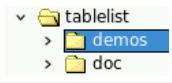
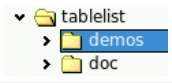
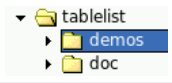
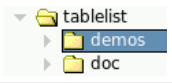
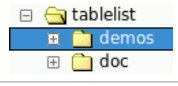
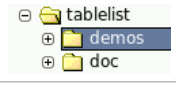
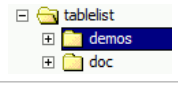
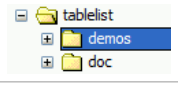
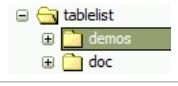
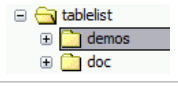
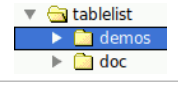
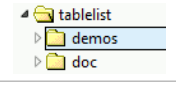
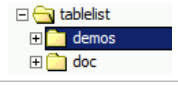
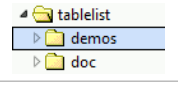
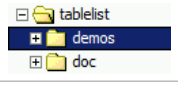
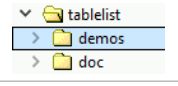
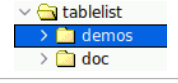
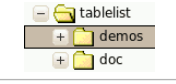
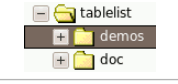
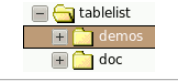
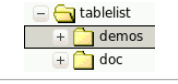
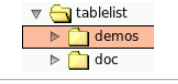
im Datenbereich einer Tablelist, die als Baum-Widget verwendet wird, angezeigt werden sollen. Der Wert der Option kann ein Spaltenindex in einer der im Abschnitt SPALTENINDIZES unten beschriebenen Formen sein. Der angegebene Index für Spalten wird durch sein numerisches Äquivalent ersetzt und automatisch aktualisiert, sobald Spalten eingefügt, gelöscht oder verschoben werden. Der Standardwert ist `0`.

HINWEIS: Bitte beachten Sie, dass die Baumstruktur nur dann wie erwartet angezeigt wird, wenn die durch diese Option angegebene Spalte linksbündig ausgerichtet ist. Es liegt in Ihrer Verantwortung, sicherzustellen, dass diese Einschränkung eingehalten wird, wenn Sie eine Tablelist als Baum-Widget verwenden.

Befehlszeilenname:	-treestyle
Name der Datenbank:	treeStyle
Datenbankklasse:	TreeStyle

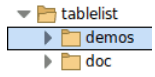
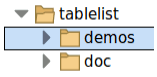
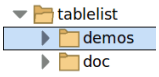
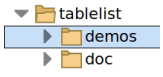
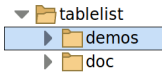
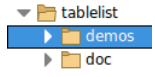
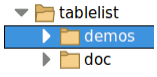
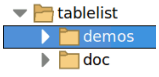
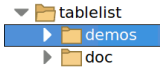
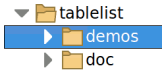
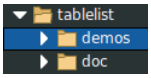
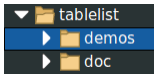
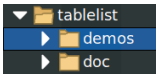
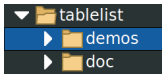
Legt das Erscheinungsbild der Spalte fest, die die Einrückungen und die Steuerelemente zum Ein- und Ausblenden im Datenbereich einer als Baum-Widget verwendeten Tablelist enthält. Dazu gehören unter anderem die zur Darstellung der Steuerelemente zum Ein- und Ausblenden verwendeten Bilder, die Einrückungsbreite sowie die Angabe, ob die Steuerelemente zum Ein- und Ausblenden und die Einrückungen bei der Auswahl einer Zeile oder Zelle geschützt werden sollen. Die derzeit unterstützten Werte sind in den beiden folgenden Tabellen auf einen Blick dargestellt:

Baumstile für ein natives Erscheinungsbild:

					
gtk	adwaita	blueMenta	ubuntu	ubuntu2	ubuntu3
					
mint	mint2	mate	menta	ubuntuMate	yuyo
					
arc	oxygen1	oxygen2	klearlooks	baghira	phase
					
plastique	plastik	winnative	winxpBlue	winxpOlive	winxpSilver
					
aqua	vistaAero	vistaClassic	win7Aero	win7Classic	win10
					
aqua11	ambiance	dust	dustSand	radiance	newWave

Die von den Baumstilen `vistaAero`, `vistaClassic`, `win7Aero`, `win7Classic` und `win10` verwendeten Größen werden *automatisch* an den Skalierungsprozentsatz des Bildschirms angepasst. Das Gleiche gilt für `aqua11`, sofern SVG-Unterstützung vorhanden ist.

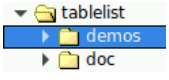
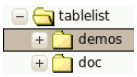
Für die folgenden Baumstile wird eine plattformunabhängige *manuelle* Skalierung unterstützt:

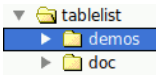
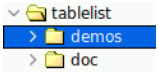
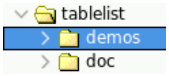
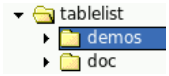
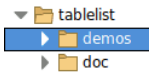
100 %	**125 %**	**150 %**	**175 %**	**200 %**
				
plain100	plain125	plain150	plain175	plain200
				
bicolor100	bicolor125	bicolor150	bicolor175	bicolor200
				
white100	white125	white150	white175	white200

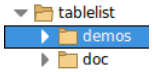
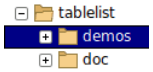
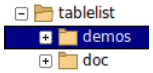
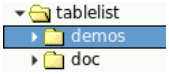
Wenn der Baumstil `gtk`, `adwaita`, `blueMenta`, `ubuntu`, `ubuntu2`, `ubuntu3`, `mint`, `mint2`, `mate`, `menta`, `ubuntuMate`, `arc`, `oxygen2`, `aqua`, `vistaAero`, `win7Aero`, `win10`, `newWave`, `plain*`, `bicolor*`, `white*` oder `classic*` ist und die Tk-Version mindestens 8.6 (mit integrierter PNG-Unterstützung) oder 8.5 beträgt und das Paket `img::png` in den Interpreter geladen werden kann, dann sind die zur Darstellung der Steuerelemente zum Ein- und Ausblenden verwendeten Bilder PNG-Bilder mit Alphakanal. Andernfalls (d. h. bei den anderen in der ersten Tabelle oben gezeigten Baumstilen oder bei fehlender PNG-Unterstützung) werden GIF-Bilder für die Steuerelemente zum Ein- und Ausblenden verwendet.

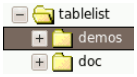
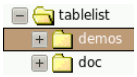
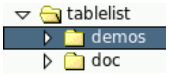
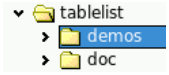
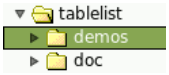
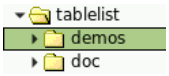
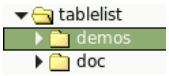
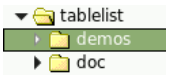
Wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) oder 8.6 ist und die Erweiterung "tksvg" im Interpreter geladen werden kann, unterstützt Tablelist die zusätzlichen Baumstile `plain`, `bicolor`, `white` und `classic`, die den Stilen `plain*`, `bicolor*`, `white*` und `classic*` sehr ähnlich sind, mit dem Unterschied, dass sie SVG-Bilder verwenden, deren Größe *automatisch* an die DPI-Skalierungsstufe des Displays und den durch `[tk scaling]` vorgegebenen Skalierungsfaktor von Tk angepasst wird. Diese unbegrenzt skalierbaren Baumstile erweisen sich insbesondere dann als nützlich, wenn Androwish auf einem Tablet oder Smartphone ausgeführt wird.

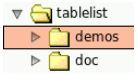
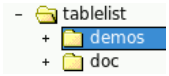
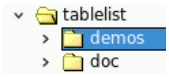
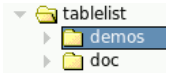
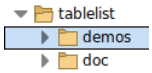
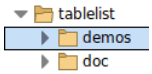
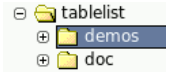
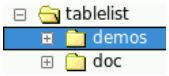
Die folgende Tabelle enthält eine detaillierte Beschreibung der Baumstile in alphabetischer Reihenfolge:

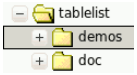
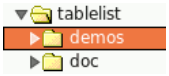
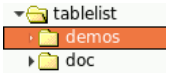
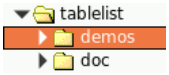
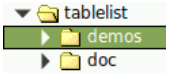
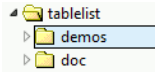
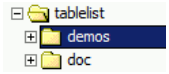
Wert	**Screenshot**	**Kommentare**
adwaita		Inspiriert vom GTK-3-Theme "Adwaita".
ambiance		Inspiriert vom GTK-2-Theme "Ambiance". Empfohlen für die Verwendung mit großen Schriftarten und Bildern.

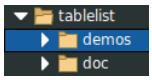
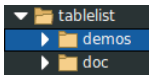
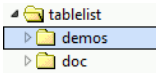
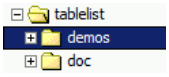
aqua		Tablelist: Standard für das Fenstersystem "Aqua" unter Mac OS X. Tablelist_tile: Standard für die Themes "Aqua" unter Mac OS X und "Aquativo".
aqua11		Verwendet nach Möglichkeit SVG-Bilder, die sich automatisch entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms anpassen. Tablelist: Standard für das Aqua-Fenstersystem unter macOS 11+. Tablelist_tile: Standard für das Aqua-Theme unter macOS 11+.
arc		Inspiriert vom Arc-Icon-Theme (des GTK 2/3-Themes "Arc").
baghira		Tablelist_tile: Standard für die Qt-Stile "baghira", "cde" und "motif" innerhalb des "tileqt"-Themes. Wird auch von einigen Varianten von "qtcurve" verwendet.
bicolor		Verwendet SVG-Bilder, die sich automatisch entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms anpassen. Tablelist_tile: Standard für die Themes "breeze", "awbreeze", "awlight", "awwinxpblue" und "sun-valley-light", wenn SVG-Unterstützung verfügbar ist.

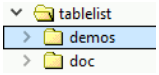
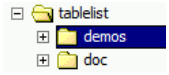
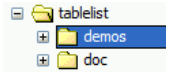
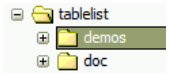
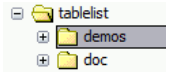
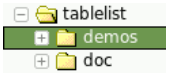
bicolor100, . . . bicolor200		Während der Baumstil bicolor100 "normale" Größen verwendet, eignen sich die vier anderen Stile für die Verwendung mit großen Schriftarten und Bildern (erforderlich auf hochauflösenden Bildschirmen). Tablelist_tile: Standard für die Themes "breeze", "awbreeze", "awlight", "awwinxpblue" und "sun-valley-light", sofern keine SVG-Unterstützung vorhanden ist.
classic		Verwendet SVG-Bilder, die sich automatisch entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms anpassen. Tablelist_tile: Standard für die Themes "arc", "awarc" und "droid", wenn SVG-Unterstützung verfügbar ist.
classic100, . . . classic200		Während der Baumstil classic100 "normale" Größen verwendet, eignen sich die vier anderen Stile für die Verwendung mit großen Schriftarten und Bildern (die auf hochauflösenden Bildschirmen benötigt werden). Tablelist_tile: Standard für die Themes "arc", "awarc" und "droid", sofern keine SVG-Unterstützung vorhanden ist.
blueMenta		Inspiriert vom GTK-3-Theme "BlueMenta" der Versionen 1.14 und höher des MATE-Desktops.

dust		Inspiriert vom GTK-2-Theme "Dust". Empfohlen für die Verwendung mit großen Schriftarten und Bildern.
dust		Inspiriert vom GTK-2-Theme "Dust". Empfohlen für die Verwendung mit großen Schriftarten und Bildern.
dustSand		Inspiriert vom GTK-2-Theme "Dust Sand". Empfohlen für die Verwendung mit großen Schriftarten und Bildern.
gtk		Tablelist: Standard für das Fenstersystem X11. Tablelist_tile: Standard für die Themes "alt", "blue", "clam", "classic", "clearlooks", "default", "kroc", "sriv", "srivlg" und "step" sowie für den Qt-Stil "gtk+" innerhalb des Themes "tileqt".
klearlooks		Inspiriert vom KDE-4-Stil "Klearlooks". Tablelist_tile: Standard für den Qt-Stil "qtcurve" im Theme "tileqt".
mate		Inspiriert von einigen GTK-2-Themes aus Versionen des MATE-Desktops, die älter sind als 1.14.
menta		Inspiriert vom GTK-3-Theme "Menta" der Versionen 1.14 und höher des MATE-Desktops.
mint		Inspiriert von einigen GTK-3-Themes, die in Versionen der Linux-Mint-Distribution vor Version 18 enthalten waren.
mint2		Inspiriert von einigen GTK-3-Themes, die in den Versionen 18 und höher der Distribution Linux Mint enthalten sind.

newWave		Inspiriert vom GTK-2-Theme "New Wave". Empfohlen für die Verwendung mit großen Schriftarten und Bildern.
oxygen1		Inspiriert vom KDE-4-Stil "Oxygen".
oxygen2		Tablelist_tile: Standard für den Qt-Stil "Oxygen" im Theme "tileqt".
phase		Tablelist_tile: Standard für den Qt-Stil "phase" innerhalb des "tileqt"-Themes.
plain		Verwendet SVG-Bilder, die sich automatisch entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms anpassen. Tablelist_tile: Standard für das "awclearlooks"-Theme, wenn SVG-Unterstützung verfügbar ist.
plain100, . . . plain200		Während der Baumstil plain100 "normale" Größen verwendet, eignen sich die vier anderen Stile für die Verwendung mit großen Schriftarten und Bildern (die auf hochauflösenden Bildschirmen benötigt werden). Tablelist_tile: Standard für das "awclearlooks"-Theme, sofern keine SVG-Unterstützung vorhanden ist.
plastik		Tablelist_tile: Standard für das "plastik"-Theme und den Qt-Stil "plastik" innerhalb des "tileqt"-Themes.
plastique		Tablelist_tile: Standard für den Qt-Stil "plastique" im Theme "tileqt".

radiance		Inspiriert vom GTK-2-Theme "Radiance". Empfohlen für die Verwendung mit großen Schriftarten und Bildern.
ubuntu		Inspiriert von einigen GTK-3-Themes, die in Versionen der Ubuntu-Linux-Distribution vor 15.04 enthalten waren.
ubuntu2		Inspiriert von einigen GTK-3-Themes, die in den Versionen 15.04 bis 16.04 der Linux-Distribution Ubuntu enthalten sind.
ubuntu3		Inspiriert von einigen GTK-3-Themes, die in den Versionen 16.10 und höher der Linux-Distribution Ubuntu enthalten sind.
ubuntuMate		Inspiriert von den Themes "Ambiant-MATE" und "Radiant-MATE", die ab Version 16.10 der Ubuntu-MATE-Distribution enthalten sind und in Ubuntu MATE 20.04 deutlich verbessert wurden.
vistaAero		Tablelist: Standard für das Fenstersystem Win32 unter Windows Vista. Tablelist_tile: Standard für das Vista-Theme unter Windows Vista mit dem Aero-Stil.
vistaClassic		Tablelist: Standard für das Fenstersystem Win32 unter Windows Vista mit dem Windows-Classic-Stil. Tablelist_tile: Standard für das Vista-Theme unter Windows Vista mit dem Windows-Classic-Stil.

white		Verwendet SVG-Bilder, die sich automatisch entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms anpassen. Tablelist_tile: Standard für die Themes "black", "awblack", "breeze-dark", "awbreezedark", "awdark" und "sun-valley-dark", wenn SVG-Unterstützung verfügbar ist.
white100, . . . white200		Während der Baumstil white100 "normale" Größen verwendet, eignen sich die vier anderen Stile für die Verwendung mit großen Schriftarten und Bildern (die auf hochauflösenden Bildschirmen benötigt werden). Tablelist_tile: Standard für die Themes "black", "awblack", "breeze-dark", "awbreezedark", "awdark" und "sun-valley-dark", sofern keine SVG-Unterstützung vorhanden ist.
win7Aero		Tablelist: Standard für das Fenstersystem Win32 unter Windows 7 und 8. Tablelist_tile: Standard für das Vista-Theme unter Windows 7 und 8 mit dem Aero-Stil.
win7Classic		Tablelist: Standard für das Fenstersystem Win32 unter Windows 7 und 8 mit dem Windows-Classic-Stil. Tablelist_tile: Standard für das Vista-Theme unter Windows 7 und 8 mit dem Windows-Classic-Stil.

win10		Tablelist: Standard für das Fenstersystem Win32 unter Windows 10 und höher. Tablelist_tile: Standard für das Vista-Theme unter Windows 10 und höher.
winnative		Tablelist: Standard für das Fenstersystem Win32 unter Windows 2000 und Windows XP mit dem Windows-Classic-Stil. Tablelist_tile: Standard für die Themes "keramik", "keramik_alt", "winnative" und "xpnative" mit dem Windows-Classic-Stil sowie für die meisten Qt-Stile innerhalb des Themes "tileqt".
winxpBlue		Tablelist: Standard für das Fenstersystem win32 unter Windows XP mit dem Farbschema "Blau". Tablelist_tile: Standard für das ursprüngliche Theme "winxpblue" und das Theme "xpnative" unter Windows XP mit dem Farbschema "Blau".
winxpOlive		Tablelist: Standard für das Fenstersystem win32 unter Windows XP mit dem Farbschema "Olive Green". Tablelist_tile: Standard für das Theme "xpnative" unter Windows XP mit dem Farbschema "Olive Green".
winxpSilver		Tablelist: Standard für das Fenstersystem Win32 unter Windows XP mit dem Farbschema "Silver". Tablelist_tile: Standard für das Theme "xpnative" unter Windows XP mit dem Farbschema "Silver".
yuyo		Inspiriert vom GTK-2-Theme "Yuyo" unter Ubuntu MATE.

Befehlszeilenname:	-width
Name der Datenbank:	width
Datenbankklasse:	Width

Legt die gewünschte Breite des Fensters in Zeichen durchschnittlicher Größe der Schriftart des Widgets fest. Wenn der Wert null oder kleiner ist, wird die gewünschte Breite des Fensters so gewählt, dass alle Spalten des Tablelist-Widgets darin Platz finden.

HINWEIS: Wenn der Wert dieser Konfigurationsoption null oder kleiner ist und die Tablelist dehnbare Spalten enthält, wird die Option `-setgrid` ignoriert. Diese geringfügige Einschränkung hat technische Gründe und ist nur unter X11 relevant.

Befehlszeilenname:	-xmousewheelwindow
Name der Datenbank:	xMouseWheelWindow
Datenbankklasse:	MouseWheelWindow

Diese Option ist nur dann relevant, wenn die Ereignisse des Mausrads und von `<TouchpadScroll>` an das Widget unter dem Mauszeiger gemeldet werden und nicht an das Widget, das den Fokus hat. Diese Bedingung ist unter X11 (für die Ereignisse des Mausrads) und unter Mac OS X/11+ erfüllt, ebenso wie unter Windows ab Tk 8.6b2 (in früheren Tk-Versionen wurden die Ereignisse des Mausrads unter Windows an das Widget mit Fokus gesendet).

Die Option `-xmousewheelwindow` legt den Pfadnamen des Fensters fest, in das die Vorgabebindungen für die Bindungs-Tags `TablelistBody` und `TablelistEdit` das Mausrad und die `<TouchpadScroll>`-Ereignisse entlang der x-Achse über `event generate` umleiten sollen, wenn sich das Fokusfenster auf dem Bildschirm, das das Tablelist-Widget enthält, außerhalb der Tablelist befindet. Der Standardwert ist eine leere Zeichenkette, was bedeutet, dass Mausrad- und `<TouchpadScroll>`-Ereignisse entlang der x-Achse, die von einem Widget mit einem der beiden oben genannten Tags empfangen werden, ein horizontales Scrollen der Tablelist oder ihres Bearbeitungsfensters auslösen, unabhängig davon, ob sich der Fokus innerhalb oder außerhalb des Tablelist-Widgets befindet.

HINWEIS 1: Die Reihe der Ereignisse des Mausrads entlang der horizontalen Achse hängt von der Tk-Version und teilweise vom Fenstersystem ab:

- *In Tk-Versionen vor 8.7a4* lauten die Ereignisse des Mausrads entlang der horizontalen Achse unter Windows `<Shift-MouseWheel>`, unter Mac OS X/11+ `<Shift-MouseWheel>` und `<Shift-Option-MouseWheel>`, sowie `<Shift-MouseWheel>`, `<Shift-Button-4>` und `<Shift-Button-5>` unter X11 (wobei `<Shift-MouseWheel>` vom X-Server nicht ausgelöst wird, aber mithilfe von `event generate` erzeugt werden kann). Unter X11 treten bei Verwendung von Tk 8.7a3 zwei weitere Mausrad-Ereignisse entlang der horizontalen Achse auf: `<Button-6>` und `<Button-7>`, die jeweils genauso behandelt werden wie `<Shift-Button-4>` bzw. `<Shift-Button-5>`. Diese Ereignisse werden üblicherweise durch das Neigen des Scrollrads nach links oder rechts bei einer Maus mit einer oder zwei zusätzlichen (Daumen-)Tasten ausgelöst. (In Tk-Versionen 8.6.x, wobei x >= 10 ist, löst das

Neigen des Scrollrads einer solchen Maus nach links oder rechts unter Windows und Mac OS X/11+ die Ereignisse `<Shift-MouseWheel>` aus und unter X11 die Ereignisse `<Shift-Button-4>` und `<Shift-Button-5>`.)

- *In den Tk-Versionen 8.7a4 und höher* sind die Ereignisse des Mausrads entlang der horizontalen Achse auf allen Fenstersystemen `<Shift-MouseWheel>` und `<Shift-Option-MouseWheel>`, wobei der Modifikator `Option` unter Mac OS X/11+ an die Taste `Option` und unter Windows sowie X11 an die Taste `Alt` gebunden ist. In diesen Tk-Versionen löst das Neigen des Scrollrads einer Maus mit einer oder zwei zusätzlichen (Daumen-)Tasten nach links oder rechts auf allen Fenstersystemen die Ereignisse `<Shift-MouseWheel>` (und `<Shift-Option-MouseWheel>`) aus.

HINWEIS 2: Wenn sich die Tablelist in einem scrollbaren Container für Widgets befindet, wie beispielsweise einem scrollutil::scrollableframe, BWidget ScrollableFrame oder iwidgets::scrolledframe, und Sie setzen diese Option auf den Pfadnamen des Toplevel-Fensters, dann wird ein horizontales Mausrad- oder `<TouchpadScroll>`-Ereignis über dem Datenbereich der Tablelist wie folgt behandelt:

- Wenn der Fokus innerhalb des "Tablelist"-Widgets liegt, wird durch das Ereignis die "Tablelist" oder deren Bearbeitungsfenster horizontal gescrollt, und es erfolgt keine weitere Verarbeitung des Ereignisses.
- Wenn der Fokus außerhalb des Tablelist-Widgets liegt, findet kein Scrollen des Tablelist-Datenbereichs oder des Bearbeitungsfensters statt. Stattdessen wird das Ereignis mithilfe des Befehls `event generate` an das Toplevel-Fenster weitergeleitet. Dies führt wiederum zu einem horizontalen Scrollen des Widget-Containers, vorausgesetzt, das Toplevel-Fenster oder das Bindungs-Tag `all` verfügt über die erforderlichen Ereignisbindungen für das Mausrad oder das Ereignis `<TouchpadScroll>`.

HINWEIS 3: Wenn Sie das "tablelist"-Widget ohne den Schalter `-ignorefocus` an den Befehl `scrollutil::adaptWheelEventHandling` übergeben, setzt dieser Befehl diese Option auf den Pfadnamen des Toplevel-Fensters.

Befehlszeilenname:	-ymousewheelwindow
Name der Datenbank:	yMouseWheelWindow
Datenbankklasse:	MouseWheelWindow

Diese Option ist nur dann relevant, wenn die Ereignisse des Mausrads und von `<TouchpadScroll>` an das Widget unter dem Mauszeiger gemeldet werden und nicht an das Widget, das den Fokus hat. Diese Bedingung ist unter X11 (für die Ereignisse des Mausrads) und unter Mac OS X/11+ erfüllt, ebenso wie unter Windows ab Tk 8.6b2 (in früheren Tk-Versionen wurden die Ereignisse des Mausrads unter Windows an das Widget mit Fokus gesendet).

Die Option `-ymousewheelwindow` legt den Pfadnamen des Fensters fest, in das die Vorgabebindungen für die Bindungs-Tags `TablelistBody` und `TablelistEdit` das Mausrad und die `<TouchpadScroll>`-Ereignisse entlang der y-Achse über `event generate` umleiten sollen, wenn sich das Fokusfenster auf dem Bildschirm, das das Tablelist-Widget enthält, außerhalb der Tablelist befindet. Der Standardwert ist eine leere Zeichenkette, was bedeutet, dass Mausrad- und `<TouchpadScroll>`-Ereignisse entlang der y-Achse, die von einem Widget

mit einem der beiden oben genannten Tags empfangen werden, ein vertikales Scrollen der Tablelist oder ihres Bearbeitungsfensters auslösen, unabhängig davon, ob sich der Fokus innerhalb oder außerhalb des Tablelist-Widgets befindet.

HINWEIS 1: Die Reihe der Ereignisse des Mausrads entlang der vertikalen Achse hängt von der Tk-Version und teilweise vom Fenstersystem ab:

- *In Tk-Versionen vor 8.7a4* lauten die Ereignisse des Mausrads entlang der vertikalen Achse unter Windows `<MouseWheel>`, unter Mac OS X/11+ `<MouseWheel>` und `<Option-MouseWheel>`, sowie `<MouseWheel>`, `<Button-4>` und `<Button-5>` unter X11 (wobei `<MouseWheel>` nicht vom X-Server ausgelöst wird, sondern mithilfe von `event generate` erzeugt werden kann).
- *In den Tk-Versionen 8.7a4 und höher* sind die Ereignisse des Mausrads entlang der vertikalen Achse auf allen Fenstersystemen `<MouseWheel>` und `<Option-MouseWheel>`, wobei der Modifikator `Option` unter Mac OS X/11+ an die Taste `Option` und unter Windows sowie X11 an die Taste `Alt` gebunden ist.

HINWEIS 2: Wenn sich die Tablelist in einem scrollbaren Container für Widgets wie einem scrollutil::scrollableframe, BWidget ScrollableFrame oder iwidgets::scrolledframe befindet, und Sie setzen diese Option auf den Pfadnamen des Toplevel-Fensters, dann wird ein vertikales Mausrad- oder `<TouchpadScroll>`-Ereignis über dem Datenbereich der Tablelist wie folgt behandelt:

- Wenn der Fokus innerhalb des "Tablelist"-Widgets liegt, wird durch das Ereignis die "Tablelist" oder deren Bearbeitungsfenster vertikal gescrollt, und es erfolgt keine weitere Verarbeitung des Ereignisses.
- Wenn der Fokus außerhalb des Tablelist-Widgets liegt, erfolgt kein Bildlauf im Datenbereich oder im Bearbeitungsfenster der Tablelist. Stattdessen wird das Ereignis mithilfe des Befehls `event generate` an das Toplevel-Fenster weitergeleitet. Dies führt wiederum zu vertikalem Scrollen des Widget-Containers, vorausgesetzt, das Toplevel-Fenster oder das Bindungs-Tag `all` verfügt über die erforderlichen Bindungen für das Mausrad oder `<TouchpadScroll>`.

HINWEIS 3: Wenn Sie das "tablelist"-Widget ohne den Schalter `-ignorefocus` an den Befehl `scrollutil::adaptWheelEventHandling` übergeben, setzt dieser Befehl diese Option auf den Pfadnamen des Toplevel-Fensters.

SPALTENKONFIGURATIONSOPTIONEN

Die folgenden Optionen werden derzeit von den Befehlen `columncget`, `columnconfigure`, `configcolumnlist` und `configcolumns` unterstützt:

-align `alignment`

Legt fest, wie die Elemente der Spalte im Daten- und Kopfbereich ausgerichtet werden sollen. Der Wert muss einer der folgenden sein: `left`, `right` oder `center`. Diese Option bezieht sich auch auf den Titel der Spalte, wenn die Option `-labelalign` für die jeweilige Spalte nicht angegeben wurde oder wenn ihr Wert eine leere Zeichenkette ist. Die Option `-align` ist mit dem Element `alignment` verknüpft, das dieser Spalte in der Liste entspricht, in der der Wert der Option `-columns` für das Tablelist-Widget festgelegt ist; Änderungen an einem der beiden Elemente werden automatisch im anderen übernommen.

-allowduplicates `boolean`

Legt fest, ob der Benutzer bei der interaktiven Zellenbearbeitung den Text einer Zelle in dieser Spalte auf den Text einer anderen Zelle derselben Spalte setzen kann. Der

Standardwert ist `1`. Der Wert dieser Option wirkt sich auch auf die Bindungen aus, die zum Beenden der Bearbeitung und zum Kopieren des neuen Zellwerts in andere Zellen der Spalte vorgesehen sind: Wenn der Wert der Option "false" ist, wird keine dieser Aktionen ausgeführt.

-background `color` **or -bg** `color`
Legt die normale Hintergrundfarbe fest, die bei der Anzeige des Inhalts der Spalte sowohl im Datenbereich als auch im Kopfbereich verwendet werden soll.

-changesnipside `boolean`
Legt fest, ob die ausrichtungsspezifische Standardposition des snip-Indikators bei der Anzeige der Elemente der Spalte im Daten- und Kopfbereich (mit Ausnahme ihres Titels) überschrieben werden soll. Der Standardwert ist `0`, was bedeutet, dass die Snip-Zeichenkette an die Elemente angehängt wird, wenn die Ausrichtung der Spalte `left` oder `center` ist, und ihnen vorangestellt wird, falls die Ausrichtung `right` ist.

-changetitlesnipside `boolean`
Legt fest, ob die ausrichtungsspezifische Standardposition des snip-Indikators bei der Anzeige des Titels der Spalte überschrieben werden soll. Der Standardwert ist `0`, was bedeutet, dass die Snip-Zeichenkette an den Titel angehängt wird, wenn die Ausrichtung der Spalte `left` oder `center` ist, und ihm vorangestellt wird, falls die Ausrichtung `right` ist.

-editable `boolean`
Legt fest, ob die Elemente der Spalte im Datenbereich der Tablelist interaktiv bearbeitet werden können. Der Standardwert lautet `0`. Der Wert dieser Option kann für einzelne Zellen mithilfe der gleichnamigen Zellenkonfigurationsoption überschrieben werden.

HINWEIS: Die Unterstützung für die interaktive Bearbeitung ist auf die Datenbereich-Komponente des Widgets beschränkt.

-editwindow `name`
Legt den Typ des temporären eingebetteten Widgets fest, das für die interaktive Bearbeitung des Inhalts der Zellen im Datenbereich der angegebenen Spalte verwendet werden soll. `name` kann einer der folgenden Werte sein: `entry` (Standard), `text`, `spinbox`, `checkbutton`, `menubutton`, `ttk::entry`, `ttk::spinbox`, `ttk::combobox`, `ttk::checkbutton` oder `ttk::menubutton` (die letzten fünf nur bei Vorhandensein der Tile-Widget-Engine) oder der Wert, der von einem der Registrierungsbefehle für Widgets aus den Paketen zurückgegeben wird: BWidget, Iwidgets, combobox (von Bryan Oakley), ctext, Mentry (oder Mentry_tile) und Tsw. Beispielsweise können Sie `-editwindow ComboBox` *nach* der Registrierung des ComboBox-Widgets für die interaktive Zellenbearbeitung mithilfe des Befehls `tablelist::addBWidgetComboBox` verwenden. Ebenso können Sie `-editwindow combobox` *nach* der Registrierung des Combobox-Widgets von Bryan Oakley für die interaktive Zellbearbeitung verwenden, indem Sie den Befehl `tablelist::addOakleyCombobox` aufrufen. Der Wert dieser Option kann für einzelne Zellen mithilfe der gleichnamigen Zellenkonfigurationsoption überschrieben werden.

-font `font`
Legt die Schriftart fest, die bei der Anzeige des Inhalts der Spalte sowohl im Datenbereich als auch im Kopfbereich verwendet werden soll.

-foreground `color` **or -fg** `color`
Legt die normale Vordergrundfarbe fest, die bei der Anzeige des Inhalts der Spalte sowohl im Datenbereich als auch im Kopfbereich verwendet werden soll.

-formatcommand command

Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn der Inhalt einer Zelle des Daten- oder Kopfbereichs innerhalb dieser Spalte angezeigt wird oder wenn der Inhalt einer Zelle des Datenbereichs der Auswahl hinzugefügt wird, während diese exportiert wird. Wenn `command` eine nicht leere Zeichenkette ist, wird sie automatisch mit dem Text der Zelle verkettet, das resultierende Skript wird im globalen Geltungsbereich ausgewertet und der Rückgabewert wird in der Zelle angezeigt oder anstelle der ursprünglichen Daten der Auswahl hinzugefügt.

Wenn beispielsweise ein Wert in Sekunden in die Zelle eingefügt wird und `command` die Prozedur `formatDate` ist, die wie folgt definiert ist:

```
proc formatDate clockVal {
    return [clock format $clockVal -format "%Y-%m-%d"]
}
```

Dann wird in der Zelle das Datum im angegebenen Format angezeigt und nicht der Wert der Zeit in Sekunden.

Diese Option wird auch von Tablelist verwendet, wenn die Rückgabewerte der Unterbefehle `getformatted`, `getformattedcolumns` und `getformattedcells` sowie ihrer kopfbereichsbezogenen Entsprechungen erstellt werden oder bei der Suche nach einem Textmuster, das mit der Option `-formatted` an den Unterbefehl `searchcolumn` übergeben wird. Alle anderen Unterbefehle, insbesondere `get`, `getcolumns`, `getcells`, `rowcget` sowie deren Entsprechungen für die Kopfzeile, `columncget`, `cellcget`, zusammen mit deren Entsprechung für die Kopfzeile, `sort`, `sortbycolumn`, `sortbycolumnlist` und `refreshsorting` wirken auf den ursprünglichen Zelltext ein, der in der internen Liste des Widgets enthalten ist. Im oben genannten Beispiel ermöglicht dies eine recht einfache Sortierung der Elemente nach Zeit mit einer Genauigkeit von einer Sekunde, selbst wenn ihre visuelle Darstellung nur das Jahr, den Monat und den Tag enthält.

Die Option `-formatcommand` ist nützlich, wenn in einer Spalte nur Bilder oder eingebettete Fenster angezeigt werden sollen, die den Zellen zugeordneten Texte jedoch nicht einfach leere Zeichenketten sein dürfen, da sie für andere Zwecke (wie Sortieren oder Bearbeiten) benötigt werden. In solchen Fällen kann eine Prozedur, die eine leere Zeichenkette zurückgibt, als Wert für die Option verwendet werden, wodurch sichergestellt wird, dass die in dieser Spalte enthaltenen Textinformationen verborgen bleiben:

```
proc emptyStr val { return "" }
```

Ein weiterer wichtiger Anwendungsfall betrifft Spalten in Tablelist, deren Option `-sortmode` auf `integer` oder `real` gesetzt wurde. Wenn einige der Zellen einer solchen Spalte leer sind, würde ein Versuch, die Einträge über `sortbycolumn` oder `sortbycolumnlist` nach dieser Spalte zu sortieren, einen Fehler auslösen, da der von Tablelist durchgeführte Aufruf von `lsort -integer` oder `lsort -real` eine Fehlermeldung ausgeben würde, weil anstelle einer Ganzzahl oder einer Gleitkommazahl eine unerwartete leere Zeichenkette erhalten wurde. Um dies zu vermeiden, stellen Sie sicher, dass die Zellen, in denen eine leere Zeichenkette angezeigt werden soll, als internen Wert eine Zahl enthalten, die entweder streng kleiner oder streng größer ist als alle anderen ganzzahligen oder reellen Zahlen in der

betreffenden Spalte, und verwenden Sie die Option `-formatcommand`, um zu steuern, was in den Zellen der Spalte angezeigt wird. Wenn beispielsweise alle Elemente der Spalte nicht-negative Werte sind, können Sie die Zahl `-1` als internen Wert für die Zellen verwenden, die eine leere Zeichenkette anzeigen sollen, und den Wert der Option `-formatcommand` auf `formatNumber` setzen, wobei `formatNumber` die wie folgt implementierte Prozedur ist:

```
proc formatNumber val { return [expr {$val == -1 ? "" : $val}] }
```

Dadurch werden die Zellen, die eine leere Zeichenkette anzeigen, bei der Sortierung der Spalte in aufsteigender Reihenfolge vor allen anderen angezeigt. Um den gegenteiligen Effekt zu erzielen, müssen Sie den Wert `-1` durch einen anwendungsspezifischen Wert ersetzen, der streng größer ist als alle anderen Zahlen in der betreffenden Spalte.

Die in der Tablelist-Distribution enthaltenen Demo-Skripte enthalten weitere Beispiele, die die Verwendung dieser Option veranschaulichen. Die interessantesten davon werden auch im Tablelist-Programmierhandbuch beschrieben.

In dem etwas komplexeren Fall, dass das Ergebnis der Formatierung auch von der Zeile der Zelle abhängen soll, müssen Sie den Unterbefehl `formatinfo` aufrufen, der die erforderlichen Informationen über die Zelle liefert, deren Inhalt formatiert wird.

Aufgrund der Konfigurationsoption `-formatcommand` unterscheiden wir zwischen dem internen Wert eines Tablelist-Elements und dessen **formatierter** Version. Letztere ist das Ergebnis des Aufrufs des Skripts, das der Option `-formatcommand` der Spalte des Elements entspricht, oder des Elements selbst, wenn diese Option für diese Spalte nicht gesetzt wurde. Folglich ist die formatierte Version eines Tablelist-Eintrags eine Liste, die sich aus den formatierten Elementen des ursprünglichen Eintrags zusammensetzt.

-hide `boolean`
Legt fest, ob die Spalte bei der Anzeige des Widgets oder beim Exportieren der Auswahl ausgeblendet werden sollen. Der Standardwert lautet `0`. Nach dem Umschalten des Ausblendstatus einer Spalte wird das virtuelle Ereignis `<<TablelistColHiddenStateChanged>>` generiert, dessen Option `-data` bei Tk-Versionen 8.5 und höher auf den numerischen Index der Spalte gesetzt ist.

-labelalign `alignment`
Legt fest, wie der Titel der Spalte ausgerichtet werden soll. Der Wert muss entweder `left`, `right` oder `center` lauten oder eine leere Zeichenkette sein. Wenn diese Option für die angegebene Spalte nicht angegeben wurde oder wenn ihr Wert eine leere Zeichenkette ist, wird der Spaltentitel genauso ausgerichtet wie die Elemente der Spalte, wie durch die Konfigurationsoption für die Spalten `-align` oder durch das dieser Spalte entsprechende Element `alignment` in der Liste, die den Wert der globalen Option `-columns` festlegt.

-labelbackground `color` or -labelbg `color`
-labelborderwidth `screenDistance` or -labelbd `screenDistance`
-labelcommand `command`
-labelcommand2 `command`
-labelfont `fontName`

-labelforeground `color` or -labelfg `color`
-labelheight `lines`
-labelpady `screenDistance`
-labelrelief `relief`
-selectfiltercommand `command`: Der Wert jeder dieser Optionen kann auch eine leere Zeichenkette sein. Diese Optionen entsprechen den spaltenspezifischen Entsprechungen der globalen Optionen mit denselben Namen, die im Abschnitt WIDGET-SPEZIFISCHE OPTIONEN beschrieben sind. Sie überschreiben die auf Widget-Ebene festgelegten gleichnamigen Optionen, wenn der angegebene Wert nicht leer ist. Wurde eine dieser Optionen für die jeweilige Spalte nicht angegeben oder ist ihr Wert eine leere Zeichenkette, wird diese Option auf Spaltenebene nicht verwendet; stattdessen kommt die gleichnamige globale Option zum Einsatz. Die Optionen `-labelactivebackground`, `-labelactiveforeground` und `-labeldisabledforeground` sind nur auf Widget-Ebene definiert; es gibt keine Spaltenkonfigurationsoptionen mit diesen Namen. Die Optionen `-labelbackground` und `-labelheight` werden nur vom Tablelist-Paket unterstützt, nicht jedoch von Tablelist_tile.

-labelimage `image`
Legt den Namen des Tk-Bildes fest, das in der Kopfbeschriftung angezeigt werden soll. `image` muss das Ergebnis eines Aufrufs des Befehls `image create` sein oder eine leere Zeichenkette, die angibt, dass kein Bild angezeigt werden soll. Wenn der Text der Beschriftung rechtsbündig ausgerichtet ist, wird das Bild rechts vom Text angezeigt, andernfalls links davon. Der Text und das Bild sind durch einen Abstand voneinander getrennt, der der Breite eines Leerzeichens in der Schriftart der jeweiligen Beschriftung entspricht. Wenn für dieselbe Spalte die Option `-labelwindow` mit einem nicht leeren Wert festgelegt wird, hat diese Vorrang vor der Option `-labelimage`.

-labelvalign `verticalAlignment`
Legt die vertikale Ausrichtung des Tk-Bildes oder -Fensters (falls vorhanden) fest, das in der Kopfbeschriftung angezeigt wird. `verticalAlignment` muss einer der folgenden Werte sein: `center` (Standardeinstellung), `top` oder `bottom`. Insbesondere wenn die Beschriftung mehrzeiligen Text enthält, ist das eingebettete Bild oder der (ttk::)checkbutton oft nicht so hoch wie der Text; in solchen Fällen ermöglicht diese Option die spaltenabhängige vertikale Ausrichtung des eingebetteten Objekts.

-labelwindow `windowType`
Legt den Typ des Fensters fest, das in die Kopfbeschriftung eingebettet werden soll. `windowType` muss `checkbutton` oder `ttk::checkbutton` lauten oder eine leere Zeichenkette sein, was festlegt, dass kein eingebettetes Fenster angezeigt werden soll. Wenn der Text des Labels rechtsbündig ausgerichtet ist, wird das (ttk::)checkbutton-Widget rechts vom Text angezeigt, andernfalls links davon. Der Text und das (ttk::)checkbutton-Widget sind durch einen Abstand voneinander getrennt, der der Breite eines Leerzeichens in der Schriftart der jeweiligen Beschriftung entspricht. Wenn diese Option mit einem nicht leeren Wert festgelegt wurde, überschreibt sie die Konfigurationsoption `-labelimage` für die Spalte.

HINWEIS 1: Die Option `-variable` des (ttk::)Checkbuttons ist auf eine globale Variable gesetzt, deren Name nur innerhalb von Tablelist bekannt ist und deren Wert beibehalten wird, wenn sich der numerische Index der Spalte ändert (aufgrund des Einfügens, Löschens oder Verschiebens von Spalten). Genauer gesagt: Wenn sich der numerische Index der Spalte ändert, wird der (ttk::)Checkbutton zerstört und automatisch an der neuen Position neu

erstellt, wobei eine andere Variable als Wert für die Option `-variable` verwendet wird; die neue Variable übernimmt jedoch den letzten Wert der alten, sodass der Markierungsstatus des (ttk::)Checkbuttons unverändert bleibt.

HINWEIS 2: Eine Anwendung, die diese Konfigurationsoption für Spalten nutzt, ruft in der Regel den Pfadnamen des (ttk::)Checkbuttons mithilfe des Unterbefehls `labelwindowpath` ab und setzt dessen Option `-command` auf ein Skript, das die Elemente der betreffenden Spalte in Abhängigkeit vom Wert der mit dem (ttk::)Checkbutton verknüpften Variablen bearbeitet. Da der Pfad des (ttk::)Checkbuttons und die zugehörige Variable vom numerischen Index der Spalte abhängen, wird dringend empfohlen, mit Spaltennamen statt mit numerischen Indizes zu arbeiten und diesen Schritt jedes Mal zu wiederholen, wenn das virtuelle Ereignis `<<TablelistColumnMoved>>` an das Tablelist-Widget gesendet wird. Das folgende Beispiel stammt aus den Demo-Skripten zur interaktiven Zellenbearbeitung:

```
# Configure the "-command" option of the checkbutton embedded into the
# header label of the column "available", and make sure that it will be
# reconfigured whenever any column is moved interactively to a new position
proc configCkbtn {tbl col} {
    set ckbtn [$tbl labelwindowpath $col]
    $ckbtn configure -command [list onCkbtnToggle $tbl $col $ckbtn]
}
proc onCkbtnToggle {tbl col ckbtn} {
    upvar #0 [$ckbtn cget -variable] var
    $tbl fillcolumn $col -text $var
    $tbl fillcolumn $col -image [expr {$var ? "checkedImg" : "uncheckedImg"}]
}
configCkbtn $tbl available
bind $tbl <<TablelistColumnMoved>> { configCkbtn %W available }
```

-maxwidth `width`

`width` muss eine Zahl sein. Ein positiver Wert legt die maximale Breite der Spalte in Zeichen durchschnittlicher Größe der Widget-Schriftart fest. Wenn `width` negativ ist, wird sein Absolutwert als maximale Breite der Spalte in Pixeln interpretiert. Schließlich legt der Wert Null (der Standardwert) fest, dass die maximale Breite der Spalte gerade so groß sein soll, dass alle Elemente der Spalte, einschließlich der Spaltenüberschrift, darin Platz finden. Diese Option ist nur relevant, wenn die angegebene Spalte eine dynamische Breite hat, d. h., wenn ihre Breite auf `0` festgelegt wurde.

-name `name`

Legt einen Namen fest, der der Spalte zugeordnet ist. Während sich der numerische Index einer Spalte durch das Einfügen, Löschen oder Verschieben von Spalten ändern kann, bleibt ihr Name unverändert und kann als sichere Alternative zum Spaltenindex verwendet werden (weitere Informationen finden Sie im Abschnitt SPALTENINDIZES). Ebenso kann er als zweite Komponente eines Zellenindexes des Datenbereichs der Form `row,col` oder eines Kopfzellenindexes der Form `headerRow,col` verwendet werden, wie in den Abschnitten ZELLENINDIZES und KOPFZELLENINDIZES beschrieben. Um Mehrdeutigkeiten zu vermeiden, sollten sich Spaltennamen von allen anderen Formen von Spaltenindizes (wie Zahlen, `active`, `anchor`, `end`, `last`, `left`, `right` oder deren Abkürzungen) unterscheiden. Sie sollten sich außerdem von der Zeichenkette `all` (und deren Abkürzungen) unterscheiden, die als Wert der Konfigurationsoption `-stretch` angegeben werden kann. Der Standardwert

ist eine leere Zeichenkette.

-resizable `boolean`
Legt fest, ob die Spalte interaktiv in der Größe angepasst werden können. Informationen zur interaktiven Größenanpassung von Spalten finden Sie im Abschnitt VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFBESCHRIFTUNGEN. Der Standardwert ist `1`. Diese Option ist nur relevant, wenn der Wert der Widget-Konfigurationsoption `-resizablecolumns` auf "true" gesetzt ist.

-selectbackground `color`
Legt die Farbe des Hintergrundes fest, die bei der Anzeige des Inhalts einer Zelle im Datenbereich der angegebenen Spalte verwendet werden soll, solange die Zelle ausgewählt ist.

-selectforeground `color`
Legt die Vordergrundfarbe fest, die bei der Anzeige des Inhalts einer Zelle des Datenbereichs in der angegebenen Spalte verwendet werden soll, solange die Zelle ausgewählt ist.

-showarrow `boolean`
Legt fest, ob der Befehl `sortbycolumn` mit dem angegebenen Spaltenindex als erstem Argument und der Befehl `sortbycolumnlist`, bei dem der angegebene Spaltenindex Bestandteil des ersten Arguments ist, einen Pfeil in die Spaltenbezeichnung einfügen sollen, der die Sortierreihenfolge angibt. Der Standardwert ist `1`. Diese Option ist nur relevant, wenn der Wert der Widget-Konfigurationsoption `-showarrow` auf "true" gesetzt ist.

-showlinenumbers `boolean`
Legt fest, ob im Datenbereich der angegebenen Spalte die Zeilennummern angezeigt werden sollen (beginnend mit 1 und endend mit der Nummer der nicht ausgeblendeten Zeilen). Der Standardwert lautet `0`.

Die folgenden Angaben gehen davon aus, dass die Option `-showlinenumbers` der angegebenen Spalte auf "true" gesetzt wurde: Die Zuordnung der Zeilennummern zu den nicht ausgeblendeten Zeilen erfolgt automatisch, sobald Elemente des Datenbereichs eingefügt, gelöscht, aktualisiert, verschoben oder sortiert werden oder ihre Option `-hide` umgeschaltet wird. Aus Gründen der Effizienz geschieht dies in einer Leerlaufphase. Werden beispielsweise mehrere Elemente in den Datenbereich eines Tablelist-Widgets eingefügt oder daraus gelöscht, erfolgt die erforderliche Neunummerierung der nicht ausgeblendeten Zeilen als Idle-Rückruffunktion, sobald die Ereignisschleife das nächste Mal durchlaufen wird und keine Ereignisse zur Verarbeitung anstehen. Die Zeilennummern überschreiben alle bisherigen Inhalte der Zellen der Spalte. Sie werden standardmäßig ohne führende Nullen angezeigt, dies (und das Anzeigeformat im Allgemeinen) kann jedoch mithilfe der Spaltenkonfigurationsoption `-formatcommand` geändert werden.

Die Unterbefehle `sortbycolumn` und `sortbycolumnlist` sowie die Befehle `tablelist::sortByColumn` und `tablelist::addToSortColumns` überprüfen die Spaltenindizes, die ihnen als Argumente übergeben werden, und führen keine Sortierung nach den Spalten durch, die für die Anzeige der Zeilennummern konfiguriert wurden (Einzelheiten finden Sie in den entsprechenden Beschreibungen).

HINWEIS: Bitte beachten Sie, dass sich die Zeilennummern auf die *nicht ausgeblendeten* Zeilen beziehen, die im Falle einer als Baum-Widget verwendeten Tablelist von den sichtbaren

abweichen können. Folglich führt das Zuklappen einer Zeile im Allgemeinen nicht zu einer Neunummerierung der Zeilen (stattdessen wird lediglich der entsprechende Bereich der Zeilennummern ausgelassen).

-sortcommand `command`

Diese Option ist nur relevant, wenn der Wert der Option `-sortmode` für die angegebene Spalte `command` lautet. Sie legt einen Befehl fest, der für den Vergleich der Elemente der Spalte verwendet werden soll, wenn der Befehl `sortbycolumn` mit dem angegebenen Spaltenindex als erstem Argument oder der Befehl `sortbycolumnlist` mit dem angegebenen Spaltenindex als Element seines ersten Arguments aufgerufen wird. Um zwei Elemente während der Operation `sortbycolumn` oder `sortbycolumnlist` zu vergleichen, wird `command` automatisch mit den beiden Elementen verkettet und das resultierende Skript ausgewertet. Das Skript sollte eine Ganzzahl zurückgeben, die kleiner als, gleich oder größer als null ist, wenn das erste Element als kleiner als, gleich oder größer als das zweite angesehen werden soll.

-sortmode `mode`

Legt fest, wie die Elemente der Spalte verglichen werden sollen, wenn der Befehl `sortbycolumn` mit dem angegebenen Spaltenindex als erstem Argument oder der Befehl `sortbycolumnlist` mit dem angegebenen Spaltenindex als Bestandteil seines ersten Arguments aufgerufen wird. `mode` kann einen der folgenden Werte annehmen:

ascii	Verwenden Sie den Zeichenkettenvergleich mit der Sortierreihenfolge nach Unicode-Codepunkten (der Name dient der Kompatibilität). Dies ist die Standardeinstellung.
asciinocase	Dies entspricht ascii, mit dem Unterschied, dass bei den Vergleichen die Groß-/Kleinschreibung nicht berücksichtigt wird.
command	Verwenden Sie den Befehl, der durch die Konfigurationsoption -sortcommand festgelegt ist, um die Elemente der Spalte zu vergleichen.
dictionary	Verwenden Sie den Vergleich im Dictionary-Stil. Dies entspricht ascii, mit folgenden Ausnahmen: (a) Die Groß-/Kleinschreibung wird ignoriert, außer bei Gleichstand; (b) wenn zwei Zeichenketten eingebettete Zahlen enthalten, werden die Zahlen als Ganzzahlen und nicht als Zeichen verglichen. Beispielsweise wird bigBoy zwischen bigbang und bigboy sortiert, und x10y wird zwischen x9y und x11y eingeordnet.
integer	Wandeln Sie die Elemente in Ganzzahlen um und verwenden Sie den Ganzzahlvergleich. Wenn eines der Elemente nicht in eine Ganzzahl konvertiert werden kann (z. B. weil es sich um eine leere Zeichenkette handelt), führt ein Sortierversuch zu einem Fehler. Informationen zur empfohlenen Vorgehensweise zur Vermeidung dieses Problems finden Sie in der Beschreibung der Option -formatcommand.

real	Konvertieren Sie die Elemente in Gleitkomma-Werte und verwenden Sie Gleitkomma-Vergleiche. Wenn eines der Elemente nicht in einen Gleitkomma-Wert konvertiert werden kann (z. B. weil es sich um eine leere Zeichenkette handelt), führt ein Sortierversuch zu einem Fehler. Informationen zur empfohlenen Vorgehensweise zur Vermeidung dieses Problems finden Sie in der Beschreibung der Option -formatcommand.

-stretchable `boolean`
Legt fest, ob die Spalte bei Bedarf gestreckt werden soll, um das Tablelist-Fenster auszufüllen. Der Wert dieser Option ist an den Wert der Option `-stretch` für das Tablelist-Widget gekoppelt; Änderungen an einer der beiden Optionen werden automatisch auf die andere übertragen.

-stretchwindow `boolean`
Legt fest, ob die Breiten der in die Zellen des Daten- und Kopfbereichs der Spalte eingebetteten Fenster dynamisch an die Spaltenbreite angepasst werden sollen, wenn diese ungleich Null (d. h. statisch) ist. Der Standardwert ist `0`. Wenn der Wert dieser Option "true" ist und die Spalte mit einer Breite ungleich null festgelegt wurde oder interaktiv in der Größe angepasst wurde, werden die Breiten der eingebetteten Fenster (sofern vorhanden) automatisch so angepasst, dass die Fenster den gesamten horizontalen Raum der Spalte ausfüllen (mit Ausnahme des linken und rechten Außenabstands). Bitte beachten Sie, dass in diesem Fall die Texte der Zellen der Spalte ausgeblendet bleiben. Wenn die Spalte hingegen eine dynamische Breite hat, wird diese Option ignoriert, und sowohl ihre Texte als auch ihre eingebetteten Fenster (sofern vorhanden) werden wie gewohnt angezeigt. Der einfachste Weg, diese Diskrepanz zu vermeiden, besteht darin, die Texte der Spalte auf leere Zeichenketten zu setzen oder sicherzustellen, dass ihre Elemente mithilfe der Spaltenkonfigurationsoption `-formatcommand` stets als leere Zeichenketten angezeigt werden. Der Wert dieser Option kann für einzelne Zellen mithilfe der gleichnamigen Zellenkonfigurationsoption überschrieben werden.

-stripebackground `color`
Legt die Farbe des Hintergrundes fest, die bei der Anzeige des Inhalts einer Zelle des Datenbereichs in der angegebenen Spalte verwendet werden soll, wenn die Zeile der Zelle zu einem Streifen gehört.

-stripeforeground `color`
Legt die Vordergrundfarbe fest, die bei der Anzeige des Inhalts einer Zelle des Datenbereichs in der angegebenen Spalte verwendet werden soll, wenn die Zeile der Zelle zu einem Streifen gehört.

-text `list`
Legt fest, welche Zeichenketten in den Zellen des Datenbereichs der angegebenen Spalte angezeigt werden sollen, d. h., aktualisiert die Elemente im Datenbereich der Spalte. Wenn der Wert `state` der Tablelist `disabled` ist, wird keine Aktualisierung durchgeführt.

-title `title`
Legt den Text fest, der in der Überschrift der Spalte angezeigt werden soll. Diese Option ist mit dem Element `title` verknüpft, das der angegebenen Spalte in der Liste entspricht, die

den Wert der Option `-columns` für das Tablelist-Widget festlegt; Änderungen an einer der beiden Optionen werden automatisch in der anderen übernommen.

-valign `verticalAlignment`
Legt die vertikale Ausrichtung von eingebetteten Bildern und Fenstern fest, die in den Zellen des Daten- und Kopfbereichs der angegebenen Spalte angezeigt werden. Der Wert `verticalAlignment` muss einer der folgenden Werte sein: `center` (Standard), `top` oder `bottom`. Insbesondere in Zeilen, die mehrzeilige Elemente enthalten, sind eingebettete Bilder oder Fenster oft nicht so hoch wie die Zeilen selbst; in solchen Fällen können Sie mit dieser Option die spaltenabhängige vertikale Ausrichtung dieser eingebetteten Objekte steuern. Der Wert dieser Option kann für einzelne Zellen mithilfe der gleichnamigen Zellenkonfigurationsoption überschrieben werden.

-width `width`
`width` muss eine Zahl sein. Ein positiver Wert legt die Spaltenbreite in Zeichen durchschnittlicher Größe der Schriftart des Widgets fest. Wenn `width` negativ ist, wird sein Absolutwert als Spaltenbreite in Pixeln interpretiert. Schließlich legt der Wert Null fest, dass die Breite der Spalte gerade so groß sein soll, dass alle Elemente der Spalte, einschließlich der Spaltenüberschrift, Platz finden (jedoch nicht größer als die durch die Spaltenkonfigurationsoption `-maxwidth` angegebene maximale Breite). Diese Option ist mit dem Element `width` verknüpft, das der jeweiligen Spalte in der Liste entspricht, die den Wert der Option `-columns` für das Tablelist-Widget festlegt; Änderungen an einer der beiden Optionen werden automatisch auf die andere übertragen.

-windowdestroy `command`
Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn ein in eine Zelle des Daten- oder Kopfbereichs der Spalte eingebettetes Fenster zerstört wird. Sofern `command` eine nicht leere Zeichenkette ist, wird sie automatisch mit dem Pfadnamen des Tablelist-Widgets, den Indizes für Zeilen und Spalten der Zelle sowie dem Pfadnamen des eingebetteten Fensters verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Diese Option bietet eine benutzerfreundliche Alternative zur Bindung eines Skripts an das Ereignis `<Destroy>` der eingebetteten Fenster der Spalte aus deren Erstellungsskripten heraus, die als Wert der Konfigurationsoption `-window` angegeben werden. Der Wert dieser Option kann für einzelne Zellen mithilfe der gleichnamigen Zellkonfigurationsoption überschrieben werden.

-windowupdate `command`
Legt einen Tcl-Befehl fest, der dafür zuständig ist, die Hintergrund- und Vordergrundfarben eines Fensters, das in eine Zelle des Daten- oder Kopfbereichs der Spalte eingebettet ist, an die aktuellen Hintergrund- und Vordergrundfarben der Zelle selbst anzupassen. Die aktuellen Farben der Zellen hängen davon ab, ob der Wert von `state` des Tablelist-Widgets `normal` oder `disabled` ist, ob die Zelle ausgewählt ist und ob ihre Zeile mit den Farben angezeigt wird, die durch die Widget- oder Spaltenkonfigurationsoptionen `-stripebackground` und `-stripeforeground` festgelegt sind. Sofern `command` eine nicht leere Zeichenkette ist, wird sie automatisch mit dem Pfadnamen des Tablelist-Widgets, den Indexen für Zeilen und Spalten der Zelle, dem Pfadnamen des eingebetteten Fensters, dem Namen der Option `-background` und der aktuellen Hintergrundfarbe der Zelle sowie dem Namen der Option `-foreground` und der aktuellen Vordergrundfarbe der Zelle verkettet. Das resultierende Skript wird anschließend im globalen Geltungsbereich ausgewertet. Der angegebene Befehl kann die ihm als Argumente übergebenen Farben verwenden, um die entsprechenden Farben

des eingebetteten Fensters auf dieselben Werte zu setzen.

Beispielsweise wenn es sich bei dem eingebetteten Fenster um ein Text- oder Ctext-Widget handelt und Sie möchten, dass dessen Hintergrund- und Vordergrundfarben automatisch an die aktuellen Hintergrund- und Vordergrundfarben der Zelle angepasst werden, können Sie dies erreichen, indem Sie die Option `-windowupdate` auf `updateWindow` setzen, wobei letzteres die wie folgt implementierte Prozedur ist:

```
proc updateWindow {tbl row col w args} {
    eval [list $w configure] $args
}
```

Ab Tcl/Tk 8.5 können Sie die kompaktere Form verwenden

```
proc updateWindow {tbl row col w args} {
    $w configure {*}$args
}
```

In diesem Beispiel ist `args` eine Liste in folgender Form:

```
-background backgroundColor -foreground foregroundColor
```

Der Wert dieser Option kann für einzelne Zellen mithilfe der gleichnamigen Zellenkonfigurationsoption überschrieben werden.

-wrap `boolean`

Legt fest, ob die Elemente des Daten- und Kopfbereichs der angegebenen Spalte, die nicht in ihre Zellen passen, mit Zeilenumbruch mehrzeilig statt abgeschnitten angezeigt werden sollen. Der Standardwert ist `0`. Wenn die angegebene Spalte eine statische Breite oder einen von Null verschiedenen `-maxwidth`-Wert hat und der Wert dieser Option "true" ist, werden Elemente der Spalte, die zu lang sind, um in einer einzigen Zeile angezeigt zu werden, in mehrere Zeilen aufgeteilt. Das Gleiche gilt für die einzelnen Zeilen der mehrzeiligen Elemente (d. h. Elemente, die Zeilenumbruchzeichen enthalten): Auch diese werden bei Bedarf umgebrochen, wodurch zusätzliche Zeilenumbrüche entstehen. In beiden Fällen werden die Zeilenumbrüche nach Möglichkeit an Wortgrenzen gesetzt und dienen ausschließlich der äußeren Darstellung der in der angegebenen Spalte enthaltenen Zeichenketten, ohne den internen Inhalt der Zellen zu beeinflussen.

Die Spaltenkonfigurationsoptionen `-background`, `-font`, `-foreground`, `-selectbackground`, `-selectforeground`, `-stripebackground` und `-stripeforeground` überschreiben die gleichnamigen Konfigurationsoptionen, die auf Widget-Ebene festgelegt wurden (jedoch nicht diejenigen, die auf Zellen- oder Zeilenebene festgelegt wurden), wenn der angegebene Wert keine leere Zeichenkette ist. Weitere Informationen zu diesen Optionen finden Sie im Abschnitt FARBEN UND SCHRIFTARTEN.

ZEILENKONFIGURATIONSOPTIONEN IM DATENBEREICH

Die folgenden Optionen werden derzeit von den Befehlen `rowcget`, `rowconfigure`, `configrowlist` und `configrows` unterstützt:

`-background color` **or** `-bg color`

Legt die normale Hintergrundfarbe fest, die bei der Anzeige des Inhalts der Zeile verwendet werden soll.

`-font font`
Legt die Schriftart fest, die bei der Anzeige des Inhalts der Zeile verwendet werden soll.

`-foreground color` **or** `-fg color`
Legt die normale Vordergrundfarbe fest, die bei der Anzeige des Inhalts der Zeile verwendet werden soll.

`-hide boolean`
Legt fest, ob die Zeile bei der Anzeige des Widgets oder beim Exportieren der Auswahl ausgeblendet werden soll. Der Standardwert lautet `0`. Nach dem Umschalten des Ausblendstatus einer Zeile wird das virtuelle Ereignis `<<TablelistRowHiddenStateChanged>>` generiert, dessen Option `-data` bei Tk-Versionen 8.5 und höher auf den numerischen Index der Zeile gesetzt ist.

ACHTUNG: Die Tk-Versionen 8.3 bis 8.4.12 wiesen einen Fehler auf, der zu einem Segmentierungsfehler führte, wenn der gesamte Inhalt eines Text-Widgets ausgeblendet (elidiert) wurde. Dieser Fehler war auch in Tk 8.5a1 – 8.5a3 vorhanden. Bei Verwendung einer dieser früheren Tk-Versionen führt dieser Fehler zu einem Absturz, wenn alle Zeilen eines Tablelist-Widgets ausgeblendet werden. Es liegt in Ihrer Verantwortung, solche Situationen bei der Verwendung einer Tk-Version mit diesem Fehler zu vermeiden!

`-name name`
Legt einen Namen fest, der der Zeile zugeordnet ist. Während sich der numerische Index einer Zeile durch Einfügen, Löschen oder Verschieben von Zeilen oder durch Sortieren der Elemente ändern kann, bleibt ihr Name konstant und kann als sichere Alternative zum Index der Zeile verwendet werden (Einzelheiten finden Sie im Abschnitt ZEILENINDIZES). Ebenso kann er als erste Komponente eines Zellenindexes der Form `row,col` verwendet werden, wie im Abschnitt ZELLENINDIZES beschrieben. Um Mehrdeutigkeiten zu vermeiden, sollten sich Zeilennamen von allen anderen Formen von Zeilenindizes (wie Zahlen, vollständige Schlüssel, `active`, `anchor`, `end`, `last`, `top`, `bottom` oder deren Abkürzungen) unterscheiden. Der Standardwert ist eine leere Zeichenkette.

`-selectable` `boolean`
Legt fest, ob die in der angegebenen Zeile angezeigten Elemente ausgewählt werden können. Der Standardwert lautet `1`. Wenn der Wert `0` angegeben ist, wird jeder Versuch, das in dieser Zeile enthaltene Element mithilfe des Widget-Befehls `selection set` oder eines seiner Elemente mithilfe des Befehls `cellselection set` auszuwählen, stillschweigend ignoriert; außerdem wird eine eventuell vorhandene alte (Zellen-)Auswahl aus der Zeile entfernt.

`-selectbackground color`
Legt die Farbe des Hintergrundes fest, die bei der Anzeige des Inhalts einer Zelle in der angegebenen Zeile verwendet werden soll, solange die Zelle ausgewählt ist.

`-selectforeground color`
Legt die Vordergrundfarbe fest, die bei der Anzeige des Inhalts einer Zelle in der angegebenen Zeile verwendet werden soll, solange die Zelle ausgewählt ist.

`-text list`
Legt fest, welche Zeichenketten in den Zellen der angegebenen Zeile angezeigt werden sollen, d. h., aktualisiert das in der Zeile enthaltene Element. Wenn der Wert `state` der Tablelist `disabled` lautet oder das neue Element mit dem alten übereinstimmt, wird keine

Aktualisierung durchgeführt.

Die Zeilenkonfigurationsoptionen `-background`, `-font`, `-foreground`, `-selectbackground`, sowie `-selectforeground` überschreiben die gleichnamigen Konfigurationsoptionen, die auf Spalten- oder Widget-Ebene festgelegt wurden (nicht jedoch diejenigen auf Zellenebene), wenn der angegebene Wert keine leere Zeichenkette ist. Weitere Informationen zu diesen Optionen finden Sie im Abschnitt FARBEN UND SCHRIFTARTEN.

ZEILENKONFIGURATIONSOPTIONEN IM KOPFBEREICH

Die folgenden Optionen werden derzeit von den Befehlen `header rowcget`, `header rowconfigure`, `header configrowlist` und `header configrows` unterstützt:

`-background color` **or** `-bg color`
Legt die Hintergrundfarbe fest, die bei der Anzeige des Inhalts der Kopfzeile verwendet werden soll.

`-font font`
Legt die Schriftart fest, die bei der Anzeige des Inhalts der Kopfzeile verwendet werden soll.

`-foreground color` **or** `-fg color`
Legt die Vordergrundfarbe fest, die bei der Anzeige des Inhalts der Kopfzeile verwendet werden soll.

`-name name`
Legt einen Namen fest, der der Kopfzeile zugeordnet ist. Während sich der numerische Index einer Kopfzeile durch das Einfügen oder Löschen von Kopfzeilen ändern kann, bleibt ihr Name konstant und kann als zuverlässiger alternativer Kopfzeilenindex verwendet werden (Einzelheiten finden Sie im Abschnitt KOPFZEILENINDIZES). Ebenso kann er als erste Komponente eines Kopfzell-Indexes der Form `headerRow,col` verwendet werden, wie im Abschnitt KOPFZELLENINDIZES beschrieben. Um Mehrdeutigkeiten zu vermeiden, sollten sich die Namen der Kopfzeilen von allen anderen Formen von Kopfzeilen-Indizes (wie Zahlen, vollständigen Schlüsseln, `end`, `last` oder deren Abkürzungen) unterscheiden. Der Standardwert ist eine leere Zeichenkette.

`-text list`
Legt eine Liste von Zeichenketten fest, die in den Zellen der angegebenen Kopfzeile angezeigt werden sollen, d. h., aktualisiert den in der Kopfzeile enthaltenen Eintrag. Wenn der Wert `state` der Tablelist `disabled` lautet oder der neue Eintrag mit dem alten übereinstimmt, wird keine Aktualisierung durchgeführt.

Die Konfigurationsoptionen für die Kopfzeile von `-background`, `-font` und `-foreground` überschreiben die gleichnamigen Konfigurationsoptionen, die auf Spalten- oder Widget-Ebene festgelegt wurden (jedoch nicht diejenigen, die auf Kopfzellenebene festgelegt wurden), wenn der angegebene Wert keine leere Zeichenkette ist. Weitere Informationen zu diesen Optionen finden Sie im Abschnitt FARBEN UND SCHRIFTARTEN.

ZELLENKONFIGURATIONSOPTIONEN IM DATENBEREICH

Die folgenden Optionen werden derzeit von den Befehlen `cellcget`, `cellconfigure`, `configcelllist` und `configcells` unterstützt:

`-background color` **or** `-bg color`
Legt die normale Hintergrundfarbe fest, die bei der Anzeige des Inhalts der Zelle verwendet werden soll.

`-editable boolean`
Legt fest, ob der Inhalt der Zelle interaktiv bearbeitet werden kann. Der Standardwert ist `0`. Diese Option überschreibt die gleichnamige Option für die Spalte, die die angegebene Zelle enthält, wenn der angegebene Wert keine leere Zeichenkette ist (was ausdrücklich zulässig ist).

`-editwindow name`
Legt den Typ des temporären eingebetteten Widgets fest, das für die interaktive Bearbeitung des Zellinhalts verwendet werden soll. Der Standardwert lautet `entry`. Diese Option überschreibt die gleichnamige Option für die Spalte, die die angegebene Zelle enthält, wenn der angegebene Wert keine leere Zeichenkette ist (was ausdrücklich zulässig ist).

`-font font`
Legt die Schriftart fest, die bei der Anzeige des Inhalts der Zelle verwendet werden soll.

`-foreground color` **or** `-fg color`
Legt die normale Vordergrundfarbe fest, die bei der Anzeige des Inhalts der Zelle verwendet werden soll.

-image `image`
Legt den Namen des Tk-Bildes fest, das in der Zelle angezeigt werden soll. `image` muss das Ergebnis eines Aufrufs des Befehls `image create` sein oder eine leere Zeichenkette, die angibt, dass kein Bild angezeigt werden soll. Wenn die Spalte, in der sich die Zelle befindet, rechtsbündig ausgerichtet ist, wird das Bild rechts neben dem Text der Zelle angezeigt, andernfalls links davon. Der Text und das Bild sind durch einen Abstand von 4 Pixeln voneinander getrennt, der entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms skaliert wird. Wenn für dieselbe Zelle die Option `-window` mit einem nicht leeren Wert festgelegt wird, hat diese Vorrang vor der Option `-image`. Wenn der Wert von `state` in der Tablelist `disabled` lautet oder der neue Name mit dem alten übereinstimmt, wird das Bild der Zelle nicht aktualisiert.

Um ein Bild in einer Zelle anzuzeigen, nutzt Tablelist ein eingebettetes Label-Widget (das bei Bedarf erstellt wird). Dies hat den wesentlichen Vorteil, dass die Breite des Labels, das das Bild enthält, so angepasst werden kann, dass es in die Spalte passt. Dies hat den visuellen Effekt, dass ein Teil des Bildes auf der rechten Seite abgeschnitten wird. Der oben erwähnte Abstand links oder rechts vom Bild ist ebenfalls Teil dieses Labels. Um sicherzustellen, dass ein Bild mit vollständig oder teilweise transparentem Hintergrund korrekt angezeigt wird, wird die Farbe des Hintergrunds des Label-Widgets, das das eingebettete Bild enthält, bei Bedarf automatisch an die Farbe des Hintergrunds der Zelle angepasst (z. B. beim Auswählen oder Aufheben der Auswahl des Zellinhalts oder beim Ändern der Farbe des Hintergrunds der Zelle), vorausgesetzt, die Option `-imagebackground` wurde nicht gesetzt oder ihr Wert ist eine leere Zeichenkette. Diese automatische Farbsynchronisation stellt zudem sicher, dass der Abstand zwischen Text und Bild in der richtigen Farbe angezeigt wird.

-imagebackground `color`
Legt die Hintergrundfarbe des Label-Widgets fest, das das eingebettete Bild enthält (wenn vorhanden). Wenn der Wert dieser Option eine gültige, nicht leere Farbanweisung ist, werden die vollständig transparenten Pixel des eingebetteten Bildes in der angegebenen Farbe angezeigt, während die teilweise transparenten Pixel entsprechend ihrem Alpha-Wert und der Hintergrundfarbe des Bildes angepasst dargestellt werden. Der Standardwert ist eine leere Zeichenkette, was bedeutet, dass die Farbe des Hintergrunds des Label-Widgets, das das

eingebettete Bild enthält, bei Bedarf automatisch aktualisiert wird.

HINWEIS: Diese Option funktioniert einwandfrei, wenn der Text der Zelle (oder zumindest dessen formatierte Version) eine leere Zeichenkette ist. Andernfalls entsteht ein Abstand zwischen dem Text und dem Bild, und dieser Abstand wird in der festgelegten Bildhintergrundfarbe angezeigt, wenn es sich dabei um eine nicht leere Zeichenkette handelt.

`-selectbackground color`
Legt die Hintergrundfarbe fest, die bei der Anzeige des Inhalts der Zelle verwendet werden soll, solange die Zelle ausgewählt ist.

`-selectforeground color`
Legt die Vordergrundfarbe fest, die bei der Anzeige des Zellinhalts verwendet werden soll, wenn die Zelle ausgewählt ist.

`-stretchwindow boolean`
Legt fest, ob die Breite des in die Zelle eingebetteten Fensters dynamisch an die Breite der Spalte der Zelle angepasst werden soll, wenn diese nicht Null ist (d. h. statisch). Der Standardwert ist `0`. Wenn der Wert dieser Option "true" ist und die Spalte mit einer Breite ungleich null festgelegt wurde oder interaktiv in der Größe angepasst wurde, wird die Breite des eingebetteten Fensters (sofern vorhanden) automatisch so angepasst, dass das Fenster den gesamten horizontalen Platz dieser Spalte ausfüllt (mit Ausnahme des linken und rechten Außenabstands). Bitte beachten Sie, dass in diesem Fall der Text der Zelle ausgeblendet bleibt. Wenn die Spalte hingegen eine dynamische Breite hat, wird diese Option ignoriert, und sowohl der Text der Zelle als auch ihr eingebettetes Fenster (falls vorhanden) werden wie gewohnt angezeigt. Der einfachste Weg, diese Diskrepanz zu vermeiden, besteht darin, den Text der Zelle auf eine leere Zeichenkette zu setzen oder sicherzustellen, dass die Elemente der Spalte stets als leere Zeichenketten angezeigt werden, indem Sie die Konfigurationsoption `-formatcommand` verwenden. Diese Option überschreibt die gleichnamige Option für die Spalte, die die betreffende Zelle enthält, wenn der angegebene Wert keine leere Zeichenkette ist (was ausdrücklich zulässig ist).

`-text text`
Legt die Zeichenkette fest, die in der angegebenen Zelle angezeigt werden soll, d. h., aktualisiert das in der Zelle enthaltene Element. Wenn der Wert `state` der Tablelist `disabled` lautet oder der neue Zelltext mit dem alten übereinstimmt, wird keine Aktualisierung durchgeführt.

`-valign verticalAlignment`
Legt die vertikale Ausrichtung des in der Zelle angezeigten eingebetteten Bildes oder Fensters fest (wenn vorhanden). Der Standardwert lautet `center`. Diese Option überschreibt die gleichnamige Option für die Spalte, die die angegebene Zelle enthält, wenn der angegebene Wert keine leere Zeichenkette ist (was ausdrücklich zulässig ist).

`-window` `command`
Legt einen Tcl-Befehl fest, mit dem das in die Zelle einzubettende Fenster erstellt wird. Der Befehl wird automatisch mit dem Pfadnamen des Tablelist-Widgets, den Zeilen- und Spaltenindizes der Zelle sowie dem Pfadnamen des zu erstellenden eingebetteten Fensters verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. `command` kann auch eine leere Zeichenkette sein, die festlegt, dass kein eingebettetes Fenster angezeigt werden soll. Wenn die Spalte, in der sich die Zelle befindet, rechtsbündig

ausgerichtet ist, wird das Fenster rechts neben dem Text der Zelle angezeigt, andernfalls links davon. Der Text und das Fenster sind durch einen Abstand von 4 Pixeln voneinander getrennt, der entsprechend dem tatsächlichen Skalierungsprozentsatz des Bildschirms skaliert wird. Wenn diese Option mit einem nicht leeren Wert festgelegt wurde, überschreibt sie die Konfigurationsoption `-image`. Wenn der Wert `state` der Tablelist `disabled` lautet, wird diese Option ignoriert.

HINWEIS: Es gibt verschiedene Situationen, in denen das eingebettete Fenster zerstört und später durch Aufruf des oben genannten Skripts neu erstellt wird. Beispielsweise beim Ändern des Werts einiger Konfigurationsoptionen des Tablelist-Widgets oder der Spalten, beim Sortieren der Elemente oder beim Verschieben einer Zeile oder Spalte wird der Inhalt des Widgets neu angezeigt, wodurch die eingebetteten Fenster neu erstellt werden müssen. Bei diesem Vorgang bleiben die Änderungen, die nach der Erstellung an den eingebetteten Fenstern vorgenommen wurden, nicht erhalten. Aus diesem Grund sollten Sie jegliche Änderungen an eingebetteten Fenstern außerhalb ihrer Erstellungsskripte vermeiden.

`-windowdestroy command`

Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn ein in die Zelle eingebettetes Fenster zerstört wird. Sofern `command` eine nicht leere Zeichenkette ist, wird sie automatisch mit dem Pfadnamen des Tablelist-Widgets, den Zeilen- und Spaltenindizes der Zelle sowie dem Pfadnamen des eingebetteten Fensters verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Diese Option bietet eine benutzerfreundliche Alternative zur Bindung eines Skripts an das `<Destroy>`-Ereignis des eingebetteten Fensters aus dessen Erstellungsskript heraus, das als Wert der Zellkonfigurationsoption `-window` festgelegt wird. Diese Option überschreibt die gleichnamige Option für die Spalte, die die angegebene Zelle enthält, wenn der angegebene Wert keine leere Zeichenkette ist.

`-windowupdate command`

Legt einen Tcl-Befehl fest, der dafür zuständig ist, die Hintergrund- und Vordergrundfarben eines in die Zelle eingebetteten Fensters an die aktuellen Hintergrund- und Vordergrundfarben der Zelle selbst anzupassen. Die aktuellen Farben der Zellen hängen davon ab, ob der Wert von `state` des Tablelist-Widgets auf `normal` oder `disabled` gesetzt ist, ob die Zelle ausgewählt ist und ob ihre Zeile mit den Farben angezeigt wird, die durch die Widget- oder Spaltenkonfigurationsoptionen `-stripebackground` und `-stripeforeground` festgelegt sind. Sofern `command` eine nicht leere Zeichenkette ist, wird sie automatisch mit dem Pfadnamen des Tablelist-Widgets, den Indexen für Zeilen und Spalten der Zelle, dem Pfadnamen des eingebetteten Fensters, dem Namen der Option `-background` und der aktuellen Farbe des Hintergrundes der Zelle sowie dem Namen der Option `-foreground` und der aktuellen Farbe des Vordergrundes der Zelle verkettet. Das resultierende Skript wird dann im globalen Geltungsbereich ausgewertet. Der angegebene Befehl kann die ihm als Argumente übergebenen Farben verwenden, um die entsprechenden Farben des eingebetteten Fensters auf dieselben Werte zu setzen. Diese Option überschreibt die gleichnamige Option für die Spalte, in der sich die angegebene Zelle befindet, sofern der angegebene Wert keine leere Zeichenkette ist.

Die Zellkonfigurationsoptionen `-background`, `-font`, `-foreground`, `-selectbackground` und `-selectforeground` überschreiben die gleichnamigen Konfigurationsoptionen, die auf Zeilen-, Spalten- oder Widget-Ebene festgelegt wurden, wenn der angegebene Wert keine leere Zeichenkette ist. Weitere Informationen zu diesen Optionen

finden Sie im Abschnitt FARBEN UND SCHRIFTEN.

ZELLENKONFIGURATIONSOPTIONEN IM KOPFBEREICH

Die folgenden Optionen werden derzeit von den Befehlen `header cellcget`, `header cellconfigure`, `header configcelllist` und `header configcells` unterstützt:

`-background color` **or** `-bg color`
Legt die normale Hintergrundfarbe fest, die bei der Anzeige des Inhalts der Kopfzeilenzelle verwendet werden soll.

`-font font`
Legt die Schriftart fest, die bei der Anzeige des Inhalts der Kopfzeilenzelle verwendet werden soll.

`-foreground color` **or** `-fg color`
Legt die Vordergrundfarbe fest, die bei der Anzeige des Inhalts der Kopfzeilenzelle verwendet werden soll.

`-image image`
Legt den Namen des Tk-Bildes fest, das in der Kopfzeilenzelle angezeigt werden soll. Die Details entsprechen denen für eine Zelle im Datenbereich der Tablelist.

`-imagebackground color`
Legt die Farbe des Hintergrunds des Beschriftungs-Widgets fest, das das eingebettete Bild enthält (falls vorhanden). Die Details entsprechen denen für eine Zelle im Datenbereich.

`-stretchwindow boolean`
Legt fest, ob die Breite des in die Kopfzelle eingebetteten Fensters dynamisch an die Breite der Spalte der Zelle angepasst werden soll, wenn diese nicht Null ist (d. h. statisch ist). Die Details entsprechen denen für eine Zelle im Datenbereich der Tablelist.

`-text text`
Legt die Zeichenkette fest, die in der angegebenen Kopfzelle angezeigt werden soll, d. h., aktualisiert das in der Zelle enthaltene Element. Wenn der Wert `state` der Tablelist `disabled` lautet oder der neue Zelltext mit dem alten übereinstimmt, wird keine Aktualisierung durchgeführt.

`-valign verticalAlignment`
Legt die vertikale Ausrichtung des eingebetteten Bildes oder Fensters fest, das in der Kopfzeilenzelle angezeigt wird (falls vorhanden). Der Standardwert lautet `center`. Diese Option überschreibt die gleichnamige Option für die Spalte, die die angegebene Zelle enthält, und kann dieselben Werte wie ihr spaltenbezogenes Pendant annehmen.

`-window command`
Legt einen Tcl-Befehl fest, mit dem das in die Kopfzeilenzelle einzubettende Fenster erstellt wird. Die Details ähneln denen für eine Zelle im Datenbereich der Tablelist.

`-windowdestroy command`
Legt einen Tcl-Befehl fest, der aufgerufen werden soll, wenn ein in die Kopfzeilenzelle eingebettetes Fenster zerstört wird. Die Details entsprechen denen für eine Zelle im Datenbereich.

`-windowupdate command`
Legt einen Tcl-Befehl fest, der dafür zuständig ist, die Hintergrund- und Vordergrundfarben eines in die Kopfzeilenzelle eingebetteten Fensters an die aktuellen Hintergrund- und Vordergrundfarben der Zelle selbst anzupassen. Die aktuellen Farben der Kopfzeilenzelle

hängen davon ab, ob der Wert von `state` des Tablelist-Widgets auf `normal` oder `disabled` gesetzt ist. Die übrigen Details entsprechen denen einer Zelle im Datenbereich der Tablelist.

Die Konfigurationsoptionen für Kopfzellen `-background`, `-font` und `-foreground` überschreiben die gleichnamigen Optionen, die auf Kopfzeilen-, Spalten- oder Widget-Ebene festgelegt wurden, wenn der angegebene Wert keine leere Zeichenkette ist. Weitere Informationen zu diesen Optionen finden Sie im Abschnitt FARBEN UND SCHRIFTEN.

FARBEN UND SCHRIFTEN

Die Optionen `-background`, `-font`, `-foreground`, `-selectbackground` und `-selectforeground` können ebenfalls auf Spalten-, Zeilen- und Zellenebene festgelegt werden, indem die Unterbefehle `columnconfigure` (oder `configcolumnlist`, oder `configcolumns`), `rowconfigure` (oder `configrowlist` oder `configrows`), und `cellconfigure` (oder `configcelllist` oder `configcells`) des Tcl-Befehls, der dem Tablelist-Widget zugeordnet ist, verwendet werden. Ebenso können die ersten drei dieser Optionen mithilfe von `header rowconfigure` (oder `header configrowlist`, oder `header configrows`) sowie `header cellconfigure` (oder `header configcelllist` oder `header configcells`) auch für den Kopfbereich des Widgets auf Zeilen- und Zellenebene festgelegt werden. Ebenso können die Optionen `-stripebackground` und `-stripeforeground` auch auf Spaltenebene festgelegt werden. Aus diesem Grund kann eine bestimmte Zelle bis zu vier Werte für eine und dieselbe Farbe oder Schriftart aufweisen. Wenn diese Werte miteinander in Konflikt stehen, wird die Option verwendet, die auf der höchsten Prioritätsstufe angegeben ist. Die absteigende Prioritätsreihenfolge lautet: Zelle, Zeile, Spalte, Widget.

Wenn eine dieser Optionen auf Zellen-, Zeilen- oder Spaltenebene nicht angegeben wurde oder wenn ihr Wert auf dieser Ebene eine leere Zeichenkette ist (was ausdrücklich zulässig ist), wird diese Option auf dieser jeweiligen Ebene nicht verwendet.

VERWENDUNG EINER TABLELIST ALS MEHRSPALTIGES BAUM-WIDGET

Wenn ein Tablelist-Widget zur Darstellung einer Baumstruktur verwendet wird, sind einige Regeln zu beachten:

1. Verwenden Sie die Option `-listvariable` des Widgets nicht, oder fügen Sie zumindest keine neuen Elemente zu der Variable hinzu, die als ihr Wert angegeben ist, da das Hinzufügen neuer Elemente zu dieser Liste dazu führt, dass diese Listenelemente in das Widget eingefügt werden, ohne die interne Struktur des Baums zu berücksichtigen.
2. Stellen Sie sicher, dass die durch die Option `-treecolumn` festgelegte Spalte linksbündig ausgerichtet ist, damit die Baumstruktur wie erwartet angezeigt werden kann.
3. Verwenden Sie zum Einfügen neuer Elemente in den Datenbereich des Widgets den Unterbefehl `insertchild(ren)` oder `insertchildlist` anstelle von `insert` oder `insertlist`.

INTERAKTIVE ZELLENBEARBEITUNG

Ob der Inhalt einer Zelle im Datenbereich eines Tablelist-Widgets interaktiv bearbeitet werden kann oder nicht, hängt von der Option `-editable` sowohl auf Zellenebene als auch auf Spaltenebene ab. Wenn die Option auf Zellenebene explizit gesetzt wurde, bestimmt ihr Wert die Bearbeitbarkeit des Zellinhalts. Andernfalls wird der Wert der Option auf Spaltenebene herangezogen, um zu entscheiden, ob die Zelle interaktiv bearbeitet werden kann. Aus dieser Regel folgt, dass Sie die interaktive Zellenbearbeitung für eine gesamte Spalte

aktivieren können, indem Sie deren Option `-editable` auf "true" setzen. Um einige Zellen der Spalte von der interaktiven Bearbeitung auszuschließen, setzen Sie einfach deren Option `-editable` auf "false".

Die interaktive Zellenbearbeitung wird durch den Unterbefehl `editcell` gestartet, der implizit durch einen Klick mit der linken Maustaste in eine bearbeitbare Zelle aufgerufen wird (Einzelheiten finden Sie unter der Option `-editselectedonly`) oder durch die Navigation mit der Tastatur von einer bearbeitbaren Zelle in eine andere. Wenn der Auswahltyp `cell` ist und sich der Positionscursor in einer bearbeitbaren Zelle befindet, kann die interaktive Bearbeitung des aktiven Elements auch durch Drücken von `Return` oder `KP_Enter` gestartet werden. Standardmäßig erstellt der Unterbefehl `editcell` ein temporäres Entry-Widget und bettet es in die Zelle ein, deren Index ihm als Argument übergeben wurde. Sie können jedoch die Spalten- oder Zellenkonfigurationsoption `-editwindow` verwenden, um anstelle eines Eingabe-Widgets ein anderes Widget anzugeben, wie beispielsweise ein Tk-Core-Text-, Spinbox-, Checkbutton oder Menubutton-Widget, ein Tile-Entry, eine Spinbox, eine Combobox, ein Checkbutton oder ein Menubutton sowie eines der 20 derzeit unterstützten Widgets aus den Paketen BWidget, Iwidgets, combobox (von Bryan Oakley), ctext, Mentry (oder Mentry_tile) sowie Tsw. Bevor Sie ein Widget aus einem dieser Bibliothekspakete als Wert der Spalten- oder Zellenkonfigurationsoption `-editwindow` angeben, müssen Sie es durch Aufruf des entsprechenden Befehls `tablelist::add*` für die interaktive Zellenbearbeitung registrieren. Die oben genannten Tk-Core- und Tile-Widgets werden automatisch für die Zellenbearbeitung registriert.

Im einfachsten Fall entspricht der automatisch in das temporäre eingebettete Widget eingefügte Text dem in der Zelle angezeigten Text, bei dem es sich wiederum um den Inhalt der Zelle oder um die Zeichenkette handeln kann, die mithilfe der Option `-formatcommand` der Spalte der Zelle aus diesem Inhalt gewonnen wird. Wenn der Wert der Konfigurationsoption `-editstartcommand` jedoch eine nicht leere Zeichenkette ist, wird der in der Zelle angezeigte Text als letztes Argument an diesen Befehl übergeben (nach dem Pfadnamen der Tablelist sowie den Zeilen- und Spaltenindizes der Zelle), das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert wird zum Inhalt des Bearbeitungsfensters. Innerhalb dieses Skripts können Sie den Unterbefehl `cancelediting` aufrufen, der das temporäre eingebettete Widget löscht und die Bearbeitung seines Inhalts abbricht. Das Hauptziel dieses Skripts besteht jedoch darin, Ihnen die Definition von Validierungen für den Bearbeitungsprozess zu ermöglichen. Dies kann entweder mithilfe der Optionen zur Eingabevalidierung (siehe die Referenzseite **entry**) oder durch Verwendung des Widget-Callback-Pakets Wcb erfolgen. Das Iwidgets-Paket stellt eigene Funktionen für die Validierung bereit, die ebenfalls verwendet werden können, wenn es sich bei dem Bearbeitungsfenster um ein Widget dieser Erweiterung handelt. In beiden Fällen benötigen Sie den Pfadnamen des temporär eingebetteten Widgets oder den seiner Entry- bzw. Entry-ähnlichen Komponente; verwenden Sie die Unterbefehle `editwinpath` und `entrypath`, um diese Pfadnamen zu ermitteln. Ein weiterer Zweck des durch die Option `-editstartcommand` angegebenen Befehls besteht darin, Ihnen die Möglichkeit zu geben, das Bearbeitungsfenster auf verschiedene andere Weisen vorzubereiten. Wenn es sich bei letzterem beispielsweise um ein Combobox-Widget handelt, können Sie dessen Option `-editable` auf "false" setzen oder (bei einer Tile-Combobox) dessen Zustand auf `readonly`

festlegen; in diesem Fall müssen Sie dessen listbox-Komponente füllen. Im selben Skript können Sie einige der visuellen Attribute des eingebetteten Widgets ändern (wie z. B. dessen Hintergrundfarbe, die Hintergrundfarbe der Auswahl oder die Vordergrundfarbe der Auswahl). (Beachten Sie, dass dies auch mithilfe der Tk-Optionen-Datenbank möglich ist.)

Die Bearbeitung des in das Bearbeitungsfenster eingefügten Textes erfolgt über die widgetspezifischen Bindungen für Maus- und Tastaturereignisse, mit einigen Erweiterungen und Änderungen, wie im Abschnitt VORGABEBINDUNGEN FÜR DIE INTERAKTIVE ZELLENBEARBEITUNG beschrieben. Beispielsweise fügt `Control-i` in Eingabekomponenten oder eingabeähnlichen Komponenten des Bearbeitungsfensters ein Tabulatorzeichen ein, `Control-j` fügt einen Zeilenumbruch ein, und wenn es sich bei dem Bearbeitungsfenster um ein Text- oder Ctext-Widget handelt, fügen `Return` und `KP_Enter` ebenfalls ein Zeilenumbruchzeichen ein. `Tab` und `Shift-Tab` dienen zur Navigation zwischen den bearbeitbaren Zellen, genau wie `Alt-Left`, `Alt-Right`, `Alt-Up`, `Alt-Down`, `Alt-Prior`, `Alt-Next`, `Alt-Home` und `Alt-End` (sowie `Control-Home` und `Control-End`, außer im Fall eines Text- oder Ctext-Widgets). Unter Mac OS X/11+ Aqua wird anstelle von `Alt` die Taste `Command` verwendet. Die Bearbeitung kann mit der Taste `Escape` (oder durch Aufruf des Unterbefehls `cancelediting`) abgebrochen werden und normal mit `Return` oder `KP_Enter` beendet werden (zusammen mit `Control` bei einem Text- oder ctext-Widget). Die Bindungen für die Tasten, die zur normalen Beendigung der Bearbeitung verwendet werden, rufen lediglich den Unterbefehl `finishediting` auf; dieser kann auch explizit aufgerufen werden, um die Bearbeitung programmgesteuert zu beenden. Ein normales Beenden wird auch ausgelöst, wenn Sie mit der linken Maustaste an einer beliebigen Stelle im Datenbereich der Tablelist außerhalb der gerade bearbeiteten Zelle klicken oder durch Tastaturnavigation in eine andere bearbeitbare Zelle wechseln. Wenn die Bearbeitung mit der linken Maustaste gestartet wurde, handelt es sich bei dem Bearbeitungsfenster um ein Tk- oder Tile-Checkbutton- oder ein Toggleswitch-Widget und ist der Wert der Option `-instanttoggle` auf "true" gesetzt, erfolgt die normale Beendigung der Bearbeitung automatisch, ohne jegliche Benutzerinteraktion. Ebenso wird die Bearbeitungssitzung automatisch beendet, wenn das Bearbeitungsfenster eines der vier von Tablelist unterstützten Combobox-Widgets oder ein (ttk::)menubutton-Widget ist und der Wert der Option `-autofinishediting` "true" ist, sobald eine Auswahl eines Combobox- oder Menüeintrags erfolgt. Schließlich wird, wenn der Wert der Option `-editendonfocusout` "true" ist, die normale Beendigung auch ausgelöst, wenn das Tablelist-Widget den Fokus verliert.

Tablelist unterstützt außerdem Bindungen, die die Bearbeitung beenden und den neuen Zellenwert in andere Zellen der Spalte kopieren: Vorausgesetzt, der Wert der Option `-allowduplicates` der Spalte ist "true" (Standardeinstellung), beenden `Alt-Return` und `Alt-KP_Enter` die Bearbeitung und füllen die ausgewählten Zellen der Spalte mit dem neuen Wert (dies ist besonders nützlich, wenn die Option `-selectmode` auf `extended` gesetzt wurde und nach dem Klicken in eine bearbeitbare Zelle durch vertikales Ziehen des Mauszeigers weitere Zellen dieser Spalte ausgewählt wurden); `Alt-a` ("Alle füllen") beendet die Bearbeitung und füllt alle Zellen der Spalte mit dem neuen Wert; `Alt-d` ("Nach unten füllen") beendet die Bearbeitung und füllt die Zellen der Spalte unterhalb der gerade bearbeiteten Zelle mit dem neuen Wert; `Alt-u` ("Nach oben füllen") beendet die

Bearbeitung und füllt die Zellen der Spalte oberhalb der gerade bearbeiteten Zelle mit dem neuen Wert. Unter Mac OS X/11+ Aqua wird die Tastenkombination `Alt-a` durch `Control-Command-a` ersetzt, und in den anderen Tastenkombinationen erfolgt die Ersetzung des Modifikators `Alt` durch `Command`.

Bei normalem Abschluss des Bearbeitungsvorgangs vergleicht der dem Tablelist-Widget zugeordnete Tcl-Befehl den endgültigen Inhalt des Bearbeitungsfensters mit dessen ursprünglichem Inhalt. Sind diese identisch, wird das eingebettete Widget gelöscht und der ursprüngliche Wert der Zelle wiederhergestellt. Wenn die beiden Zeichenketten unterschiedlich sind und der Wert der Konfigurationsoption `-editendcommand` eine nicht leere Zeichenkette ist, wird der endgültige Text des Bearbeitungsfensters als letztes Argument an diesen Befehl übergeben (nach dem Pfadnamen der Tablelist sowie den Zeilen- und Spaltenindizes der Zelle), das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert wird nach dem Löschen des temporären eingebetteten Widgets zum neuen internen Inhalt der Zelle. Das Hauptziel dieses Skripts besteht darin, Ihnen eine abschließende Validierung des Inhalts des Bearbeitungsfensters zu ermöglichen. Innerhalb dieses Skripts können Sie den Unterbefehl `rejectinput` aufrufen, der verhindert, dass der Wert des Skripts zum neuen Inhalt der Zelle wird; dieser Unterbefehl verhindert zudem die Löschung des temporären eingebetteten Widgets. Ein weiterer Zweck des durch die Option `-editendcommand` angegebenen Befehls besteht darin, den Text des Bearbeitungsfensters in den neuen *internen* Inhalt der Zelle umzuwandeln, was erforderlich ist, wenn aufgrund der Konfigurationsoption `-formatcommand` der interne Wert der Zelle von ihrer externen Darstellung abweicht. Weitere Informationen zur Aufrufweise des oben genannten Befehls finden Sie in der Beschreibung der Option `-forceeditendcommand`.

UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN

Wie im Abschnitt VORGABE- UND EINZELBINDUNGEN FÜR DEN DATENBEREICH ausführlich beschrieben, sorgen die Standard-Skripte für eine automatische, drag-freundliche Verarbeitung der Auswahl und der Mauszeiger-Bewegungen bei gedrückter Maustaste 1, wenn die Option `-customdragsource` des Tablelist-Widgets auf "true" gesetzt wurde oder seine Datenbereich-Komponente über den Befehl `tkdnd::drag_source register` oder den BWidget-Befehl `DragSite::register` als Ziehquelle für Maustaste 1 registriert wurde.

Andererseits können Sie ein Tablelist-Widget auch als TkDND-Ablegeziel registrieren, indem Sie den Befehl `tkdnd::drop_target register` aufrufen. Ebenso können Sie die Datenbereich-Komponente eines Tablelist-Widgets mithilfe des Befehls `DropSite::register` als Ablegeziel für das in BWidget enthaltene Framework zum Ziehen und Ablegen registrieren. (Beachten Sie, dass bei BWidget der *Datenbereich* der Tablelist und nicht das Widget selbst registriert werden muss.)

Bevor der Benutzer das gezogene Objekt in den Datenbereich des Tablelist-Widgets ablegt, benötigt er möglicherweise eine Möglichkeit, die gewünschte Stelle für den Ablegevorgang in den sichtbaren Bereich zu bringen. Durch Aufruf des Unterbefehls `autoscrolltarget` können Sie sicherstellen, dass, wenn der Mauszeiger den Bereich verlässt, der durch Abzug einiger Pixel an allen Seiten des scrollbaren Teils des Tablelist-Datenbereichs entsteht, ein automatisches Wegscrollen von der Maus aus gestartet wird, das so lange andauert, bis es durch den Unterbefehl `stopautoscroll` abgebrochen wird, der Mauszeiger diesen Bereich

wieder betritt oder kein weiteres Scrollen in diese Richtung mehr möglich ist.

Die Aktion, die durch ein "Drop"-Ereignis über einem Tablelist-Widget ausgelöst wird, kann von der Zeile unter dem Mauszeiger und der vertikalen Position des Mauszeigers innerhalb der umschließenden Zeile (falls vorhanden) abhängen. Beispielsweise könnte Ihre Anwendung je nach vertikaler Position des Mauszeigers innerhalb der umschließenden Zeile ein oder mehrere neue gleichrangige Elemente vor oder hinter der betreffenden Zeile einfügen oder ein untergeordnetes Element, dessen übergeordnetes Element diese Zeile ist. Alternativ könnte das Drop-Ereignis das Element unter dem Mauszeiger unter Verwendung der mit der Ziehquelle verknüpften Daten aktualisieren oder eine andere, anwendungsspezifische Aktion ausführen. In jedem Fall können Sie die Benutzerfreundlichkeit Ihrer Anwendung erheblich steigern, indem Sie vor oder innerhalb der von der Ablage betroffenen Zeile eine Zielanzeige (einen horizontalen Abstand oder einen vertikalen Balken) anzeigen. Die folgenden Beispiele zeigen, wie dies für ein Tablelist-Widget `$tbl` mithilfe der Unterbefehle `showtargetmark`, `hidetargetmark`, `targetmarkpos` und `targetmarkpath` erreicht wird.

TkDND-BEISPIEL: Eine Einführung in TkDND finden Sie im Online- TkDND-Tutorial von Georgios Petasis. Für das folgende Beispiel wird dringend empfohlen, TkDND Version 2.7 oder höher zu verwenden, die unter folgender Adresse heruntergeladen werden kann:

https://sourceforge.net/projects/tkdnd/files/

```
package require tkdnd 2.7

tkdnd::drop_target register $tbl DND_Text

bind $tbl <<DropEnter>>    { onTblDropEnterOrPos %W %e %X %Y %a %b }
bind $tbl <<DropPosition>> { onTblDropEnterOrPos %W %e %X %Y %a %b }
bind $tbl <<DropLeave>>    { %W hidetargetmark }
bind $tbl <<Drop>>         { onTblDrop %W %A %D }

proc onTblDropEnterOrPos {tbl event rootX rootY actions buttons} {
    # Scroll the target if the mouse is about to leave its scrollable part
    set x [expr {$rootX - [winfo rootx $tbl]}]
    set y [expr {$rootY - [winfo rooty $tbl]}]
    $tbl autoscrolltarget $event $x $y

    global place row
    foreach {place row} [$tbl targetmarkpos $y] {}

    if {some_optional_excluding_condition} {
        $tbl hidetargetmark
        return refuse_drop
    } else {
        $tbl showtargetmark $place $row
        return copy ;# for example
    }
}

proc onTblDrop {tbl action data} {
    # Cancel the automatic scrolling of the target
```

```
    $tbl stopautoscroll

    handleTblDrop $tbl $data
    return $action
}
```

BWidget-BEISPIEL: Eine Einführung zum Ziehen und Ablegen mit BWidget finden Sie im Online-Tutorial BWidget-Beispiel: Drag-and-Drop-Demo von Kevin Walzer. Die im folgenden Beispiel verwendeten `DropSite::*`-Befehle werden auf der Referenzseite "DropSite" beschrieben, die im BWidget-Paket enthalten ist. Bei der Verwendung dieses Frameworks für Ziehen und Ablegen müssen Sie nicht nur den Datenbereich der Tablelist, sondern auch deren Zielindikator als Ablegeorte registrieren, um sicherzustellen, dass die Zielmarkierung nicht verdeckt wird, wenn der Mauszeiger bei gedrückter Maustaste 1 darüber bewegt wird:

```
package require BWidget

foreach w [list [$tbl bodypath] [$tbl targetmarkpath]] {
    DropSite::register $w -dropovercmd tblDropOverCmd -dropcmd tblDropCmd \
        -droptypes ...
}

proc tblDropOverCmd {dropTarget dragSrc event rootX rootY op dataType data} {
    # Scroll the target if the mouse is about to leave its scrollable part
    set tbl [tablelist::getTablelistPath $dropTarget]
    set x [expr {$rootX - [winfo rooty $tbl]}]
    set y [expr {$rootY - [winfo rooty $tbl]}]
    $tbl autoscrolltarget $event $x $y

    # $event may be "enter", "motion", or "leave"
    if {[string equal $event "leave"]} {
        set newWidget [winfo containing -displayof $dropTarget $rootX $rootY]
        if {![string equal $newWidget [$tbl targetmarkpath]] &&
            ![string equal $newWidget [$tbl bodypath]]} {
            $tbl hidetargetmark
            return 2 ;# refuse the drop and re-invoke the callback on motion events
        }
    }

    global place row
    foreach {place row} [$tbl targetmarkpos $y] {}

    if {some_optional_excluding_condition} {
        $tbl hidetargetmark
        DropSite::setcursor dot
        return 2 ;# refuse the drop and re-invoke the callback on motion events
    } else {
        $tbl showtargetmark $place $row
        DropSite::setcursor based_arrow_down
        return 3 ;# accept the drop and re-invoke the callback on motion events
    }
}
```

```
proc tblDropCmd {dropTarget dragSrc rootX rootY op dataType data} {
    # Cancel the automatic scrolling of the target
    set tbl [tablelist::getTablelistPath $dropTarget]
    $tbl stopautoscroll

    handleTblDrop $tbl $data
    return 1 ;# accept the drop
}
```

In beiden oben genannten Beispielen können Sie den Typ des unterstützten Ablegeziel-Indikators einschränken, indem Sie dem Aufruf von `targetmarkpos` das Argument `-horizontal` oder `-vertical` anhängen. Wenn Sie die Option `-vertical` übergeben, müssen Sie die Anweisung `if`, die auf diesen Unterbefehlsaufruf folgt, wie folgt erweitern:

```
    if {$row < 0 || some_optional_excluding_condition} {
```

In beiden oben genannten Beispielen wird die Prozedur `handleTblDrop` aufgerufen, die wie folgt implementiert ist:

```
proc handleTblDrop {tbl data} {
    $tbl hidetargetmark
    global place row

    if {[string equal $place "before"]}
        if {$row < [$tbl size]} {
            # Insert a sibling of the item indicated by $row, built from $data
            # (For a flat tablelist replace the following 2 lines with
            # "set key [$tbl insert $row ...]")
            set key [$tbl insertchild [$tbl parentkey $row] \
                     [$tbl childindex $row] ...]
        } else {
            # Append a top-level item built from $data
            # (For a flat tablelist replace the following line with
            # "set key [$tbl insert end ...]")
            set key [$tbl insertchild root end ...]
        }
    } else { ;# "inside"
        # Insert a child of the item indicated by $row, built from $data
        # Alternatively, update the item indicated by $row, using $data
        set key [$tbl insertchild $row end ...]
        $tbl expand $row -partly
    }

    . . .
}
```

VIRTUELLE EREIGNISSE

"Tablelist" definiert einige virtuelle Ereignisse zu Benachrichtigungszwecken und nutzt den Befehl `event generate`, um diese an das betreffende "Tablelist"-Widget zu senden (dessen Pfadname dem Wert der Ersetzung `%W` in Ereignis-Skripten entspricht). Bei Verwendung von Tk 8.5 oder höher werden die meisten dieser virtuellen Ereignisse so generiert, dass ihre Option `-data` auf einen geeigneten ereignisspezifischen Wert gesetzt ist. Bindungsskripte

können auf diese Benutzerdaten als Wert der Ersetzung `%d` zugreifen. Leider wurde die Option `-data` für `event generate` von Tk-Versionen vor 8.5 nicht unterstützt. Bei diesen Tk-Versionen wird der Befehl `event generate` ohne die Option `-data` aufgerufen.

Die von Tablelist definierten und an die Tablelist-Widgets gesendeten virtuellen Ereignisse lauten wie folgt:

Virtuelles Ereignis	**Beschreibung**	**Benutzerdaten (für Tk 8.5 oder höher)**
<<TablelistActivate>>	Wird von den Vorgabebindungen generiert, sobald sich die aktive Zeile oder Zelle im Tablelist-Widget aufgrund einer Benutzeraktion ändert.	Wird nicht verwendet.
<<TablelistCellRestored>>	Erstellt durch den Unterbefehl canceledit ing.	Eine Liste, die aus den numerischen Zeilen-und Spaltenindizes der Zelle besteht, deren Inhalt gerade bearbeitet wird.
<<TablelistCellUpdated>>	Wird vom Unterbefehl finishediting generiert, wenn der Inhalt der Zelle während der Bearbeitungssitzung tatsächlich geändert wurde.	Eine Liste, die aus den numerischen Zeilen-und Spaltenindizes der Zelle besteht, deren Inhalt gerade bearbeitet wird.
<<TablelistColHiddenStateChanged>>	Wird immer dann generiert, wenn der ausgeblendete Status einer oder mehrerer Spalten mithilfe des Unterbefehls columnconfigure, configcolumnlist, configcolumns oder togglecolumnhide umgeschaltet wird.	Eine Liste, die aus den numerischen Indizes der Spalten besteht, deren Ausblendstatus sich geändert hat.

Virtuelles Ereignis	**Beschreibung**	**Benutzerdaten (für Tk 8.5 oder höher)**
<<TablelistColumnMoved>>	Wird von den Vorgabebindungen generiert, sobald eine Spalte interaktiv an eine neue Position verschoben wird.	Eine Liste der Länge 4, deren erste beiden Elemente mit den beiden Argumenten identisch sind, die an den Unterbefehl movecolumn übergeben wurden, der zum Verschieben der Quellspalte an die Zielposition aufgerufen wurde, und deren letzte beiden Elemente die entsprechenden Spaltennamen sind, die mit Hilfe des Unterbefehls columncget ... -name abgerufen wurden.
<<TablelistColumnResized>>	Wird von den Vorgabebindungen generiert, sobald die Größe einer Spalte interaktiv geändert wird.	Der numerische Index der Spalte, deren Breite geändert wurde.
<<TablelistColumnSorted>>	Erstellt mit dem Befehl tablelist::sortByColumn.	Eine Liste, bestehend aus dem numerischen Index für die Spalten und der Sortierreihenfolge (decreasing oder increasing).
<<TablelistColumnsSorted>>	Erstellt mit dem Befehl tablelist::addToSortColumns.	Eine Liste, die aus den beiden Listen besteht, die an den Unterbefehl sortbycolumnlist übergeben wurden, der zur Durchführung der mehrspaltigen Sortierung aufgerufen wurde.

Virtuelles Ereignis	**Beschreibung**	**Benutzerdaten (für Tk 8.5 oder höher)**
<<TablelistCopyFinished>>	Erzeugt durch die Vorgabebindungen für die Tastenfolgen Alt-Return, Alt-KP_Enter, Alt-a, Alt-d und Alt-u im Bearbeitungsfenster, wenn das Kopieren des neuen Werts in andere Zellen der Spalte abgeschlossen ist. Unter Mac OS X/11+ Aqua wird die Tastenkombination Alt-a durch Control-Command-a ersetzt, und in den anderen Tastenfolgen erfolgt die Ersetzung des Modifikators Alt durch Command.	Der numerische Spaltenindex der Zelle, deren Inhalt bearbeitet wurde.
<<TablelistHeaderHeightChanged>>	Wird immer dann generiert, wenn sich die (angeforderte) Höhe des Kopfbereichs der Tablelist ändert. Wird vom Widget scrollarea des Pakets Scrollutil verwendet.	Die (angeforderte) Höhe des Kopfbereichs der Tablelist.
<<TablelistRowHiddenStateChanged>>	Wird immer dann generiert, wenn der Ausblendstatus einer oder mehrerer Zeilen mithilfe der Unterbefehle rowconfigure, configrowlist, configrows oder togglerowhide umgeschaltet wird.	Eine Liste, die aus den numerischen Indizes der Zeilen besteht, deren versteckter Status sich geändert hat.

Virtuelles Ereignis	**Beschreibung**	**Benutzerdaten (für Tk 8.5 oder höher)**
<<TablelistRowMoved>>	Wird von den Vorgabebindungen generiert, sobald eine Zeile interaktiv an eine neue Position verschoben wird.	Eine Liste der Länge 3, deren Elemente aus den Argumenten abgeleitet werden, die an die zweite Form des Unterbefehls move übergeben wurden, der zum Verschieben der Quellzeile an die Zielposition aufgerufen wurde. Das erste Listenelement ist der vollständige Schlüssel, der dem ersten Argument entspricht, das zweite ist entweder root oder der vollständige Schlüssel, der dem zweiten Argument entspricht, und das dritte Listenelement ist identisch mit dem dritten Argument, das an den Unterbefehl move übergeben wurde.
<<TablelistSelect>>, <<ListboxSelect>>	Wird von den Vorgabebindungen generiert, sobald sich die Auswahl im Tablelist-Widget aufgrund einer Benutzeraktion ändert.	Wird nicht verwendet.
<<TablelistSelectionLost>>	Wird an das "tablelist"-Widget gesendet, sobald es die Eigentümerschaft an der PRIMARY-Auswahl verliert.	Wird nicht verwendet.
<<TablelistTitleColsWidthChanged>>	Wird immer dann generiert, wenn sich die Gesamtbreite der nicht ausgeblendeten Titelspalten ändert. Wird vom Widget scrollarea des Pakets Scrollutil verwendet.	Die Gesamtbreite der nicht ausgeblendeten Titelspalten.

Virtuelles Ereignis	**Beschreibung**	**Benutzerdaten (für Tk 8.5 oder höher)**
<<TablelistViewUpdated>>	Wird immer dann generiert, wenn das "tablelist"-Widget die Aktualisierung seiner Ansicht abgeschlossen hat (was aus Leistungsgründen in Zeiten ohne Benutzeraktivität erfolgt).	Wird nicht verwendet.
<<TablelistXViewChanged>>	Wird immer dann generiert, wenn sich die horizontale Ansicht im Fenster des Tablelist-Widgets ändert (und die horizontale Scrollbar, falls vorhanden, aktualisiert wird).	Eine Liste, die aus den numerischen Indizes der Spalten ganz links und ganz rechts besteht, die im Tablelist-Fenster sichtbar sind, d. h. aus den numerischen Entsprechungen der Spaltenindizes, left und right.
<<TablelistYViewChanged>>	Wird immer dann generiert, wenn sich die vertikale Ansicht im Fenster des Tablelist-Widgets ändert (und die vertikale Scrollbar, falls vorhanden, aktualisiert wird).	Eine Liste, die aus den numerischen Indizes der obersten und untersten im Tablelist-Fenster sichtbaren Elemente des Datenbereichs besteht, d. h. aus den numerischen Entsprechungen der Zeilenindizes, top und bottom.

Darüber hinaus definiert "Tablelist_tile" das virtuelle Ereignis `<<TablelistThemeDefaultsChanged>>` und sendet es (ohne Benutzerdaten) *an das Hauptfenster* (!), nachdem der Befehl `tablelist::setThemeDefaults` bei der Verarbeitung der virtuellen Ereignisse `<<ThemeChanged>>`, `<<LightAppearance>>` (früher: `<<LightAqua>>`) und `<<DarkAppearance>>` (früher: `<<DarkAqua>>`) aufgerufen wurde. Verwenden Sie zur Behandlung das Bindungs-Tag `TablelistMain`, das von Tablelist_tile dem Hauptfenster zugewiesen wird.

ZEILENINDIZES

Viele der Widget-Befehle für Tablelist-Widgets nehmen einen oder mehrere Zeilenindizes als Argumente entgegen. Ein Zeilenindex legt ein bestimmtes Element der Datenbereich-Komponente der Tablelist fest auf eine der folgenden Weisen:

number	Legt das Element als numerischen Index fest, wobei 0 dem ersten Element im Datenbereich der Tablelist entspricht.

knumber	Legt fest, welches Element es ist, anhand seines vollständigen Schlüssels, der sich aus dem Präfix k und der dem Element zugeordneten Sequenznummer zusammensetzt. Sie können den Widget-Befehl getkeys verwenden, um diese Sequenznummer abzurufen, oder den Widget-Befehl getfullkeys, um den vollständigen Schlüssel abzurufen. Darüber hinaus geben die Unterbefehle insert, insertlist, insertchildren und insertchildlist die Liste der vollständigen Schlüssel zurück, die den gerade eingefügten Elementen zugeordnet sind.
active	Gibt den Eintrag an, der das Element enthält, auf dem sich der Positionscursor befindet. Je nach Auswahltyp wird dieses Element als Ganzes oder nur das Element, auf dem sich der Positionscursor befindet, entsprechend dem Wert der Konfigurationsoption -activestyle angezeigt, wenn die Tablelist den Tastaturfokus hat. Dieses Element wird mit dem Widget-Befehl activate oder als die Zeile festgelegt, die das mit dem Widget-Befehl activatecell angegebene Element enthält.
anchor	Gibt den Ankerpunkt für die Zeilenauswahl an, der mit dem Widget-Befehl selection anchor festgelegt wird, oder die Zeile, die den Ankerpunkt für die Zellenauswahl enthält, der mit dem Widget-Befehl cellselection anchor festgelegt wird.
end	Zeigt das Ende der Tablelist an. Bei den meisten Befehlen bezieht sich dies auf den letzten Eintrag in der Tablelist, bei einigen wenigen Befehlen wie index, insert, insertlist und showtargetmark before sowie für das Ziel des Befehls move bezieht es sich auf den Eintrag unmittelbar nach dem letzten.
last	Bezeichnet das letzte Element der Tablelist.
top	Gibt das oberste, im Tablelist-Fenster sichtbare Element an.
bottom	Gibt das unterste, im Tablelist-Fenster sichtbare Element an.
@x,y	Gibt das Element des Datenbereichs an, das den durch x und y (in Pixelkoordinaten) angegebenen Punkt im Tablelist-Fenster abdeckt. Wenn kein Element diesen Punkt überdeckt, wird das diesem Punkt am nächsten liegende Element verwendet. Die Koordinaten x und y beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Datenbereich-Komponente).
name	Legt die Zeile anhand des Werts der Konfigurationsoption -name fest. name muss sich von allen oben genannten Zeilenindizes unterscheiden und sollte eindeutig sein (wenn mehrere Zeilen denselben Namen haben, wird dieser Wert als Hinweis auf die erste übereinstimmende Zeile angesehen).

In den nachstehenden Beschreibungen der Widget-Befehle enthalten die Argumente mit den

Namen `index`, `firstIndex`, `lastIndex`, `sourceIndex` und `targetIndex` immer Zeilenindizes in einer der oben genannten Formen.

KNOTENINDIZES

Viele der Widget-Befehle für Tablelist-Widgets nehmen einen Knotenindex als Argument entgegen. Ein Knotenindex ist entweder das Wort `root`, das den unsichtbaren Stammknoten einer als Baum-Widget verwendeten Tablelist bezeichnet, oder ein regulärer Zeilenindex in einer der oben genannten Formen, der ein bestimmtes Element der Tablelist festlegt.

In den nachstehenden Beschreibungen der Widget-Befehle enthalten die Argumente mit den Namen `nodeIndex`, `parentNodeIndex` und `targetParentNodeIndex` stets Knotenindizes.

SPALTENINDIZES

Viele der Widget-Befehle für Tablelist-Widgets nehmen einen oder mehrere Spaltenindizes als Argumente entgegen. Ein Spaltenindex legt eine bestimmte Spalte der Tablelist fest auf eine der folgenden Weisen:

number	Legt die Spalte als numerischen Index fest, wobei 0 der ersten Spalte in der Tablelist entspricht.
active	Gibt die Spalte an, in der sich das Element befindet, auf dem der Positionscursor steht. Wenn der Auswahltyp cell lautet, wird dieses Element entsprechend dem Wert der Konfigurationsoption -activestyle angezeigt, wenn die Tablelist den Tastaturfokus hat. Dieses Element wird mit dem Widget-Befehl activatecell festgelegt.
anchor	Gibt die Spalte an, die den Anker für die Auswahl der Zellen enthält, der mit dem Widget-Befehl cellselection anchor festgelegt wird.
end	Bezeichnet die letzte Spalte der Tablelist, mit Ausnahme der Befehle insertcolumns und insertcolumnlist, sowie für das Ziel des Befehls movecolumn; in diesen Fällen bezieht es sich auf die Spalte unmittelbar nach der letzten.
last	Bezeichnet die letzte Spalte der Tablelist.
left	Gibt die Spalte ganz links an, die im Tablelist-Fenster sichtbar ist.
right	Gibt die Spalte ganz rechts an, die im Tablelist-Fenster sichtbar ist.
@x,y	Gibt die Spalte an, die den durch x und y (in Pixelkoordinaten) angegebenen Punkt im Tablelist-Fenster abdeckt. Wenn keine Spalte diesen Punkt abdeckt, wird die diesem Punkt am nächsten gelegene Spalte verwendet. Die Koordinaten x und y beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Datenbereich-Komponente).

name	Legt die Spalte anhand des Werts ihrer Konfigurationsoption -name fest. name muss sich von allen oben genannten Spaltenindizes unterscheiden und sollte eindeutig sein (wenn mehrere Spalten denselben Namen haben, wird dieser Wert so interpretiert, dass er auf die erste übereinstimmende Spalte verweist).

In den folgenden Beschreibungen der Widget-Befehle enthalten die Argumente mit den Namen `columnIndex`, `firstColumn`, `lastColumn`, `sourceColumn` und `targetColumn` immer Spaltenindizes in einer der oben genannten Formen.

ZELLENINDIZES

Viele der Widget-Befehle für Tablelist-Widgets nehmen einen oder mehrere Zellenindizes als Argumente entgegen. Ein Zellenindex legt eine bestimmte Zelle der Datenbereich-Komponente der Tablelist fest auf eine der folgenden Weisen:

row,col	Bezeichnet die Zelle mit dem Index für die Zeile row und dem Index für die Spalte col. row kann eine Zahl, ein vollständiger Schlüssel (in der Form knumber), active, anchor, end (wobei letzteres die letzte Zeile in der Tablelist bezeichnet), last, top, bottom oder ein Zeilenname sein. col kann eine Zahl sein, active, anchor, end, last, left, right oder ein Spaltenname sein.
active	Gibt das Element an, auf dem sich der Positionscursor befindet. Wenn der Auswahltyp cell lautet, wird dieses Element entsprechend dem Wert der Konfigurationsoption -activestyle angezeigt, wenn die Tablelist den Tastaturfokus hat. Dieses Element wird mit dem Widget-Befehl activatecell festgelegt.
anchor	Gibt den Anker für die Zellenauswahl an, der mit dem Widget-Befehl cellselection anchor festgelegt wird.
end	Bezeichnet die letzte Zelle in der letzten Zeile der Tablelist.
last	Bezeichnet die letzte Zelle in der letzten Zeile der Tablelist (entspricht end).
@x,y	Gibt die Zelle des Datenbereichs an, die den durch x und y (in Pixelkoordinaten) angegebenen Punkt im Tablelist-Fenster abdeckt. Wenn keine Zelle des Datenbereichs diesen Punkt abdeckt, wird die diesem Punkt am nächsten gelegene Zelle des Datenbereichs verwendet. Die Koordinaten x und y beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Datenbereich-Komponente).

In den nachstehenden Beschreibungen der Widget-Befehle enthalten die Argumente mit den Namen `cellIndex`, `firstCell` und `lastCell` stets Zellenindizes in einer der oben genannten Formen.

KOPFZEILENINDIZES

Viele der Unterbefehle des Befehls `header` für das Widget "tablelist" nehmen einen oder mehrere Indizes der Kopfzeile als Argumente entgegen. Ein Index der Kopfzeile legt ein bestimmtes Element der Kopfbereich-Komponente der Tablelist fest auf eine der folgenden Weisen:

number	Legt den Eintrag des Kopfbereichs als numerischen Index fest, wobei 0 dem ersten Eintrag im Kopfbereich der Tablelist entspricht.
hknumber	Legt den Eintrag des Kopfbereichs anhand seines vollständigen Schlüssels fest, der sich aus dem Präfix hk und der dem Eintrag zugeordneten Sequenznummer zusammensetzt. Sie können den Widget-Befehl header getkeys verwenden, um diese Sequenznummer abzurufen, oder den Widget-Befehl header getfullkeys , um den vollständigen Schlüssel abzurufen. Darüber hinaus geben die Unterbefehle header insert und header insertlist die Liste der vollständigen Schlüssel zurück, die den gerade eingefügten Kopfposten zugeordnet sind.
end	Zeigt das Ende des Kopfbereichs der Tablelist an. Bei den meisten Befehlen bezieht sich dies auf den letzten Eintrag in der Kopfzeilenkomponente, bei einigen wenigen Befehlen wie beispielsweise header index, header insert und header insertlist bezieht es sich auf den Eintrag des Kopfbereichs unmittelbar nach dem letzten.
last	Bezeichnet den letzten Kopfzeilenposten der Tablelist.
@x,y	Gibt den Eintrag des Kopfbereichs an, der den durch x und y (in Pixelkoordinaten) angegebenen Punkt im Tablelist-Fenster abdeckt. Wenn kein Kopfzeilenelement diesen Punkt abdeckt, wird das diesem Punkt am nächsten liegende Kopfzeilenelement verwendet. Die Koordinaten x und y beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Kopfbereich-Komponente).
name	Legt die Kopfzeile anhand des Werts der Konfigurationsoption -name fest. name muss sich von allen oben genannten Indizes der Kopfzeilen unterscheiden und sollte eindeutig sein (wenn mehrere Kopfzeilen denselben Namen haben, wird dieser Wert so interpretiert, dass er auf die erste übereinstimmende Kopfzeile verweist).

In den nachstehenden Beschreibungen der Widget-Befehle enthalten die Argumente mit den Namen `headerIndex`, `firstHeaderIndex` und `lastHeaderIndex` stets Indizes für Kopfzeilen in einer der oben genannten Formen.

KOPFZELLENINDIZES

Viele der Unterbefehle des Befehls `header` für das Widget "Tablelist" nehmen einen oder mehrere Indizes von Kopfzellen als Argumente entgegen. Ein Index einer Kopfzelle legt eine bestimmte Zelle der Kopfbereich-Komponente der Tablelist fest auf eine der folgenden

Weisen:

headerRow,col	Bezeichnet die Zelle mit dem Kopfzeilenindex headerRow und dem Spaltenindex col. headerRow kann eine Zahl, ein vollständiger Schlüssel (in der Form hknumber), end (wobei Letzteres die letzte Kopfzeile in der Tablelist bezeichnet), last oder ein Kopfzeilenname sein. col kann eine Zahl, active, anchor, end, last, left, right oder ein Spaltenname sein.
end	Bezeichnet die letzte Zelle in der letzten Zeile des Kopfbereichs der Tablelist.
last	Bezeichnet die letzte Zelle in der letzten Zeile des Kopfbereichs der Tablelist (entspricht end).
@x,y	Gibt die Kopfzelle an, die den durch x und y festgelegten Punkt im Tablelist-Fenster abdeckt (in Pixelkoordinaten). Wenn keine Kopfzeilenzelle diesen Punkt abdeckt, wird die diesem Punkt am nächsten gelegene Kopfzeilenzelle verwendet. Die Koordinaten x und y beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Datenbereich-Komponente).

In den nachstehenden Beschreibungen der Widget-Befehle enthalten die Argumente mit den Namen `headerCellIndex`, `firstHeaderCell` und `lastHeaderCell` stets Kopfzellenindizes in einer der oben genannten Formen.

WIDGET-BEFEHL

Der Befehl `tablelist::tablelist` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und `arg` legen das genaue Verhalten des Befehls fest. Für Tablelist-Widgets sind die folgenden Befehle möglich:

`pathName` **activate** `index`

Setzt das aktive Element auf das durch `index` angegebene Element, wenn `state` der Tablelist nicht `disabled` ist. Wenn `index` außerhalb des Bereichs der Elemente in der Tablelist liegt oder es sich um ein nicht sichtbares Element handelt, wird das nächstgelegene sichtbare Element aktiviert. Das aktive Element wird gemäß der Konfigurationsoption `-activestyle` gezeichnet, wenn das Widget den Fokus hat und der Auswahltyp `row` ist. Sein Index kann mit dem Index `active` abgerufen werden. Gibt eine leere Zeichenkette zurück.

`pathName` **activatecell** `cellIndex`

Setzt das aktive Element auf das durch `cellIndex` angegebene Element, wenn `state` der Tablelist nicht `disabled` ist. Wenn `cellIndex` außerhalb des Bereichs der Elemente in der Tablelist liegt oder es sich um ein nicht sichtbares Element handelt, wird das nächstgelegene sichtbare Element aktiviert. Das aktive Element wird gemäß der Konfigurationsoption

`-activestyle` gezeichnet, wenn das Widget den Fokus hat und der Auswahltyp `cell` ist. Sein Index kann mit dem Zellenindex `active` abgerufen werden. Gibt eine leere Zeichenkette zurück.

pathName **applysorting** `itemList`
Sortiert die Elemente von `itemList` gemäß den Parametern des letzten Aufrufs von `sort`, `sortbycolumn` oder `sortbycolumnlist` und gibt eine neue, sortierte Liste zurück. Das Argument `itemList` soll eine wohlgeformte Liste von Listen sein, wobei die Länge jeder Unterliste mindestens der Anzahl der Spalten des Tablelist-Widgets entsprechen muss. Wenn die Elemente noch gar nicht sortiert wurden oder die Sortierinformationen durch den Aufruf von `resetsortinfo` zurückgesetzt wurden, findet keine Sortierung statt und der Rückgabewert entspricht `itemList`.

HINWEIS: Dieser Unterbefehl wird in der Regel aus dem Befehl heraus aufgerufen, der als Wert der Konfigurationsoption `-expandcommand` angegeben ist, und zwar für eine Zeile der Tablelist, deren untergeordnete Elemente noch nicht in das Widget eingefügt wurden. Die effizienteste Methode, die untergeordneten Elemente in der richtigen Reihenfolge einzufügen, besteht darin, sie einer Liste hinzuzufügen, diese Liste mit `applysorting` zu sortieren und anschließend die Elemente der sortierten Liste durch Aufruf des Unterbefehls `insertchildlist` einzufügen.

pathName **attrib**`?name? ?value name value ...?`
Fragt die Attribute des Widgets ab oder ändert sie. Ist kein `name` angegeben, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Ist `name` ohne `value` angegeben, gibt der Befehl den Wert des genannten Attributs zurück, oder eine leere Zeichenkette, wenn kein entsprechender Wert existiert (mit dem Unterbefehl `hasattrib` unterscheiden Sie diesen Fall von dem, in dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Sind ein oder mehrere Paare aus `name` und `value` angegeben, setzt der Befehl die genannten Widget-Attribute auf die genannten Werte; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `name` kann eine beliebige Zeichenkette sein.

pathName **autoscrolltarget** `event x y`
Dieser Befehl ist für die Verwendung während eines Vorgangs des Ziehens und Ablegens vorgesehen, bei dem das Tablelist-Widget (oder dessen Datenbereich-Komponente) als Ablegeziel für TkDND oder das in BWidget enthaltene Drag-&-Drop-Framework registriert wurde. Im ersten Fall (TkDND) wird davon ausgegangen, dass der Befehl aus den Bindungsskripten aufgerufen wurde, die den virtuellen Ereignissen `<<DropEnter>>` und `<<DropPosition>>` zugeordnet sind, und dass seine Argumente wie folgt lauten: *event* ist der Wert des Feldes `%e`, während *x* und *y* durch Transformation der durch die Felder `%X` und `%Y` angegebenen Stammkoordinaten in Koordinaten relativ zum Tablelist-Fenster ermittelt wurden. Im zweiten Fall (BWidget) wird davon ausgegangen, dass der Befehl aus dem Befehl heraus aufgerufen wurde, der als Wert der Option `-dropovercmd` von `DropSite::register` angegeben ist, und seine Argumente lauten wie folgt: *event* ist der Wert des dritten Arguments, das an diesen Befehl übergeben wurde, während *x* und *y* durch Transformation der Stammkoordinaten ermittelt wurden, die als viertes und fünftes Argument an diesen Befehl übergeben wurden.

Ein Aufruf dieses Unterbefehls wie oben beschrieben bewirkt, dass, wenn der Mauszeiger den

Bereich verlässt, der sich ergibt, wenn Sie auf jeder Seite einige Pixel vom scrollbaren Teil des Datenbereichs der Tablelist abziehen, ein automatisches Scrollen weg von der Maus gestartet wird, das so lange fortgesetzt wird, bis es durch den Unterbefehl `stopautoscroll` abgebrochen wird, der Mauszeiger diesen Bereich wieder betritt oder kein weiteres Scrollen in diese Richtung mehr möglich ist. Der scrollbare Teil des Datenbereichs der Tablelist ist der Datenbereich selbst oder der Bereich rechts von den Titelspalten, wenn der Wert der Option `-titlecolumns` positiv ist.

Beispiele für die Verwendung dieses Unterbefehls beim Ziehen und Ablegen finden Sie im Abschnitt UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN.

`pathName` **bbox** `index`

Gibt eine Liste mit vier Zahlen zurück, die den Begrenzungsrahmen der durch `index` angegebenen Zeile beschreiben. Die ersten beiden Elemente der Liste geben die x- und y-Koordinaten der oberen linken Ecke des von der Zeile eingenommenen Bildschirmbereichs an (angegeben in Pixeln relativ zum Widget), und die letzten beiden Elemente geben die Breite und Höhe des Bereichs in Pixeln an. Wenn kein Teil der durch `index` angegebenen Zeile auf dem Bildschirm sichtbar ist oder wenn `index` auf eine nicht vorhandene Zeile verweist, dann ist das Ergebnis eine leere Zeichenkette; Wenn die Zeile teilweise sichtbar ist, gibt das Ergebnis den gesamten Bereich der Zeile an, einschließlich aller nicht sichtbaren Teile.

`pathName` **bodypath**

Gibt den Pfadnamen der Datenbereich-Komponente des Widgets zurück.

`pathName` **bodytag**

Gibt den Namen eines Bindungstags zurück, dessen Name vom Pfadnamen des Tablelist-Widgets abhängt und das dem Datenbereich der Tablelist, den Trennlinien sowie den Message- und Label-Widgets zugeordnet ist, die jeweils zur Anzeige mehrzeiliger Elemente und Bilder verwendet werden. Dieses Bindungs-Tag steht in der Liste der Bindungs-Tags der oben genannten "Tablelist"-Nachkommen vor dem Tag `TablelistBody` und ist für die Verwendung bei der Definition individueller Bindungs-Skripte für "Tablelist"-Widgets vorgesehen. Der Hauptvorteil der Verwendung dieses Tags anstelle des Pfadnamens des Datenbereichs der Tablelist besteht darin, dass Sie damit Skripte zur Ereignisbehandlung schreiben können, die nicht nur für den Datenbereich der Tablelist, sondern auch für die Trennlinien, mehrzeiligen Zellen und eingebettete Bilder gültig sind.

Um beispielsweise den Datenbereich eines Tablelist-Widgets `$tbl` als Ziehquelle für Maustaste 1 so zu registrieren, dass das Ziehen auch für Trennlinien, mehrzeilige Zellen und eingebettete Bilder funktioniert, können Sie wie in den folgenden Code-Beispielen vorgehen:

TkDND-BEISPIEL:

```
package require tkdnd 2.7

bind [$tbl bodytag] <Button-1> { registerDragSrc %W }

proc registerDragSrc w {
    # $w can be the tablelist's body, a separator, a message widget
    # displaying a multi-line element, or a label widget displaying an
```

```
    # image.  Register this widget as a drag source for mouse button 1
    tkdnd::drag_source register $w DND_Text 1
    bind $w <<DragInitCmd>> { onTblDragInit %W }
    bind $w <<DragEndCmd>>  { onTblDragEnd %W %A }
}

proc onTblDragInit w {
    set tbl [tablelist::getTablelistPath $w]
    . . .
}

proc onTblDragEnd {w action} {
    if {![string equal $action "refuse_drop"]} {        ;# drop accepted
        set tbl [tablelist::getTablelistPath $w]
        . . .
    }
}

# Make sure to register the tablelist's body explicitly as a drag source:
registerDragSrc [$tbl bodypath]
```

BWidget-BEISPIEL:

```
package require BWidget

bind [$tbl bodytag] <Button-1> { registerDragSrc %W }

proc registerDragSrc w {
    # $w can be the tablelist's body, a separator, a message widget
    # displaying a multi-line element, or a label widget displaying an
    # image.  Register this widget as a drag source for mouse button 1
    DragSite::register $w -dragevent 1 \
        -draginitcmd tblDragInitCmd -dragendcmd tblDragEndCmd
}

proc tblDragInitCmd {dragSrc rootX rootY top} {
    set tbl [tablelist::getTablelistPath $dragSrc]
    . . .
}

proc tblDragEndCmd {dragSrc dropTarget op dataType data result} {
    if {$result != 0} {                                 ;# drop accepted
        set tbl [tablelist::getTablelistPath $dragSrc]
        . . .
    }
}

# Make sure to register the tablelist's body explicitly as a drag source:
registerDragSrc [$tbl bodypath]
```

In beiden oben genannten Beispielen rufen wir die Prozedur `registerDragSrc` auch explizit für den Datenbereich der Tablelist auf, da das Widget, das durch den Aufruf bei einem `<Button-1>`-Ereignis als Ziehquelle registriert wurde, sich von der

Datenbereich-Komponente unterscheiden kann, und die Vorgabebindungen benötigen die Information, ob *der Datenbereich der Tablelist* als Ziehquelle registriert wurde.

pathName **canceledediting**
Gibt den Wert 1 zurück, wenn die letzte interaktive Zellenbearbeitung abgebrochen wurde (durch den Benutzer oder programmgesteuert mithilfe des Unterbefehls `canceledediting`), andernfalls 0.

HINWEIS: Durch Aufruf dieses Unterbefehls innerhalb des durch die Konfigurationsoption `-editendcommand` festgelegten Tcl-Befehls können Sie feststellen, ob die Bearbeitungssitzung abgebrochen oder normal beendet wurde. Dies könnte von Interesse sein, wenn die Option `-forceeditendcommand` auf "true" gesetzt wurde, da in diesem Fall der Tcl-Befehl auch dann aufgerufen wird, wenn die Bearbeitung abgebrochen wird (in diesem Fall ist der als letztes Argument übergebene Text der ursprüngliche Inhalt des Bearbeitungsfensters und nicht dessen endgültiger Inhalt).

pathName **cancelediting**
Dieser Unterbefehl bricht die interaktive Bearbeitung des Inhalts der Zelle ab, deren Index an den Unterbefehl `editcell` übergeben wurde, löscht das in die Zelle eingebettete temporäre Widget und stellt den ursprünglichen Zellinhalt wieder her. Dieser Befehl ermöglicht es Ihnen, die interaktive Zellenbearbeitung aus dem durch die Konfigurationsoption `-editstartcommand` festgelegten Tcl-Befehl heraus abzubrechen, wenn bei der Rückruffunktion vor der Bearbeitung ein Fehler bei der Vorbereitung des in das Bearbeitungsfenster einzufügenden Textes auftritt. Der Befehl wird außerdem implizit durch Drücken der Taste `Escape` aufgerufen, wenn eine Zelle bearbeitet wird. Der Rückgabewert ist eine leere Zeichenkette. Bevor dieser Wert zurückgegeben wird, stellt der Befehl den ursprünglichen Wert der Option `-exportselection` des Tablelist-Widgets wieder her und generiert das virtuelle Ereignis `<<TablelistCellRestored>>`. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis so generiert, dass seine Option `-data` auf eine Liste gesetzt ist, die aus den numerischen Zeilen- und Spalten-Indizes der Zelle besteht, deren Inhalt gerade bearbeitet wird. Wurde zum Zeitpunkt der Ausführung des Befehls keine Zelle bearbeitet, gibt der Befehl eine leere Zeichenkette zurück, ohne dass ein virtuelles Ereignis generiert wird.

pathName **cellattrib** `cellIndex ?name? ?value name value ...?`
Fragt die Attribute der durch `cellIndex` angegebenen Zelle ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs der Zelle enthält. Wenn `name` festgelegt wird, ohne `value`, gibt der Befehl den Wert des einen benannten Zellenattributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (Sie können den Unterbefehl `hascellattrib` verwenden, um diesen Fall von dem zu unterscheiden, bei dem der Wert eines *vorhandenen* Zellenattributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Zellattribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `name` kann eine beliebige Zeichenkette sein.

pathName **cellbbox** `cellIndex`
Gibt eine Liste mit vier Zahlen zurück, die den Begrenzungsrahmen der durch `cellIndex` angegebenen Zelle beschreiben. Die ersten beiden Elemente der Liste geben die x- und y-Koordinaten der oberen linken Ecke des von der Zelle abgedeckten Bildschirmbereichs an

(angegeben in Pixeln relativ zum Widget), und die letzten beiden Elemente geben die Breite und Höhe des Bereichs in Pixeln an. Wenn kein Teil der durch `cellIndex` angegebenen Zelle auf dem Bildschirm sichtbar ist oder wenn `cellIndex` auf eine nicht vorhandene Zelle verweist, dann ist das Ergebnis eine leere Zeichenkette; Wenn die Zelle teilweise sichtbar ist, gibt das Ergebnis den gesamten Bereich der Zelle an, einschließlich aller nicht sichtbaren Teile.

`pathName` **cellcget** `cellIndex option`
Gibt den aktuellen Wert der durch `option` angegebenen Zellenkonfigurationsoption für die durch `cellIndex` angegebene Zelle zurück. `option` kann einen beliebigen Wert annehmen, der vom Befehl `cellconfigure` akzeptiert wird.

`pathName` **cellconfigure** `cellIndex ?option? value option value ...?`
Ruft die Konfigurationsoptionen der durch `cellIndex` angegebenen Zelle ab oder ändert sie. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle für die Zelle verfügbaren Optionen beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Werts, wenn kein `option` angegeben wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Zellenoptionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann einen beliebigen der im Abschnitt ZELLENKONFIGURATIONSOPTIONEN IM DATENBEREICH beschriebenen Werte annehmen.

`pathName` **cellindex** `cellIndex`
Gibt den kanonischen Zellenindex-Wert zurück, der `cellIndex` entspricht, und zwar in der Form `row,col`, wobei `row` und `col` Ganzzahlen sind.

`pathName` **cellselection** `option args`
Dieser Befehl dient dazu, die Auswahl der Zellen innerhalb des Datenbereichs eines Tablelist-Widgets anzupassen. Je nach `option` gibt es verschiedene Formen dieses Befehls:

`pathName` **cellselection anchor** `cellIndex`
Setzt den Anker der Zellenauswahl auf das durch `cellIndex` angegebene Element. Wenn `cellIndex` auf ein nicht vorhandenes oder nicht sichtbares Element verweist, wird das nächstgelegene sichtbare Element verwendet. Der Zellauswahl-Anker ist das Ende der Zellauswahl, das beim Ziehen einer Zellauswahl mit der Maus fixiert bleibt, wenn der Auswahltyp `cell` ist. Der Zellindex `anchor` kann verwendet werden, um auf den Anker zu verweisen.

`pathName` cellselection clear `firstCell lastCell`
`pathName cellselection clear cellIndexList`: Wenn eines der Elemente zwischen `firstCell` und `lastCell` (einschließlich) oder eines der Elemente, die den in der Liste `cellIndexList` angegebenen Zellenindizes entsprechen, ausgewählt ist, wird die Auswahl aufgehoben. Die Auswahl der Elemente, die außerhalb des in der ersten Form des Befehls angegebenen Bereichs liegen oder sich von den in der zweiten Form des Befehls angegebenen Zellenindexen unterscheiden, wird nicht geändert.

`pathName` **cellselection includes** `cellIndex`
Gibt `1` zurück, wenn das durch `cellIndex` angegebene Element derzeit ausgewählt ist, andernfalls `0`.

`pathName cellselection set firstCell lastCell`
`pathName cellselection set cellIndexList`

Wählt alle auswählbaren Elemente im Bereich zwischen `firstCell` und `lastCell` (einschließlich) aus oder diejenigen, die den in der Liste `cellIndexList` festgelegten Indizes entsprechen, ohne die Auswahl anderer Elemente zu beeinflussen. Ein Element gilt genau dann als auswählbar, wenn der Wert der Option `-selectable` der Zeile, in der es enthalten ist, "true" ist.

Wenn der Wert `state` der Tablelist `disabled` entspricht und sich `option` von `includes` unterscheidet, gibt der Befehl lediglich eine leere Zeichenkette zurück, ohne eine der oben genannten Aktionen auszuführen.

`pathName` **`cget`** `option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen der vom Befehl `tablelist::tablelist` akzeptierten Werte annehmen kann.

`pathName` **`childcount`** `nodeIndex`
Gibt die Anzahl der untergeordneten Elemente des durch `nodeIndex` angegebenen Baumknotens zurück. Wird dieses Argument als `root` festgelegt, entspricht der Rückgabewert der Anzahl der Elemente der obersten Ebene des Tablelist-Widgets.

`pathName` **`childindex`** `index`
Gibt den numerischen Index der Zeile zurück, die durch `index` in der Liste der untergeordneten Elemente ihres übergeordneten Knotens angegeben ist.

`pathName` **`childkeys`** `nodeIndex`
Gibt die Liste der vollständigen Schlüssel der untergeordneten Elemente des durch `nodeIndex` angegebenen Baumknotens zurück. Wenn dieses Argument als `root` angegeben wird, ist der Rückgabewert die Liste der vollständigen Schlüssel der Elemente der obersten Ebene, die im Tablelist-Widget enthalten sind.

`pathName` **`collapse`** `indexList ?-fully|-partly?`
Dieser Unterbefehl klappt die angegebenen Zeilen einer als Baum-Widget verwendeten Tablelist zu, d. h., er blendet alle deren untergeordneten Elemente aus. Das optionale Argument `-fully` (das standardmäßig verwendet wird) gibt an, dass der Befehl rekursiv ausgeführt wird, d. h., alle Nachkommen der durch `indexList` angegebenen Knoten werden zugeklappt, sodass bei einem nachfolgenden Aufruf der nicht-rekursiven Version des Unterbefehls `expand(all)` nur deren Kinder, aber keine weiteren Nachkommen davon angezeigt werden. Die Option `-partly` beschränkt die Operation auf nur eine Hierarchieebene, was bedeutet, dass bei einem nachfolgenden Aufruf der nicht-rekursiven Version des Unterbefehls `expand(all)` genau dieselben Nachkommen wieder angezeigt werden, die vor dem Zuklappen der Zeilen sichtbar waren.

Bevor die Nachkommen einer Zeile ausgeblendet werden, wird der als Wert der Option `-collapsecommand` angegebene Befehl (sofern vorhanden) automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das daraus resultierende Skript wird im globalen Geltungsbereich ausgewertet.

HINWEIS: Wenn Sie mehrere Zeilen zusammenklappen möchten, ist es wesentlich effizienter, diesen Unterbefehl für die Liste ihrer Indizes aufzurufen, als ihn in einer Schleife für die einzelnen Zeilen auszuführen.

pathName **collapseall**?`-fully|-partly`?
Dieser Unterbefehl klappt alle Zeilen der obersten Ebene einer als Baum-Widget verwendeten Tablelist zu, d. h., er blendet alle deren untergeordneten Elemente aus. Das optionale Argument `-fully` (das standardmäßig verwendet wird) gibt an, dass der Befehl rekursiv ausgeführt wird, d. h., alle Nachkommen der Knoten der obersten Ebene werden zugeklappt, sodass bei einem anschließenden Aufruf der nicht-rekursiven Version des Unterbefehls `expandall` nur deren Kinder, aber keine weiteren Nachkommen mehr angezeigt werden. Die Option `-partly` beschränkt die Operation auf nur eine Hierarchieebene, was bedeutet, dass bei einem nachfolgenden Aufruf der nicht-rekursiven Version des Unterbefehls `expandall` genau dieselben Elemente erneut angezeigt werden, die vor dem Zuklappen der obersten Knoten sichtbar waren.

Bevor die Nachkommen einer Zeile ausgeblendet werden, wird der als Wert der Option `-collapsecommand` angegebene Befehl (sofern vorhanden) automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das daraus resultierende Skript wird im globalen Geltungsbereich ausgewertet.

pathName **columnattrib** `columnIndex ?name? ?value name value ...?`
Fragt die Attribute der durch `columnIndex` angegebenen Spalte ab oder ändert sie. Wenn kein `name` festgelegt ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für die Spalte enthält. Wenn `name` ohne `value` angegeben wird, gibt der Befehl den Wert des einen benannten Spaltenattributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (Sie können den Unterbefehl `hascolumnattrib` verwenden, um diesen Fall von dem zu unterscheiden, bei dem der Wert eines *vorhandenen* Spaltenattributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, setzt der Befehl das/die angegebene(n) Spaltenattribut(e) auf den/die angegebenen Wert(e); in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `name` kann eine beliebige Zeichenkette sein.

pathName **columncget** `columnIndex option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption für die durch `columnIndex` angegebene Spalte zurück. `option` kann einen beliebigen Wert annehmen, der vom Befehl `columnconfigure` akzeptiert wird.

pathName **columnconfigure** `columnIndex ?option? ?value option value ...?`
Ruft die Konfigurationsoptionen der durch `columnIndex` angegebenen Spalte ab oder ändert sie. Wenn kein `option` festgelegt ist, gibt der Befehl eine Liste zurück, die alle für die Spalte verfügbaren Optionen beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des Werts, der zurückgegeben wird, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Spaltenoptionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann einen beliebigen der im Abschnitt SPALTENKONFIGURATIONSOPTIONEN beschriebenen Werte annehmen.

pathName **columncount**
Gibt die Anzahl der Spalten im Tablelist-Widget zurück.

pathName **columnindex** `columnIndex`
Gibt den ganzzahligen Spaltenindex zurück, der `columnIndex` entspricht.

HINWEIS: Wenn `columnIndex` gleich `end` ist, ist der Rückgabewert der numerische Index der letzten Spalte (nicht die Anzahl der Spalten). Dieses Verhalten unterscheidet sich von dem des Unterbefehls `index`, der (aus Gründen der Kompatibilität mit der Tk-Core-Listbox) für das Argument `end` die Anzahl der Elemente zurückgibt (nicht den Index des letzten Elements).

pathName **columnwidth** `columnIndex ?-requested|-stretched|-total?`

Gibt die aktuelle Breite in Pixeln der durch `columnIndex` angegebenen Spalte zurück. Wenn das optionale Argument `-requested` ist (was der Standardwert ist), dann ist der Rückgabewert die Anzahl der Pixel, die der Spaltenbreite entspricht, die durch die Widget-Konfigurationsoption `-columns` oder die Spaltenkonfigurationsoption `-width` festgelegt wurde (möglicherweise überschrieben durch interaktive Spaltenanpassung); Wenn die Spaltenbreite als `0` festgelegt wurde (und nicht durch interaktive Spaltenanpassung geändert wurde), dann ist der Rückgabewert die tatsächliche Anzahl an Pixeln, die dem breitesten sichtbaren Element der Spalte entspricht, einschließlich ihrer Kopfzeile. Bei Verwendung der Option `-stretched` gibt der Befehl die Spaltenbreite zurück, die sich ergibt, wenn der oben beschriebene Wert um die Anzahl der zusätzlichen Pixel erhöht wird, die möglicherweise durch eine Streckoperation zur angeforderten Spaltenbreite hinzugefügt wurden (siehe die Widget-Konfigurationsoption `-stretch` und die Spaltenkonfigurationsoption `-stretchable`). Wenn das optionale Argument schließlich `-total` ist, so ist der Rückgabewert die gestreckte Spaltenbreite, erhöht um die Anzahl der Pixel, die den linken und rechten Außenabständen innerhalb der Spalte entsprechen; dieser Wert entspricht der Breite der Kopfzeilenbeschriftung, wenn das "tablelist"-Widget abgebildet ist. HINWEIS: Die Breite des linken und rechten Außenabstands innerhalb der Spalte entspricht der Breite des Zeichens `0` in der Schriftart des Widgets.

pathName **configcelllist** `cellConfigSpecList`

Für jeden Zellindex, jede Option und jeden Wert, die durch die Liste `cellConfigSpecList` angegeben sind, ändert der Befehl die angegebene Option der angegebenen Zelle so, dass sie den angegebenen Wert annimmt. Das Argument `cellConfigSpecList` muss eine Liste der folgenden Form sein:

```
cellIndex option value cellIndex option value ...
```

wobei jedes `option` einen der im Abschnitt ZELLENKONFIGURATIONSOPTIONEN IM DATENBEREICH beschriebenen Werte annehmen kann. Der Rückgabewert ist eine leere Zeichenkette.

Dieser Befehl hat dieselbe Wirkung wie

```
eval [list pathName configcells] cellConfigSpecList
```

aber es ist effizienter und einfacher zu bedienen.

pathName **configcells**`?cellIndex option value cellIndex option value ...?`

Für jedes `cellIndex`, `option` und `value` ändert der Befehl die angegebene Option der angegebenen Zelle so, dass sie den angegebenen Wert annimmt. Jedes `option` kann einen der im Abschnitt ZELLENKONFIGURATIONSOPTIONEN IM DATENBEREICH beschriebenen Werte annehmen. Der Rückgabewert ist eine leere Zeichenkette.

pathName **configcolumnlist** `columnConfigSpecList`

Für jeden Spaltenindex, jede Option und jeden Wert, die in der Liste `columnConfigSpecList` angegeben sind, ändert der Befehl die angegebene Option der angegebenen Spalte so, dass sie den angegebenen Wert annimmt. Das Argument `columnConfigSpecList` muss eine Liste der folgenden Form sein:

```
columnIndex option value columnIndex option value ...
```

wobei jedes `option` einen der im Abschnitt SPALTENKONFIGURATIONSOPTIONEN beschriebenen Werte annehmen kann. Der Rückgabewert ist eine leere Zeichenkette.

Dieser Befehl hat dieselbe Wirkung wie

```
eval [list pathName configcolumns] columnConfigSpecList
```

aber es ist effizienter und einfacher zu bedienen.

`pathName` **`configcolumns`**`?columnIndex option value columnIndex option value ...?`

Für jedes `columnIndex`, `option` und `value` ändert der Befehl die angegebene Option der angegebenen Spalte so, dass sie den angegebenen Wert annimmt. Jedes `option` kann einen der im Abschnitt SPALTENKONFIGURATIONSOPTIONEN beschriebenen Werte annehmen. Der Rückgabewert ist eine leere Zeichenkette.

`pathName` **`configrowlist`** `rowConfigSpecList`

Für jeden Zeilenindex, jede Option und jeden Wert, die durch die Liste `rowConfigSpecList` angegeben sind, ändert der Befehl die angegebene Option der angegebenen Zeile so, dass sie den angegebenen Wert annimmt. Das Argument `rowConfigSpecList` muss eine Liste der folgenden Form sein:

```
index option value index option value ...
```

wobei jedes `option` einen der im Abschnitt ZEILENKONFIGURATIONSOPTIONEN IM DATENBEREICH beschriebenen Werte annehmen kann. Der Rückgabewert ist eine leere Zeichenkette.

Dieser Befehl hat dieselbe Wirkung wie

```
eval [list pathName configrows] rowConfigSpecList
```

aber es ist effizienter und einfacher zu bedienen.

`pathName` **`configrows`**`?index option value index option value ...?`

Für jedes `index`, `option` und `value` ändert der Befehl die angegebene Option der angegebenen Zeile so, dass sie den angegebenen Wert annimmt. Jedes `option` kann einen der im Abschnitt ZEILENKONFIGURATIONSOPTIONEN IM DATENBEREICH beschriebenen Werte annehmen. Der Rückgabewert ist eine leere Zeichenkette.

`pathName` **`configure`**`?option? ?value option value ...?`

Ruft die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des

zurückgegebenen Werts, wenn kein `option` angegeben wurde). Wenn ein oder mehrere Paare aus `option` und `value` angegeben werden, ändert der Befehl die angegebenen Widget-Optionen so, dass sie die angegebenen Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann jeden der vom Befehl `tablelist::tablelist` akzeptierten Werte annehmen.

`pathName` **containing** `y`
Bei Angabe einer y-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl den Index des Elements im Datenbereich der Tablelist zurück, das diese y-Koordinate enthält. Wird kein entsprechendes Element gefunden, lautet der Wert `-1`. Die Koordinate `y` muss relativ zum Tablelist-Fenster selbst (nicht zu dessen Datenbereich-Komponente) angegeben werden.

`pathName` **containingcell** `x y`
Bei Angabe einer x- und einer y-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl den Index der Zelle im Datenbereich zurück, die den Punkt mit diesen Koordinaten enthält. Wird keine entsprechende Zelle gefunden, ist die Zeilen- oder Spaltenkomponente (oder beide) des Rückgabewerts `-1`. Die Koordinaten `x` und `y` beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Datenbereich).

`pathName` **containingcolumn** `x`
Bei Angabe einer x-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl den Index der Spalte in der Tablelist zurück, die diese x-Koordinate enthält. Wird keine entsprechende Spalte gefunden, lautet der Rückgabewert `-1`. Die Koordinate `x` muss relativ zum Tablelist-Fenster selbst (nicht zu dessen Datenbereich) sein.

`pathName` **cornerlabelpath**
Gibt den Pfadnamen des Beschriftungs-Widgets zurück, das sich im Container der nordöstlichen Ecke befindet (siehe den Unterbefehl `cornerpath`). Bei Tablelist_tile ist der Rückgabewert ein ttk::label-Widget im gleichen Stil wie die Kopfzeilenbeschriftungen. Die für die Kopfzeilenbeschriftungen festgelegten globalen visuellen Optionen werden automatisch auch auf dieses (ttk::)label-Widget angewendet.

`pathName` **cornerpath**`?-ne|-sw?`
Gibt den Pfadnamen eines von zwei Frame-Widgets zurück, die Geschwisterelemente der Tablelist sind und automatisch zusammen mit dieser erstellt und gelöscht werden. Einer der beiden Frames entspricht der nordöstlichen Ecke der Tablelist, das andere ihrer südwestlichen Ecke. Die Ecke wird über das optionale Argument festgelegt: `-ne` (das ist der Standardwert) steht für Nordosten und `-sw` für Südwesten.

Wenn die Eckspezifikation den Wert `-ne` hat, gibt der Befehl den Pfadnamen des **nordöstlichen Eckrahmens** zurück, der so ausgelegt ist, dass er bei der Verwaltung der vertikalen Scrollbar unter Mac OS X/11+ oder X11 rechts neben der oberen rechten Ecke des Tablelist-Widgets angezeigt wird (siehe Anmerkung 1 unten). Dieser Eckrahmen enthält ein Label oder ein ttk::label-Widget, je nachdem, ob das Paket "Tablelist" oder "Tablelist_tile" verwendet wird (siehe den Unterbefehl `cornerlabelpath`). Die Höhe dieses Eckrahmens wird automatisch an die Höhe der Kopfzeilen-Komponente der Tablelist angepasst, und die Höhe des oben genannten (ttk::)label-Widgets wird automatisch an die Höhe der Kopfzeilenbeschriftungen angepasst. Immer wenn sich die (angeforderte) Höhe der Kopfzeile der Tablelist (und damit die des nordöstlichen Eckrahmens) ändert, wird das virtuelle Ereignis `<<TablelistHeaderHeightChanged>>` ausgelöst, dessen Option `-data` bei Tk-Versionen 8.5 und höher auf die (angeforderte) Höhe der Kopfzeile gesetzt ist.

Wenn die Ecke als `-sw` festgelegt wurde, ist der Rückgabewert der Pfadname des **südwestlichen Eckrahmens**, der so ausgelegt ist, dass er bei der Verwaltung der horizontalen Scrollbar direkt unterhalb der unteren linken Ecke der Tablelist angezeigt wird, sofern der Wert der Option `-titlecolumns` positiv ist (siehe Anmerkung 2 unten). Die Breite dieses Eckrahmens wird automatisch an die Gesamtbreite der nicht ausgeblendeten Titelspalten angepasst. Bei Tablelist_tile ist der südwestliche Eckrahmen ein ttk::frame-Widget. Immer wenn sich die Gesamtbreite der nicht ausgeblendeten Titelspalten (und damit die erforderliche Breite des südwestlichen Eckrahmens) ändert, wird das virtuelle Ereignis `<<TablelistTitleColsWidthChanged>>` generiert, dessen Option `-data` bei Tk-Versionen 8.5 und höher auf die Gesamtbreite der nicht ausgeblendeten Titelspalten gesetzt ist.

HINWEIS 1: Mit diesem Unterbefehl können Sie festlegen, dass die vertikale Scrollbar (wenn vorhanden) unterhalb der Kopfzeile des Tablelist-Widgets angezeigt wird, wodurch das native Look & Feel unter Mac OS X/11+ und auf vielen modernen Linux-Systemen beibehalten wird. Wie im folgenden Beispiel gezeigt, wird empfohlen, eine scrollbare Tablelist zusammen mit der/den Scrollbar(n) stets als untergeordnete Elemente eines (ttk::)frame-Widgets zu erstellen:

```
# Add some entries to the Tk option database
option add *ScrollArea.borderWidth                      1
option add *ScrollArea.relief                           sunken
option add *ScrollArea.Tablelist.borderWidth            0
option add *ScrollArea.Tablelist.highlightThickness     0

. . .

# Create the tablelist and the scrollbars as
# children of a frame of the class ScrollArea
set frm [(ttk::)frame ... -class ScrollArea]
set tbl $frm.tbl
set vsb $frm.vsb
set hsb $frm.hsb
tablelist::tablelist $tbl ... \
    -xscrollcommand [list $hsb set] -yscrollcommand [list $vsb set]
(ttk::)scrollbar $vsb -orient vertical   -command [list $tbl yview]
(ttk::)scrollbar $hsb -orient horizontal -command [list $tbl xview]

. . .

# Manage the widgets within the frame
grid $tbl -row 0 -rowspan 2 -column 0 -sticky news
if {[string compare [tk windowingsystem] "win32"] == 0} {
    grid $vsb -row 0 -rowspan 2 -column 1 -sticky ns
} else {
    grid [$tbl cornerpath] -row 0 -column 1 -sticky ew
    grid $vsb              -row 1 -column 1 -sticky ns
}
grid $hsb -row 2 -column 0 -sticky ew
```

```
grid rowconfigure    $frm 1 -weight 1
grid columnconfigure $frm 0 -weight 1

# Manage the frame
pack $frm -expand yes -fill both -padx 10 -pady 10
```

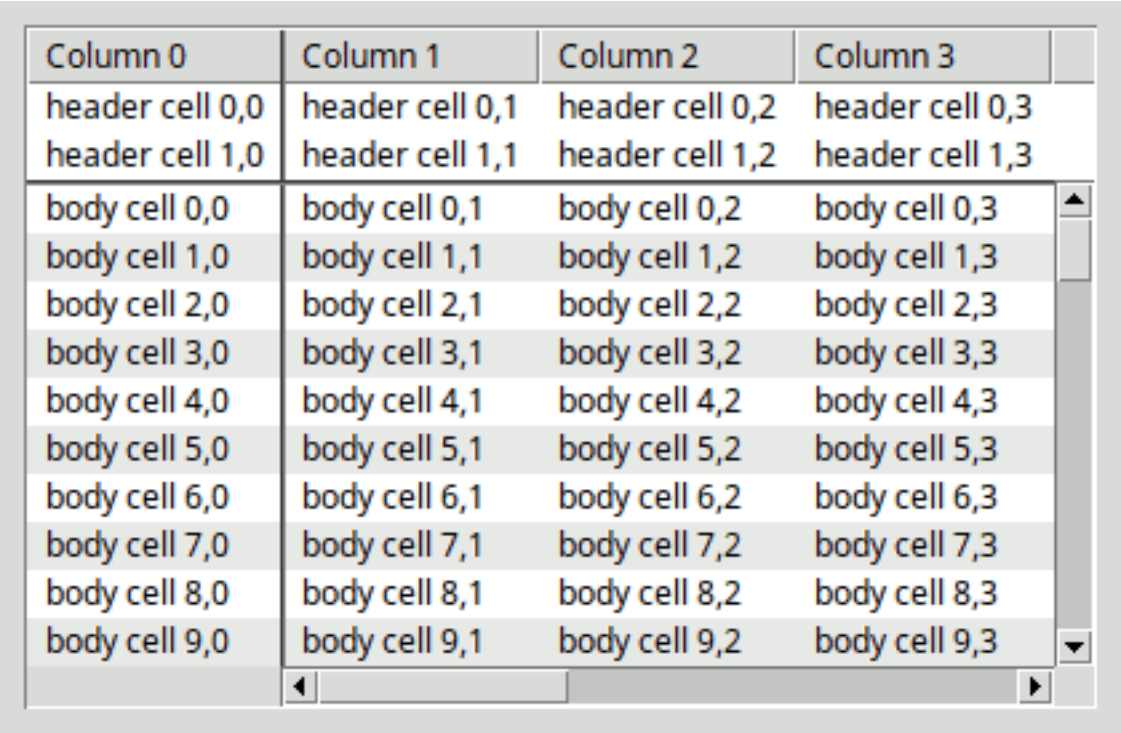
HINWEIS 2: Mit diesem Unterbefehl können Sie außerdem festlegen, dass die horizontale Scrollbar (wenn vorhanden) rechts neben dem nicht scrollbaren Bereich der Titelspalten des Tablelist-Widgets beginnt, wenn der Wert der Option `-titlecolumns` positiv ist. Im obigen Beispiel lässt sich dies erreichen, indem die zweite Hälfte wie folgt geändert wird:

```
. . .

# Manage the widgets within the frame
grid $tbl -row 0 -rowspan 2 -column 0 -columnspan 2 -sticky news
if {[string compare [tk windowingsystem] "win32"] == 0} {
    grid $vsb -row 0 -rowspan 2 -column 2 -sticky ns
} else {
    grid [$tbl cornerpath] -row 0 -column 2 -sticky ew
    grid $vsb              -row 1 -column 2 -sticky ns
}
grid [$tbl cornerpath -sw] -row 2 -column 0 -sticky ns
grid $hsb                  -row 2 -column 1 -sticky ew
grid rowconfigure    $frm 1 -weight 1
grid columnconfigure $frm 1 -weight 1

# Manage the frame
pack $frm -expand yes -fill both -padx 10 -pady 10
```

Bei einem Tablelist-Widget mit zwei Kopfzeilen und einer Titelspalte sehen die Scrollbars des angepassten Beispiels wie im folgenden Screenshot aus (unter Windows gilt dies nur für die horizontale Scrollbar):



Scrollbars

HINWEIS 3: Sie können die Erstellung von scrollbaren Tablelist (und anderen scrollbaren Widgets) erheblich vereinfachen, indem Sie ein scrollarea-Widget verwenden, das Teil des Scrollutil-Pakets ist. Es unterstützt sowohl statische als auch dynamische Scrollbars und berücksichtigt die Kopfzeilen-Komponente sowie die Titelspalten von Tablelist-Widgets. Mithilfe des scrollarea-Widgets lässt sich der recht technische Code aus dem obigen Beispiel durch nur wenige Zeilen ersetzen:

```
package require scrollutil_tile

. . .

# Create the tablelist within a scrollarea
set sa [scrollutil::scrollarea ...]
set tbl $sa.tbl
tablelist::tablelist $tbl ...
$sa setwidget $tbl

. . .

# Manage the scrollarea
pack $sa -expand yes -fill both -padx 10 -pady 10
```

`pathName` **`curcellselection`**`?-all|-nonhidden|-viewable?`

Gibt eine Liste zurück, die die kanonischen Indizes (in der Form `row,col`, wobei `row` und `col` Zahlen sind) aller derzeit ausgewählten Elemente in der Tablelist enthält. Wenn keine solchen Elemente in der Tablelist vorhanden sind, wird eine leere Zeichenkette zurückgegeben.

Das optionale Argument kann verwendet werden, um die Menge der aktuell ausgewählten

Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` in der entsprechenden Zeile oder in der entsprechenden Spalte auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Menge der aktuell ausgewählten Elemente auf die viewable Elemente.

`pathName` **curselection**`?-all|-nonhidden|-viewable?`

Gibt eine Liste zurück, die die numerischen Indizes aller Elemente in der Tablelist enthält, die mindestens ein ausgewähltes Element enthalten. Wenn es in der Tablelist keine solchen Elemente gibt, wird eine leere Zeichenkette zurückgegeben.

Das optionale Argument kann verwendet werden, um die Menge der derzeit (teilweise) ausgewählten Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` in der jeweiligen Zeile auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Menge der aktuell (teilweise) ausgewählten Elemente auf die viewable Elemente.

`pathName delete firstIndex lastIndex`

`pathName delete indexList`: Löscht ein oder mehrere Elemente der Tablelist, wenn deren `state` nicht `disabled` ist. In der ersten Form des Befehls sind `firstIndex` und `lastIndex` Indizes, die das erste bzw. letzte Element des zu löschenden Bereichs festlegen. Die zweite Form des Befehls akzeptiert eine Liste `indexList` mit Indizes, die die zu löschenden Elemente festlegen. In beiden Fällen werden auch alle untergeordneten Elemente der angegebenen Elemente gelöscht. Der Rückgabewert ist eine leere Zeichenkette.

HINWEIS: Wenn das Tablelist-Widget keine Elemente enthält, geben beide Formen des Befehls sofort zurück, ohne dass weitere Aktionen (wie die Validierung der Indizes) durchgeführt werden. Beispielsweise gibt `pathName delete 0 end` sofort zurück, ohne einen Fehler auszulösen, obwohl in diesem Fall beide Indizes ungültig sind.

`pathName deletecolumns firstColumn lastColumn`

`pathName deletecolumns columnIndexList`: Löscht eine oder mehrere Spalten der Tablelist, wenn deren `state` nicht `disabled` ist. In der ersten Form des Befehls sind `firstColumn` und `lastColumn` Indizes, die die erste und letzte Spalte des zu löschenden Bereichs festlegen. Die zweite Form des Befehls akzeptiert eine Liste `columnIndexList` mit Indizes, die die zu löschenden Spalten festlegen. Gibt eine leere Zeichenkette zurück.

HINWEIS: Wenn das Tablelist-Widget keine Spalten enthält, geben beide Formen des Befehls sofort zurück, ohne dass weitere Aktionen (wie die Validierung der Spaltenindizes) durchgeführt werden. Beispielsweise gibt `pathName deletecolumns 0 end` sofort zurück, ohne einen Fehler auszulösen, obwohl in diesem Fall beide Spaltenindizes ungültig sind.

`pathName` **depth** `nodeIndex`

Gibt die Anzahl der Schritte vom durch `nodeIndex` angegebenen Baumknoten zum Knoten `root` zurück. Die Tiefe des letzteren beträgt 0, die der Zeilen auf oberster Ebene 1 und so weiter.

`pathName` **descendantcount** `nodeIndex`

Gibt die Anzahl der Nachkommen des durch `nodeIndex` angegebenen Baumknotens zurück.

Wenn dieses Argument als `root` angegeben wird, entspricht der Rückgabewert der Anzahl der Elemente des Tablelist-Widgets.

pathName **dicttoitem** `dictionary`

Gibt ein Tablelist-Element zurück, das dem durch `dictionary` angegebenen Dictionary entspricht. Die Werte, die den Schlüsseln im angegebenen Dictionary entsprechen und gültige Spaltenindizes in einer der im Abschnitt SPALTENINDIZES beschriebenen Formen sind, werden als Elemente des resultierenden Tablelist-Eintrags interpretiert. Schlüssel, die keine gültigen Spaltenindizes sind, werden stillschweigend ignoriert. Wenn die Anzahl der unterschiedlichen numerischen Entsprechungen der Schlüssel, die gültige Spaltenindizes sind (siehe den Unterbefehl `columnindex`), kleiner ist als die Anzahl der Spalten, werden die Elemente, die den fehlenden Spaltenindizes entsprechen, auf leere Zeichenketten gesetzt.

Wenn Ihr Tablelist-Widget beispielsweise 3 Spalten hat, von denen die ersten beiden mit `-name forename` bzw. `-name surname`konfiguriert wurden, und Sie das Dictionary `{forename Joe surname Black}` an den Unterbefehl `dicttoitem` übergeben, dann ist der Rückgabewert die Liste `{Joe Black {}}`. Wenn der Benutzer die ersten beiden Spalten vertauscht, gibt der Unterbefehl für dasselbe Dictionary die Liste `{Black Joe {}}` zurück.

Dieser Unterbefehl ist nur verfügbar, wenn der Befehl `dict` vorhanden ist. Dies ist der Fall, wenn Tcl Version 8.5 oder höher verwendet wird oder eine der alternativen reinen Tcl-Dict-Implementierungen verwendet wird.

pathName **dumptofile** `fileName`

Schreibt die in der internen Liste des Widgets enthaltenen Elemente des Kopf- und Datenbereichs sowie Informationen zu den Spalten, der Sortierung und der Baumstruktur in die Datei, deren Pfadname durch `fileName` angegeben wird. Bevor die Daten in die Datei geschrieben werden, erweitert der Unterbefehl alle Zeilen der Tablelist durch Aufruf von `expandall -fully`. Den Inhalt der Datei können Sie später mit Hilfe des Unterbefehls `loadfromfile` laden.

HINWEIS: Da die Ausführung des Unterbefehls `expandall` recht lange dauern kann (mehrere Sekunden oder sogar Minuten), wird empfohlen, den Aufruf von `dumptofile` zwischen `setbusycursor` und `restorecursor` einzufügen.

pathName **dumptostring**

Gibt eine Zeichenkette zurück, die aus den in der internen Liste des Widgets enthaltenen Elementen des Kopf- und Datenbereichs sowie aus Informationen zu den Spalten, der Sortierung und der Baumstruktur besteht. Vor der Erstellung dieser Zeichenkette erweitert der Unterbefehl alle Zeilen der Tablelist durch Aufruf von `expandall -fully`. Sie können die zurückgegebene Zeichenkette später mit Hilfe des Unterbefehls `loadfromstring` laden.

HINWEIS: Da die Ausführung des Unterbefehls `expandall` recht lange dauern kann (mehrere Sekunden oder sogar Minuten), wird empfohlen, den Aufruf von `dumptostring` zwischen `setbusycursor` und `restorecursor` einzufügen.

pathName **editcell** `cellIndex`

Startet die interaktive Bearbeitung des Inhalts der Zelle, wenn die Option `state` der Tablelist nicht auf `disabled` gesetzt ist und die Zelle sowohl viewable als auch bearbeitbar ist. Gibt

eine leere Zeichenkette zurück. Bevor dieser Wert zurückgegeben wird, setzt der Befehl die Option `-exportselection` des Tablelist-Widgets auf `0` und stellt so sicher, dass das Widget die Auswahl nicht verliert, während der Inhalt der Zelle bearbeitet wird. (HINWEIS: Nach Beendigung der Bearbeitungssitzung wird der ursprüngliche Wert der Option wiederhergestellt.) Weitere Informationen zur Bearbeitbarkeit und zum Bearbeitungsprozess finden Sie im Abschnitt INTERAKTIVE ZELLENBEARBEITUNG.

`pathName` **editinfo**

Gibt eine Liste mit drei Elementen zurück, die Informationen über die Tablelist-Zelle enthält, deren Inhalt gerade bearbeitet wird. Das erste Element der Liste ist der vollständige Schlüssel (in der Form `knumber`), der dem Element zugeordnet ist, das das gerade bearbeitete Tablelist-Element enthält. Das zweite und dritte Element sind die Zeilen- und Spalten-Indizes der Zelle. Wird derzeit keine Zelle bearbeitet, ist der Rückgabewert die Liste `{"" -1 -1}`.

`pathName` **editwinpath**

Gibt den Pfadnamen des temporären eingebetteten Widgets zurück, das für die interaktive Zellbearbeitung verwendet wird und durch den Unterbefehl `editcell` erstellt wurde. Wenn derzeit keine Zelle bearbeitet wird, ist der Rückgabewert eine leere Zeichenkette. Mit diesem Unterbefehl können Sie innerhalb der durch die Konfigurationsoptionen `-editstartcommand` und `-editendcommand` festgelegten Befehle auf das Bearbeitungsfenster zugreifen.

`pathName` **editwintag**

Gibt den Namen eines Bindungs-Tags zurück, dessen Name vom Pfadnamen des Tablelist-Widgets abhängt und das einigen Komponenten des temporären eingebetteten Widgets zugeordnet ist, das für die interaktive Zellenbearbeitung verwendet wird. Diese Komponenten hängen vom Bearbeitungsfenster ab: das Widget selbst im Falle eines Tk- oder Tile-Checkbuttons oder -Menubuttons; die "Entry"-Unterelemente des Bearbeitungsfensters im Falle eines "Mentry"-Widgets; das in einem als Bearbeitungsfenster verwendeten Toggle-Switch enthaltene ttk::scale-Widget; in allen anderen Fällen die einzige "Entry"-Eingabekomponente oder eine "Entry"-ähnliche Komponente des Bearbeitungsfensters (siehe auch den Unterbefehl `entrypath`). Dieses Bindungs-Tag steht in der Liste der Bindungs-Tags der oben genannten Komponenten des Bearbeitungsfensters vor dem Tag `TablelistEdit` und ist für die Verwendung bei der Definition individueller Bindungs-Skripte zur Steuerung der interaktiven Zellbearbeitung vorgesehen.

Der folgende Befehl ersetzt beispielsweise das Standardverhalten der Taste `Return` bei der Bearbeitung von Zellen im Tablelist-Widget `.tbl` durch das der Taste `Tab`:

```
bind [.tbl editwintag] <Return> "[bind TablelistEdit <Tab>]; break"
```

`pathName` **embedcheckbutton** `cellIndex ?command?`

Setzt die Option `-window` der durch `cellIndex` angegebenen Zelle auf eine Prozedur, die einen Frame mit einem untergeordneten checkbutton erstellt, dessen Option `-variable` auf eine globale Variable gesetzt ist, deren Name privat für Tablelist ist und die mit dem internen Wert der Zelle initialisiert wird (der der boolesche Wert `0` oder `1` sein sollte). Jede Änderung des Auswahlzustands des checkbutton setzt den internen Wert der Zelle automatisch auf den entsprechenden booleschen Wert `0` oder `1`. Wenn das optionale Argument `command` eine nicht leere Zeichenkette ist, wird die Option `-command` des checkbutton-Widgets auf ein Skript gesetzt, das dieses Argument mit dem Pfadnamen des Tablelist-Widgets und den

aktuellen Zeilen- und Spaltenindizes der Zelle (die sich von denen zum Zeitpunkt der Erstellung des checkbutton unterscheiden können) verknüpft und ruft das resultierende Skript dann im Leerlauf auf, wodurch sichergestellt wird, dass zum Zeitpunkt des Aufrufs der interne Wert der Zelle bereits aktualisiert wurde, um den Auswahlzustand des checkbutton widerzuspiegeln.

HINWEIS 1: Beachten Sie, dass der Unterbefehl `windowpath` den Pfadnamen des eingebetteten Fensters zurückgibt, bei dem es sich um einen Frame handelt; um den Pfadnamen des checkbuttons zu erhalten, fügen Sie `.ckbtn` an den Pfadnamen des Frames an.

HINWEIS 2: Wie oben beschrieben, spiegelt der *anfängliche* Auswahlzustand des checkbutton den internen Wert der Zelle wider, und jede Änderung des Auswahlzustands des checkbutton führt automatisch zu einer Aktualisierung der Zelle. Wenn Sie hingegen den internen Wert der Zelle programmgesteuert ändern, liegt es in Ihrer Verantwortung, den Markierungszustand des checkbutton zu synchronisieren. Zu diesem Zweck müssen Sie den Pfadnamen des checkbuttons wie oben beschrieben abrufen, den Namen der zugehörigen Variable über `cget -variable` ermitteln und diese Variable auf den internen Wert der Zelle setzen.

`pathName` **embedcheckbuttons** `columnIndex ?command?`
Dieser Befehl entspricht logisch dem Aufruf von `embedcheckbutton` für alle Zellen im Datenbereich der durch `columnIndex` angegebenen Spalte.

`pathName` **embedttkcheckbutton** `cellIndex ?command?`
Ähnlich wie `embedcheckbutton`, mit dem einzigen Unterschied, dass der in die Zelle eingebettete Rahmen ein untergeordnetes Element vom Typ ttk::checkbutton enthält.

`pathName` **embedttkcheckbuttons** `columnIndex ?command?`
Dieser Befehl entspricht logisch gesehen dem Aufruf von `embedttkcheckbutton` für alle Zellen im Datenbereich der durch `columnIndex` angegebenen Spalte.

`pathName` **entrypath**
Gibt den Pfadnamen der Entry- oder Entry-ähnlichen Komponente des temporären eingebetteten Widgets zurück, das für die interaktive Zellenbearbeitung verwendet wird und durch den Unterbefehl `editcell` erstellt wurde. Wenn derzeit keine Zelle bearbeitet wird oder die Bearbeitung mithilfe eines Tk- oder Tile-Checkbuttons, eines Tk- oder Tile-Menübuttons, eines Mentry oder eines Toggleswitch-Widgets erfolgt, ist der Wert eine leere Zeichenkette; ansonsten ist es der Pfadname eines Tk- oder Tile-"Entry"-Widgets, eines "text"- oder "ctext"-Widgets, eines Tk- oder Tile-"spinbox"-Widgets oder eines BWidget-"Entry"-Widgets, bei dem es sich um das Bearbeitungsfenster selbst oder eines seiner untergeordneten Elemente handeln kann. Mit diesem Unterbefehl können Sie innerhalb der durch die Konfigurationsoptionen `-editstartcommand` und `-editendcommand` festgelegten Befehle auf das "Entry"-Widget oder eine "Entry"-ähnliche Eingabekomponente des temporär eingebetteten Widgets zugreifen.

`pathName` **expand** `indexList ?-fully|-partly?`
Dieser Unterbefehl klappt die angegebenen Zeilen einer Tablelist auf, die als Baum-Widget verwendet wird, d. h., er macht alle deren untergeordneten Elemente sichtbar. Das optionale Argument `-fully` (das standardmäßig verwendet wird) gibt an, dass der Befehl rekursiv ausgeführt wird, d. h., alle Nachkommen der durch `indexList` angegebenen Knoten werden

angezeigt. Die Option `-partly` beschränkt die Operation auf nur eine Hierarchieebene und gibt an, dass nur die untergeordneten Elemente der angegebenen Knoten angezeigt werden, ohne den Ein- bzw. Ausklappzustand der untergeordneten Knoten zu ändern.

Bevor die untergeordneten Elemente einer Zeile angezeigt werden, wird der als Wert der Option `-expandcommand` angegebene Befehl (sofern vorhanden) automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das daraus resultierende Skript wird im globalen Geltungsbereich ausgewertet. Auf diese Weise können Sie die untergeordneten Elemente eines Baumknotens bei Bedarf einfügen, unmittelbar bevor dieser erweitert wird.

HINWEIS: Wenn Sie mehrere Zeilen aufklappen möchten, ist es wesentlich effizienter, diesen Unterbefehl für die Liste ihrer Indizes aufzurufen, als ihn in einer Schleife für die einzelnen Zeilen auszuführen.

`pathName` **expandall**`?-fully|-partly?`
Dieser Unterbefehl klappt alle Zeilen der obersten Ebene einer Tablelist auf, die als Baum-Widget verwendet wird, d. h., er macht alle ihre untergeordneten Elemente sichtbar. Das optionale Argument `-fully` (das standardmäßig verwendet wird) gibt an, dass der Befehl rekursiv ausgeführt wird, d. h., alle Nachkommen der Knoten der obersten Ebene werden angezeigt. Die Option `-partly` beschränkt den Vorgang auf nur eine Hierarchieebene und gibt an, dass nur die untergeordneten Elemente der Knoten der obersten Ebene angezeigt werden, ohne den Ein- oder Ausklappzustand der untergeordneten Knoten zu ändern.

Bevor die untergeordneten Elemente einer Zeile angezeigt werden, wird der als Wert der Option `-expandcommand` angegebene Befehl (sofern vorhanden) automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das daraus resultierende Skript wird im globalen Geltungsbereich ausgewertet. Auf diese Weise können Sie die untergeordneten Elemente eines Baumknotens bei Bedarf einfügen, unmittelbar bevor dieser erweitert wird.

`pathName` **expandedkeys**
Gibt die Liste der vollständigen Schlüssel der aufgeklappten Elemente zurück.

`pathName` **fillcolumn** `columnIndex ?-text|-image|-window? value`
Dieser Unterbefehl entspricht logisch dem Aufruf des Unterbefehls `cellconfigure` für alle Zellen im Datenbereich der angegebenen Spalte, wobei `-text` (die Standardeinstellung), `-image` oder `-window` sowie `value` als Argumente verwendet werden.

`pathName` **findcolumnname** `name`
Gibt den Index der ersten Spalte zurück, deren Name (gemäß den Konfigurationsoptionen `-name`) dem Wert `name` entspricht. Wenn keine Spalten den angegebenen Namen haben, lautet der Rückgabewert `-1`.

`pathName` **findrowname** `name ?-descend? ?-parent nodeIndex?`
Gibt den Index der ersten untergeordneten Zeile des durch `nodeIndex` angegebenen Baumknotens zurück, deren Name (der durch die Konfigurationsoption `-name` festgelegt wird) dem Wert `name` entspricht. Die Option `-descend` erweitert die Suche auf alle Nachkommen des durch die Option `-parent` angegebenen Baumknotens. Standardmäßig ist die Suche auf die Kinder des übergeordneten Knotens beschränkt. Der Standard-Überknoten

ist `root`. Bei der Suche nach dem angegebenen Namen werden die Elemente in der Reihenfolge ihrer Zeilenindizes durchlaufen. Wenn keine der relevanten Zeilen den angegebenen Namen enthält, ist der Rückgabewert `-1`.

`pathName` **finishediting**

Dieser Unterbefehl versucht, die interaktive Bearbeitung des Inhalts der Zelle zu beenden, deren Index an den Unterbefehl `editcell` übergeben wurde, indem er das in die Zelle eingebettete temporäre Widget löscht und den Inhalt der Zelle aktualisiert. Die genauen Schritte sind wie folgt: Zunächst wird der endgültige Text des Widgets mit seinem ursprünglichen Text verglichen. Wenn diese identisch sind, wird das Bearbeitungsfenster gelöscht und der ursprüngliche Inhalt der Zelle wiederhergestellt. Wenn die beiden Zeichenketten unterschiedlich sind und der Wert der Konfigurationsoption `-editendcommand` eine nicht leere Zeichenkette ist, wird der endgültige Text des Widgets als letztes Argument an diesen Befehl übergeben (nach dem Pfadnamen der Tablelist sowie den Zeilen- und Spaltenindizes der Zelle), das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert wird nach dem Schließen des Bearbeitungsfensters zum neuen Inhalt der Zelle. Wurde jedoch innerhalb dieses Skripts der Unterbefehl `rejectinput` aufgerufen, wird der Wert der Zelle nicht geändert und das eingebettete Widget bleibt in der Zelle angezeigt; in diesem Fall gibt der Befehl den booleschen Wert `0` zurück. In allen anderen Fällen lautet der Wert `1`. Bevor der Wert `1` zurückgegeben wird, stellt der Befehl den ursprünglichen Wert der Option `-exportselection` des Tablelist-Widgets wieder her und generiert das virtuelle Ereignis `<<TablelistCellUpdated>>`, wenn der Bearbeitungsvorgang den Inhalt der Zelle tatsächlich verändert hat. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis so generiert, dass seine Option `-data` auf eine Liste gesetzt ist, die aus den numerischen Zeilen- und Spalten-Indizes der Zelle besteht, deren Inhalt gerade bearbeitet wird. Wurde zum Zeitpunkt des Aufrufs des Befehls keine Zelle bearbeitet, gibt der Befehl den selben Wert `1` zurück, es wird jedoch kein virtuelles Ereignis generiert. Ebenso gibt der Unterbefehl, wenn der Zustand des Widgets `disabled` ist, sofort den Wert `1` zurück, ohne die Bearbeitung zu beenden und ein virtuelles Ereignis zu erzeugen.

Dieser Unterbefehl wird *implizit* durch Drücken von `Return` oder `KP_Enter` (zusammen mit `Control`, wenn es sich bei dem Bearbeitungsfenster um ein Text- oder Ctext-Widget handelt) beim Bearbeiten einer Zelle, oder durch Klicken mit der linken Maustaste an einer beliebigen Stelle im Datenbereich der Tablelist außerhalb der gerade bearbeiteten Zelle oder durch Wechseln in eine andere bearbeitbare Zelle mittels Tastaturnavigation aufgerufen. Wenn die Bearbeitung mit der linken Maustaste gestartet wurde, handelt es sich bei dem Bearbeitungsfenster um ein Tk- oder Tile-Checkbutton- oder ein Toggleswitch-Widget und ist der Wert der Option `-instanttoggle` "true", dann wird dieser Unterbefehl automatisch und ohne jegliche Benutzerinteraktion aufgerufen. Ähnlich verhält es sich, wenn das Bearbeitungsfenster eines der vier von Tablelist unterstützten Combobox-Widgets oder ein (ttk::)menubutton-Widget ist und der Wert der Option `-autofinishediting` "true" ist, wird durch die Auswahl eines Combobox- oder Menüeintrags automatisch dieser Unterbefehl aufgerufen und damit die Bearbeitungssitzung beendet. Schließlich wird dieser Unterbefehl auch automatisch aufgerufen, wenn der Wert der Option `-editendonfocusout` "true" ist und das Tablelist-Widget den Fokus verliert.

Wenn der Wert der Option `-allowduplicates` der Spalte der Zelle "true" ist (Standardeinstellung), dann sind weitere Tastenkombinationen im Bearbeitungsfenster, die einen Aufruf dieses Unterbefehls auslösen, `Alt-Return`, `Alt-KP_Enter`, `Alt-a`, `Alt-d` und `Alt-u`, die zusätzlich den neuen Wert in einige andere Zellen der Spalte kopieren. Anschließend erzeugen die Bindungen für diese Tastenfolgen das virtuelle Ereignis `<<TablelistCopyFinished>>`. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis so erzeugt, dass seine Option `-data` auf den numerischen Index der Spalte der Zelle gesetzt ist, deren Inhalt bearbeitet wurde. Wenn der Wert der Konfigurationsoption `-aftercopycommand` eine nicht leere Zeichenkette ist, übergeben die oben genannten Bindungen nach der Erzeugung dieses virtuellen Ereignisses den Pfadnamen der Tablelist und den Index der Spalte der Zelle an diesen Befehl und führen die Auswertung des resultierenden Skripts im globalen Geltungsbereich aus. Unter Mac OS X/11+ Aqua wird die Tastenfolge `Alt-a` durch `Control-Command-a` ersetzt, und in den anderen Tastenfolgen erfolgt die Ersetzung des Modifikators `Alt` durch `Command`.

HINWEIS: Es gibt auch Situationen, in denen ein *expliziter* Aufruf dieses Unterbefehls erforderlich ist, um sicherzustellen, dass die gerade bearbeitete Zelle mit dem vom Benutzer eingegebenen Text aktualisiert wird. Wenn beispielsweise ein Tablelist-Widget Teil eines Dialogfelds ist, das zur Bearbeitung von Daten dient, sollte der Befehl, der der Schaltfläche zum Übernehmen der Daten zugeordnet ist, diesen Unterbefehl aufrufen; andernfalls geht der im Bearbeitungsfenster eingegebene Text verloren, wenn die Schaltfläche während der interaktiven Zellenbearbeitung gedrückt wird (es sei denn, es handelt sich bei dieser Schaltfläche um ein "ttk::button"-Widget; in diesem Fall verliert die "Tablelist" durch das Drücken der Schaltfläche den Fokus, was wiederum diesen Unterbefehl aufruft, sofern die Option `-editendonfocusout` auf "true" gesetzt wurde).

`pathName` **formatinfo**

Dieser Befehl ist dafür vorgesehen, aus einem Tcl-Befehl heraus aufgerufen zu werden, der als Wert der Konfigurationsoption `-formatcommand` angegeben ist. Er gibt eine Liste mit drei Elementen zurück, die Informationen über die Datenbereichs- oder Kopfzeilenzelle enthält, deren Inhalt mithilfe dieses Befehls formatiert wird. Das erste Element der Liste ist der vollständige Schlüssel (in der Form `knumber` oder `hknumber`), der dem Element des Daten- oder Kopfbereichs zugeordnet ist, das das zu formatierende Tablelist-Element enthält. Das zweite und dritte Element der Liste sind die Zeilen- (bzw. Kopfzeilen-) und Spaltenindizes der Zelle.

HINWEIS: Dieser Unterbefehl wird benötigt, wenn das Ergebnis der Formatierung von der Zeile der Zelle abhängen soll. Um ihn verwenden zu können, legen Sie den Wert der Konfigurationsoptionen für die Spalte `-formatcommand` auf `[list formatCommand pathName]` fest, wie im folgenden Beispiel gezeigt:

```
.tbl columnconfigure 1 -formatcommand [list formatValue .tbl]

proc formatValue {tbl val} {
    # Get information about the cell whose content is being formatted
    foreach {key row col} [$tbl formatinfo] {}

    # Return a string depending on $val and $row (or $key)
    . . .
}
```

`pathName get firstIndex lastIndex ?-all|-nonhidden|-viewable?`
`pathName get indexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente alle Elemente der Tablelist (d. h. Zeileninhalte) zwischen `firstIndex` und `lastIndex` (einschließlich) sind. Das optionale Argument kann verwendet werden, um die Einträge beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` der jeweiligen Zeile auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Elemente auf die viewable Elemente.

Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `indexList` ab: Enthält diese genau einen Index, so ist der Rückgabewert das durch diesen Index angegebene Element der Tablelist (oder eine leere Zeichenkette, wenn der Index auf ein nicht vorhandenes Element verweist); ansonsten gibt der Befehl die Liste aller Tablelist-Elemente zurück, die den durch `indexList` angegebenen Indizes entsprechen.

`pathName getcells firstCell lastCell ?-all|-nonhidden|-viewable?`
`pathName getcells cellIndexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente alle Elemente der Tablelist (d. h. Zellinhalte) zwischen `firstCell` und `lastCell` (einschließlich) sind. Das optionale Argument kann verwendet werden, um die Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` für die jeweilige Zeile oder Spalte auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Elemente auf die viewable Elemente.

Die zweite Form gibt einen Wert zurück, der von der Anzahl der Elemente in der Liste `cellIndexList` abhängt: Wenn diese genau einen Zellenindex enthält, ist der Wert der durch diesen Zellenindex angegebene Tablelist-Element; andernfalls gibt der Befehl die Liste aller Tablelist-Elemente zurück, die den durch `cellIndexList` angegebenen Zellenindizes entsprechen.

`pathName getcolumns firstColumn lastColumn`
`pathName getcolumns columnIndexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente wiederum Listen sind, wobei jede dieser Unterlisten genau einer Spalte zwischen `firstColumn` und `lastColumn` (einschließlich) entspricht und aus allen in dieser Spalte enthaltenen Elementen des Datenbereichs besteht. Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `columnIndexList` ab: Wenn diese genau einen Spaltenindex enthält, so ist der Rückgabewert eine Liste, die aus allen Elementen des Datenbereichs besteht, die in der durch diesen Spaltenindex angegebenen Spalte enthalten sind; ansonsten gibt der Befehl eine Liste zurück, deren Elemente selbst Listen sind, wobei jede dieser Unterlisten genau einem Index in `columnIndexList` entspricht und

aus allen in dieser Spalte enthaltenen Elementen des Datenbereichs besteht.

`pathName getformatted firstIndex lastIndex ?-all|-nonhidden|-viewable?`
`pathName getformatted indexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente alle formatierten Tablelist-Elemente (d. h. formatierte Zeileninhalte) zwischen `firstIndex` und `lastIndex` (einschließlich) sind. Das optionale Argument kann verwendet werden, um die Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, deren Zeile die Option `-hide` auf "true" gesetzt hat. Schließlich beschränkt der Wert `-viewable` die Elemente auf die viewable Elemente.

Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `indexList` ab: Enthält diese genau einen Index, so ist der Wert das durch diesen Index angegebene formatierte Tablelist-Element (oder eine leere Zeichenkette, wenn der Index auf ein nicht vorhandenes Element verweist); ansonsten gibt der Befehl die Liste aller formatierten Tablelist-Elemente zurück, die den durch `indexList` festgelegten Indizes entsprechen.

`pathName getformattedcells firstCell lastCell ?-all|-nonhidden|-viewable?`
`pathName getformattedcells cellIndexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente alle formatierten Tablelist-Elemente sind (d. h. formatierte Zellinhalte) zwischen `firstCell` und `lastCell`, einschließlich. Das optionale Argument kann verwendet werden, um die Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` für die Zeile oder die Spalte auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Elemente auf die viewable Elemente.

Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `cellIndexList` ab: Wenn diese genau einen Zellenindex enthält, ist der Wert das formatierte Tablelist-Element, auf das dieser Zellenindex verweist; andernfalls gibt der Befehl die Liste aller formatierten Tablelist-Elemente zurück, die den durch `cellIndexList` angegebenen Zellenindizes entsprechen.

`pathName getformattedcolumns firstColumn lastColumn`
`pathName getformattedcolumns columnIndexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente wiederum Listen sind, wobei jede dieser Unterlisten genau einer Spalte zwischen `firstColumn` und `lastColumn` (einschließlich) entspricht und aus allen formatierten Elementen des Datenbereichs besteht, die in dieser Spalte enthalten sind. Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `columnIndexList` ab: Wenn diese genau einen Spaltenindex enthält, so ist der Rückgabewert eine Liste, die aus allen formatierten Elementen des Datenbereichs besteht, die in der durch diesen Spaltenindex angegebenen Spalte enthalten sind; ansonsten gibt der Befehl eine Liste zurück, deren Elemente selbst Listen sind, wobei jede dieser Unterlisten genau einem Spaltenindex in `columnIndexList` entspricht und aus allen formatierten "tablelist"-Körperelementen besteht, die in dieser Spalte enthalten sind.

`pathName getfullkeys firstIndex lastIndex ?-all|-nonhidden|-viewable?`
`pathName getfullkeys indexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente alle vollständigen Schlüssel sind, die den Tablelist-Elementen zwischen `firstIndex` und `lastIndex` (einschließlich) zugeordnet sind. Das optionale Argument kann verwendet

werden, um die Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` in der jeweiligen Zeile auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Elemente auf die viewable Elemente.

Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `indexList` ab: Enthält diese genau einen Index, so ist der Rückgabewert der vollständige Schlüssel, der dem durch diesen Index angegebenen Tablelist-Element zugeordnet ist (oder eine leere Zeichenkette, falls der Index auf ein nicht vorhandenes Element verweist); ansonsten gibt der Befehl die Liste aller vollständigen Schlüssel zurück, die den Tablelist-Elementen zugeordnet sind, die den durch `indexList` angegebenen Indizes entsprechen.

Jedes Element eines Tablelist-Widgets verfügt über eine eindeutige Sequenznummer, die unverändert bleibt, bis das Element gelöscht wird, und somit als Schlüssel dient, der das Element eindeutig identifiziert, selbst wenn sich dessen Position (d. h. der numerische Index der Zeile) ändert. Dieser Befehl bietet schreibgeschützten Zugriff auf die vollständigen Schlüssel, die durch Voranstellen des Buchstabens `k` an diese internen Element-IDs erhalten werden.

`pathName` getkeys `firstIndex lastIndex ?-all|-nonhidden|-viewable?`
`pathName getkeys indexList`: Die erste Form des Befehls gibt eine Liste zurück, deren Elemente alle Laufnummern sind, die den Tablelist-Elementen zwischen `firstIndex` und `lastIndex` (einschließlich) zugeordnet sind. Das optionale Argument kann verwendet werden, um die Elemente beim Erstellen der Ergebnisliste einzuschränken. Der Standardwert `-all` bedeutet: keine Einschränkung. Der Wert `-nonhidden` filtert die Elemente heraus, bei denen die Option `-hide` in der jeweiligen Zeile auf "true" gesetzt ist. Schließlich beschränkt der Wert `-viewable` die Elemente auf die viewable anzeigbaren.

Der von der zweiten Form zurückgegebene Wert hängt von der Anzahl der Elemente in der Liste `indexList` ab: Enthält diese genau einen Index, so ist der Rückgabewert die Sequenznummer, die dem durch diesen Index angegebenen Tablelist-Element zugeordnet ist (oder eine leere Zeichenkette, falls der Index auf ein nicht vorhandenes Element verweist); ansonsten gibt der Befehl die Liste aller Sequenznummern zurück, die den Tablelist-Elementen zugeordnet sind, welche den durch `indexList` festgelegten Indizes entsprechen.

Jedes Element eines Tablelist-Widgets verfügt über eine eindeutige Sequenznummer, die bis zum Löschen des Elements unverändert bleibt und somit als Schlüssel dient, der das Element eindeutig identifiziert, selbst wenn sich dessen Position (d. h. der numerische Zeilenindex) ändert. Dieser Befehl ermöglicht den schreibgeschützten Zugriff auf diese internen Element-IDs.

`pathName` **hasattrib** `name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName` **hascellattrib** `cellIndex name`
Gibt `1` zurück, wenn das Attribut `name` für die durch `cellIndex` angegebene Zelle vorhanden ist, andernfalls `0`.

`pathName` **hascolumnattrib** `columnIndex name`

Gibt `1` zurück, wenn das Attribut `name` für die durch `columnIndex` angegebene Spalte vorhanden ist, andernfalls `0`.

pathName **hasrowattrib** `index name`
Gibt `1` zurück, wenn das Attribut `name` für die durch `index` angegebene Zeile vorhanden ist, andernfalls `0`.

pathName **header** `option ?arg arg ...?`
Dieser Befehl wird nur von Tk-Versionen ab 8.5 unterstützt. Er dient dazu, verschiedene Operationen auf die Elemente in der Kopfzeilenkomponente des Widgets auszuführen. Die unterstützten Unterbefehle sind im Abschnitt UNTERBEFEHLE FÜR DEN KOPFBEREICH beschrieben.

pathName **headerpath**
Dieser Befehl wird nur von Tk-Versionen ab 8.5 unterstützt. Er gibt den Pfadnamen des Text-Widgets zurück, das die Kopfzeilenelemente enthält.

pathName **headertag**
Dieser Befehl wird nur für Tk-Versionen ab 8.5 unterstützt. Er gibt den Namen eines Bindungstags zurück, dessen Name vom Pfadnamen des Tablelist-Widgets abhängt und das dem Text-Widget zugeordnet ist, das die Kopfzeilenelemente enthält, sowie den Message- und Label-Widgets, die zur Anzeige von mehrzeiligen Elementen bzw. Bildern in den Kopfzeilenzellen verwendet werden. Dieses Bindungs-Tag steht in der Liste der Bindungs-Tags der oben genannten Tablelist-Nachkommen vor dem Tag `TablelistHeader` und ist für die Verwendung bei der Definition individueller Bindungs-Skripte für Tablelist-Widgets vorgesehen. Der Hauptvorteil der Verwendung dieses Tags anstelle des Pfadnamens des Text-Widgets, das die Kopfzeilenelemente enthält, besteht darin, dass Sie damit Ereignisbehandlungsskripte schreiben können, die nicht nur für das oben genannte Text-Widget gelten, sondern auch für die darin enthaltenen mehrzeiligen Zellen und eingebetteten Bilder.

pathName **hidetargetmark**
Blendet den horizontalen Abstand oder den vertikalen Balken aus, der durch den Befehl `showtargetmark` angezeigt wird. Es wird kein Fehler ausgegeben, wenn die Zielmarkierung zum Zeitpunkt der Ausführung des Befehls nicht sichtbar ist.

Dieser Befehl ist für die Verwendung während eines Vorgangs des Ziehens und Ablegens vorgesehen, bei dem das Tablelist-Widget (oder dessen Datenbereich) als Ablegeziel registriert wurde. Beispiele finden Sie im Abschnitt UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN.

pathName **imagelabelpath** `cellIndex`
Gibt den Pfadnamen des Label-Widgets zurück, das das in die durch `cellIndex` angegebene Zelle eingebettete Bild enthält, wie mit der Option `-image` des Unterbefehls `cellconfigure` festgelegt. Wenn kein Bild in die Zelle eingebettet ist, ist der Rückgabewert eine leere Zeichenkette.

HINWEIS: Beachten Sie, dass die Label-Widgets, die eingebettete Bilder enthalten, bei Bedarf erstellt werden. Aus diesem Grund gibt dieser Unterbefehl nur dann einen nicht leeren Pfad zurück, wenn die Zeile, die die angegebene Zelle mit einem eingebetteten Bild enthält, derzeit im Tablelist-Fenster sichtbar ist. Beachten Sie dies immer dann, wenn Sie auf die

Label-Widgets mit eingebetteten Bildern zugreifen müssen. Um beispielsweise Mausklicks auf eingebettete Bilder zu verarbeiten, sollten Sie wie im folgenden Code-Beispiel gezeigt vorgehen:

```
bind [.tbl bodytag] <Button-1> {printClickedImage %W %x %y}
proc printClickedImage {w x y} {
    foreach {tbl x y} [tablelist::convEventFields $w $x $y] {}
    set cellIdx [$tbl containingcell $x $y]
    scan $cellIdx "%d,%d" row col
    if {$row >= 0 && $col >= 0 &&
        [string compare $w [$tbl imagelabelpath $cellIdx]] == 0} {
        puts "clicked the image of cell $cellIdx"
    }
}
```

pathName **index** `index`
Gibt den ganzzahligen Index-Wert der Zeile zurück, der `index` entspricht. Aus Gründen der Kompatibilität mit der Tk-Core-Listbox gilt: Wenn `index` gleich `end` ist, ist der Rückgabewert die Anzahl der Elemente in der Tablelist (und nicht den Index des letzten Elements). Um den numerischen Index des letzten Elements zu erhalten, können Sie das Wort `last` als Argument für `index` übergeben.

pathName **insert** `index ?item item ...?`
Fügt null oder mehr neue Elemente in die interne Liste des Widgets ein, und zwar direkt vor dem durch `index` angegebenen Element, wenn der Wert `state` der Tablelist nicht `disabled` ist. Wenn `index` der Anzahl der Elemente entspricht oder als `end` festgelegt wird, werden die neuen Elemente am Ende der Liste des Widgets hinzugefügt. Tabulatorzeichen werden als `\t` angezeigt (d. h. ein Backslash gefolgt von einem `t`), werden jedoch unverändert in die interne Liste eingefügt. Zeilenumbruchzeichen erzwingen Zeilenumbrüche, d. h., sie führen zu mehrzeiligen Elementen (die in eingebetteten Nachrichten-Widgets angezeigt werden, die bei Bedarf erstellt werden). Der Wert ist die Liste der vollständigen Schlüssel, die den gerade eingefügten Elementen zugeordnet sind.

HINWEIS: Dieser Unterbefehl eignet sich nicht zum Einfügen von Elementen in eine Tablelist, die für die Darstellung einer Baumstruktur vorgesehen ist. Verwenden Sie für solche Tablelist-Widgets den Unterbefehl `insertchildren` oder `insertchildlist`.

pathName **insertchildlist** `parentNodeIndex childIndex itemList`
Fügt die Elemente der Liste `itemList` in die interne Liste der untergeordneten Elemente des durch `parentNodeIndex` angegebenen Knotens ein, und zwar unmittelbar vor dem durch `childIndex` angegebenen Element, wenn der Wert `state` der Tablelist nicht `disabled` ist. `childIndex` muss eine Zahl, `last` (Legt das letzte untergeordnete Element des übergeordneten Elements fest) oder `end` sein; Wenn es der Anzahl der durch `parentNodeIndex` angegebenen Kinder des Knotens entspricht oder als `end` festgelegt wird, werden die neuen Elemente am Ende der Kinderliste des übergeordneten Knotens hinzugefügt. Tabulatorzeichen werden als `\t` angezeigt (d. h. ein Backslash gefolgt von einem `t`), werden jedoch unverändert in die interne Liste eingefügt. Zeilenumbruchzeichen erzwingen Zeilenumbrüche, d. h., sie führen zu mehrzeiligen Elementen (die in eingebetteten Nachrichten-Widgets angezeigt werden, die bei Bedarf erstellt werden). Der Wert ist die Liste der vollständigen Schlüssel, die den gerade eingefügten Elementen zugeordnet sind.

Dieser Befehl hat dieselbe Wirkung wie

```
eval [list pathName insertchildren parentNodeIndex childIndex] itemList
```

aber er ist effizienter und einfacher zu verwenden.

HINWEIS: Sie können eine erhebliche Zeitersparnis erzielen, indem Sie diesen Unterbefehl verwenden, um eine ganze Liste von Elementen einzufügen, anstatt den Befehl `insertchildren` mehrfach aufzurufen.

pathName **insertchildren** `parentNodeIndex childIndex ?item item ...?`
Fügt null oder mehr neue Elemente in die interne Liste der untergeordneten Elemente des durch `parentNodeIndex` festgelegten Knotens ein, und zwar direkt vor dem durch `childIndex` festgelegten Element, wenn der Wert `state` der Tablelist nicht `disabled` ist. `childIndex` muss eine Zahl, `last` (legt das letzte untergeordnete Element des übergeordneten Elements fest) oder `end` sein; Wenn es der Anzahl der durch `parentNodeIndex` angegebenen Kinder des Knotens entspricht oder als `end` festgelegt wird, werden die neuen Elemente am Ende der Kinderliste des übergeordneten Knotens hinzugefügt. Tabulatorzeichen werden als `\t` angezeigt (d. h. ein Backslash gefolgt von einem `t`), werden jedoch unverändert in die interne Liste eingefügt. Zeilenumbruchzeichen erzwingen Zeilenumbrüche, d. h., sie führen zu mehrzeiligen Elementen (die in eingebetteten Nachrichten-Widgets angezeigt werden, die bei Bedarf erstellt werden). Der Wert ist die Liste der vollständigen Schlüssel, die den gerade eingefügten Elementen zugeordnet sind.

HINWEIS: Es ist ausdrücklich erlaubt, den Namen `insertchildren` als `insertchild` abzukürzen. Dies ist praktisch, wenn Sie diesen Unterbefehl verwenden, um nur ein untergeordnetes Element einzufügen.

pathName **insertcolumnlist** `columnIndex columnList`
Fügt die durch die Liste `columnList` festgelegten Spalten unmittelbar vor der durch `columnIndex` festgelegten Spalte ein, wenn der Wert `state` der Tablelist nicht `disabled` ist. Wenn `columnIndex` der Anzahl der Spalten entspricht oder als `end` festgelegt wird, werden die neuen Spalten am Ende der Spaltenliste angefügt. Das Argument `columnList` muss eine Liste sein, die die Breite, den Titel und optionale Angaben zur Ausrichtung für die neuen Spalten enthält, und zwar in derselben Form wie bei der Konfigurationsoption `-columns`. Der Rückgabewert ist eine leere Zeichenkette. Die Elemente der neuen Spalten sind zunächst leere Zeichenketten; am einfachsten lassen sich diese Werte mit dem Unterbefehl `fillcolumn` oder der Konfigurationsoption `-text` ändern.

Dieser Befehl hat dieselbe Wirkung wie

```
eval [list pathName insertcolumns columnIndex] columnList
```

aber er ist effizienter und einfacher zu verwenden.

pathName **insertcolumns** `columnIndex ?width title ?alignment? width title ?alignment? ...?`
Fügt null oder mehr neue Spalten unmittelbar vor der durch `columnIndex` angegebenen Spalte ein, wenn der Wert `state` der Tablelist nicht `disabled` ist. Wenn `columnIndex` der Anzahl der Spalten entspricht oder als `end` festgelegt wird, werden die neuen Spalten am

Ende der Spaltenliste hinzugefügt. Die auf den Spaltenindex folgenden Argumente haben dieselbe Bedeutung wie bei der Konfigurationsoption `-columns`. Der Rückgabewert ist eine leere Zeichenkette. Die Elemente der neuen Spalten sind anfangs leere Zeichenketten; am einfachsten lassen sich diese Werte mit dem Unterbefehl `fillcolumn` oder der Konfigurationsoption `-text` ändern.

`pathName` **`insertlist`** `index itemList`

Fügt die Elemente der Liste `itemList` in die interne Liste des Widgets direkt vor dem durch `index` angegebenen Element ein, wenn der Wert `state` der Tablelist nicht `disabled` ist. Wenn `index` der Anzahl der Elemente entspricht oder als `end` festgelegt wird, werden die neuen Elemente am Ende der Liste des Widgets angehängt. Tabulatorzeichen werden als `\t` angezeigt (d. h. ein Backslash gefolgt von einem `t`), werden jedoch unverändert in die interne Liste eingefügt. Zeilenumbruchzeichen erzwingen Zeilenumbrüche, d. h., sie führen zu mehrzeiligen Elementen (die in eingebetteten Nachrichten-Widgets angezeigt werden, die bei Bedarf erstellt werden). Der Wert ist die Liste der vollständigen Schlüssel, die den gerade eingefügten Elementen zugeordnet sind.

Dieser Befehl hat dieselbe Wirkung wie

```
eval [list pathName insert index] itemList
```

aber er ist effizienter und einfacher zu verwenden.

HINWEIS 1: Sie können eine erhebliche Beschleunigung erzielen, indem Sie diesen Unterbefehl verwenden, um eine ganze Liste von Elementen einzufügen, anstatt `insert` mehrfach aufzurufen.

HINWEIS 2: Dieser Unterbefehl eignet sich nicht zum Einfügen von Elementen in eine Tablelist, die für die Darstellung einer Baumstruktur vorgesehen ist. Verwenden Sie für solche Tablelist-Widgets den Unterbefehl `insertchildlist` oder `insertchildren`.

`pathName` **`iselemsnipped`** `cellIndex fullTextName`

Gibt den Wert `1` zurück, wenn der in der durch `cellIndex` angegebenen Zelle angezeigte Text gekürzt ist, andernfalls `0`. In beiden Fällen wird der vollständige (nicht gekürzte) Zelltext in der Variable gespeichert, deren Name durch `fullTextName` angegeben wird; dieser vollständige Text kann entweder der Inhalt der Zelle sein oder die Zeichenkette, die unter Verwendung der Option `-formatcommand` der Spalte der Zelle aus diesem Inhalt gewonnen wird. Am häufigsten wird dieser Befehl innerhalb der Prozedur aufgerufen, die als Wert der Konfigurationsoption `-tooltipaddcommand` angegeben ist.

`pathName` **`isexpanded`** `index`

Gibt den Wert `1` zurück, wenn die durch `index` angegebene Zeile aufgeklappt ist, andernfalls den Wert `0`.

`pathName` **`istitlesnipped`** `columnIndex fullTextName`

Gibt den Wert `1` zurück, wenn der in der durch `columnIndex` angegebenen Kopfzeilenbeschriftung angezeigte Text gekürzt ist, andernfalls `0`. In beiden Fällen wird der vollständige (nicht gekürzte) Beschriftungstext in der Variablen gespeichert, deren Name durch `fullTextName` vorgegeben ist. Der häufigste Aufruf dieses Befehls erfolgt innerhalb der Prozedur, die als Wert der Konfigurationsoption `-tooltipaddcommand` angegeben ist.

`pathName` **isviewable** `index`

Gibt den Wert `1` zurück, wenn die durch `index` angegebene Zeile **sichtbar** ist, andernfalls `0`. Eine Zeile einer Tablelist gilt als sichtbar, wenn der Wert ihrer Option `-hide` "false" ist und alle ihre übergeordneten Elemente (teilweise) aufgeklappt sind. Ebenso wird eine Zelle der Tablelist als sichtbar bezeichnet, wenn ihre Zeile sichtbar ist und der Wert der Option `-hide` ihrer Spalte "false" ist.

`pathName` **itemlistvar**

Gibt den Namen einer Variablen zurück, die von Tablelist verwendet wird, um die dem Datenbereich zugeordnete interne Liste zu speichern. Die empfohlene Vorgehensweise bei der Verwendung dieser Variablen besteht darin, mithilfe des Befehls `upvar` einen Verweis darauf zu erstellen, wie im folgenden Beispiel gezeigt:

```
upvar #0 [.tbl itemlistvar] itemList
```

In diesem Beispiel ist der Wert der Variablen `itemList` die interne Liste, die dem Datenbereich des Tablelist-Widgets `.tbl` zugeordnet ist. Jedes Element der Liste entspricht einem Eintrag und ist wiederum eine Liste, deren Elemente den Elementen dieses Eintrags entsprechen, mit dem Unterschied, dass sie ein zusätzliches Element enthält, das den vollständigen Schlüssel des Eintrags enthält.

HINWEIS: Der Befehl `itemlistvar` bietet eine effiziente Möglichkeit, auf diese interne Liste zuzugreifen, anstatt die Elemente mit dem Unterbefehl `get` abzurufen oder die Option `-listvariable` zu verwenden (diese Methoden verbrauchen deutlich mehr Speicher). Dies kann in Situationen nützlich sein, in denen auf die Elemente eines Tablelist-Widgets zugegriffen werden muss, um Textdateien, HTML-Ausgaben, XML-Daten, Datenbankbefehle usw. zu erstellen. Dies sollte jedoch ein streng schreibgeschützter Zugriff sein; andernfalls sind die Ergebnisse unvorhersehbar!

`pathName` **itemtodict** `item ?excludedColumnIndexList?`

Gibt ein Dictionary zurück, das aus dem durch `item` angegebenen Element der Tablelist erstellt wurde. Das optionale Argument `excludedColumnIndexList` (das eine gültige Liste sein muss) legt jene Spaltenindizes fest (in einer der im Abschnitt SPALTENINDIZES beschriebenen Formen), die beim Erstellen des Dictionary ignoriert werden sollen. Die Schlüssel des Dictionary sind Spaltennummern oder -namen, und die Werte sind die entsprechenden Elemente des angegebenen Eintrags. Für jede Spaltennummer `col` gilt: Wenn der entsprechende Spaltenname (zurückgegeben von `pathName columncget col -name`) eine nicht leere Zeichenkette ist, wird dieser Name als Schlüssel im Dictionary verwendet, andernfalls der numerische Spaltenindex `col`. In beiden Fällen gilt: Wenn das optionale Argument `excludedColumnIndexList` einen Spaltenindex enthält, dessen numerisches Äquivalent (das vom Unterbefehl `columnindex` zurückgegeben wird) gleich `col` ist, wird diese Spalte stillschweigend ignoriert, d. h., die Zuordnung vom Schlüssel zum entsprechenden Tablelist-Element wird nicht in das Dictionary eingefügt.

Wenn Ihr "Tablelist"-Widget beispielsweise 3 Spalten hat, von denen die ersten beiden mit `-name forename` bzw. `-name surname` konfiguriert wurden, und Sie das Element übergeben

```
{Joe Black "Young Man in Coffee Shop"}
```

an den Unterbefehl `itemtodict`, dann ist der Rückgabewert das Dictionary

```
{forename Joe  surname Black  2 {Young Man in Coffee Shop}}
```

Wenn Sie die Liste `{2}`, oder `{end}` oder `{last}` als zusätzliches Argument an den Unterbefehl `itemtodict` übergeben, dann ist der Rückgabewert das Dictionary

```
{forename Joe  surname Black}
```

Dieser Unterbefehl ist nur verfügbar, wenn der Befehl `dict` vorhanden ist. Dies ist der Fall, wenn Tcl Version 8.5 oder höher verwendet wird oder eine der alternativen reinen Tcl-Dict-Implementierungen verwendet wird.

`pathName` **labelpath** `columnIndex`
Gibt den Pfadnamen der Kopfzeilenbeschriftung zurück, die der durch `columnIndex` angegebenen Spalte entspricht.

`pathName` **labels**
Gibt eine Liste zurück, die die Pfadnamen aller Kopfzeilenbezeichnungen des Widgets enthält.

`pathName` **labeltag**
Gibt den Namen eines Bindungs-Tags zurück, dessen Name vom Pfadnamen des Tablelist-Widgets abhängt und das sowohl mit den Kopfzeilenbeschriftungen als auch mit den zusätzlichen Widgets verknüpft ist, die von Tablelist in diese platziert werden, um Kopfzeilenbilder, (ttk::)Checkbuttons und Sortierpfeile anzuzeigen. Dieses Bindungs-Tag ist für die Verwendung bei der Definition von nicht standardmäßigen Bindungsskripten für die Kopfzeilenbeschriftungen vorgesehen. Innerhalb solcher Ereignisbehandlungsskripte können Sie die Spaltennummer und den Pfadnamen des Tablelist-Widgets mithilfe der Hilfsbefehle `tablelist::getTablelistColumn` und `tablelist::getTablelistPath` abrufen.

`pathName` **labelwindowpath** `columnIndex`
Gibt den Pfadnamen des Tk-Core-Checkbuttons oder des ttk::checkbutton-Fensters zurück, das mithilfe der Spaltenkonfigurationsoption `-labelwindow` in die Kopfzeilenbeschriftung der angegebenen Spalte eingebettet wurde. Wenn diese Option nicht gesetzt oder auf eine leere Zeichenkette gesetzt wurde, ist der Wert eine leere Zeichenkette.

`pathName` **loadfromfile** `fileName ?-fully|-partly?`
Füllt das "tablelist"-Widget mit den Elementen und Informationen zu den Spalten, der Sortierung und der Baumstruktur, die aus der Datei gelesen wurden, deren Name durch `fileName` angegeben wird. Diese Datei muss mit Hilfe des Unterbefehls `dumptofile` erstellt worden sein.

Nachdem der aktuelle Inhalt des Widgets (sofern vorhanden) gelöscht und die aus der Datei abgerufenen Sortierinformationen darauf angewendet wurden, fügt der Unterbefehl die Kopfzeilenelemente (sofern vorhanden) über `header insertlist` ein und prüft anschließend die in der Datei enthaltenen Elemente des Datenbereichs. Wenn alle diese Elemente direkte Unterelemente des Knotens `root` der ursprünglichen (Quell-)Tablelist sind, fügt der Unterbefehl sie über `insertlist` als Elemente der obersten Ebene ein. Andernfalls:
- Wenn das optionale Argument als `-partly` angegeben wurde (was der Standardwert ist), fügt der Unterbefehl jene Elemente des Datenbereichs, die in der ursprünglichen Tablelist direkte Unterelemente des Knotens `root` waren, über `insertchildlist` als Elemente der obersten Ebene ein und setzt die Optionen `-populatecommand` und `-expandcommand` des Widgets auf `::tablelist::populateCmd` bzw.

`::tablelist::expandCmd`, und stellt so sicher, dass die Elemente später sowohl programmgesteuert als auch interaktiv erweitert werden können. Diese Prozeduren, die in der Tablelist-Distributionsdatei `tablelistWidget.tcl` implementiert sind, verwenden ausschließlich dokumentierte Tablelist-Unterbefehle und sind ausführlich kommentiert. Sie funktionieren wie erwartet, unabhängig davon, ob die aktuellen Sortierspalten und/oder Sortierreihenfolgen noch mit den aus der Datei abgerufenen übereinstimmen. Andererseits müssen die Anzahl und die Position der Spalten unverändert bleiben (zu diesem Zweck setzt der Unterbefehl auch die Option `-movablecolumns` des Widgets auf `0`).

- Wenn das optionale Argument als `-fully` angegeben wurde, fügt der Unterbefehl alle Elemente des Datenbereichs über `insertchild` unter die entsprechenden übergeordneten Knoten ein, und klappt anschließend alle Zeilen der Tablelist ein, indem er `collapseall -fully` aufruft.

HINWEIS 1: Die empfohlene Vorgehensweise ist, diesen Unterbefehl mit der Standardoption `-partly` aufzurufen, da dabei nur die Elemente der obersten Ebene eingefügt werden, wodurch das Widget wesentlich reaktionsschneller ist als eines, bei dem alle Elemente tatsächlich eingefügt wurden.

HINWEIS 2: Da das vollständige Laden des Widgets recht lange dauern kann (mehrere Sekunden oder sogar Minuten), wird empfohlen, einen Aufruf von `loadfromfile` mit der Option `-fully` zwischen `setbusycursor` und `restorecursor` einzufügen.

pathName **loadfromstring** `string ?-fully|-partly?`

Füllt das "Tablelist"-Widget mit den Elementen und Informationen zu den Spalten, der Sortierung und der Baumstruktur, die aus der Zeichenkette `string` abgerufen wurden, die vom Unterbefehl `dumptostring` zurückgegeben worden sein muss.

Die übrigen Details entsprechen nahezu denen des Unterbefehls `loadfromfile` (ersetzen Sie einfach "file" durch "string", wo dies erforderlich ist).

pathName move `sourceIndex targetIndex`
pathName `move sourceIndex targetParentNodeIndex targetChildIndex`: Die erste Form des Befehls verschiebt das durch `sourceIndex` angegebene Element direkt vor das durch `targetIndex` angegebene Element, wenn der Wert `state` der Tablelist nicht `disabled` ist. Wenn `targetIndex` der Anzahl der Elemente entspricht oder als `end` festgelegt ist, wird das Quellelement hinter das letzte Element verschoben. Das durch `targetIndex` festgelegte Element muss denselben übergeordneten Knoten haben wie das durch `sourceIndex` festgelegte Element oder es muss das Element sein, das direkt unter dem letzten Nachkommen des übergeordneten Knotens des Quellknotens liegt.

Die zweite Form des Befehls verschiebt das durch `sourceIndex` angegebene Element direkt vor den Knoten, dessen übergeordneter Knoten durch `targetParentNodeIndex` angegeben ist und der den Index `targetChildIndex` in der Kinderliste des Elternknotens hat, wenn der Wert `state` der Tablelist nicht `disabled` ist. `targetChildIndex` muss eine Zahl, `last` (die das letzte Kind des Ziel-Elternknotens festlegt) oder `end` sein; Wenn es der Anzahl der Kinder des Ziel-Elternknotens entspricht oder als `end` festgelegt ist, wird das Quellelement hinter das letzte Kind des Ziel-Elternknotens verschoben.

Beide Formen des Befehls bewahren die Knotenhierarchie unter dem Quellelement bei, indem

sie dessen untergeordnete Elemente entsprechend verschieben. Der Rückgabewert ist eine leere Zeichenkette.

`pathName` **movecolumn** `sourceColumn targetColumn`
Verschiebt die durch `sourceColumn` angegebene Spalte direkt vor die durch `targetColumn` angegebene Spalte, wenn der Wert von `state` in der Tablelist nicht `disabled` ist. Wenn `targetColumn` der Anzahl der Spalten entspricht oder als `end` festgelegt ist, wird die Quellspalte hinter die letzte Spalte verschoben. Gibt eine leere Zeichenkette zurück.

`pathName` **nearest** `y`
Bei Angabe einer y-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl den Index des viewable Elements im Datenbereich der Tablelist zurück, das dieser y-Koordinate am nächsten liegt. Es wird erwartet, dass die Koordinate `y` relativ zum Tablelist-Fenster selbst (und nicht zu dessen Datenbereich) angegeben wird.

`pathName` **nearestcell** `x y`
Bei Angabe einer x- und einer y-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl den Index der Zelle viewable im Datenbereich zurück, die dem Punkt mit diesen Koordinaten am nächsten liegt. Es wird erwartet, dass die Koordinaten `x` und `y` relativ zum Tablelist-Fenster selbst (und nicht zu dessen Datenbereich) angegeben werden.

`pathName` **nearestcolumn** `x`
Bei Angabe einer x-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl den Index der nicht ausgeblendeten Tablelist-Spalte zurück, die dieser x-Koordinate am nächsten liegt. Die Koordinate `x` muss sich relativ zum Tablelist-Fenster selbst (nicht zu dessen Datenbereich) beziehen.

`pathName` **noderow** `parentNodeIndex childIndex`
Gibt den numerischen Index der Zeile des Knotens zurück, dessen übergeordneter Knoten durch `parentNodeIndex` angegeben ist und der in der Liste der untergeordneten Knoten des übergeordneten Knotens den Index `childIndex` hat. `childIndex` muss eine Zahl, `last` (legt das letzte untergeordnete Element des übergeordneten Elements fest) oder `end` sein; Wenn er der Anzahl der Kinder des übergeordneten Knotens entspricht oder als `end` festgelegt wird, ist der Rückgabewert der Index der Zeile des Elements, das auf den letzten Nachkommen des übergeordneten Knotens folgt.

`pathName` **parentkey** `nodeIndex`
Gibt den vollständigen Schlüssel des übergeordneten Elements des durch `nodeIndex` angegebenen Baumknotens zurück. Wenn dieses Argument als `root` festgelegt wird, ist der Rückgabewert eine leere Zeichenkette. Wenn `nodeIndex` ein Element der obersten Ebene identifiziert, gibt der Unterbefehl `root` zurück. Bei allen anderen Elementen gibt der Unterbefehl einen vollständigen Schlüssel der Form `knumber` zurück.

`pathName` **refreshsorting**`?parentNodeIndex?`
Sortiert die untergeordneten Elemente des durch `parentNodeIndex` festgelegten Tablelist-Knotens gemäß den Parametern des zuletzt aufgerufenen `sort`, `sortbycolumn` oder `sortbycolumnlist`. Wenn die Elemente noch gar nicht sortiert wurden oder die Sortierinformationen durch den Aufruf von `resetsortinfo` zurückgesetzt wurden, findet keine Sortierung statt. Das optionale Argument `parentNodeIndex` ist standardmäßig auf `root` gesetzt, was bedeutet, dass standardmäßig alle Elemente sortiert werden sollen. Der Wert ist eine leere Zeichenkette.

pathName **rejectinput**
Wird dieser Unterbefehl aus dem durch die Konfigurationsoption `-editendcommand` festgelegten Tcl-Befehl heraus aufgerufen, verhindert er die Beendigung der interaktiven Bearbeitung des Inhalts der Zelle, deren Index an den Unterbefehl `editcell` übergeben wurde. Er ruft den Unterbefehl `seecell` auf, um sicherzustellen, dass die betreffende Zelle sichtbar wird (falls sie aus dem Sichtbereich herausgescrollt wurde), und setzt den Fokus auf das in die Zelle eingebettete temporäre Widget. Dieser Befehl ermöglicht es Ihnen, den Text des Widgets bei der abschließenden Validierung der Zeichenkette, die als neuer Zellinhalt vorgesehen ist, abzulehnen. Der Rückgabewert ist eine leere Zeichenkette.

pathName **resetsortinfo**
Setzt die Informationen zur Sortierung der Elemente zurück. Bei nachfolgenden Aufrufen von `sortcolumn` und `sortorder` wird `-1` bzw. eine leere Zeichenkette zurückgegeben. Ebenso geben nachfolgende Aufrufe von `sortcolumnlist` und `sortorderlist` eine leere Zeichenkette zurück. Dieser Befehl entfernt außerdem alle vorhandenen Aufwärts- oder Abwärtspfeile, die durch einen früheren Aufruf von `sortbycolumn` oder `sortbycolumnlist` angezeigt wurden. Der Wert ist eine leere Zeichenkette.

pathName **restorecursor**
Setzt die Konfigurationsoption `-cursor` auf den Wert zurück, den sie vor dem Aufruf des Unterbefehls `setbusycursor` hatte, und gibt den booleschen Wert `1` zurück. Wenn zuvor kein `setbusycursor` aufgerufen wurde oder dies nicht der erste Aufruf von `restorecursor` seit dem letzten Aufruf von `setbusycursor` ist, gibt der Befehl den Wert `0` zurück, ohne zu versuchen, den Cursor wiederherzustellen.

pathName **rowattrib** `index ?name? ?value name value ...?`
Fragt die Attribute der durch `index` angegebenen Zeile ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs der Zeile enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des einen benannten Zeilenattributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (Sie können den Unterbefehl `hasrowattrib` verwenden, um diesen Fall von dem zu unterscheiden, bei dem der Wert eines *vorhandenen* Zeilenattributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, setzt der Befehl das/die angegebene(n) Zeilenattribut(e) auf den/die angegebenen Wert(e); in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `name` kann eine beliebige Zeichenkette sein.

pathName **rowcget** `index option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption für die durch `index` angegebene Zeile zurück. `option` kann einen beliebigen Wert annehmen, der vom Befehl `rowconfigure` akzeptiert wird.

pathName **rowconfigure** `index ?option? ?value option value ...?`
Ruft die Konfigurationsoptionen der durch `index` angegebenen Zeile ab oder ändert sie. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle für die Zeile verfügbaren Optionen beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des zurückgegebenen Werts, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Zeilenoptionen so, dass sie die festgelegten Werte annehmen; in diesem Fall ist

der Rückgabewert eine leere Zeichenkette. `option` kann einen beliebigen der im Abschnitt ZEILENKONFIGURATIONSOPTIONEN IM DATENBEREICH beschriebenen Werte annehmen.

`pathName` **rowcount**
Gibt die Gesamtanzahl der Elemente im Datenbereich der Tablelist zurück. Synonym für `size`.

`pathName` **scan** `option args`
Dieser Befehl dient dazu, das Scannen in Tablelist-Widgets zu implementieren. Je nach `option` gibt es zwei Varianten:

`pathName` **scan mark** `x y`
Speichert `x` und `y` sowie die aktuelle Ansicht im Tablelist-Fenster; wird in Verbindung mit den nachfolgenden Befehlen `scan dragto` verwendet. In der Regel ist dieser Befehl mit einem Mausklick im Datenbereich des Widgets verknüpft. Er gibt eine leere Zeichenkette zurück. Die Koordinaten `x` und `y` sollten relativ zum Tablelist-Fenster selbst (nicht zu dessen Datenbereich) angegeben werden.

`pathName` **scan dragto** `x y`
Dieser Befehl berechnet die Differenz zwischen seinen Argumenten `x` und `y` und dem letzten `scan mark`-Befehl für das Widget. Anschließend passt er die Ansicht (im Datenbereich nur die vertikale) um das Zehnfache der Koordinatendifferenz an. Dieser Befehl wird typischerweise mit Mausbewegungsereignissen im Datenbereich des Widgets verknüpft, um den Effekt zu erzeugen, dass die Tabelle mit hoher Geschwindigkeit durch das Fenster gezogen wird. Der Wert, der zurückgegeben wird, ist eine leere Zeichenkette. Die Koordinaten `x` und `y` beziehen sich auf das Tablelist-Fenster selbst (nicht auf dessen Datenbereich).

`pathName` **searchcolumn** `columnIndex pattern ?options?`
Dieser Unterbefehl durchsucht die Elemente im Datenbereich der durch `columnIndex` angegebenen Spalte, um festzustellen, ob eines davon mit `pattern` übereinstimmt. Wird eine Übereinstimmung gefunden, wird der Zeilenindex des ersten übereinstimmenden Elements zurückgegeben (sofern nicht die Option `-all` festgelegt ist). Andernfalls ist der Rückgabewert `-1`. Zur Steuerung der Suche können eine oder mehrere der folgenden Optionen festgelegt werden:

-all	Ändert das Ergebnis in eine Liste aller übereinstimmenden Zeilenindizes, die in numerischer Reihenfolge sortiert sind (bzw. in umgekehrter numerischer Reihenfolge, wenn die Option -backwards verwendet wird).
-backwards	Die Suche erfolgt rückwärts durch die Elemente der angegebenen Spalte im Datenbereich der Tablelist.

-check command	Legt eine zusätzliche Bedingung fest, die von den übereinstimmenden Elementen erfüllt werden muss. Wenn ein Element der angegebenen Spalte dem angegebenen Muster entspricht und command eine nicht leere Zeichenkette ist, wird diese automatisch mit dem Pfadnamen des Tablelist-Widgets, dem Index des Elements, dem numerischen Äquivalent von columnIndex sowie dem Element selbst oder dessen formatierter Version (abhängig vom Vorhandensein der Option -formatted) verkettet, das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und der Rückgabewert (der ein Boolescher Wert sein muss) bestimmt, ob das betreffende Element weiterhin als übereinstimmend angesehen wird oder nicht. Der Standardwert für command ist eine leere Zeichenkette. Mit dieser Option können Sie beliebige zusätzliche Übereinstimmungskriterien an den Suchprozess übergeben.
-descend	Durchsucht die Elemente der angegebenen Spalte in allen Nachkommen des Baumknotens, der durch die Option -parent festgelegt wird. Die Elemente werden in der Reihenfolge ihrer Zeilenindizes durchlaufen (bzw. in umgekehrter Reihenfolge ihrer Zeilenindizes, wenn die Option -backwards verwendet wird). Standardmäßig ist die Suche auf die Kinder des übergeordneten Knotens beschränkt.
-exact	Das/die übereinstimmende(n) Element(e) muss/müssen mit der literalen Zeichenkette pattern identisch sein.
-formatted	Betrachten Sie die formatierten Versionen der Elemente und nicht die internen Werte der Zellen.
-glob	Behandeln Sie pattern als "Glob"-Muster und vergleichen Sie es mit den Elementen nach denselben Regeln wie beim Befehl string match.
-nocase	Bewirkt, dass Vergleiche ohne Berücksichtigung der Groß-/Kleinschreibung durchgeführt werden. Hat keine Auswirkung, wenn diese Option mit der Option -numeric kombiniert wird.

-not	Diese Option kehrt die Bedeutung der Übereinstimmung um und gibt den Index der ersten nicht übereinstimmenden Zeile (oder, bei aktivierter Option -all, die Liste der Indizes aller nicht übereinstimmenden Zeilen) der angegebenen Spalte zurück.
-numeric	Die Elemente sollen mit pattern als Ganzzahl-oder Gleitkomma-Werte verglichen werden, wobei der Vergleichsoperator == verwendet wird. Diese Option ist nur in Verbindung mit -exact sinnvoll.
-parent nodeIndex	Diese Option beschränkt die Suche auf die Kinder (oder Nachkommen, bei Verwendung mit -descend) des durch nodeIndex angegebenen Baumknotens. Der Standard-Elternknoten ist root.
-regexp	Behandeln Sie pattern als regulären Ausdruck und vergleichen Sie ihn anhand der auf der Referenzseite zu re_syntax beschriebenen Regeln mit den Elementen.
-start index	Die Elemente der angegebenen Spalte sollen (vorwärts oder rückwärts) beginnend mit der durch index angegebenen Zeile durchsucht werden. Diese Option erleichtert die Umsetzung einer inkrementellen Suche.

Wenn alle Optionen für die Vergleichsart, `-exact`, `-glob` und `-regexp`, weggelassen werden, wird standardmäßig die Vergleichsart `-glob` verwendet. Wenn mehr als eine davon festgelegt wird, hat die zuletzt angegebene Vergleichsart Vorrang.

Bevor die untergeordneten Elemente (bzw. Nachkommen, bei Verwendung der Option `-descend`) einer Zeile untersucht werden, deren untergeordnete Elemente noch nicht eingefügt wurden, wird der als Wert der Option `-populatecommand` angegebene Befehl (sofern vorhanden) automatisch mit dem Pfadnamen des Tablelist-Widgets und dem Zeilenindex verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Auf diese Weise können Sie die untergeordneten Elemente bei Bedarf einfügen, unmittelbar bevor Sie sie nach dem angegebenen Muster durchsuchen.

`pathName` **see** `index`

Passt die Ansicht im Datenbereich der Tablelist so an, dass das durch `index` angegebene Element sichtbar ist. Wenn das Element bereits sichtbar ist, hat der Befehl keine Auswirkung; befindet sich das Element nahe einem Rand des Fensters, wird die Tablelist so gescrollt, dass das Element am Rand sichtbar wird; andernfalls wird die Tablelist so gescrollt, dass das Element zentriert wird.

`pathName` **seecell** `cellIndex`

Passt die Ansicht im Datenbereich der Tablelist so an, dass die durch `cellIndex` angegebene Zelle sichtbar ist. Wenn die Zelle bereits sichtbar ist, hat der Befehl keine Auswirkung; befindet sich die Zelle nahe einem Rand des Fensters, wird die Tablelist so

gescrollt, dass die Zelle am Rand sichtbar wird; andernfalls wird die Tablelist so gescrollt, dass die Zelle zentriert wird. Wenn der Wert der Option `-titlecolumns` positiv ist, erfolgt die Zentrierung der Zelle nur vertikal; das horizontale Scrollen (das in diesem Fall spaltenweise erfolgt) bringt die Zelle lediglich neben den Titelspalten oder am rechten Rand des Fensters in den sichtbaren Bereich.

`pathName` **seecolumn** `columnIndex`
Passt die Ansicht in der Tablelist so an, dass die durch `columnIndex` angegebene Spalte sichtbar ist. Wenn die Spalte bereits sichtbar ist, hat der Befehl keine Auswirkung; befindet sich die Spalte nahe einem Rand des Fensters, wird die Tablelist horizontal gescrollt, um die Spalte am Rand in den sichtbaren Bereich zu bringen; andernfalls wird die Tablelist horizontal gescrollt, um die Spalte zu zentrieren. Wenn der Wert der Option `-titlecolumns` positiv ist, wird die Spalte durch das horizontale Scrollen (das in diesem Fall spaltenweise erfolgt) lediglich neben den Titelspalten oder am rechten Rand des Fensters in den sichtbaren Bereich gebracht.

`pathName` **selection** `option args`
Dieser Befehl dient dazu, die Auswahl innerhalb des Datenbereichs eines Tablelist-Widgets anzupassen. Er kann je nach `option` verschiedene Formen annehmen:

`pathName` **selection anchor** `index`
Setzt den Anker der Auswahl auf das durch `index` angegebene Element. Wenn `index` auf ein nicht vorhandenes oder nicht viewable Element verweist, wird das nächstgelegene sichtbare Element verwendet. Der Anker der Auswahl ist das Ende der Auswahl, das beim Ziehen einer Auswahl mit der Maus fixiert bleibt, wenn der Auswahltyp `row` ist. Der Index `anchor` kann verwendet werden, um auf das Ankerelement zu verweisen.

`pathName` selection clear `firstIndex lastIndex`
`pathName selection clear indexList`: Wenn eines der Elemente zwischen `firstIndex` und `lastIndex` (einschließlich) oder eines der Elemente, die den in der Liste `indexList` angegebenen Indizes entsprechen, mindestens eine ausgewählte Zelle enthält, wird die Auswahl aufgehoben. Der Auswahlstatus von Elementen, die außerhalb des in der ersten Form des Befehls angegebenen Bereichs liegen oder sich von den in der zweiten Form des Befehls durch die Indexliste angegebenen Elementen unterscheiden, wird nicht geändert.

`pathName` **selection includes** `index`
Gibt `1` zurück, wenn das durch `index` angegebene Element mindestens eine ausgewählte Zelle enthält, andernfalls `0`.

`pathName` selection set `firstIndex lastIndex`
`pathName selection set indexList`: Wählt alle auswählbaren Elemente im Bereich zwischen `firstIndex` und `lastIndex` (einschließlich) aus oder diejenigen, die den in der Liste `indexList` festgelegten Indizes entsprechen, ohne die Auswahl anderer Elemente zu beeinflussen.

Wenn der Wert `state` der Tablelist `disabled` entspricht und sich `option` von `includes` unterscheidet, gibt der Befehl lediglich eine leere Zeichenkette zurück, ohne eine der oben genannten Aktionen auszuführen.

`pathName` **separatorpath**`?columnIndex?`
Wenn das optionale Argument nicht festgelegt wird, gibt dieser Befehl den Pfadnamen der speziellen Trennlinie zurück, die zur Markierung des Endes der Titelspalten angezeigt wird,

sofern der Wert der Option `-titlecolumns` positiv ist; andernfalls wird eine leere Zeichenkette zurückgegeben. Wenn das optionale Argument vorhanden ist, gibt der Befehl den Pfadnamen des Trennzeichens zurück, das am rechten Rand der durch `columnIndex` angegebenen Kopfzeilenbeschriftung angefügt ist, sofern der Wert der Konfigurationsoption `-showseparators` "true" ist; andernfalls wird eine leere Zeichenkette zurückgegeben.

`pathName` **separators**
Gibt eine Liste zurück, die die Pfadnamen aller Spaltentrennlinien enthält. Wenn der Wert der Option `-titlecolumns` positiv ist, so ist das erste Element der Liste der Pfadname des speziellen Trennzeichens, das zur Markierung des Endes der Titelspalten angezeigt wird. Ob die Pfadnamen der anderen Trennzeichen in der Liste enthalten sind, hängt vom Wert der Konfigurationsoption `-showseparators` ab.

`pathName` **setbusycursor**
Speichert den aktuellen Wert der Konfigurationsoption `-cursor` und setzt diese Option anschließend auf den Namen eines für das jeweilige Fenstersystem spezifischen "Busy"-Cursors (`watch` oder `wait`). Da dieser Unterbefehl dafür vorgesehen ist, unmittelbar vor dem Start eines Vorgangs aufgerufen zu werden, dessen Ausführung viel Zeit in Anspruch nehmen kann, stellt er durch den Aufruf von `update idletasks` außerdem sicher, dass der neue Cursor sofort sichtbar wird. In dem seltenen, aber möglichen Fall, dass als Nebeneffekt dieses Aufrufs das Tablelist-Widget durch eine anwendungsspezifische Aktion zerstört wird, die zur Ausführung im Leerlauf geplant ist, gibt der Befehl den booleschen Wert `0` zurück, ansonsten den Wert `1`.

`pathName` **showtargetmark** `before|inside index`
Zeigt eine Anzeige für das Ablegeziel in Form einer horizontalen Lücke oder eines vertikalen Balkens vor oder innerhalb der durch `index` festgelegten Zeile an. Wenn auf den Namen des Unterbefehls `before` folgt und `index` der Anzahl der Elemente entspricht oder `end` festgelegt wird, wird der horizontale Abstand direkt *hinter* der letzten Zeile der Tablelist angezeigt. Wenn der Unterbefehl mit der Option `inside` verwendet wird, wird der Index `end` als Hinweis auf das letzte Element des Widgets interpretiert.

Dieser Befehl ist für die Verwendung während eines Vorgangs des Ziehens und Ablegens vorgesehen, bei dem das Tablelist-Widget (oder dessen Datenbereich) als Ablegeziel registriert wurde. Beispiele finden Sie im Abschnitt UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN.

`pathName` **size**
Gibt die Gesamtanzahl der Elemente im Datenbereich der Tablelist zurück.

`pathName` **sort**`?-increasing|-decreasing?`
Sortiert die Elemente des Datenbereichs in aufsteigender oder absteigender Reihenfolge, je nach Angabe des optionalen Arguments. Die Standardeinstellung ist `-increasing`. Verwendet den Wert der Widget-Konfigurationsoption `-sortcommand` als Vergleichsbefehl. `sort` entfernt außerdem alle vorhandenen Auf- oder Abwärtspfeile, die durch einen früheren Aufruf von `sortbycolumn` oder `sortbycolumnlist` angezeigt wurden. Nach dem Sortieren der Elemente passt der Befehl die vertikale Ansicht bedingt wie folgt an: (a) Wenn gerade eine interaktive Zellbearbeitung stattfindet, wird die bearbeitete Zelle in den sichtbaren Bereich gebracht; (b) andernfalls, wenn genau ein Element ausgewählt ist, wird die Ansicht so verschoben, dass dieses Element sichtbar wird; (c) andernfalls wird, wenn der Datenbereich der Tablelist das Fenster ist, das zuletzt den Fokus unter allen Fenstern auf

demselben Toplevel-Fenster hatte, das aktuell aktive Element in den sichtbaren Bereich gebracht.

pathName **sortbycolumn** `columnIndex ?-increasing|-decreasing?`

Sortiert die Elemente des Datenbereichs anhand der Elemente der durch `columnIndex` angegebenen Spalte in aufsteigender oder absteigender Reihenfolge, je nach Angabe des optionalen Arguments. Der Standardwert ist `-increasing`. Der Sortierprozess wird durch die Werte der Optionen `-sortmode` und `-sortcommand` für die angegebene Spalte gesteuert. Wenn sowohl der Wert der Konfigurationsoption `-showarrow` als auch der der Option `-showarrow` für die angegebene Spalte "wahr" sind, wird ein Aufwärts- oder Abwärtspfeil, der die Sortierreihenfolge anzeigt, in die Spaltenbezeichnung eingefügt. Die Form des Pfeils hängt vom optionalen Argument des Befehls und vom Wert der Konfigurationsoption `-incrarrowtype` ab. Wenn der Text der Beschriftung rechtsbündig ausgerichtet ist, wird der Pfeil auf der linken Seite der Beschriftung angezeigt, andernfalls auf der rechten Seite – mit Ausnahme von Windows Vista, 7, 8 und 10+, wo der Pfeil horizontal zentriert in der Kopfzeilenbeschriftung und an deren Oberkante angebracht wird. Nach dem Sortieren der Elemente wird die vertikale Ansicht auf dieselbe Weise angepasst wie beim Unterbefehl `sort`.

Die oben beschriebenen Aktionen werden nur ausgeführt, wenn die Option `-showlinenumbers` für die angegebene Spalte nicht auf "true" gesetzt wurde.

pathName **sortbycolumnlist** `columnIndexList ?sortOrderList?`

Sortiert die Elemente des Datenbereichs anhand der Elemente der Spalten, die durch das Argument `columnIndexList` angegeben werden, wobei es sich um eine Liste eindeutiger Spaltenindizes handeln muss. Als relevant gelten nur diejenigen Elemente dieser Liste, die Spalten identifizieren, deren Option `-showlinenumbers` nicht auf "true" gesetzt wurde.

Die Einträge werden zunächst nach der Spalte sortiert, die durch das letzte signifikante Element von `columnIndexList` festgelegt wird, dann nach der Spalte, die durch das vorletzte signifikante Element festgelegt wird, und so weiter. Die Reihenfolge jedes Sortiervorgangs wird aus dem optionalen Argument `sortOrderList` entnommen, dessen Elemente (Abkürzungen von) `increasing` oder `decreasing` sein müssen. Wurde dieses Argument nicht angegeben oder enthält es weniger Elemente als `columnIndexList`, so wird für die fehlenden Sortierreihenfolgen `increasing` angenommen. Jeder Sortierprozess wird durch die Werte der Optionen `-sortmode` und `-sortcommand` für die jeweilige Spalte gesteuert. Wenn der Index der Spalte unter den ersten 9 signifikanten Elementen von `columnIndexList` festgelegt wurde und sowohl der Wert der Konfigurationsoption `-showarrow` als auch der der Option `-showarrow` für diese Spalte wahr sind, wird ein Aufwärts-oder ein Abwärtspfeil, der die Sortierreihenfolge anzeigt, in die Spaltenbezeichnung eingefügt. Die Form des Pfeils hängt von der jeweiligen Sortierreihenfolge und vom Wert der Konfigurationsoption `-incrarrowtype` ab. Wenn der Text der Beschriftung rechtsbündig ausgerichtet ist, wird der Pfeil auf der linken Seite der Beschriftung angezeigt, andernfalls auf der rechten Seite – mit Ausnahme von Windows Vista, 7, 8 und 10+, wo der Pfeil horizontal zentriert in der Kopfzeilenbeschriftung und an deren Oberkante angebracht wird. Wenn mehrere Sortierpfeile angezeigt werden sollen, werden die ersten 9 Sortierränge (1 für das erste signifikante Element von `columnIndexList`, 2 für das zweite usw.) ebenfalls rechts neben den Pfeilen angezeigt. Nach dem Sortieren der Elemente wird die vertikale Ansicht auf

dieselbe Weise angepasst wie beim Unterbefehl `sort`.

`pathName` **sortcolumn**
Gibt den numerischen Index der Spalte zurück, nach der die Einträge zuletzt mit Hilfe des Befehls `sortbycolumn` oder `sortbycolumnlist` sortiert wurden, oder `-1`, wenn sie zuletzt mit dem Befehl `sort` sortiert wurden oder gar nicht sortiert wurden oder die Sortierinformationen durch den Aufruf von `resetsortinfo` zurückgesetzt wurden. Wenn dieser Unterbefehl aus dem Befehl heraus aufgerufen wird, der als Wert der Widget- oder Spalten-Konfigurationsoption `-sortcommand` angegeben wurde, bezieht sich der Wert dieses Unterbefehls auf die gerade laufende Sortierung und nicht auf die zuletzt durchgeführte.

`pathName` **sortcolumnlist**
Gibt eine Liste zurück, die aus den numerischen Indizes der Spalten besteht, nach denen die Einträge zuletzt mit Hilfe des Befehls `sortbycolumnlist` oder `sortbycolumn` sortiert wurden (im zweiten Fall enthält die Liste genau ein Element), oder eine leere Zeichenkette, wenn sie zuletzt mit dem Befehl `sort` sortiert wurden oder gar nicht sortiert wurden oder die Sortierinformationen durch den Aufruf von `resetsortinfo` zurückgesetzt wurden. Wenn dieser Unterbefehl innerhalb des Befehls aufgerufen wird, der als Wert der Konfigurationsoption `-sortcommand` (Widget- oder Spalten-Option) angegeben wurde, bezieht sich der Wert dieses Unterbefehls auf die gerade stattfindende Sortierung und nicht auf die zuletzt durchgeführte.

`pathName` **sortorder**
Gibt die Sortierreihenfolge (als `increasing` oder `decreasing`) der letzten Sortierung zurück, die mit dem Befehl `sort`, `sortbycolumn` oder `sortbycolumnlist` durchgeführt wurde, oder eine leere Zeichenkette, wenn die Elemente noch gar nicht sortiert wurden oder die Sortierinformationen durch den Aufruf von `resetsortinfo` zurückgesetzt wurden. Wenn dieser Unterbefehl innerhalb des Befehls aufgerufen wird, der als Wert der Widget- oder Spalten-Konfigurationsoption `-sortcommand` angegeben wurde, bezieht sich der Wert dieses Unterbefehls auf die aktuell laufende Sortierung und nicht auf die zuletzt durchgeführte.

`pathName` **sortorderlist**
Gibt eine Liste zurück, die aus den Sortierreihenfolgen (wie `increasing` oder `decreasing`) des letzten Aufrufs des Befehls `sortbycolumnlist` oder `sortbycolumn` besteht (im zweiten Fall enthält die Liste genau ein Element) oder eine leere Zeichenkette, wenn die Elemente zuletzt mit dem Befehl `sort` sortiert wurden oder gar nicht sortiert wurden oder die Sortierinformationen durch den Aufruf von `resetsortinfo` zurückgesetzt wurden. Wenn dieser Unterbefehl innerhalb des Befehls aufgerufen wird, der als Wert der Widget- oder Spalten-Konfigurationsoption `-sortcommand` angegeben wurde, bezieht sich der Wert dieses Unterbefehls auf die gerade stattfindende Sortierung und nicht auf die zuletzt durchgeführte.

`pathName` **stopautoscroll**
Bricht den durch den Unterbefehl `autoscrolltarget` ausgelösten automatischen Bildlauf ab.

Dieser Befehl ist für die Verwendung während eines Vorgangs des Ziehens und Ablegens vorgesehen, bei dem das Tablelist-Widget (oder dessen Datenbereich) als Ablegeziel für TkDND oder das in BWidget enthaltene Drag-&-Drop-Framework registriert wurde. Im ersten Fall (TkDND) sollte er aus dem Skript der Bindung, das dem virtuellen Ereignis `<<Drop>>` zugeordnet ist, aufgerufen werden. Im zweiten Fall (BWidget) sollte er aus dem Befehl heraus aufgerufen werden, der als Wert der Option `-dropcmd` von `DropSite::register`

angegeben ist.

Beispiele für die Verwendung dieses Unterbefehls beim Ziehen und Ablegen finden Sie im Abschnitt UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN.

pathName **targetmarkpath**
Gibt den Pfadnamen des Ablegeziel-Indikators zurück (der als horizontaler Spalt oder vertikaler Balken angezeigt wird).

pathName **targetmarkpos** `y ?-any|-horizontal|-vertical?`
Bei Angabe einer y-Koordinate innerhalb des Tablelist-Fensters gibt dieser Befehl eine Liste mit zwei Elementen zurück, die die Argumente enthält, die an den Unterbefehl `showtargetmark` übergeben werden müssen, um die dieser y-Koordinate entsprechende Ablegeziel-Anzeige anzuzeigen. Das erste Listenelement ist die Zeichenkette `before` oder `inside`, das zweite ein numerischer Index für eine Zeile. Diese Listenelemente hängen von der relativen Position von `y` innerhalb der umgebenden Zeile (sofern vorhanden) sowie vom optionalen zweiten Argument ab: Die Standardoption `-any` lässt sowohl `before` als auch `inside` als erstes Element der Ergebnisliste zu, während die Optionen `-horizontal` und `-vertical` den Wert des ersten Listenelements auf `before` bzw. `inside` beschränken. Wenn die Option `-vertical` festgelegt wurde und kein Element des Datenbereichs der Tablelist die angegebene y-Position enthält, dann ist der Rückgabewert die Liste `{inside -1}`. Die Koordinate `y` wird relativ zum Fenster der Tablelist selbst (nicht zu deren Datenbereich) erwartet.

Dieser Befehl ist für die Verwendung während eines Vorgangs des Ziehens und Ablegens vorgesehen, bei dem das Tablelist-Widget (oder dessen Datenbereich) als Ablegeziel registriert wurde. Beispiele finden Sie im Abschnitt UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN.

pathName togglecolumnhide `firstColumn lastColumn`
`pathName togglecolumnhide columnIndexList`: Schaltet den Wert der Option `-hide` für eine oder mehrere Spalten des Tablelist-Widgets um. In der ersten Form des Befehls sind `firstColumn` und `lastColumn` Indizes, die die erste und letzte Spalte des Bereichs festlegen, dessen Sichtbarkeit umgeschaltet werden soll. Die zweite Form des Befehls akzeptiert eine Liste `columnIndexList` mit Indizes, die die Spalten festlegen, deren Sichtbarkeit umgeschaltet werden soll. Gibt eine leere Zeichenkette zurück. Nach dem Umschalten des Ausblendstatus der angegebenen Spalten wird das virtuelle Ereignis `<<TablelistColHiddenStateChanged>>` ausgelöst. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis mit der Option `-data` erzeugt, die auf eine Liste gesetzt ist, die aus den numerischen Spaltenindizes der Spalten besteht, deren Option `-hide` umgeschaltet wurde. Der Hauptvorteil der Verwendung dieses Befehls gegenüber dem Aufruf von `columnconfigure` für jede der angegebenen Spalten besteht darin, dass dadurch nur eine einzige Neuanzeige des Widget-Inhalts verursacht wird, was den Vorgang deutlich beschleunigt.

pathName togglerowhide `firstIndex lastIndex`
`pathName togglerowhide indexList`: Schaltet den Wert der Option `-hide` für eine oder mehrere Zeilen des Datenbereichs des Tablelist-Widgets um. In der ersten Form des Befehls sind `firstIndex` und `lastIndex` Indizes, die die erste und letzte Zeile des Bereichs festlegen, dessen Sichtbarkeit umgeschaltet werden soll. Die zweite Form des Befehls akzeptiert eine Liste

`indexList` mit Indizes, die die Zeilen festlegen, deren Sichtbarkeit umgeschaltet werden soll. Gibt eine leere Zeichenkette zurück. Nach dem Umschalten des Ausblendstatus der angegebenen Zeilen wird das virtuelle Ereignis `<<TablelistRowHiddenStateChanged>>` ausgelöst. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis mit der Option `-data` erzeugt, die auf eine Liste gesetzt ist, die aus den numerischen Zeilenindizes der Zeilen besteht, deren Option `-hide` umgeschaltet wurde.

ACHTUNG: Die Tk-Versionen 8.3 bis 8.4.12 wiesen einen Fehler auf, der zu einem Segmentierungsfehler führte, wenn der gesamte Inhalt eines Text-Widgets ausgeblendet (elided) wurde. Dieser Fehler war auch in Tk 8.5a1 – 8.5a3 vorhanden. Bei Verwendung einer dieser früheren Tk-Versionen führt dieser Fehler zu einem Absturz, wenn alle Zeilen eines Tablelist-Widgets ausgeblendet werden. Es liegt in Ihrer Verantwortung, solche Situationen bei der Verwendung einer Tk-Version mit diesem Fehler zu vermeiden!

`pathName` **`toplevelkey`** `index`

Wenn es sich bei dem durch `index` identifizierten Element um ein Element der obersten Ebene handelt, gibt der Unterbefehl den vollständigen Schlüssel dieses Elements zurück. Andernfalls gibt der Unterbefehl den vollständigen Schlüssel des eindeutigen Elements der obersten Ebene zurück, zu dessen Nachkommen das angegebene Element gehört.

`pathName` **`unsetattrib`** `name`

Hebt das Attribut `name` auf. Gibt eine leere Zeichenkette zurück.

`pathName` **`unsetcellattrib`** `cellIndex name`

Hebt das Attribut `name` für die durch `cellIndex` angegebene Zelle auf. Gibt eine leere Zeichenkette zurück.

`pathName` **`unsetcolumnattrib`** `columnIndex name`

Hebt das Attribut `name` für die durch `columnIndex` angegebene Spalte auf. Gibt eine leere Zeichenkette zurück.

`pathName` **`unsetrowattrib`** `index name`

Hebt das Attribut `name` für die durch `index` angegebene Zeile auf. Gibt eine leere Zeichenkette zurück.

`pathName` **`viewablerowcount`**`?firstIndex lastIndex?`

Gibt die Anzahl der viewable Zeilen im durch `firstIndex` und `lastIndex` angegebenen Indexbereich zurück. Wenn diese optionalen Indizes nicht angegeben werden, wird für `firstIndex` standardmäßig `0` und für `lastIndex` standardmäßig die Gesamtzahl der Elemente im Datenbereich minus eins verwendet, d. h., der Indexbereich umfasst alle Zeilen des Datenbereichs der Tablelist.

`pathName` **`windowpath`** `cellIndex`

Gibt den Pfadnamen des Fensters zurück, das in der durch `cellIndex` angegebenen Zelle enthalten ist und mit der Option `-window` des Unterbefehls `cellconfigure` erstellt wurde. Wenn derzeit kein Fenster in die Zelle eingebettet ist, ist der Rückgabewert eine leere Zeichenkette.

`pathName` **`xview`**`?args?`

Dieser Befehl dient dazu, die horizontale Position der Informationen im Fenster des Widgets abzufragen und zu ändern. Er kann eine der folgenden Formen annehmen:

`pathName xview`

Gibt eine Liste mit zwei Elementen zurück. Jedes Element ist eine reelle Bruchzahl zwischen

`0` und `1`; zusammen beschreiben sie den im Fenster sichtbaren horizontalen Bereich. Wenn beispielsweise das erste Element `0.2` und das zweite Element `0.6` ist, befinden sich 20 % des scrollbaren Textes der Tablelist links außerhalb des Bildschirms, die mittleren 40 % sind im Fenster sichtbar und 40 % des scrollbaren Textes befinden sich rechts außerhalb des Bildschirms. Dies sind dieselben Werte, die über die Option `-xscrollcommand` an die Scrollbars übergeben werden.

`pathName xview units`
Wenn der Wert der Option `-titlecolumns` positiv ist, passt dieser Befehl die Ansicht im Fenster so an, dass die Spalte, deren Versatz vom Ende des Titel-Spaltenbereichs der Anzahl der nicht ausgeblendeten Spalten gemäß `units` entspricht, neben den Titelspalten angezeigt wird. Andernfalls passt der Befehl die Ansicht im Fenster so an, dass die durch `units` angegebene Zeichenposition am linken Rand des Fensters angezeigt wird. Zeichenpositionen werden durch die Breite des Zeichens `0` definiert.

`pathName xview moveto fraction`
Passt die Ansicht im Fenster so an, dass `fraction` der Gesamtbreite des scrollbaren Tablelist-Textes links außerhalb des Bildschirms liegt. `fraction` muss ein Bruchwert zwischen `0` und `1` sein.

`pathName xview scroll number what`
Dieser Befehl verschiebt die Ansicht im Fenster entsprechend den Parametern `number` und `what` nach links oder rechts. `number` muss eine Ganzzahl oder eine Gleitkommazahl sein; wenn es sich um eine Gleitkommazahl handelt, wird sie in eine Ganzzahl umgewandelt und dabei von `0` weg gerundet. `what` muss entweder `units` oder `pages` oder eine Abkürzung einer dieser Bezeichnungen sein. Wenn `what` gleich `units` ist, wird die Ansicht auf dem Bildschirm um `number` nicht ausgeblendete Spalten oder Zeicheneinheiten (die Breite des Zeichens `0`) nach links oder rechts verschoben, je nach dem Wert der Option `-titlecolumns`; wenn `what` gleich `pages` ist, wird die Ansicht um `number` Bildschirmseiten angepasst. Ist `number` negativ, werden Spalten oder Zeichen weiter links sichtbar; ist der Wert positiv, werden Spalten oder Zeichen weiter rechts sichtbar.

`pathName` **`yview`**`?args?`
Dieser Befehl dient dazu, die vertikale Position des Textes im Fenster des Datenbereichs des Widgets abzufragen und zu ändern. Er kann eine der folgenden Formen annehmen:

`pathName yview`
Gibt eine Liste mit zwei Elementen zurück, bei denen es sich jeweils um reelle Brüche zwischen `0` und `1` handelt. Das erste Element gibt die Position des viewable Tablelist-Elements am oberen Rand des Fensters relativ zur gesamten Tablelist an (`0.5` bedeutet beispielsweise, dass es sich in der Mitte der Tablelist befindet). Das zweite Element gibt die Position des sichtbaren Tablelist-Elements unmittelbar nach dem letzten Element im Fenster relativ zur gesamten Tablelist an. Dies sind dieselben Werte, die über die Option `-yscrollcommand` an die Scrollbars übergeben werden.

`pathName yview units`
Passt die Ansicht im Fenster so an, dass das Element, dessen Versatz `units` viewable Zeilen entspricht, oben im Fenster angezeigt wird.

`pathName yview moveto fraction`
Passt die Ansicht im Fenster so an, dass das durch `fraction` angegebene Element viewable oben im Fenster angezeigt wird. `fraction` ist ein Bruch zwischen `0` und `1`; `0` bezeichnet

das erste sichtbare Element in der Tablelist, `0.33` bezeichnet das sichtbare Element im ersten Drittel der Tablelist und so weiter.

`pathName yview scroll number what`
Dieser Befehl verschiebt die Ansicht im Fenster entsprechend den Parametern `number` und `what` nach oben oder unten. `number` muss eine Ganzzahl oder eine Gleitkommazahl sein; wenn es sich um eine Gleitkommazahl handelt, wird diese in eine Ganzzahl umgewandelt und dabei von `0` weg gerundet. `what` muss entweder `units` oder `pages` oder eine Abkürzung eines dieser Werte sein. Wenn `what` gleich `units` ist, wird die Ansicht um `number` viewable Zeilen nach oben oder unten angepasst; wenn es `pages` ist, dann wird die Ansicht um `number` Bildschirmseiten angepasst. Wenn `number` negativ ist, werden Elemente weiter oben sichtbar; wenn der Wert positiv ist, werden Elemente weiter unten sichtbar.

UNTERBEFEHLE FÜR DEN KOPFBEREICH
Der Tablelist-Befehl `header`, mit dem verschiedene Operationen auf die Kopfzeilenelemente ausgeführt werden können, hat die folgende allgemeine Form:

```
pathName header option ?arg arg ...?
```

`option` und `arg` legen das genaue Verhalten des Befehls fest. Für den Kopfbereich der Tablelist werden die folgenden Unterbefehle unterstützt:

`pathName` **`header bbox`** `headerIndex`
Ähnlich wie der Befehl `bbox` für den Datenbereich.

`pathName` **`header cellattrib`** `headerCellIndex ?name? ?value name value ...?`
Ähnlich wie der Befehl `cellattrib` für den Datenbereich.

`pathName` **`header cellbbox`** `headerCellIndex`
Ähnlich wie der Befehl `cellbbox` für den Datenbereich.

`pathName` **`header cellcget`** `headerCellIndex option`
Ähnlich wie der Befehl `cellcget` für den Datenbereich. `option` kann jeden der Werte annehmen, die vom Unterbefehl `header cellconfigure` akzeptiert werden.

`pathName` **`header cellconfigure`** `headerCellIndex ?option? value option value ...?`
Ähnlich wie der Befehl `cellconfigure` für den Datenbereich. Jeder `option`-Eintrag kann einen der Werte annehmen, die im Abschnitt ZELLENKONFIGURATIONSOPTIONEN IM KOPFBEREICH beschrieben sind.

`pathName` **`header cellindex`** `headerCellIndex`
Ähnlich wie der Befehl `cellindex` für den Datenbereich.

`pathName` **`header configcelllist`** `headerCellConfigSpecList`
Ähnlich wie der Befehl `configcelllist` für den Datenbereich. Das Argument `headerCellConfigSpecList` muss eine Liste in folgender Form sein:

```
headerCellIndex option value headerCellIndex option value ...
```

wobei jedes `option` einen der Werte annehmen kann, die im Abschnitt ZELLENKONFIGURATIONSOPTIONEN IM KOPFBEREICH beschrieben sind.

`pathName` **`header configcells`**`?headerCellIndex option value headerCellIndex option value ...?`
Ähnlich wie der Befehl `configcells` für den Datenbereich. Jede `option` kann einen der

Werte annehmen, die im Abschnitt ZELLENKONFIGURATIONSOPTIONEN IM KOPFBEREICH beschrieben sind.

pathName **header configrowlist** `headerRowConfigSpecList`
Ähnlich wie der Befehl `configrowlist` für den Datenbereich. Das Argument `headerRowConfigSpecList` muss eine Liste in folgender Form sein:

```
headerIndex option value headerIndex option value ...
```

wobei jedes `option` einen der Werte annehmen kann, die im Abschnitt ZEILENKONFIGURATIONSOPTIONEN IM KOPFBEREICH beschrieben sind.

pathName **header configrows**`?headerIndex option value headerIndex option value ...?`
Ähnlich wie der Befehl `configrows` für den Datenbereich. Jeder `option`-Eintrag kann einen der Werte annehmen, die im Abschnitt ZEILENKONFIGURATIONSOPTIONEN IM KOPFBEREICH beschrieben sind.

pathName **header containing** `y`
Ähnlich wie der Befehl `containing` für den Datenbereich.

pathName **header containingcell** `x y`
Ähnlich wie der Befehl `containingcell` für den Datenbereich.

pathName header delete `firstHeaderIndex lastHeaderIndex`
`pathName header delete headerIndexList`: Ähnlich wie der Befehl `delete` für den Datenbereich.

pathName **header embedcheckbutton** `headerCellIndex ?command?`
Ähnlich wie der Befehl `embedcheckbutton` für den Datenbereich.

pathName **header embedcheckbuttons** `columnIndex ?command?`
Ähnlich wie der Befehl `embedcheckbuttons` für den Datenbereich. Logisch gesehen entspricht dies dem Aufruf von `header embedcheckbutton` für alle Kopfzellen der durch `columnIndex` angegebenen Spalte.

pathName **header embedttkcheckbutton** `headerCellIndex ?command?`
Ähnlich wie der Befehl `embedttkcheckbutton` für den Datenbereich.

pathName **header embedttkcheckbuttons** `columnIndex ?command?`
Ähnlich wie der Befehl `embedttkcheckbutton` für den Datenbereich. Logisch gesehen entspricht dies dem Aufruf von `header embedttkcheckbutton` für alle Kopfzellen der durch `columnIndex` angegebenen Spalte.

pathName **header fillcolumn** `columnIndex ?-text|-image|-window? value`
Ähnlich wie der Befehl `fillcolumn` für den Datenbereich. Logisch gesehen entspricht dies dem Aufruf des Unterbefehls `header cellconfigure` für alle Kopfzellen der angegebenen Spalte, wobei `-text` (Standard), `-image` oder `-window` sowie `value` als Argumente verwendet werden.

pathName **header findrowname** `name`
Ähnlich wie der Befehl `findrowname` für den Datenbereich. Gibt den Index der Kopfzeile des ersten Kopfzeilenelements zurück, dessen Name `name` lautet.

pathName header get `firstHeaderIndex lastHeaderIndex`
`pathName header get headerIndexList`: Ähnlich wie der Befehl `get` für den Datenbereich.

pathName header getcells `firstHeaderCell lastHeaderCell`
`pathName header getcells headerCellIndexList`: Ähnlich wie der Befehl `getcells` für den Datenbereich.

pathName header getcolumns `firstColumn lastColumn`
`pathName header getcolumns columnIndexList`: Ähnlich wie der Befehl `getcolumns` für den Datenbereich.

pathName header getformatted `firstHeaderIndex lastHeaderIndex`
`pathName header getformatted headerIndexList`: Ähnlich wie der Befehl `getformatted` für den Datenbereich.

pathName header getformattedcells `firstHeaderCell lastHeaderCell`
`pathName header getformattedcells headerCellIndexList`: Ähnlich wie der Befehl `getformattedcells` für den Datenbereich.

pathName header getformattedcolumns `firstColumn lastColumn`
`pathName header getformattedcolumns columnIndexList`: Ähnlich wie der Befehl `getformattedcolumns` für den Datenbereich.

pathName header getfullkeys `firstHeaderIndex lastHeaderIndex`
`pathName header getfullkeys headerIndexList`: Ähnlich wie der Befehl `getfullkeys` für den Datenbereich. Beachten Sie, dass die vollständigen Schlüssel der Kopfzeilenelemente mit hk beginnen.

pathName header getkeys `firstHeaderIndex lastHeaderIndex`
`pathName header getkeys headerIndexList`: Ähnlich wie der Befehl `getkeys` für den Datenbereich.

pathName **header hascellattrib** `headerCellIndex name`
Ähnlich wie der Befehl `hascellattrib` für den Datenbereich.

pathName **header hasrowattrib** `headerIndex name`
Ähnlich wie der Befehl `hasrowattrib` für den Datenbereich.

pathName **header imagelabelpath** `headerCellIndex`
Ähnlich wie der Befehl `imagelabelpath` für den Datenbereich.

pathName **header index** `headerIndex`
Ähnlich wie der Befehl `index` für den Datenbereich.

pathName **header insert** `headerIndex ?item item ...?`
Ähnlich wie der Befehl `insert` für den Datenbereich.

pathName **header insertlist** `headerIndex itemList`
Ähnlich wie der Befehl `insertlist` für den Datenbereich.

pathName **header iselemsnipped** `headerCellIndex fullTextName`
Ähnlich wie der Befehl `iselemsnipped` für den Datenbereich.

pathName **header itemlistvar**
Ähnlich wie der Befehl `itemlistvar` für den Datenbereich.

pathName **header nearest** `y`
Ähnlich wie der Befehl `nearest` für den Datenbereich.

pathName **header nearestcell** `x y`
Ähnlich wie der Befehl `nearestcell` für den Datenbereich.

pathName **header rowattrib** `headerIndex ?name? ?value name value ...?`

Ähnlich wie der Befehl `rowattrib` für den Datenbereich.

pathName **header rowcget** `index option`
Ähnlich wie der Befehl `rowcget` für den Datenbereich. `option` kann jeden der Werte annehmen, die vom Unterbefehl `header rowconfigure` akzeptiert werden.

pathName **header rowconfigure** `headerIndex ?option? ?value option value ...?`
Ähnlich wie der Befehl `rowconfigure` für den Datenbereich. Jede `option` kann einen der Werte annehmen, die im Abschnitt ZEILENKONFIGURATIONSOPTIONEN IM KOPFBEREICH beschrieben sind.

pathName **header rowcount**
Ähnlich wie der Befehl `rowcount` für den Datenbereich. Synonym für `header size`.

pathName **header size**
Ähnlich wie der Befehl `size` für den Datenbereich. Synonym für `header rowcount`.

pathName **header unsetcellattrib** `headerCellIndex name`
Ähnlich wie der Befehl `unsetcellattrib` für den Datenbereich.

pathName **header unsetrowattrib** `headerIndex name`
Ähnlich wie der Befehl `unsetrowattrib` für den Datenbereich.

pathName **header windowpath** `headerCellIndex`
Ähnlich wie der Befehl `windowpath` für den Datenbereich.

VORGABE- UND EINZELBINDUNGEN FÜR DEN DATENBEREICH

Der Datenbereich einer Tablelist ist als Text-Widget implementiert, dessen Bindungs-Tag `Text` durch ein neues Bindungs-Tag namens `TablelistBody` ersetzt wurde. Letzteres verfügt über alle Ereignisse der Widget-Klasse `Listbox`, und einige seiner Bindungsskripte werden aus denen von `Listbox` abgeleitet, indem die Ereignisfelder `%W`, `%x` und `%y` durch den Pfadnamen des Tablelist-Widgets sowie die relativ zu diesem geltenden x- und y-Koordinaten ersetzt werden. Diese Werte werden den Hilfsvariablen `tablelist::W`, `tablelist::x` und `tablelist::y` zugewiesen, indem der Hilfsbefehl `tablelist::convEventFields` wie folgt aufgerufen wird:

```
foreach {tablelist::W tablelist::x tablelist::y} \
    [tablelist::convEventFields %W %x %y] {}
```

Diese Umwandlung der Ereignisfelder ist erforderlich, da der einer Tablelist zugeordnete Tcl-Befehl Koordinaten erwartet, die sich auf das Widget selbst beziehen, nicht auf dessen Datenbereich. Dabei werden Hilfsvariablen aus dem Namespace `tablelist` verwendet, um Konflikte mit globalen Variablen zu vermeiden.

Bei einigen der Ereignisse fehlen die Felder `%x` und `%y`; in diesem Fall wird ein weiterer Hilfsbefehl `tablelist::getTablelistPath` verwendet, um die Hilfsvariable `tablelist::W` auf den Pfadnamen des Tablelist-Widgets zu setzen:

```
set tablelist::W [tablelist::getTablelistPath %W]
```

Das Bindungs-Tag `TablelistBody` ersetzt ebenfalls den Namen der Klasse (`Frame` oder `TSeparator`) der Trennzeichen-Widgets. Außerdem ersetzt es das Bindungs-Tag `Message` der Nachrichten-Widgets, die zur Anzeige mehrzeiliger Elemente verwendet werden, sowie das Bindungs-Tag `Label` der Beschriftungs-Widgets, die zur Anzeige eingebetteter Bilder verwendet werden. Dadurch wird sichergestellt, dass die Standardbehandlung der

Mausereignisse bei Spaltentrennzeichen, mehrzeiligen Zellen und eingebetteten Bildern dieselbe ist wie im restlichen Datenbereich der Tablelist.

Bei der Definition individueller Bindungen für Tablelist-Widgets ist dieselbe Konvertierung der Ereignisfelder erforderlich wie bei den Vorgabebindungen. Das folgende Skript für das Tablelist-Widget `.tbl` gibt beispielsweise den Index der Zelle aus, in der mit Maustaste 1 geklickt wurde:

```
bind [.tbl bodytag] <Button-1> {printClickedCell %W %x %y}
proc printClickedCell {w x y} {
    foreach {tbl x y} [tablelist::convEventFields $w $x $y] {}
    puts "clicked on cell [$tbl containingcell $x $y]"
```

Indem wir das Skript mit dem vom Unterbefehl `bodytag` zurückgegebenen Bindungs-Tag verknüpfen, anstatt nur mit dem Pfadnamen des Hauptteils der Tablelist, stellen wir sicher, dass die Ereignisbehandlung für Trennzeichen, mehrzeilige Zellen und eingebettete Bilder dieselbe ist wie für den restlichen Datenbereich der Tablelist.

KOMPATIBILITÄT MIT DEM LISTBOX-WIDGET: Die mit dem Bindungs-Tag `TablelistBody` verknüpften Standardbindungen stellen sicher, dass sich der Datenbereich einer Tablelist genauso verhält wie ein Tk-Core-Listbox-Widget. Ob dies streng genommen gültig ist, wird durch die boolesche Variable `tablelist::strictTk` gesteuert. Zur Verbesserung der Benutzerfreundlichkeit und der Kompatibilität mit anderen Rahmen hat diese Variable den Standardwert `0`, wodurch einige Tablelist-spezifische Erweiterungen und Änderungen aktiviert werden:

- Wenn `tablelist::strictTk` wahr ist, führen alle Vorgabebindungen, die mit dem Tag `TablelistBody` verknüpft sind, zu genau demselben Verhalten wie die Vorgabebindungen, die mit dem Tag `Listbox` verknüpft sind.
- Wenn `tablelist::strictTk` auf "false" gesetzt ist, kann *in allen Auswahlmodi* der Auswahlstatus einzelner Tablelist-Einträge oder -Elemente (je nach Auswahltyp) mithilfe von `Control-Button-1`, `Control-space` und `Control-Select` (siehe unten) umgeschaltet werden.

BINDUNGEN FÜR DEN AUSWAHLTYP row: Wenn der Auswahltyp `row` ist (was der Standardwert ist), dann gilt alles, was im Abschnitt "VORGABEBINDUNGEN" des Handbucheintrags zur **listbox** beschrieben ist, auch für den Datenbereich einer Tablelist. Der einzige Unterschied besteht darin, dass das Wort "Element" auf dieser Handbuchseite durch "Element" ersetzt werden muss, wenn die Beschreibung auf den Datenbereich eines Tablelist-Widgets angewendet wird. Außerdem:

1. Wenn `tablelist::strictTk` auf "false" gesetzt ist (was die Standardeinstellung ist), wird durch Drücken der Maustaste 1 bei gedrückter Taste `Control` die Auswahl des Elements unter der Maus umgeschaltet. Wenn der Auswahlmodus `extended` ist, gelten zusätzliche Aktionen, wie im Handbucheintrag zu **listbox** beschrieben. Wenn der Auswahlmodus `single` oder `browse` ist und sich der Auswahlstatus des Elements von "nicht ausgewählt" zu "ausgewählt" ändert, werden alle anderen ausgewählten Elemente abgewählt, genau so, als wäre Maustaste 1 gedrückt worden, ohne dass die Taste `Control` gedrückt wurde.
2. Auch hier gilt: Wenn `tablelist::strictTk` auf "false" gesetzt ist, dann schalten

`Control-space` und `Control-Select` die Auswahl des aktiven Elements um, genau so, als wäre Maustaste 1 über diesem Element gedrückt worden, während die Taste `Control` gedrückt gehalten wurde.

BINDUNGEN FÜR DEN AUSWAHLTYP cell: Wenn der Auswahltyp `cell` lautet, gilt alles, was im Abschnitt "VORGABEBINDUNGEN" des Handbucheintrags zur **listbox** beschrieben ist, auch für den Datenbereich einer Tablelist, mit folgenden Erweiterungen und Änderungen:

1. Wenn `Tab` oder `Shift-Tab` gedrückt wird, springt der Positionscursor (aktives Element) zum nächsten bzw. vorherigen Element. Wenn der Auswahlmodus `browse` oder `extended` ist, wird das neue aktive Element ebenfalls ausgewählt und die Auswahl aller anderen Elemente aufgehoben. Im Modus `extended` wird das neue aktive Element zum Anker der Auswahl.

   Beachten Sie, dass diese Bindungen die übliche Navigation zwischen Widgets über `Tab` und `Shift-Tab` durch eine Navigation zwischen Zellen ersetzen. Genau wie beim Text-Widget sind `Control-Tab` und `Control-Shift-Tab` für die Tastaturnavigation von Widget zu Widget vorgesehen. Leider funktioniert dies nicht immer, da einige Fenstermanager die letztgenannten Tastenfolgen abfangen und für ihre eigenen Zwecke verwenden (z. B. für die Navigation zwischen Arbeitsbereichen). Aus diesem Grund unterstützt Tablelist die zusätzlichen Tastenfolgen `Meta-Tab` und `Meta-Shift-Tab` als Ersatz für `Control-Tab` bzw. `Control-Shift-Tab`.
2. Wenn die Taste `Left` oder `Right` gedrückt wird, springt der Positionscursor (aktives Element) zum vorherigen bzw. nächsten Element der aktiven Zeile. Wenn der Auswahlmodus `browse` oder `extended` ist, wird das neue aktive Element ebenfalls ausgewählt und die Auswahl aller anderen Elemente wird aufgehoben. Im Modus `extended` wird das neue aktive Element zum Anker der Auswahl.
3. Im Modus `extended` verschieben `Shift-Left` und `Shift-Right` den Positionscursor (aktives Element) zum vorherigen bzw. nächsten Element der aktiven Zeile und erweitern zudem die Auswahl auf dieses Element, ähnlich wie beim Ziehen mit Maustaste 1.
4. Wenn die Taste `Home` oder `End` gedrückt wird, springt der Positionscursor (aktives Element) zum ersten bzw. letzten Element der aktiven Zeile, das neue aktive Element wird ausgewählt und alle anderen Elemente werden abgewählt.
5. Im Modus `extended` erweitern `Shift-Home` und `Shift-End` die Auswahl auf das erste bzw. letzte Element der aktiven Zeile. Im Modus `multiple` bewegen `Shift-Home` und `Shift-End` den Positionscursor zum ersten bzw. letzten Element der aktiven Zeile.
6. Wenn der Positionscursor in einer bearbeitbaren Zelle ist, starten die Tasten `Return` und `KP_Enter` die interaktive Bearbeitung des aktiven Elements.
7. Wenn `tablelist::strictTk` auf "false" gesetzt ist (was die Standardeinstellung ist), wird durch Drücken der Maustaste 1 bei gedrückter Taste `Control` die Auswahl des Elements unter der Maus umgeschaltet. Wenn der Auswahlmodus `extended` ist, gelten zusätzliche Aktionen, wie im Handbucheintrag **listbox** beschrieben. Wenn der Auswahlmodus `single` oder `browse` ist und sich der Auswahlstatus des Elements von "nicht ausgewählt" zu "ausgewählt" ändert, werden alle anderen ausgewählten Elemente abgewählt, genau so, als wäre Maustaste 1 gedrückt worden, ohne dass die Taste `Control` gedrückt wäre.
8. Auch hier gilt: Wenn `tablelist::strictTk` auf "false" gesetzt ist, dann schalten `Control-space` und `Control-Select` die Auswahl des aktiven Elements um, genau so, als wäre Maustaste 1 über diesem Element gedrückt worden, während die Taste `Control`

gedrückt gehalten wurde.

Bindungen FÜR "ALLES AUSWÄHLEN" UND "ALLES ABWÄHLEN": Die folgenden zusätzlichen Bindungen, die mit dem Bindungs-Tag `TablelistBody` verknüpft sind, sind auf den Fenstersystemen `x11` und `win32` gültig:

1. `Control-a` verhält sich genauso wie `Control-slash`, d. h., es wählt den gesamten Inhalt des Widgets aus, außer in den Modi `single` und `browse`; in diesen Fällen wählt es das aktive Element (je nach Auswahltyp) aus und hebt die Auswahl aller anderen Elemente auf.

   HINWEIS 1: Die Standard-Widget-Bindungen in aktuellen Tk-Versionen unter Windows unterstützen bereits `Control-a` als Alternative zu `Control-slash`. In Tablelist gilt dies auch unter X11.

   HINWEIS 2: Unter Mac OS X/11+ verwenden die Vorgabebindungen für Widgets in aktuellen Tk-Versionen die Tastenkombination `Command-a` *anstelle von* `Control-slash`.
2. `Shift-Control-A` verhält sich genauso wie `Control-backslash`, d. h., es hebt die Auswahl aller Elemente im Widget auf, außer im Modus `browse`, in dem es keine Wirkung hat.

   HINWEIS 1: Diese Tastenkombination ist unter Windows nützlich, wenn Sie beispielsweise eine französische oder deutsche Tastatur verwenden, da Tk in diesem Fall die Tastenfolge `Control-backslash` nicht erkennt (wofür Sie `Control`, `AltGr` sowie eine dritte Taste: `_` auf einer französischen und `ß` auf einer deutschen Tastatur drücken müssen). Zudem ist es auf vielen Tastaturen recht schwierig (wenn nicht sogar unmöglich), die Tastenfolge `Control-backslash` zu erzeugen, was die Unterstützung dieser alternativen Tastenkombination sowohl unter X11 als auch unter Windows umso nützlicher macht.

   HINWEIS 2: Unter Mac OS X/11+ verwenden die Vorgabebindungen für Widgets in aktuellen Tk-Versionen die Tastenkombination `Option-Command-a` *anstelle von* `Control-backslash`.

   Genau wie beim listbox-Widget löst jede Änderung der Auswahl automatisch das virtuelle Ereignis `<<ListboxSelect>>` aus. Anstelle dieses Ereignisses (das aus Gründen der Kompatibilität unterstützt wird) kann das virtuelle Ereignis `<<TablelistSelect>>` verwendet werden, um über Änderungen an der Auswahl in der Tablelist informiert zu werden. Beide Ereignisse werden unabhängig vom Typ der Auswahl ausgelöst. Ein Skript für eines dieser Ereignisse kann die Unterbefehle `curselection` und `curcellselection` verwenden, um die Indizes der ausgewählten Zeilen bzw. Zellen abzurufen.

   Ebenso wird bei jeder Änderung an der aktiven Zeile oder Zelle automatisch das virtuelle Ereignis `<<TablelistActivate>>` ausgelöst. Ein Skript für die Bindung dieses Ereignisses kann auf das aktive Element zugreifen, indem es das Wort `active` verwendet (das ein gültiger Zeilen- und Zellenindex ist) oder dessen kanonische Entsprechung `[pathName index active]` bzw. `[pathName cellindex active]` verwendet.

   LOKALES ZIEHEN UND ABLEGEN: Die folgende Bindung, die mit dem Bindungs-Tag `TablelistBody` verknüpft ist, ist nur gültig, wenn der Auswahlmodus `single` oder `multiple` lautet:

Wenn mit Maustaste 1 auf ein Element geklickt und die Maus anschließend außerhalb dieses

Elements gezogen wird, und der Wert der Konfigurationsoption `-movablerows` auf "true" gesetzt ist, nimmt der Mauszeiger die durch die Option `-movecursor` festgelegte Form an, um anzuzeigen, dass das betreffende Element an eine andere Position verschoben wird. Die neue Position des Elements (falls vorhanden) wird mithilfe einer Lücke vor der Zielzeile oder eines Balkens innerhalb dieser Zeile (abhängig von der aktuellen Mausposition) visualisiert, was anzeigt, ob das Quellelement vor diese Zeile verschoben oder zu einem untergeordneten Element dieser Zeile werden würde. Dieser **lokale Vorgang zum Ziehen und Ablegen** endet, wenn die Maustaste 1 losgelassen wird, und kann durch Drücken der Taste `Escape` abgebrochen werden. In beiden Fällen wird der Mauszeiger auf seinen ursprünglichen Wert zurückgesetzt, der durch die Konfigurationsoption `-cursor` festgelegt ist. Nach dem Loslassen von Maustaste 1 bei Vorhandensein eines gültigen Ziels wird das Quellelement vor die Zielzeile oder direkt vor deren erstes untergeordnetes Element verschoben, und das virtuelle Ereignis `<<TablelistRowMoved>>` wird generiert. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis mit der Option `-data` erzeugt, die auf eine Liste der Länge 3 gesetzt ist, deren Elemente aus den Argumenten abgeleitet werden, die an die zweite Form des Unterbefehls `move` übergeben wurden, der zum Verschieben der Quellzeile an ihre neue Position aufgerufen wurde. Das erste Listenelement ist der vollständige Schlüssel, der dem ersten Argument entspricht, das zweite ist `root` oder der vollständige Schlüssel, der dem zweiten Argument entspricht, und das dritte Listenelement ist identisch mit dem dritten Argument, das an den Unterbefehl `move` übergeben wurde.

Beachten Sie, dass es je nach aktueller Mausposition während des lokalen Ziehens und Ablegens eine entsprechende potenzielle Zielzeile geben kann oder auch nicht. Beispielsweise kann ein Baumelement kein gleichrangiges Element eines seiner Nachkommen werden, und möglicherweise dürfen nicht alle Elemente Kinder haben oder Elemente der obersten Ebene werden (Beispiel: In einem Dateimanager können normale Dateielemente keine übergeordneten Elemente anderer Elemente sein und sollten nicht zur obersten Ebene gehören dürfen). Um zu entscheiden, ob die Zeile, die der y-Koordinate der aktuellen Mausposition entspricht, ein gültiges potenzielles Ziel darstellt, prüft der Tablelist-Code zunächst, ob das Verschieben des Quellelements vor diese Zeile oder dessen Umwandlung in ein untergeordnetes Element dieser Zeile aus Sicht der allgemeinen Baumstruktur zulässig ist. Ist dies der Fall und würde die Verschiebungsoperation den übergeordneten Knoten des Quellelements ändern, und ist der als Wert der Konfigurationsoption `-acceptchildcommand` angegebene Befehl eine nicht leere Zeichenkette, dann wird dieser Befehl mit dem Pfadnamen des Tablelist-Widgets, dem Knotenindex des potenziellen neuen übergeordneten Knotens und dem Zeilenindex des verschobenen Elements verkettet; das resultierende Skript wird im globalen Geltungsbereich ausgewertet, und wenn der Rückgabewert (der ein Boolescher Wert sein muss) "false" ist, darf das Quellelement nicht an die aktuelle Mausposition verschoben werden. Ebenso gilt: Wenn der als Wert der Konfigurationsoption `-acceptdropcommand` angegebene Befehl eine nicht leere Zeichenkette ist, so wird dieser Befehl mit dem Pfadnamen des Tablelist-Widgets, dem Zeilenindex der potenziellen neuen Zielzeile und dem Zeilenindex des verschobenen Elements verkettet; das resultierende Skript wird im globalen Geltungsbereich ausgewertet; und wenn der Rückgabewert (der ein Boolescher Wert sein muss) "false" ist, darf das Quellelement nicht an die aktuelle Mausposition verschoben werden.

Beachten Sie, dass wenn der Auswahlmodus `multiple` aktiv ist, das *Drücken* der Maustaste 1 auf einem ausgewählten Element dieses Element normalerweise wieder abwählt (abhängig vom Auswahltyp). Wenn jedoch zusätzlich der Wert der Konfigurationsoption `-movablerows` auf

"true" gesetzt ist, stellt die angeklickte Zeile eine potenzielle Ziehquelle für den oben beschriebenen lokalen Vorgang des Ziehens und Ablegens dar, und aus diesem Grund wird das angeklickte Element erst dann abgewählt, wenn die linke Maustaste über demselben Element *losgelassen* wird.

UNTERSTÜTZUNG VON ZIEHQUELLEN FÜR GLOBALES ZIEHEN UND ABLEGEN: Neben dem lokalen Ziehen und Ablegen unterstützen die Vorgabebindungen auch die kompilierte Erweiterung "TkDND" und den in BWidget enthaltenen Ziehen-und-Ablegen-Rahmen sowie benutzerdefinierte Implementierungen für Ziehen und Ablegen. Ein Tablelist-Widget wird als **Ziehquelle für Maustaste 1** betrachtet, wenn sein Datenbereich über den Befehl `tkdnd::drag_source register` oder den BWidget-Befehl `DragSite::register` als solche registriert wurde oder die Option `-customdragsource` der Tablelist auf "true" gesetzt wurde. Die VORGABEBINDUNGEN bieten Unterstützung für Ziehquellen wie unten beschrieben:

1. Wenn der Auswahlmodus `extended` ist, führt das *Drücken* der Maustaste 1 auf einem ausgewählten Element normalerweise dazu, dass alle anderen Elemente die Auswahl verlieren (abhängig vom Auswahltyp). Wenn die Tablelist jedoch eine Ziehquelle für Maustaste 1 ist, werden die anderen Elemente erst dann abgewählt, wenn Maustaste 1 über dem angeklickten Element *losgelassen* wird. Dies liegt daran, dass auf den Mausklick möglicherweise ein Ziehvorgang folgt, der für alle aktuell ausgewählten Elemente bestimmt ist.
2. Ebenso gilt: Wenn der Auswahlmodus `multiple` ist, wird durch das *Drücken* der Maustaste 1 auf einem ausgewählten Element dieses Element normalerweise abgewählt (abhängig vom Auswahltyp). Wenn die Tablelist jedoch eine Ziehquelle für Maustaste 1 ist, wird das angeklickte Element erst dann abgewählt, wenn Maustaste 1 über demselben Element *losgelassen* wird. Auch dies liegt daran, dass auf den Mausklick möglicherweise ein Ziehvorgang folgt, der für alle aktuell ausgewählten Elemente bestimmt ist.
3. Immer wenn die Maus das Tablelist-Fenster verlässt, während Maustaste 1 gedrückt ist, führen die Vorgabebindungen normalerweise ein automatisches Scrollen durch, genau wie beim Tk-Listbox-Widget. Wenn die Tablelist jedoch eine Ziehquelle für Maustaste 1 ist, wird das automatische Scrollen unterdrückt, um Konflikte mit der Drag-Operation zu vermeiden.

Bindungen FÜR BAUM-WIDGETS: Die folgenden Bindungen, die mit dem Bindungs-Tag `TablelistBody` verknüpft sind, gelten für Tablelists, die als Baum-Widgets verwendet werden:

1. Durch Klicken mit Maustaste 1 auf ein Ein-/Ausklapp-Steuerelement wird der Ein- bzw. Ausklappzustand der entsprechenden Zeile umgeschaltet, indem die nicht-rekursive Version des Unterbefehls `expand` oder `collapse` (die mit dem Argument `-partly`) aufgerufen wird. Durch Drücken der Maustaste 1 über einem Ein-/Ausklapp-Steuerelement bei gedrückter `Alt`-Taste wird der Ein-/Ausklappzustand der entsprechenden Zeile mithilfe der rekursiven Version des Unterbefehls `expand` oder `collapse` (die mit dem Argument `-fully`) umgeschaltet. Unter Mac OS X/11+ Aqua wird anstelle von `Alt` die Taste `Command` verwendet.
2. Wenn die aktuell aktive Zeile ein Ein-/Ausklapp-Steuerelement im eingeklappten Zustand enthält, klappen die Tasten `Right`, `plus` und `KP_Add` diese Zeile auf, indem sie die nicht-rekursive Version des Unterbefehls `expand` aufrufen, während `Alt-Right`, `Alt-plus` und `Alt-KP_Add` (`Command-Right`, `Command-plus`, und `Command-KP_Add` unter Mac OS X/11+ Aqua) die Zeile mithilfe der rekursiven Version des Unterbefehls `expand` aufklappen.

3. Wenn die aktuell aktive Zeile ein Ein-/Ausklapp-Steuerelement im aufgeklappten Zustand enthält, dann klappen die Tasten `Left`, `minus` und `KP_Subtract` diese Zeile durch Aufruf der nicht-rekursiven Version des Unterbefehls `collapse` einklappen, während `Alt-Left`, `Alt-minus` und `Alt-KP_Subtract` (`Command-Left`, `Command-minus` und `Command-KP_Subtract` unter Mac OS X/11+ Aqua) die Zeile mithilfe der rekursiven Version des Unterbefehls `collapse` zusammenklappen.

VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFZEILEN

Die Kopfzeilenelemente sind in einem Text-Widget enthalten, dessen Bindungs-Tag `Text` durch ein neues Bindungs-Tag namens `TablelistHeader` ersetzt wird. Dieses Tag ersetzt auch das Bindungs-Tag `Message` der Nachrichten-Widgets, die zur Anzeige mehrzeiliger Kopfzeilenelemente verwendet werden, sowie das Bindungs-Tag `Label` der Beschriftungs-Widgets, die zur Anzeige eingebetteter Bilder in den Kopfzeilenzellen verwendet werden. Dadurch wird sichergestellt, dass die Standardbehandlung der Mausereignisse bei den mehrzeiligen Zellen und eingebetteten Bildern in der Kopfzeile der Tablelist der im restlichen Kopfzeilen-Text-Widget entspricht.

Kopfzeilenelemente dienen ausschließlich der Anzeige und sind nicht für die Interaktion vorgesehen. Aus diesem Grund sind die einzigen Vorgabebindungen, die mit dem Tag `TablelistHeader` verknüpft sind, diejenigen, die von Tablelist intern für die Tooltip-Unterstützung verwendet werden. Sie können jedoch individuelle Bindungen für die Kopfzeilenkomponente definieren, wie im folgenden Beispiel gezeigt (dafür ist Tk 8.5 oder höher erforderlich):

```
bind [.tbl headertag] <Button-1> {printClickedCell %W %x %y}
proc printClickedCell {w x y} {
    foreach {tbl x y} [tablelist::convEventFields $w $x $y] {}
    puts "clicked on header cell [$tbl header containingcell $x $y]"
```

VORGABE- UND EINZELBINDUNGEN FÜR DIE KOPFBESCHRIFTUNGEN

Tablelist erstellt automatisch die folgenden Bindungen für die Kopfzeilenbeschriftungen eines Tablelist-Widgets:

1. Wenn der Mauszeiger am rechten Rand einer Spaltenüberschrift oder innerhalb weniger Pixel von deren rechtem Rand liegt und sowohl der Wert der Konfigurationsoption `-resizablecolumns` als auch der der Option `-resizable` für die dieser Überschrift entsprechende Spalte auf "true" gesetzt sind, dann nimmt der Mauszeiger die durch die Option `-resizecursor` festgelegte Form an. Durch Klicken mit Maustaste 1 in diesem Bereich und Bewegen der Maus bei gedrückter Maustaste 1 wird die dieser Beschriftung entsprechende Spalte um den Betrag der Cursorbewegung in der Größe angepasst. Die interaktive Anpassung der Spaltengrößen endet, wenn die linke Maustaste losgelassen wird, und kann durch Drücken der Taste `Escape` abgebrochen werden. In beiden Fällen wird der Mauszeiger auf seinen ursprünglichen Wert zurückgesetzt, der durch die Konfigurationsoption `-cursor` festgelegt ist. Wenn die Spaltengrößenanpassung abgeschlossen ist, wird das virtuelle Ereignis `<<TablelistColumnResized>>` ausgelöst, dessen Option `-data` bei Tk-Versionen 8.5 und höher auf den numerischen Index der Spalte gesetzt ist.
2. Wenn die Maustaste 1 über einer Kopfzeilenbeschriftung, jedoch außerhalb des oben beschriebenen Größenänderungsbereichs gedrückt und anschließend außerhalb der Beschriftung gezogen wird, und der Wert der Konfigurationsoption `-movablecolumns` auf

"true" gesetzt ist, nimmt der Mauszeiger die durch die Option `-movecolumncursor` festgelegte Form an, was anzeigt, dass die betreffende Spalte an eine andere Position verschoben wird; dies wird durch eine Lücke vor der Beschriftung der Zielspalte visualisiert. Dieser Vorgang endet, wenn die Maustaste 1 losgelassen wird, und kann durch Drücken der Taste `Escape` abgebrochen werden, wenn sich der Mauszeiger außerhalb der Beschriftung befindet. In beiden Fällen wird der Mauszeiger auf seinen ursprünglichen Wert zurückgesetzt, der durch die Konfigurationsoption `-cursor` festgelegt ist. Nach dem Loslassen der Maustaste 1 wird die Quellspalte vor die durch die oben erwähnte Lücke angezeigte Spalte verschoben, und das virtuelle Ereignis `<<TablelistColumnMoved>>` wird generiert. Bei Tk-Versionen 8.5 und höher wird dieses virtuelle Ereignis mit der Option `-data` erzeugt, die auf eine Liste der Länge 4 gesetzt ist, deren erste beiden Elemente mit den beiden numerischen Spaltenindizes identisch sind, die an den Unterbefehl `movecolumn` übergeben wurden, der zum Verschieben der Quellspalte an ihre neue Position aufgerufen wurde, wobei die letzten beiden Elemente die entsprechenden Namen der Spalten sind, die mit Hilfe des Unterbefehls `columncget ... -name` abgerufen werden.

3. Wenn die Maustaste 1 über einer Kopfzeilenbezeichnung, jedoch außerhalb des oben beschriebenen Größenänderungsbereichs gedrückt und später über derselben Bezeichnung wieder losgelassen wird, und der durch die Option `-labelcommand` angegebene Befehl eine nicht leere Zeichenkette ist, wird dieser Befehl mit dem Pfadnamen des Tablelist-Widgets und dem Index der jeweiligen Beschriftung verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Wurde auf Spaltenebene mithilfe der Option `columnconfigure` des mit dem Tablelist-Widget verknüpften Tcl-Befehls ein anderer nicht leerer Beschriftungsbefehl angegeben, so wird dieser spaltenspezifische Befehl anstelle des globalen Befehls verwendet. Wenn die Maustaste 1 zusammen mit der Taste `Shift` gedrückt wird, wird der oben genannte Widget- oder spaltenspezifische Befehl durch den Befehl ersetzt, der durch die Option `-labelcommand2` auf Widget- oder Spaltenebene festgelegt wurde.
4. Das Paket "Tablelist" definiert das virtuelle Ereignis `<<Button3>>` für alle Fenstersysteme als `<Button-3>` und zusätzlich für Mac OS X/11+ Aqua als `<Control-Button-1>`. Wenn dieses Ereignis über einer Kopfzeilenbeschriftung auftritt und sowohl der Wert der Konfigurationsoption `-resizablecolumns` als auch der Wert der Option `-resizable` für die dieser Kopfzeile entsprechende Spalte wahr sind, wird die Breite dieser Spalte auf Null gesetzt, d. h., sie wird gerade so breit gemacht, dass alle Elemente der Spalte, einschließlich der Kopfzeile, Platz finden (jedoch nicht breiter als die durch die Konfigurationsoption `-maxwidth` angegebene maximale Breite), und das virtuelle Ereignis `<<TablelistColumnResized>>` wird generiert, wobei seine Option `-data` bei Tk-Versionen 8.5 und höher auf den numerischen Index der Spalte gesetzt ist. Die gleiche Aktion wird durch einen Doppelklick auf den Größenänderungsbereich einer Kopfzeilenbeschriftung ausgelöst.
5. Das Paket "Tablelist" definiert das virtuelle Ereignis `<<ShiftButton3>>` für alle Fenstersysteme als `<Shift-Button-3>` und zusätzlich für Mac OS X/11+ Aqua als `<Shift-Control-Button-1>`. Tritt dieses Ereignis über einer Kopfzeilenbeschriftung auf und sind sowohl der Wert der Konfigurationsoption `-resizablecolumns` als auch der der Option `-resizable` für die dieser Kopfzeile entsprechende Spalte wahr, dann wird die Breite dieser Spalte auf ihre letzte statische Breite (falls vorhanden) gesetzt und das virtuelle Ereignis `<<TablelistColumnResized>>` generiert, wobei dessen Option `-data` für

Tk-Versionen 8.5 und höher auf den numerischen Spaltenindex gesetzt ist. Die gleiche Aktion wird durch einen Doppelklick auf den Größenänderungsbereich einer Kopfzeilenbeschriftung ausgelöst, während die Taste `Shift` gedrückt gehalten wird.

BINDUNGEN FÜR DIE SPALTENWEISE ZELLENAUSWAHL: Wenn der Auswahltyp `cell` lautet, gelten einige weitere Bindungen für die Kopfzeilenbezeichnungen. In der folgenden Beschreibung kann die Taste `Alt` durch `Meta` ersetzt werden; unter Mac OS X/11+ Aqua wird die Taste `Command` anstelle von `Alt` und `Meta` verwendet. Beachten Sie außerdem, dass Sie die Menge der Zellen einschränken können, die ausgewählt werden, wenn diese Bindungen eine spaltenweise Auswahl durchführen: Wenn der durch die Option `-selectfiltercommand` angegebene Befehl eine nicht leere Zeichenkette ist, wird dieser Befehl mit dem Pfadnamen des Tablelist-Widgets und dem numerischen Index der jeweiligen Spalte verkettet, und das resultierende Skript wird im globalen Geltungsbereich ausgewertet. Wenn auf Spaltenebene als Wert der gleichnamigen Option ein anderer nicht leerer Befehl festgelegt wird, wird dieser spaltenspezifische Befehl anstelle des globalen Befehls verwendet. In beiden Fällen werden nur jene Zellen dieser Spalte ausgewählt, deren Zeilennummern in der vom Skript zurückgegebenen Liste enthalten sind.

1. Durch Drücken der Maustaste 1 auf eine Kopfzeilenbeschriftung bei gedrückter Taste `Alt` werden die Zellen der entsprechenden Spalte ausgewählt, alle anderen Zellen werden abgewählt und der Anker wird auf die erste sichtbare ausgewählte Zelle dieser Spalte gesetzt; Durch Ziehen der Maus bei gedrückter Maustaste 1 wird die Auswahl auf alle Spalten zwischen dem Anker und der Spalte unter dem Mauszeiger (einschließlich) erweitert.
2. Wenn die Ankerspalte mindestens eine ausgewählte Zelle enthält, kann der Bereich der ausgewählten Spalten angepasst werden, indem Sie Maustaste 1 auf einer Kopfzeilenbeschriftung drücken, während die Tasten `Alt` und `Shift` gedrückt gehalten werden: Dadurch wird die Auswahl so geändert, dass sie aus den Zellen der Spalten zwischen dem Anker und der Spalte unter dem Mauszeiger (einschließlich dieser beiden) besteht. Das nicht verankerte Ende dieser neuen spaltenweisen Zellauswahl kann ebenfalls bei gedrückter Maustaste gezogen werden.
3. Durch Drücken der Maustaste 1 auf einer Kopfzeilenbeschriftung bei gleichzeitig gedrückten Tasten `Alt` und `Control` wird eine Umschaltfunktion ausgelöst: Die Zellen der Spalte, die der angeklickten Beschriftung entspricht, werden abgewählt, wenn zuvor mindestens eine davon ausgewählt war, andernfalls werden sie ausgewählt; Im ersten Fall wird der Anker auf die erste sichtbare Zelle dieser Spalte gesetzt, im zweiten Fall auf die erste sichtbare ausgewählte Zelle der Spalte. die Auswahl der anderen Zellen wird nicht verändert. Wenn anschließend die Maus bei gedrückter Maustaste 1 gezogen wird, wird der Status der Auswahl der Zellen zwischen der Ankerspalte und der Spalte unter dem Mauszeiger so gesetzt, dass er dem Umschaltstatus der Ankerspalte entspricht; der Status der Auswahl aller anderen Zellen bleibt so, wie er vor Beginn des Umschaltvorgangs war.

Wenn der Wert `state` der Tablelist `disabled` lautet, tritt keine der oben genannten Aktionen ein: Die Beschriftungen sind vollständig unempfindlich.

Wenn Sie andere als die Vorgabebindungen für die Kopfzeilenbezeichnungen festlegen möchten, empfiehlt es sich, diese mit dem Bindungs-Tag zu verknüpfen, dessen Name vom Unterbefehl `labeltag` zurückgegeben wird, und die Hilfsbefehle `tablelist::getTablelistColumn` und `tablelist::getTablelistPath` zu

verwenden. Um beispielsweise die Standardbindung für <Button-3> durch ein Skript zu ersetzen, das eine spaltenabhängige Aktion ausführt, können Sie wie im folgenden Code gezeigt vorgehen:

```
bind [.tbl labeltag] <Button-3> {
    puts "right-clicked on header label no. [tablelist::getTablelistColumn %W]"
    break
}
```

VORGABEBINDUNGEN FÜR DIE INTERAKTIVE ZELLENBEARBEITUNG

Tablelist erweitert und definiert die Bindungen einiger Komponenten des temporären eingebetteten Widgets, das für die interaktive Zellbearbeitung verwendet wird, teilweise neu. Die Zellenbearbeitung wird durch Drücken der linken Maustaste in einer bearbeitbaren Zelle gestartet (Einzelheiten finden Sie unter der Option `-editselectedonly`) oder durch die Navigation mit der Tastatur von einer bearbeitbaren Zelle zur nächsten. Wenn der Auswahltyp `cell` ist und sich der Positionscursor in einer bearbeitbaren Zelle befindet, kann die interaktive Bearbeitung des aktiven Elements auch durch Drücken von `Return` oder `KP_Enter` gestartet werden.

Die betroffenen Komponenten des temporär eingebetteten Widgets hängen vom Bearbeitungsfenster ab: das Widget selbst im Falle eines Tk- oder Tile-Checkbuttons oder -Menubuttons; die "Entry"-Unterelemente des Bearbeitungsfensters im Falle eines "Mentry"-Widgets; das in einem als Bearbeitungsfenster verwendeten Toggle-Switch enthaltene ttk::scale-Widget; in allen anderen Fällen die einzige "Eingabekomponente" oder eine "Entry"-ähnliche Komponente des Bearbeitungsfensters (siehe auch den Unterbefehl `entrypath`). Die Liste der Bindungs-Tags dieser Bearbeitungsfenster-Komponenten enthält zwei zusätzliche Tags, die unmittelbar vor ihren Pfadnamen eingefügt werden: das Bindungs-Tag, dessen Name vom Unterbefehl `editwintag` zurückgegeben wird, gefolgt vom Tag `TablelistEdit`. Die im Folgenden beschriebenen Bindungen sind mit dem Tag `TablelistEdit` verknüpft und können für einzelne Tablelist-Widgets überschrieben werden, indem das vom Unterbefehl `editwintag` zurückgegebene Bindungstag verwendet wird.

1. `Control-i` fügt an der Position des Einfügecursors ein Tabulatorzeichen in die Eingabefelder oder eingabefeldähnlichen Komponenten (sofern vorhanden) des Bearbeitungsfensters ein.
2. `Control-j` fügt an der Position des Einfügecursors ein Zeilenumbruchzeichen in die Entry- oder Entry-ähnlichen Komponenten (sofern vorhanden) des Bearbeitungsfensters ein.
3. Wenn es sich bei dem Bearbeitungsfenster um ein Text- oder Ctext-Widget handelt, fügen `Return` und `KP_Enter` an der Position des Einfügecursors ein Zeilenumbruchzeichen ein. Andernfalls beenden sie die Bearbeitung und löschen das Bearbeitungsfenster.
4. `Control-Return` und `Control-KP_Enter` beenden die Bearbeitung und schließen das Bearbeitungsfenster.
5. Wenn der Wert der Option `-allowduplicates` der Spalte "true" ist (Standardeinstellung), dann beenden `Alt-Return` und `Alt-KP_Enter` (`Command-Return`/`Command-KP_Enter` unter Mac OS X/11+ Aqua) die Bearbeitung, schließen das Bearbeitungsfenster und füllen die ausgewählten Zellen der Spalte mit dem neuen Wert. Dies ist besonders dann nützlich, wenn die Option `-selectmode` der Tablelist auf `extended` gesetzt war und nach dem Klicken in eine bearbeitbare Zelle durch vertikales Ziehen des Mauszeigers weitere Zellen dieser Spalte ausgewählt wurden.

6. Wenn der Wert der Option `-allowduplicates` der Spalte "true" ist (Standardeinstellung), dann beendet `Alt-a` (`Control-Command-a` unter Mac OS X/11+ Aqua) die Bearbeitung, schließt das Bearbeitungsfenster und füllt alle Zellen der Spalte mit dem neuen Wert.
7. Wenn der Wert der Option `-allowduplicates` der Spalte "true" ist (Standardeinstellung), dann beendet `Alt-d` (`Command-d` unter Mac OS X/11+ Aqua) die Bearbeitung, schließt das Bearbeitungsfenster und füllt die Zellen der Spalte unterhalb der gerade bearbeiteten Zelle mit dem neuen Wert.
8. Wenn der Wert der Option `-allowduplicates` der Spalte "true" ist (Standardeinstellung), dann beendet `Alt-u` (`Command-u` unter Mac OS X/11+ Aqua) die Bearbeitung, schließt das Bearbeitungsfenster und füllt die Zellen der Spalte oberhalb der gerade bearbeiteten Zelle mit dem neuen Wert.
9. `Escape` bricht die Bearbeitung ab und schließt das Bearbeitungsfenster.
10. Ein Klick mit der linken Maustaste an einer beliebigen Stelle im Datenbereich der Tablelist außerhalb der gerade bearbeiteten Zelle beendet die Bearbeitung in der aktuellen Zelle und schließt das Bearbeitungsfenster oder verschiebt es in die Zelle, in die gerade geklickt wurde, wenn diese bearbeitbar ist.
11. Wenn eine Zelle beim Bearbeiten nicht die einzige bearbeitbare Zelle des Tablelist-Widgets ist, beenden `Tab` und `Shift-Tab` die Bearbeitung in der aktuellen Zelle, verschieben das Bearbeitungsfenster in die nächste bzw. vorherige bearbeitbare Zelle der Tablelist, wählen den Inhalt der ersten Entry- oder Entry-ähnlichen Komponente des Bearbeitungsfensters (wenn vorhanden) aus und setzen den Einfügecursor an dessen Ende. Wenn das neue Bearbeitungsfenster ein Text- oder Ctext-Widget ist, bleibt dessen Inhalt unmarkiert.

    Beachten Sie, dass diese Bindungen die übliche Navigation zwischen Widgets über `Tab` und `Shift-Tab` durch eine Navigation zwischen Zellen ersetzen. Genau wie beim Text-Widget sind `Control-Tab` und `Control-Shift-Tab` für die Tastaturnavigation von Widget zu Widget während der interaktiven Zellenbearbeitung vorgesehen. Leider funktioniert dies nicht immer, da einige Fenstermanager die letztgenannten Tastenfolgen abfangen und für ihre eigenen Zwecke verwenden (z. B. für die Navigation zwischen Arbeitsbereichen). Aus diesem Grund unterstützt Tablelist die zusätzlichen Tastenfolgen `Meta-Tab` und `Meta-Shift-Tab` als Ersatz für `Control-Tab` bzw. `Control-Shift-Tab`.
12. Wenn eine Zelle bearbeitet wird, die nicht die erste bzw. letzte bearbeitbare Zelle innerhalb ihrer Zeile ist, beendet `Alt-Left`/`Alt-Right` (`Command-Left`/`Command-Right` unter Mac OS X/11+ Aqua) die Bearbeitung in der aktuellen Zelle, verschiebt das Bearbeitungsfenster in die vorherige bzw. nächste bearbeitbare Zelle der Zeile, wählt den Inhalt der ersten Entry- oder Entry-ähnlichen Komponente (falls vorhanden) im Bearbeitungsfenster aus und setzt den Einfügecursor an dessen Ende. Wenn das neue Bearbeitungsfenster ein Text- oder Ctext-Widget ist, bleibt dessen Inhalt unmarkiert. Die Tastenkombination `Meta-Left`/`Meta-Right` hat dieselbe Wirkung wie `Alt-Left`/`Alt-Right`. Wenn `tk_strictMotif` falsch ist und das Bearbeitungsfenster nicht ein Text- oder Ctext-Widget ist, verhalten sich `Meta-b` und `Meta-f` jeweils genauso wie `Alt-Left` und `Alt-Right`. Wenn das Bearbeitungsfenster ein Tk- oder Tile-Checkbutton oder -Menubutton oder ein Toggleswitch-Widget ist, dann haben `Left`/`Right` die gleiche Wirkung wie `Alt-Left`/`Alt-Right`.
13. Wenn Sie eine Zelle bearbeiten, die nicht die erste bzw. letzte bearbeitbare Zelle innerhalb ihrer Spalte ist, beendet `Alt-Up`/`Alt-Down` (`Command-Up`/`Command-Down` unter Mac OS X/11+ Aqua) die Bearbeitung in der aktuellen Zelle, verschiebt das Bearbeitungsfenster

innerhalb der Spalte um eine Zeile nach oben oder unten, wählt den Inhalt der ersten Entry- oder Entry-ähnlichen Komponente (falls vorhanden) im Bearbeitungsfenster aus und setzt den Einfügecursor an dessen Ende. Wenn das neue Bearbeitungsfenster ein Text- oder Ctext-Widget ist, bleibt dessen Inhalt unmarkiert. Die Tastenkombination `Meta-Up`/`Meta-Down` hat dieselbe Wirkung wie `Alt-Up`/`Alt-Down`. Wenn `tk_strictMotif` auf "false" gesetzt ist und das Bearbeitungsfenster nicht ein Text- oder Ctext-Widget oder eine Iwidgets-Combobox ist, dann verhalten sich `Control-p` und `Control-n` jeweils genauso wie `Alt-Up` und `Alt-Down`. Wenn das Bearbeitungsfenster ein Tk- oder Tile-Entry, ein Tk- oder Tile-Checkbutton, ein Tk- oder Tile-Menubutton, ein Toggleswitch, ein BWidget-Entry, ein Iwidgets-Entryfield/Spinner/Spinint oder ein Mentry-Widget vom Typ `"FixedPoint"` ist, dann haben `Up`/`Down` dieselbe Wirkung wie `Alt-Up`/`Alt-Down`.

14. Wenn Sie eine Zelle bearbeiten, die nicht die erste bzw. letzte bearbeitbare Zelle innerhalb ihrer Spalte ist, beendet `Alt-Prior`/`Alt-Next` (`Command-Prior`/`Command-Next` unter Mac OS X/11+ Aqua) die Bearbeitung in der aktuellen Zelle, verschiebt das Bearbeitungsfenster innerhalb der Spalte um eine Seite nach oben bzw. unten, wählt den Inhalt der ersten Entry- oder Entry-ähnlichen Komponente (falls vorhanden) im Bearbeitungsfenster aus und setzt den Einfügecursor an dessen Ende. Wenn das neue Bearbeitungsfenster ein Text- oder Ctext-Widget ist, bleibt dessen Inhalt unmarkiert. Die Tastenkombination `Meta-Prior`/`Meta-Next` hat dieselbe Wirkung wie `Alt-Prior`/`Alt-Next`. Wenn das Bearbeitungsfenster kein Text- oder Ctext-Widget, keine BWidget-SpinBox, keine Oakley-Combobox und kein Mentry-Widget vom Typ `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"` oder `"IPv6Addr"` ist, dann haben `Prior`/`Next` dieselbe Wirkung wie `Alt-Prior`/`Alt-Next`.
15. Wenn Sie eine Zelle bearbeiten, die nicht die einzige bearbeitbare Zelle des Tablelist-Widgets ist, beendet `Alt-Home`/`Alt-End` (`Command-Home`/`Command-End` unter Mac OS X/11+ Aqua) die Bearbeitung in der aktuellen Zelle, verschiebt das Bearbeitungsfenster in die erste bzw. letzte bearbeitbare Zelle der Tablelist, wählt den Inhalt der ersten Entry- oder Entry-ähnlichen Komponente des Bearbeitungsfensters aus (sofern vorhanden) und setzt den Einfügecursor an dessen Ende. Wenn das neue Bearbeitungsfenster ein Text- oder Ctext-Widget ist, bleibt dessen Inhalt unmarkiert. Die Tastenkombination `Meta-Home`/`Meta-End` hat denselben Effekt wie `Alt-Home`/`Alt-End`. Wenn `tk_strictMotif` falsch ist und das Bearbeitungsfenster nicht ein Text- oder Ctext-Widget ist, verhalten sich `Meta-<` und `Meta->` jeweils genauso wie `Alt-Home` und `Alt-End`. Wenn das Bearbeitungsfenster kein Text- oder Ctext-Widget ist, haben `Control-Home`/`Control-End` dieselbe Wirkung wie `Alt-Home`/`Alt-End`.

Bevor das Bearbeitungsfenster verschoben wird, verschieben die unter 11–15 genannten Tastenkombinationen das aktive Element und ändern die (Zellen-)Auswahl sowie den (Zellen-)Auswahlanker im Datenbereich des Tablelist-Widgets. Wenn beispielsweise `Alt-Up`/`Alt-Down` oder `Meta-Up`/`Meta-Down` (`Command-Up`/`Command-Down` unter Mac OS X/11+ Aqua) gedrückt wird, während eine Zelle bearbeitet wird, die nicht die erste bzw. letzte bearbeitbare Zelle innerhalb ihrer Spalte ist, dann bewegt sich das aktive Element (je nach Auswahltyp) eine Zeile nach oben bzw. unten. Wenn der Auswahlmodus in `browse` oder `extended` ist, wird das neue aktive Element ebenfalls ausgewählt und die Auswahl aller anderen Elemente aufgehoben. Im Modus `extended` wird das neue aktive Element zum (Zellen-)Anker der Auswahl. Dies entspricht genau dem Verhalten der Tasten `Up` und `Down` im Datenbereich der Tablelist.

Wenn der Wert `state` der Tablelist `disabled` lautet, tritt keine der oben genannten Aktionen ein.

SCHLÜSSELWÖRTER

tablelist, multi-column, listbox, tree, widget

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung entstand mit Unterstützung von DeepL und KI-Werkzeugen und wurde von Hand durchgesehen und überarbeitet.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Multi-column listbox and tree widget package Tablelist, version 7.12 Copyright (c) 2000-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Stichwortverzeichnis

-

-align 157
-allowduplicates 157
-background 158
-bg 158
-changesnipside 158
-changetitlesnipside 158
-editable 158
-editwindow 158
-fg 158
-font 158
-foreground 158
-formatcommand 159
-hide 160
-image 170
-imagebackground 170
-labelalign 160
-labelbackground 160
-labelbd 160
-labelbg 160
-labelborderwidth 160
-labelcommand 160
-labelcommand2 160
-labelfg 161
-labelfont 160
-labelforeground 161
-labelheight 161
-labelimage 161
-labelpady 161
-labelrelief 161
-labelvalign 161
-labelwindow 161
-maxwidth 162
-name 162
-resizable 163
-selectable 168
-selectbackground 163
-selectfiltercommand 161
-selectforeground 163
-showarrow 163
-showlinenumbers 163
-sortcommand 164
-sortmode 164

-stretchable 165
-stretchwindow 165
-stripebackground 165
-stripeforeground 165
-text 165
-title 165
-valign 166
-width 166
-window 171
-windowdestroy 166
-windowupdate 166
-wrap 167

A

activate 190
activatecell 190
applysorting 191
attrib 191
autoscrolltarget 191

B

bbox 192
bodypath 192
bodytag 192

C

canceledediting 194
canceledediting 194
cellattrib 194
cellbbox 194
cellcget 195
cellconfigure 195
cellindex 195
cellselection 195
cellselection anchor 195
cellselection clear 195
cellselection includes 195
cget 196
childcount 196
childindex 196
childkeys 196
collapse 196
collapseall 197
columnattrib 197
columncget 197
columnconfigure 197
columncount 197
columnindex 197

columnwidth 198
configcelllist 198
configcells 198
configcolumnlist 198
configcolumns 199
configrowlist 199
configrows 199
configure 199
containing 200
containingcell 200
containingcolumn 200
cornerlabelpath 200
cornerpath 200
curcellselection 203
curselection 204

D

Das Erscheinungsbild eines Tablelist-Widgets verbessern 40
delete 204
deletecolumns 204
depth 204
descendantcount 204
dicttoitem 205
dumptofile 205
dumptostring 205

E

editcell 205
editinfo 206
editwinpath 206
editwintag 206
Ein Tablelist-Widget mit skalierbaren eingebetteten Fenstern 51
embedcheckbutton 206
embedcheckbuttons 207
embedttkcheckbutton 207
embedttkcheckbuttons 207
entrypath 207
expand 207
expandall 208
expandedkeys 208

F

FARBEN UND SCHRIFTEN 174
fillcolumn 208
findcolumnname 208
findrowname 208
finishediting 209
formatinfo 210

G

get 211
getcells 211
getcolumns 211
getformatted 212
getformattedcells 212
getformattedcolumns 212
getfullkeys 212
getkeys 213

H

hasattrib 213
hascellattrib 213
hascolumnattrib 213
hasrowattrib 214
header 214
header bbox 233
header cellattrib 233
header cellbbox 233
header cellcget 233
header cellconfigure 233
header cellindex 233
header configcelllist 233
header configcells 233
header configrowlist 234
header configrows 234
header containing 234
header containingcell 234
header delete 234
header embedcheckbutton 234
header embedcheckbuttons 234
header embedttkcheckbutton 234
header embedttkcheckbuttons 234
header fillcolumn 234
header findrowname 234
header get 234
header getcells 235
header getcolumns 235
header getformatted 235
header getformattedcells 235
header getformattedcolumns 235
header getfullkeys 235
header getkeys 235
header hascellattrib 235
header hasrowattrib 235
header imagelabelpath 235
header index 235

header insert 235
header insertlist 235
header iselemsnipped 235
header itemlistvar 235
header nearest 235
header nearestcell 235
header rowattrib 235
header rowcget 236
header rowconfigure 236
header rowcount 236
header size 236
header unsetcellattrib 236
header unsetrowattrib 236
header windowpath 236
headerpath 214
headertag 214
hidetargetmark 214

I

imagelabelpath 214
index 3, 215
insert 215
insertchildlist 215
insertchildren 216
insertcolumnlist 216
insertcolumns 216
insertlist 217
Interaktive Zellbearbeitung mit dem checkbutton-Widget 90
Interaktive Zellbearbeitung mit dem Menüschaltflächen-Widget 91
Interaktive Zellbearbeitung mit dem Text-Widget 89
Interaktive Zellbearbeitung mit dem Widget "ttk::combobox" 86
Interaktive Zellbearbeitung mit dem Widget "ttk::entry" 86
Interaktive Zellbearbeitung mit dem Widget "ttk::menubutton" 87
Interaktive Zellbearbeitung mit dem Widget "ttk::spinbox" 86
Interaktive Zellbearbeitung mithilfe des Eingabe-Widgets 89
Interaktive Zellbearbeitung mithilfe des Spinbox-Widgets 90
INTERAKTIVE ZELLENBEARBEITUNG 174
iselemsnipped 217
isexpanded 217
istitlesnipped 217
isviewable 218
itemlistvar 218
itemtodict 218

K

KNOTENINDIZES 187
KOPFZEILENINDIZES 188

KOPFZELLENINDIZES 189

L

labelpath 219
labels 219
labeltag 219
labelwindowpath 219
loadfromfile 219
loadfromstring 220

M

Mehr zu Tablelist_tile 12
move 220
movecolumn 221

N

nearest 221
nearestcell 221
nearestcolumn 221
noderow 221

P

parentkey 221

R

refreshsorting 221
rejectinput 222
resetsortinfo 222
restorecursor 222
rowattrib 222
rowcget 222
rowconfigure 222
rowcount 223

S

scan 223
scan dragto 223
scan mark 223
searchcolumn 223
see 225
seecell 225
seecolumn 226
selection 226
selection anchor 226
selection clear 226
selection includes 226
selection set 226
separatorpath 226
separators 227
setbusycursor 227

showtargetmark 227
size 227
sort 227
sortbycolumn 228
sortbycolumnlist 228
sortcolumn 229
sortcolumnlist 229
sortorder 229
sortorderlist 229
SPALTENINDIZES 187
SPALTENKONFIGURATIONSOPTIONEN 157
stopautoscroll 229

T

tablelist 3
tablelist::addBWidgetComboBox 62
tablelist::addBWidgetEntry 61
tablelist::addBWidgetSpinBox 62
tablelist::addCtext 70
tablelist::addDateMentry 76
tablelist::addDateTimeMentry 78
tablelist::addFixedPointMentry 79
tablelist::addIncrCombobox 75
tablelist::addIncrDateTimeWidget 73
tablelist::addIncrEntryfield 73
tablelist::addIncrSpinint 74
tablelist::addIncrSpinner 74
tablelist::addIPAddrMentry 80
tablelist::addIPv6AddrMentry 80
tablelist::addOakleyCombobox 69
tablelist::addTimeMentry 77
tablelist::addToggleswitch 92
tablelist::addToSortColumns 67
tablelist::convEventFields 64
tablelist::delaySashPosUpdates 65
tablelist::getCurrentTheme 82
tablelist::getScalingPct 11
tablelist::getTablelistColumn 65
tablelist::getTablelistPath 64
tablelist::getThemes 82
tablelist::scalingpct 7
tablelist::setTheme 81
tablelist::setThemeDefaults 83
tablelist::sortByColumn 67
tablelist::svgfmt 11
tablelistBinding 63
tablelistBWidget 61

tablelistColSort 66
tablelistCombobox 68
tablelistCtext 70
tablelistIwidgets 72
tablelistMentry 76
tablelistThemes 81
tablelistTile 84
tablelistTkCore 88
tablelistTsw 91
tablelistWidget 92
targetmarkpath 230
targetmarkpos 230
Tile-basierte Demo-Skripte 56
togglecolumnhide 230
togglerowhide 230
toplevelkey 231

U

unsetattrib 231
unsetcellattrib 231
unsetcolumnattrib 231
unsetrowattrib 231
UNTERBEFEHLE FÜR DEN KOPFBEREICH 233
UNTERSTÜTZUNG FÜR ZIEHEN UND ABLEGEN 177

V

viewablerowcount 231
VIRTUELLE EREIGNISSE 180

W

windowpath 231

X

xview 231

Y

yview 232

Z

ZEILENINDIZES 185
ZELLENINDIZES 188
Zwei skalierbare Widget-Browser auf Basis einer Tablelist 21